

机器学习原理及应用

李石峰

2024

目录

第一章 绪论	17
1.1 基本概念	17
1.1.1 核心要素	18
1.1.2 机器学习分类	18
1.1.3 应用领域	18
1.2 发展历程	18
1.2.1 萌芽与发展初期	18
1.2.2 关键理论与技术的突破期	19
1.2.3 深度学习爆发期	19
1.2.4 未来展望	19
1.3 重要性和挑战	19
1.3.1 机器学习的重要性	19
1.3.2 机器学习的挑战	20
1.4 外延阅读材料	20
1.4.1 机器学习参考书	20
1.4.2 深度学习参考书	21
1.4.3 学术论文与综述文章	21
1.4.4 在线课程与视频教程	22
1.4.5 大语言模型	22
第二章 基本术语及程序运行环境	23
2.1 核心术语的直观理解	23
2.2 核心术语的专业定义	24
2.3 程序环境安装及配置	25
2.3.1 Anaconda 安装	25
2.3.2 环境配置	25
2.3.3 Python 库安装	26
2.4 集成开发环境	26
2.4.1 VSCode 安装和配置	26
2.4.2 PyCharm 安装和配置	27
2.5 免费在线开发资源	28
2.5.1 Kaggle	28
2.5.2 阿里天池	29
2.5.3 AWS SageMaker Studio (基础版)	30
2.5.4 选择与使用建议	30

2.6	实践练习	31
2.6.1	Numpy 练习	31
2.6.2	Pandas 练习	32
2.6.3	Matplotlib 练习	33
2.6.4	Scikit-learn 练习	34
2.6.5	综合练习	35
2.7	习题	37
2.7.1	简答题	37
2.7.2	计算题	37
第三章	模型评估	39
3.1	分类问题评估指标	39
3.1.1	基本术语定义	39
3.1.2	准确率 (Accuracy)	40
3.1.3	精确率 (Precision) 与召回率 (Recall)	40
3.1.4	F1 分数 (F1 Score)	41
3.1.5	混淆矩阵 (Confusion Matrix)	41
3.2	回归问题评估指标	41
3.2.1	均方误差 (Mean Squared Error, MSE)	41
3.2.2	均方根误差 (Root Mean Squared Error, RMSE)	42
3.2.3	平均绝对误差 (Mean Absolute Error, MAE)	42
3.2.4	R 平方 (R-Squared)	42
3.3	性能度量方法	43
3.3.1	留出法 (Holdout Method)	43
3.3.2	K 折交叉验证 (K-Fold CV)	43
3.3.3	留一法交叉验证 (Leave One out CV)	43
3.3.4	自助法 (Bootstrap Methods)	43
3.3.5	交叉验证示例	44
3.3.6	超参数调优	44
3.3.7	超参数调优示例	45
3.3.8	模型对比与选择	46
3.4	实践练习	46
3.4.1	示例: 评估指标计算	46
3.4.2	示例: 模型调优	47
3.5	习题	48
3.5.1	简答题	48
3.5.2	计算题	49
3.5.3	编程题	49
第四章	线性模型	51
4.1	房价问题	51
4.1.1	问题提出	51
4.1.2	基本概念与应用	52
4.1.3	房价计算	52
4.2	简单线性模型	53

目录	5
4.2.1 问题设定	53
4.2.2 斜率的推导	54
4.2.3 截距项的推导	54
4.3 多元线性模型	54
4.3.1 基本概念与应用	55
4.3.2 多元房价问题	55
4.3.3 一般形式说明	57
4.4 逻辑回归模型	57
4.4.1 问题提出	57
4.4.2 成绩问题	58
4.4.3 问题求解	59
4.4.4 一般形式说明	60
4.4.5 牛顿法求解	61
4.4.6 逻辑回归的应用与总结	62
4.5 线性判别分析	63
4.5.1 录取预测问题	63
4.5.2 LDA 应用求解	63
4.5.3 一般形式说明	64
4.6 线性模型中的正则化	65
4.6.1 问题提出	65
4.6.2 正则化引入	65
4.6.3 求解过程	66
4.6.4 一般形式说明	66
4.6.5 直观认识	66
4.7 实践练习	67
4.7.1 示例：有无正则化对比	67
4.7.2 示例：房价预测	68
4.8 补充知识	70
4.8.1 矩阵逆、行列式和伴随矩阵	70
4.8.2 最小二乘法原理及其应用	73
4.8.3 矩阵求导及其应用	74
4.8.4 梯度	78
4.8.5 牛顿法的基础数学知识	79
4.8.6 协方差矩阵	81
4.8.7 广义特征值问题	82
4.9 习题	84
4.9.1 简答题	84
4.9.2 计算题	84
4.9.3 编程题	84
第五章 决策树模型	85
5.1 活动计划问题	85
5.1.1 周末计划问题	85
5.1.2 活动决策树的构建	85
5.2 决策树构建	87

5.2.1	数据集纯度的概念	87
5.2.2	信息熵：衡量不确定性	87
5.2.3	信息增益：减少不确定性	87
5.2.4	基尼指数：衡量不纯度	88
5.2.5	信息增益和基尼指数计算	88
5.3	剪枝操作	90
5.3.1	剪枝的必要性	90
5.3.2	预剪枝	91
5.3.3	后剪枝	93
5.4	连续属性与缺失值处理	94
5.4.1	连续属性的节点划分	94
5.4.2	缺失值的处理方法	96
5.5	实践练习	97
5.5.1	示例：鸢尾花分类问题	97
5.5.2	示例：加利福尼亚房价预测问题	99
5.5.3	示例：特征空间的划分	101
5.6	习题	103
5.6.1	简答题	103
5.6.2	计算题	103
5.6.3	编程题	104
第六章	支持向量机	105
6.1	水果分类问题	105
6.2	SVM 基本概念	106
6.2.1	SVM 的目标	106
6.2.2	间隔	106
6.2.3	硬间隔	107
6.2.4	软间隔	107
6.2.5	核函数	107
6.2.6	支持向量	108
6.2.7	对偶问题	108
6.3	硬间隔 SVM	108
6.3.1	基础概念	108
6.3.2	硬间隔 SVM 优化目标	109
6.4	凸优化直接求解	110
6.4.1	梯度下降法	111
6.4.2	牛顿法	111
6.4.3	内点法	111
6.5	对偶问题	112
6.5.1	原问题到对偶问题	112
6.5.2	KKT 条件	113
6.6	SMO 算法求解对偶问题	114
6.7	软间隔 SVM	115
6.8	SVM 直观理解	116
6.8.1	点到超平面的距离	117

6.8.2	平行超平面之间的距离	117
6.8.3	点沿某维度到超平面距离	117
6.9	软间隔 SVM 求解	119
6.10	支持向量回归	120
6.10.1	SVR 原始问题	120
6.10.2	SVR 对偶问题	121
6.10.3	对偶问题求解	121
6.11	核函数	122
6.11.1	核函数的定义	122
6.11.2	常用的核函数	123
6.11.3	核函数的选择	123
6.12	核技巧 (Kernel Trick)	123
6.12.1	核技巧直观理解	124
6.12.2	高斯径向基函数理解	124
6.13	实践练习	126
6.13.1	示例: 支持向量可视化	126
6.13.2	示例: 不同 γ 值高斯核函数可视化	127
6.13.3	示例: 手写数字识别	128
6.14	补充知识	130
6.14.2	向量范式基础知识	131
6.14.3	拉格朗日求解方法	133
6.14.6	张量积	136
6.15	习题	138
6.15.1	简答题	138
6.15.2	计算题	138
6.15.3	编程题	138
第七章	贝叶斯模型	139
7.1	信件问题	139
7.2	贝叶斯公式	139
7.2.1	贝叶斯决策论	139
7.2.2	贝叶斯公式推导	140
7.2.3	贝叶斯决策规则	140
7.3	0/1 损失最优贝叶斯	141
7.3.1	最优贝叶斯分类器	141
7.3.2	后验概率直接计算的挑战	142
7.3.3	贝叶斯定理引入	142
7.3.4	贝叶斯定理引入的问题	142
7.3.5	相关解决方案	143
7.4	朴素贝叶斯方法	143
7.4.1	邮件分类问题	143
7.4.2	直接计算的挑战	143
7.4.3	朴素贝叶斯假设	144
7.4.4	朴素贝叶斯解决邮件分类	144
7.4.5	朴素贝叶斯一般形式	144

7.5	参数估计	144
7.5.1	离散特征参数估计	144
7.5.2	连续特征	145
7.6	半朴素贝叶斯	146
7.6.1	半朴素贝叶斯的基本思想	146
7.6.2	依赖关系引入	146
7.6.3	SPODE 半朴素贝叶斯分类器	147
7.6.4	AODE 半朴素贝叶斯分类器	148
7.6.5	TAN 半朴素贝叶斯分类器	149
7.7	实践练习	150
7.7.1	示例：电子邮件分类	150
7.7.2	示例：疾病诊断	152
7.7.3	示例：半朴素贝叶斯	154
7.8	补充知识	155
7.8.1	概率论的基础概念	155
7.8.2	拉普拉斯平滑	158
7.8.3	最大似然估计正态分布的参数	159
7.9	习题	160
7.9.1	简答题	160
7.9.2	计算题	161
7.9.3	编程题	161
第八章	聚 类	163
8.1	照片分类问题	163
8.2	聚类性能度量	163
8.2.1	内部指标	163
8.2.2	外部指标	165
8.2.3	可视化辅助度量	167
8.2.4	综合评价	167
8.2.5	性能度量小结	167
8.3	聚类中的距离度量	167
8.3.1	欧氏距离	167
8.3.2	曼哈顿距离	168
8.3.3	闵可夫斯基距离	168
8.3.4	余弦相似度	168
8.3.5	马氏距离	168
8.3.6	汉明距离	169
8.3.7	杰卡德距离	169
8.3.8	编辑距离	169
8.3.9	小结	169
8.4	原型聚类	169
8.4.1	原型聚类的基本思想	169
8.4.2	K-Means	170
8.4.3	K-Means++	171
8.4.4	模糊 C 均值聚类	173

8.4.5	高斯混合模型	175
8.5	密度聚类	176
8.5.1	密度聚类的基本概念	176
8.5.2	DBSCAN 算法	177
8.5.3	OPTICS 算法	178
8.6	分层聚类	180
8.6.1	分层聚类的基本概念	180
8.6.2	凝聚层次聚类	180
8.6.3	分裂层次聚类	182
8.7	实践练习	184
8.7.1	示例：基于 K-Means 的客户分群	184
8.7.2	示例：基于 DBSCAN 的异常检测	184
8.7.3	示例：基于凝聚层次聚类的文档分类	186
8.7.4	示例：照片分类	186
8.8	习题	188
8.8.1	简答题	188
8.8.2	计算题	189
8.8.3	编程题	189
第九章	数据降维	191
9.1	水果营养分析问题	191
9.1.1	从故事到原理	191
9.1.2	水果降维	192
9.2	主成分分析	193
9.2.1	PCA 目标函数	193
9.2.2	PCA 拉格朗日求解	194
9.3	PCA 核心要素直观理解	194
9.3.1	特征值分解理解	195
9.3.2	特征向量可视化	196
9.4	核化线性降维	197
9.4.1	PCA 的局限性	197
9.4.2	核化线性降维	197
9.4.3	核化 PCA	197
9.4.4	核化 PCA 实例	198
9.5	线性判别分析降维	201
9.6	实践练习	201
9.6.1	示例：PCA 降维	201
9.6.2	示例：LDA 在鸢尾花数据集上的应用	202
9.6.3	示例：核化 PCA 在 Iris 数据集的应用	204
9.7	补充知识	206
9.7.1	协方差矩阵	206
9.7.2	特征值和特征向量	206
9.7.3	协方差矩阵	207
9.7.4	矩阵特征分解	210
9.8	习题	213

9.8.1	简答题	213
9.8.2	计算题	213
9.8.3	编程题	213
第十章	集成学习	215
10.1	竞赛问题	215
10.1.1	竞赛策略问题	215
10.1.2	集成学习的核心思想	216
10.1.3	Bagging: 独立训练, 集体决策	216
10.1.4	Boosting: 相互补强, 构建更强预测力	216
10.1.5	Stacking: 多元视角, 统一判断	216
10.2	集成学习策略及经典实现	217
10.2.1	Bagging 和随机森林	217
10.2.2	Boosting 和 AdaBoost 模型	218
10.2.3	Stacking 和元模型整合	219
10.3	集成学习数学描述	221
10.3.1	随机森林回归的数学描述	221
10.3.2	AdaBoost 的数学描述	222
10.3.3	Stacking 模型的数学描述	224
10.4	实践练习	225
10.4.1	示例: 鸢尾花二分类	225
10.4.2	示例: 房价预测	227
10.4.3	示例: 集成学习方法对比	228
10.5	补充知识	230
10.6	习题	231
10.6.1	简答题	231
10.6.2	计算题	232
10.6.3	编程题	232
第十一章	特征选择	233
11.1	图像分类问题	233
11.2	过滤法	234
11.2.1	皮尔逊相关系数	234
11.2.2	互信息	234
11.2.3	卡方检验	235
11.3	包裹法	235
11.3.1	递归特征消除	236
11.3.2	前向选择	236
11.3.3	后向消除	237
11.4	嵌入法	237
11.4.1	L1 正则化	237
11.4.2	决策树及其变体	238
11.5	实践练习	239
11.5.1	示例: 乳腺癌分类	239
11.5.2	示例: 各类特征选择方法对比	240

目录	11
11.5.3 示例：泰坦尼克号生存预测	242
11.6 习题	245
11.6.1 简答题	245
11.6.2 计算题	245
11.6.3 编程题	245
第十二章 参考文献	247

内容简介

机器学习作为人工智能领域的核心支撑技术，已成为现代智能系统开发的重要方法论。本教材构建“基础理论-数学模型-工程实现”三位一体的教学体系，着力培养读者系统性解决实际问题的能力。全书采用模块化知识架构，共分三大部分 11 个教学单元：基础理论篇（第 1-3 章）系统阐释机器学习的基础范式，涵盖学科发展脉络、基本术语体系、模型评估方法论以及数据预处理关键技术，为后续学习奠定理论基础。经典方法篇（第 4-10 章）深入解析七大核心算法体系：从线性回归、决策树等监督学习经典模型，到支持向量机、贝叶斯分类器等概率推断方法，延伸至聚类分析、维度压缩等无监督技术，并详解集成学习框架的构建原理。各章均配备公式推导图解、Python 实现案例及典型应用场景分析。进阶应用篇（第 11 章）聚焦工业级特征工程，系统讲解特征构造、选择与优化的工程化方法，培养读者处理高维复杂数据的实战能力。教材编写突出以下特色：采用生活化案例导入理论模型，通过数学推导揭示算法本质，配合可复现的编程实例强化工程实践。每章设置课后习题和延伸阅读指南，重要公式均辅以推导注释，关键算法提供完整代码框架。通过“原理-推导-实现”的递进式教学设计，帮助读者构建完整的知识图谱。

本教材适用于计算机科学、智能科学与技术等相关专业的本科及研究生教学，亦可作为人工智能工程师的参考用书。配套提供教学课件、实验指导手册及扩展数据集，满足不同层次学习者的需求。

前言

在这个日新月异的时代，技术的每一次飞跃都深刻地改变着整个世界。其中，机器学习作为人工智能领域的一颗璀璨明珠，正以前所未有的速度引领着科技的进步与变革。它不仅重塑了人类对数据处理、模式识别及决策制定的理解，更在医疗健康、金融分析、自动驾驶、智能制造、自然语言处理等众多领域展现出了巨大的应用潜力和价值。因此，撰写一本全面而深入的机器学习教材，不仅是对当前技术潮流的积极响应，更是对未来人才培养和社会发展的深远布局。本书旨在为读者提供一条清晰、系统的学习路径，从机器学习的基础理论出发，逐步深入到算法原理、模型构建、实践应用及前沿探索等多个层面。我们力求做到内容全面、结构严谨、实例丰富、讲解透彻，既适合作为高等院校计算机科学与技术、人工智能、数据科学等相关专业的学生教材，也可作为科研工作者、工程技术人员及广大机器学习爱好者的自学参考书。在编写过程中，特别注重以下几个方面的平衡与融合：

理论与实践并重：理论学习是构建知识体系的基础，而实践操作则是检验理论、深化理解的必要途径。因此，本书在详细介绍机器学习基本理论的同时，也提供了大量的实验案例和代码实现，帮助读者在动手实践中掌握核心技能。

经典与现代融合：机器学习领域的发展日新月异，新技术、新方法层出不穷。本书在保留经典算法和理论精髓的基础上，也积极引入最新的研究成果和应用案例，使读者能够紧跟时代步伐，把握领域前沿。

广度与深度兼顾：机器学习是一个涉及数学、统计学、计算机科学等多个学科的交叉领域。本书在保持内容广度的同时，也针对关键知识点进行了深入的剖析和拓展，以满足不同读者的学习需求。

启发式教学：本书深知，学习的最终目的是培养学生的创新思维和解决问题的能力。因此，本书在介绍知识的同时，也注重启发读者的思考，鼓励读者在理解的基础上进行探索和创新。

本书在内容构建与呈现上，力求体现以下几大特色：

1. 配套资源丰富，实战性强

为了让读者能够更好地理解和掌握机器学习技术，本书配套了丰富的资源。所有实例均附有详细的代码实现，读者可以直接在本地环境中运行这些代码，通过实践加深对理论知识的理解。此外，本书还提供了数据集、实验环境配置指南等辅助材料，确保读者能够顺利地开展学习和研究工作，相关程序和配套电子资源可以在<https://github.com/SVIL2024/Book-MLPA.git>上获取。

2. 语言简单易懂，易于入门

对于初学者而言，复杂的数学公式和术语往往构成学习障碍。因此，本书在编写过程中特别注重语言的简洁性和易懂性，以主人公石榴同学学习机器学习的故事为主线，力求用平实的语言阐述复杂的机器学习概念。同时，各章节之间相对独立，读者可以根据自身的学习进度和兴趣选择性地阅读，降低了学习门槛，提高了学习效率。

3. 注重思想解析，深化理解

在介绍机器学习算法和模型时，本书不仅关注数学公式的推导和计算过程的展示，更注重对算法背后思想和直觉的解析。只有深入理解算法的本质和原理，才能真正掌握其精髓，并灵活运用到实际问题中去。因此，在每个章节中，本书都努力挖掘算法背后的思想脉络，帮助读者建立清晰的

知识框架和思维方式。更进一步，为了帮助读者深入理解算法的数学本质，特别在部分章节末尾提供了与该章节相关的数学基础，并进行了系统而通俗易懂的讲解。

4. 数学概念适度，强调应用

虽然机器学习离不开数学的支持，但本书并不一味罗列数学公式和定理。相反，根据实际应用的需要，适度引入必要的数学概念和工具，并注重通过实例和案例来展示这些数学工具在机器学习中的应用。这样既能满足读者对理论知识的需求，又能增强其实践能力和创新思维。

最后，我要感谢所有参与本书编写和审校工作的同仁们，是你们的辛勤付出和无私奉献，才使得这本书能够顺利问世。同时，我也要感谢广大读者对本书的关注和期待，你们的支持是我们不断前进的动力。愿这本书能够成为你探索机器学习领域的得力助手，引领你走向更加辉煌的未来。

第一章 绪论

在当今这个信息爆炸的时代，数据已成为驱动社会进步和经济发展的新燃料。随着互联网的普及和物联网技术的飞速发展，数据量以前所未有的速度增长，如何从这些海量数据中提取有价值的信息，成为了一个亟待解决的重要问题。正是在这样的背景下，机器学习作为一门跨学科领域，迅速崛起并展现出强大的生命力和广泛的应用前景。机器学习，作为人工智能的一个重要分支，其核心思想在于让计算机系统能够从数据中自动学习并改进自身的性能，而无需进行明确的编程。这一过程不仅模拟了人类从经验中学习的能力，还极大地扩展了计算机处理复杂任务的范围和精度。通过机器学习，可以从大量数据中发现隐藏的模式和规律，进而解决传统方法难以应对的问题，如图像识别、语音识别、自然语言处理等。机器学习的兴起，不仅推动了计算机科学的进步，也深刻改变了众多行业的发展轨迹。在医疗健康领域，机器学习技术被用于疾病预测、药物研发等方面；在金融领域，它帮助银行和企业实现风险评估、欺诈检测等功能；在智能制造领域，机器学习则助力实现生产过程的自动化和智能化。这些应用实例充分展示了机器学习的巨大潜力和社会价值。然而，机器学习的研究与应用并非一帆风顺。面对复杂多变的数据环境和不断涌现的新技术挑战，需要不断探索和创新，以应对日益增长的数据处理需求和更加智能化的应用场景。因此，掌握机器学习的基础知识、理解其背后的原理和方法论、熟悉其在实际问题中的应用实践，对于每一位从事计算机科学、数据科学及相关领域的研究者、工程师和从业者来说都至关重要。正是基于这样的认识和理解，编写了这本机器学习教材。本书旨在通过系统而深入的讲解，帮助读者建立起对机器学习的全面认识，掌握其核心技术和方法，并激发他们在该领域的探索精神和创新能力。通过本书的学习和实践，读者将能够开启一段充满挑战与机遇的机器学习之旅。

学习目标

本章旨在引导读者全面了解机器学习的基本概念、发展历程、主要分类及其应用领域，明确其在人工智能体系中的关键地位。通过对比传统编程方法，突出机器学习以数据驱动、自动优化模型核心理念。梳理其从起源至今的发展脉络，帮助读者构建整体认知框架。系统介绍监督学习、无监督学习、半监督学习与强化学习的基本原理、适用场景及优劣，为后续学习奠定理论基础。结合医疗、金融、制造、自动驾驶等领域的实际案例，展现机器学习的广泛应用与实践价值。最后，明确学习目标与路径，引导读者根据自身需求制定合理的学习计划，为深入掌握相关技术做好准备。

1.1 基本概念

机器学习是人工智能领域中的一个重要分支，它专注于开发能够自动从数据中学习并改进其性能的算法和模型。其核心思想在于，通过让计算机系统分析大量数据，自动识别出数据中的模式和规律，从而无需进行显式的编程即可对新数据进行预测或分类。定义机器学习可以被定义为一种使计算机系统能够自动地从经验中改进的方法。这里的“经验”通常指的是数据，而“改进”则指的是提高某项任务的性能。在机器学习的上下文中，任务可以是分类、回归、聚类、异常检测等。

1.1.1 核心要素

数据：机器学习算法的训练和测试都依赖于数据。数据可以是结构化的（如数据库中的表格数据），也可以是非结构化的（如文本、图像、音频等）。

模型：模型是机器学习算法的核心组成部分，它定义了如何从数据中提取特征和规律。模型可以是简单的线性回归模型，也可以是复杂的深度神经网络。

目标函数：目标函数用于评估模型在给定数据上的性能。在监督学习中，目标函数通常与真实值和预测值之间的差异有关（如均方误差）；在无监督学习中，则可能涉及数据内部结构的某种度量（如聚类内部的紧密度和聚类间的分离度）。

优化算法：优化算法用于调整模型参数以最小化目标函数。常见的优化算法包括梯度下降法、随机梯度下降法、牛顿法等。

1.1.2 机器学习分类

根据学习方式的不同，机器学习可以分为以下几类：

监督学习：在监督学习中，算法从一组包含输入和对应输出的训练数据中学习。目标是学习一个函数，该函数能够将输入映射到输出。例如，在图像识别任务中，输入是图像，输出是图像中的对象类别。

无监督学习：与监督学习不同，无监督学习算法处理的数据没有明确的标签或输出。算法的任务是发现数据中的内在结构或模式。聚类是无监督学习的一个典型例子。

半监督学习：半监督学习介于监督学习和无监督学习之间。它使用部分标记的数据和大量未标记的数据进行训练。这种方法利用了未标记数据的分布信息来提高学习性能。

强化学习：强化学习是一种通过试错来学习的方法。在强化学习中，算法（称为代理）在环境中采取行动，并根据这些行动的结果（奖励或惩罚）来学习如何最大化长期累积奖励。

1.1.3 应用领域

机器学习的应用领域非常广泛，包括但不限于图像识别、语音识别、自然语言处理、推荐系统、金融风控、医疗健康等。随着技术的不断进步和数据量的持续增长，机器学习的应用前景将更加广阔。

1.2 发展历程

机器学习的发展历程可以追溯到多个学科领域的交叉融合与不断探索之中。从最初的萌芽到如今的蓬勃发展，机器学习经历了从理论构建到实践应用的深刻转变，成为了推动科技进步的重要力量。

1.2.1 萌芽与发展初期

机器学习的起源可以追溯到 20 世纪 40 年代至 60 年代，这一时期是人工智能和机器学习的早期探索阶段。在这一时期，计算机科学的先驱们开始思考如何让机器能够像人类一样学习和决策。1950 年，艾伦·图灵提出了著名的“图灵测试”，为人工智能的发展设定了一个明确的目标。随后，塞缪尔开发的跳棋程序成为了早期机器学习的经典案例，它通过自我对弈不断改进策略，展示了机器从经验中学习的潜力。

1.2.2 关键理论与技术的突破期

进入 20 世纪 70 年代至 90 年代，机器学习领域迎来了关键理论与技术的突破。这一时期，统计学习理论、神经网络、决策树等算法和技术相继涌现，为机器学习的发展奠定了坚实的基础。支持向量机（SVM）作为一种有效的分类方法被提出，并在文本分类、图像识别等领域展现出优异性能。同时，随着计算能力的提升和数据的日益丰富，神经网络开始展现出强大的学习能力，尽管当时受限于计算资源和训练技巧，但其潜力已初露锋芒。

1.2.3 深度学习爆发期

进入 21 世纪，随着大数据时代的到来和计算能力的飞跃式发展，深度学习技术迅速崛起并引领了机器学习的新一轮革命。2006 年，Hinton 等人提出了深度信念网络（DBN）及其训练算法，为深度学习的复兴奠定了基础。随后几年里，卷积神经网络（CNN）、循环神经网络（RNN）等深度学习模型相继被提出并应用于图像识别、语音识别、自然语言处理等领域，取得了前所未有的成功。特别是 2012 年 ImageNet 竞赛中，AlexNet 模型以巨大优势夺冠，标志着深度学习技术在计算机视觉领域的全面领先。

1.2.4 未来展望

随着深度学习技术的不断成熟和普及，机器学习已经广泛应用于医疗健康、金融风控、智能制造、自动驾驶等多个领域。在医疗健康领域，机器学习技术被用于疾病预测、辅助诊断等方面；在金融领域，它则帮助银行和企业实现风险评估、欺诈检测等功能。未来，随着技术的不断进步和应用场景的持续拓展，机器学习有望在更多领域发挥重要作用，推动社会进步和经济发展。总之，机器学习的发展历程是一个从理论探索到实践应用的不断演进过程。随着技术的不断进步和创新，机器学习将在未来发挥更加重要的作用，为人类社会的进步贡献更大的力量。

1.3 重要性和挑战

1.3.1 机器学习的重要性

在当今这个数据驱动的时代，机器学习的重要性日益凸显。它不仅推动了科技的飞速发展，也深刻改变了我们的生活方式和商业模式。以下是机器学习重要性的几个关键方面：

数据洞察与决策支持：随着大数据时代的到来，企业积累了海量数据。机器学习技术能够从这些数据中挖掘出有价值的信息和模式，为企业的决策提供有力支持。通过智能分析，企业可以更准确地预测市场趋势、评估客户需求、优化运营流程，从而提高竞争力和盈利能力。

创新产品与服务：机器学习技术为新产品和服务的开发提供了无限可能。通过自动学习用户的偏好和行为模式，企业可以定制化地推出更符合市场需求的产品和服务。例如，智能推荐系统可以根据用户的浏览历史和购买记录，精准推送个性化的商品信息，提升用户体验和满意度。

效率提升与成本降低：机器学习能够自动化处理大量重复性任务，减轻人工负担，提高工作效率。同时，通过优化资源配置和预测维护需求，企业可以降低运营成本，提高整体效益。例如，在制造业中，机器学习可以实时监测设备运行状态，提前预警潜在故障，减少停机时间和维修成本。

解决复杂问题：许多传统方法难以解决的复杂问题，如图像识别、语音识别、自然语言处理等，在机器学习的帮助下取得了突破性进展。这些技术的应用不仅提高了处理速度和准确性，还极大地扩展了人工智能的应用范围。

1.3.2 机器学习的挑战

尽管机器学习带来了诸多优势，但其发展也面临着诸多挑战：

数据质量与隐私保护：高质量的数据是机器学习模型有效性的基础。然而，在实际应用中，数据往往存在缺失、错误或不一致等问题。此外，随着隐私保护意识的增强，如何在保护用户隐私的同时有效利用数据成为了一个亟待解决的问题。

模型可解释性与透明度：虽然机器学习模型在预测和分类方面表现出色，但其决策过程往往缺乏可解释性。这导致企业在应用机器学习技术时难以理解和信任模型的输出结果。因此，提高模型的可解释性和透明度是机器学习领域的一个重要研究方向。

算法公平性与偏见：机器学习模型在训练过程中可能会受到训练数据中的偏见影响，导致模型在实际应用中产生不公平的结果。例如，在招聘系统中，如果训练数据中存在性别或种族偏见，那么模型可能会倾向于推荐某些特定群体而非其他群体。因此，确保算法的公平性和无偏见是机器学习领域面临的一个重大挑战。

计算资源与能耗：深度学习等复杂机器学习模型的训练需要消耗大量的计算资源和能源。这不仅增加了企业的运营成本，还可能对环境造成负面影响。因此，如何在保证模型性能的同时降低计算复杂度和能耗是机器学习领域需要解决的一个问题。

综上所述，机器学习的重要性不言而喻，但其发展也面临着诸多挑战。只有通过不断创新和优化技术、加强数据管理和隐私保护、提高模型可解释性和公平性等措施，才能充分发挥机器学习的潜力并推动其可持续发展。

1.4 外延阅读材料

1.4.1 机器学习参考书

- 《机器学习》（周志华著）：这本书被誉为“西瓜书”，是机器学习领域的经典教材之一。它不仅详细介绍了机器学习的基础理论、算法和应用，还通过丰富的实例和案例帮助读者理解复杂的概念。绪论部分对机器学习的定义、重要性、发展历程等进行了全面阐述，是理解机器学习全貌的重要入口。
- 《统计学习方法》（李航著）：这本书系统地介绍了统计学习理论和方法，包括感知机、k 近邻法、朴素贝叶斯法、决策树、支持向量机等多种算法。与“西瓜书”相辅相成，为读者提供了更全面的视角。
- 《Machine Learning》是 Tom M. Mitchell 的经典教材，系统阐述机器学习理论与方法，涵盖监督、无监督、强化学习及决策树、神经网络、贝叶斯方法等核心算法，结合数学推导与实例解析其原理与性能边界，理论与实践并重，逻辑严谨，适合计算机科学、统计学等领域入门学习与研究，被誉为机器学习领域的奠基性著作。
- 《Pattern Recognition and Machine Learning》（PRML）是 Christopher M. Bishop 的经典著作，以概率统计为核心框架，系统讲解机器学习与模式识别理论，涵盖贝叶斯方法、图形模型、监督与无监督学习、高斯过程等核心内容，结合数学推导与直观图表解析算法原理，适合具备统计与数学基础的学习者深入理解机器学习的统计建模思路。
- 《The Elements of Statistical Learning: Data Mining, Inference, and Prediction》由 Trevor Hastie、Robert Tibshirani 和 Jerome Friedman 合著，是统计学习领域的权威经典教材。第二版（2009 年）系统覆盖监督学习框架下的核心方法，包括线性回归、分类、广义线性模型、核方法及概率图模型等，并深入探讨流形学习等前沿技术的数学原理。书中以矩阵论与多元统计为基础构建理论体系，通过严谨推导与直观案例结合的方式，帮助读者理解数据建模与预测的

核心逻辑。其内容难度较高，既适合作为高年级本科生和研究生的教材，也为研究者提供了从统计决策理论到实际算法实现的完整分析框架。

- 《Machine Learning: A Probabilistic Perspective》由 Kevin P. Murphy 撰写，以概率理论为统一框架，全面系统地阐述了机器学习的核心方法与技术，涵盖贝叶斯推理、概率图模型、监督与无监督学习、马尔可夫链、深度学习等主题，通过严谨的数学推导与直观的案例解析算法原理，强调概率建模在复杂问题中的应用，适合具备数学基础的学习者深入掌握机器学习的理论与实践。
- 《Machine Learning: The Art and Science of Algorithms that Make Sense of Data》由 Peter Flach 所著，以独特视角将机器学习视为算法解析数据的艺术与科学，系统阐述了分类、聚类、回归、矩阵分解等核心方法，通过统一框架比较不同算法的原理与适用场景，结合实际案例与直观解释降低理解门槛，强调算法设计的逻辑与数据洞察的平衡，适合非数学背景的学习者及实践者建立全面的机器学习思维框架。

1.4.2 深度学习参考书

- 《Deep Learning》由 Ian Goodfellow、Yoshua Bengio 和 Aaron Courville 合著，是深度学习领域的权威教材，系统阐述了神经网络与深度学习的核心理论与技术，涵盖数学基础、前馈网络、卷积网络、循环网络、生成模型、强化学习等主题，结合数学推导与实际案例解析算法原理与工程实践，内容全面且前沿，适合具备数学基础的研究人员、工程师及高阶学习者作为深度学习的系统性参考。
- 《Deep Learning with Python》由 François Chollet (Keras 框架作者) 撰写，以实战为导向，通过 Keras 和 TensorFlow 讲解深度学习核心概念与技术，涵盖图像分类、自然语言处理、生成模型等典型任务，结合代码示例解析模型设计与训练流程，强调快速实现项目与调优技巧，适合编程基础扎实的学习者及工程师快速上手深度学习应用开发。
- 《Reinforcement Learning: An Introduction》由 Richard S. Sutton 和 Andrew G. Barto 合著，是强化学习领域的经典教材。该书第二版在第一版（1998 年）基础上更新，系统阐述了强化学习的核心理论与算法，涵盖从基础概念（如马尔可夫决策过程、动态规划）到前沿方法（如深度强化学习）的完整框架，并通过丰富的示例与数学推导帮助读者建立直观理解。其内容注重理论与实践结合，既适合计算机科学、人工智能等领域的研究者，也为工程应用提供了清晰的指导路径。

1.4.3 学术论文与综述文章

- 《Deep Learning in Neural Networks: An Overview》由 Schmidhuber 发表于《Neural Networks》（2015），系统阐述了深度神经网络的核心理论与技术进展，涵盖反向传播、优化算法（如梯度下降、Adam）及正则化方法（如 Dropout）等训练技术，分析了前馈网络、卷积神经网络（CNN）、循环神经网络（RNN）及生成模型（如 VAE、GAN）的架构与原理，结合图像识别、语音处理、自然语言处理等应用案例，探讨了过拟合、计算复杂度等挑战，并展望了迁移学习、无监督学习等方向，为理解深度学习的理论基础、技术实现与应用前景提供了权威综述。
- 《A Comprehensive Survey on Graph Neural Networks》由 Zonghan Wu 等发表于《IEEE Transactions on Neural Networks and Learning Systems》（2021），系统综述了图神经网络（GNN）的研究进展，涵盖模型架构（如图卷积网络 GCN、图注意力网络 GAT、图生成模型）、训练方法（如图嵌入、消息传递机制）及优化挑战（如过平滑、计算效率），并结合社交网络分析、推荐系统、化学分子预测等应用案例，探讨了 GNN 在图结构数据建模中的优势与局限。

性，为研究者与工程师提供了图神经网络的理论基础、技术实现与应用方向的权威参考。

- 建议研读 Yann LeCun 等学者发表的权威综述文章，此类文献系统性梳理了深度学习的理论基础、核心技术、实际应用及前沿方向，能够帮助学习者全面理解深度学习的发展历程、数学原理、跨领域应用范式以及未来研究趋势，为构建系统性知识框架提供理论支撑。
- 深度学习领域的经典研究论文（如 AlexNet [Krizhevsky et al., 2012]、ResNet [He et al., 2016] 等）是理解模型创新与研究方法的核心文献。这些论文不仅系统阐述了模型的创新架构设计，还通过严谨的实验设计、定量与定性结果分析，以及对技术局限性和未来研究方向的前瞻性探讨，为学习者提供了从理论到实践的完整研究范式。深入研读此类原始文献，能够帮助研究者掌握模型设计原理、工程实现逻辑，以及科学问题的系统性探索方法，是机器学习领域研究能力培养的重要基础。

1.4.4 在线课程与视频教程

- MOOCs（大型开放式在线课程）：许多知名大学和在线教育平台提供了机器学习的 MOOCs，如 Coursera、edX、中国大学 MOOC 等。这些课程通常由该领域的专家教授，内容涵盖机器学习的基础理论、算法、编程实践等方面。
- 技术论坛与博客：关注一些技术论坛（如 CSDN 博客、知乎等）上的机器学习相关讨论，可以获取最新的研究进展、技术难题和解决方案。

1.4.5 大语言模型

近年来，大语言模型（Large Language Model, LLM），特别是 DeepSeek 系列的出现，作为机器学习领域的突破性成果，正在重新定义人工智能的边界。其强大的表征能力和多任务处理能力，使其成为推动机器学习技术落地的核心工具之一。大语言模型正通过以下方式推动机器学习发展：

- 大语言模型（如 ChatGPT、DeepSeek）被整合到课程设计中，形成“人机协同学习模式”。例如，通过 LLM 辅助课程答疑、案例分析和实验指导，读者则可以借助 LLM 的实时反馈获得个性化学习支持。
- 通过快速生成多样化教学案例（如分类任务、神经网络调试场景），模拟真实实验环境，并实时整合最新技术动态更新教学内容，显著提升了机器学习教育的实践性与时效性。其多模态能力支持个性化案例定制，而动态数据接入能力则确保教学内容始终贴近技术前沿。
- 传统机器学习课程常因数学理论与编程实践的割裂导致学习困难。学生可要求 LLM 生成逻辑回归、神经网络等算法的代码框架，并解释其原理，减少“从零开始”的挫败感。通过对话形式提问（如“如何优化过拟合？”），LLM 能提供分步骤解答，这种交互式的学习方法能够帮助读者建立系统性知识网络。

第二章 基本术语及程序运行环境

机器学习作为人工智能领域的核心分支，正在深刻地改变我们的生活和工作方式。从推荐系统到自动驾驶，从医疗诊断到金融预测，机器学习技术已经渗透到各个行业，并展现出强大的潜力。然而，要真正掌握机器学习，首先需要对其基本术语和核心概念有深入的理解。这些术语不仅是构建机器学习知识体系的基石，也是读者在实际项目中分析问题、设计解决方案的重要工具。

本章将围绕机器学习领域中的核心基本术语展开学习。通过深入探讨监督学习、无监督学习、半监督学习、强化学习等不同学习范式，以及特征、模型、过拟合、欠拟合、交叉验证等关键概念，帮助读者建立起对机​​器学习的整体认知框架。同时，还将关注这些术语之间的关联与区别，理解它们在实际应用中的作用和意义。

此外，本章不仅注重理论知识的讲解，还鼓励读者培养对新术语和概念的好奇心，激发进一步探索的兴趣。通过对核心术语的全面理解，读者将能够更好地应对复杂的机器学习任务，并为后续深入学习打下坚实的基础。无论您是初学者还是希望巩固基础知识的从业者，本章都将为您提供有价值的指导和启发。

学习目标

掌握机器学习的核心理论与实践技能，包括监督、无监督、半监督和强化学习等范式，理解特征、模型、过拟合与欠拟合等关键概念，并熟悉交叉验证和性能评估指标。具备使用 Python 配置开发环境、运用 Scikit-learn、numpy 等工具进行数据预处理、模型训练与调优的能力，能完成从数据到部署的完整项目流程并提出优化建议。

2.1 核心术语的直观理解

在探索未知的征途中，石榴是一名实验学校的学生，她与机器学习结下了不解之缘。这一路上，她逐渐揭开了机器学习神秘面纱的一角，以下是她对几个核心术语的直观理解。

监督学习想象一下，石榴是一位园艺师，正在训练她的新助手识别不同种类的花卉。她展示给助手一朵朵花卉，并逐一告诉它们各自的名字（即标签）。助手通过观察花卉的颜色、形状等特征（输入），逐渐学会了如何准确地将新花卉归类。这个过程，正是监督学习的精髓所在：通过已知特征和对应标签的数据集，训练模型学会预测新数据的标签。

无监督学习：有一天，石榴在花园里发现了一堆混杂在一起的种子。她不清楚每颗种子的种类，但想要将它们分类整理。于是，她让助手根据种子的外观特征（如大小、颜色、纹理）自行分组。助手没有明确的分类标准，只能依靠自己的观察力和判断力。这便是无监督学习的魅力：在没有标签的数据集中，自动发现隐藏的结构和模式。

半监督学习：随着对花园管理的深入，石榴意识到完全标记所有种子既耗时又费力。因此，她采用了一种折中的方法——半监督学习。她标记了一部分种子作为“种子选手”，并让助手利用这些已知信息去推测未知种子的种类。这种方法结合了监督学习和无监督学习的优点，既利用了少量标

记数据的准确性，又利用了大量未标记数据的丰富性。

强化学习：石榴设计了一个自动浇水系统，该系统需要根据土壤湿度和植物需求来智能调节浇水量。她采用强化学习算法来训练这个系统。系统通过不断尝试不同的浇水量，并根据植物的生长情况（奖励或惩罚信号）来调整策略。这个过程就像是一个不断试错和优化的循环，旨在找到最优的浇水方案。强化学习让机器在不确定的环境中通过探索和利用来最大化长期回报。

特征、模型与预测：在机器学习中，“特征”是描述数据的关键属性，如同植物的叶子形状、颜色等；“模型”则是基于这些特征构建的函数或算法，用于预测未知数据的输出。石榴通过提取花卉的特征并训练模型，能够预测新花卉的种类。模型的准确性取决于特征的选择和模型的复杂度，合理的特征工程和模型优化是提高预测精度的关键。

过拟合与欠拟合：在训练模型的过程中，石榴遇到了两个常见问题：过拟合和欠拟合。过拟合是指模型在训练数据上表现完美但在新数据上表现糟糕的情况，就像助手过分记住了每颗种子的具体特征而忽略了泛化能力；欠拟合则是指模型在训练数据和新数据上都表现不佳的情况，如同助手无法从种子特征中提取出有用的分类信息。通过调整模型复杂度、增加数据量或使用正则化等方法可以有效避免过拟合和欠拟合问题。

交叉验证：为了确保模型的稳定性和可靠性，石榴采用了交叉验证的方法。她将花园中的种子分成多个子集进行多次训练和测试循环操作。通过比较不同子集上的测试结果来评估模型的泛化能力并调整参数以达到最优性能。交叉验证为石榴提供了一个客观评价模型性能的标准并帮助她选择出最佳的模型配置。

2.2 核心术语的专业定义

在深入探索机器学习的过程中，理解其核心术语的专业定义至关重要。以下是几个关键术语的详细解析。

监督学习 (Supervised Learning)：监督学习是一种机器学习方法，它使用一组包含输入特征和对应输出标签的训练数据集来训练模型。模型通过学习输入特征与输出标签之间的映射关系，能够对新的输入数据进行预测或分类。在监督学习中，每个输入样本都配有一个明确的输出标签，用于指导模型的训练过程。

无监督学习 (Unsupervised Learning)：与监督学习不同，无监督学习处理的数据集不包含明确的输出标签。无监督学习的目标是从数据中发现隐藏的结构或模式，如聚类分析、异常检测等。在无监督学习中，模型需要自行探索数据的内在特性，并通过相似性度量等方法将数据分组或排序。

半监督学习 (Semi-Supervised Learning)：半监督学习结合了监督学习和无监督学习的特点，它利用部分标记的数据和大量未标记的数据进行训练。通过利用未标记数据提供的分布信息，半监督学习能够提高模型的泛化能力，同时减少对数据标记的依赖。这种方法在处理标记数据稀缺的场景中尤为有用。

强化学习 (Reinforcement Learning, RL)：强化学习是一种通过试错来学习的方法，它模拟了生物体在环境中通过交互来优化自身行为的过程。在强化学习中，智能体 (agent) 在环境中执行动作，并根据动作的结果（奖励或惩罚）来调整其策略以最大化长期累积奖励。强化学习适用于解决序贯决策问题，如游戏对战、自动驾驶等复杂场景。

特征 (Feature)：在机器学习中，特征是指用于描述数据实例的属性或变量。特征的选择和工程对模型的性能有重要影响。有效的特征能够捕捉数据的本质信息，帮助模型更好地理解和预测数据。特征工程是机器学习项目中的关键环节，它涉及特征的提取、选择、转换和降维等操作。

模型 (Model)：模型是机器学习算法的核心组成部分，它定义了输入特征与输出预测之间的映射关系。模型可以是简单的线性回归模型，也可以是复杂的深度神经网络。在训练过程中，模型通过不断调整其参数以最小化损失函数（即预测误差），从而实现对新数据的准确预测。

过拟合 (Overfitting) 与欠拟合 (Underfitting): 过拟合和欠拟合是机器学习中常见的两个问题。过拟合是指模型在训练数据上表现非常好但在新数据上表现糟糕的情况,这通常是因为模型过于复杂而捕捉到了训练数据中的噪声或异常值。欠拟合则是指模型在训练数据和新数据上都表现不佳的情况,这可能是因为模型过于简单无法捕捉数据的内在规律。通过调整模型复杂度、使用正则化技术或增加数据量等方法可以有效缓解过拟合和欠拟合问题。

交叉验证 (Cross-Validation): 交叉验证是一种评估模型性能的技术,它将数据集分成多个子集进行多次训练和测试循环操作。在每次循环中,选择一个子集作为测试集用于评估模型性能,其余子集作为训练集用于训练模型。通过比较不同子集上的测试结果可以评估模型的稳定性和泛化能力。交叉验证有助于避免模型在特定训练集上过拟合的问题,并提供了一个更可靠的模型性能评估标准。

2.3 程序环境安装及配置

本书中所有的实战示例和代码都是使用 Python 语言完成的。Python 是一种广泛使用的高级编程语言,以其清晰的语法和代码可读性而闻名。它在数据科学、机器学习、网络服务器和桌面及网络应用程序开发中尤为流行。选择 Python 是因为它强大的数据处理能力,以及拥有丰富的库和框架,如 NumPy、Pandas、Matplotlib 和 Scikit-learn,这些工具使得数据分析和机器学习任务变得更加简单高效。

本节将介绍如何设置 Python 运行环境,特别是使用 Anaconda 进行环境安装和配置。Anaconda 是一个流行的 Python 发行版,它集成了大量用于科学计算、数据分析和机器学习的库和工具。

2.3.1 Anaconda 安装

为了提升下载速度,建议从国内镜像源下载 Anaconda。可以从清华大学开源软件镜像站或其他国内镜像源下载 Anaconda 安装包。安装完成后,可以通过 Anaconda Navigator 或 Anaconda Prompt 进行环境配置。因为 Anaconda Navigator 程序启动较慢,如果电脑硬件配置一般,建议大家使用 Anaconda Prompt,通过命令行的方式对环境进行配置。

2.3.2 环境配置

如果已经成功安装了 Anaconda,它会自带一个名为“base”的虚拟环境。你可以直接基于此环境进行开发,因为它已经安装了绝大多数常用的 Python 库,能够让你快速开始开发。然而,在某些情况下,你可能会遇到使用的 Python 库对 Python 版本有严格要求的问题。为避免这种情况,建议你创建自己的虚拟环境。在一般开发过程中,建议大家首先创建自己独立的虚拟环境,这样可以有效隔离项目依赖,避免版本冲突。创建虚拟环境的方法如下:

在程序菜单中打开 Anaconda Prompt,使用以下命令创建一个新的虚拟环境:

```
1 conda create -n myenv python=3.9
```

其中 myenv 是创建虚拟环境的名称,可以根据自己需要进行修改,这里指定 Python 的版本为 3.9,如果没有特殊要求,建议版本不要过高,过高的话可能和其他工具包出现兼容性问题。

激活虚拟环境:

```
1 conda activate myenv
```

为了加速第三方库的下载,建议使用国内镜像源,包括清华镜像源、中科院镜像源、阿里镜像源等。在 Anaconda 环境中配置镜像源通常涉及到修改 Anaconda 的配置文件,这个配置文件被称为 `.condarc`。该文件位于用户的主目录下,具体路径可能因操作系统而异:

- 在 Windows 系统中，通常位于 C:\Users\你的用户名\.condarc。
- 在 macOS 和 Linux 系统中，通常位于 ~/.condarc（即用户主目录下的\.condarc 文件）。

当使用 Anaconda 命令行工具（如 `conda`）时，可以通过以下命令直接修改配置，而不需要手动编辑 `.condarc` 文件，可以通过以下命令配置 `conda` 使用清华大学镜像源：

```
1 conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/free/
2 conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/pkg/main/
3 conda config --add channels https://mirrors.tuna.tsinghua.edu.cn/anaconda/cloud/conda-forge/
4 conda config --set show_channel_urls yes
```

这些命令会更新 Anaconda 的配置，使得在安装包时会优先使用指定的镜像源。如果你需要手动编辑 `condarc` 文件，可以使用文本编辑器打开并修改它。但通常建议使用 `conda config` 命令来避免手动编辑错误。在 `.condarc` 文件中，你可以设置多个 channels，Anaconda 会按照你在文件中设置的顺序来搜索和安装包。

注意：

- 修改配置文件可能会影响系统中所有 Anaconda 环境的包安装行为。
- 建议在修改前备份原 `.condarc` 文件。

2.3.3 Python 库安装

在虚拟环境中，可以使用以下命令安装所需的 Python 库：

```
1 conda install pandas matplotlib scikit-learn numpy
```

或者使用 `pip` 命令，并指定国内镜像源，例如使用清华大学镜像源：

```
1 pip install pandas matplotlib scikit-learn numpy -i https://pypi.tuna.tsinghua.edu.cn/simple
```

安装完成后，可以通过 Python 解释器验证库是否安装成功：

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3 from sklearn import svm
4 import numpy as np
5
6 print(pd.__version__)
7 print(plt.__version__)
8 print(svm.__version__)
9 print(np.__version__)
```

2.4 集成开发环境

对于集成开发环境（IDE），可以选择使用 Visual Studio Code（VSCode）或 Jupyter Notebook。VSCode 是一个功能强大的编辑器，支持 Python 开发，并且可以通过安装扩展来增强其功能。Jupyter Notebook 则提供了一个交互式的计算环境，特别适合数据分析和可视化。

2.4.1 VSCode 安装和配置

Visual Studio Code（VSCode）是一款由微软开发的免费、开源代码编辑器，支持多种编程语言，具有轻量级、高性能和高度可定制的特点。其主要优点包括：跨平台支持（Windows、macOS、Linux），丰富的扩展生态以满足多样化需求，内置 Git 集成简化版本控制，强大的调试功能提升开发效率，以及支持实时预览、内联代码执行和远程开发能力。此外，VSCode 提供了流畅的用户体

验，可通过自定义界面和快捷键适应个人偏好，是开发者理想的编码工具。本书将使用 vscode 为主要开发工具，对机器学习进行实战，以下将介绍如何对 vscode 进行安装和配置。

如果你还没有安装 VSCode，可以从官方网站下载并安装。因为安装步骤较为简单，这里不再赘述，安装完成后，打开 VSCode 就可以按照 Python 扩展了。如下图右边2.1所示，在 VSCode 中，点击左侧的 Extensions 图标，搜索并安装官方提供的 Python 扩展，Notebook 相关的扩展，以及需要的扩展。

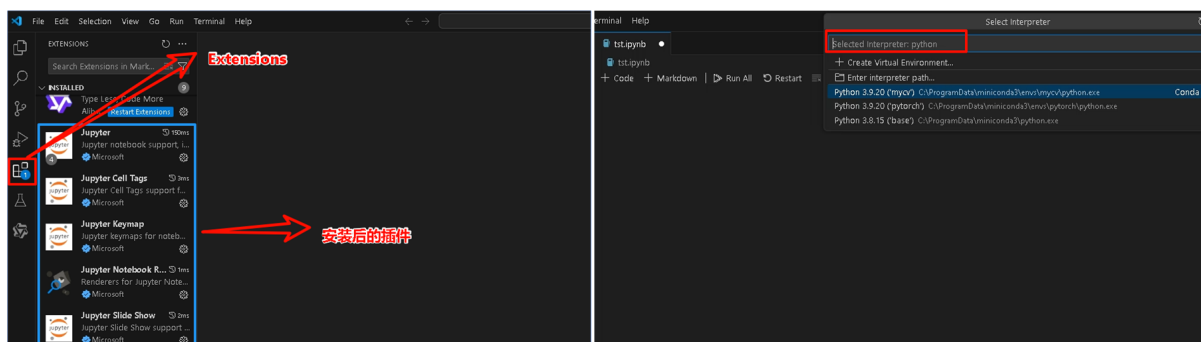


图 2.1: 在 VSCode 中安装 Python 扩展和配置 Python 解释器

通过安装 python 和 Jupyter Notebook 相关的扩展，可以在 VSCode 中使用 python 语言进行程序设计，并且可以通过 jupyter Notebook 方式进行编程。

安装 Python 扩展后，VSCode 将提示您选择一个 Python 解释器。您也可以通过命令面板（按下快捷键 **Ctrl+Shift+P**）输入“Python: Select Interpreter”来手动选择解释器，具体操作可参考图 2.1 左侧的示意图。在选择解释器时，您需要从系统中已安装的选项中挑选合适的版本。如果您之前按照前文所述的方法创建了虚拟环境，那么这些自定义环境的名称将会出现在解释器选择的下拉菜单中。如图 2.1 右侧所示，在本例中，系统上已成功配置了名为 *mycv* 和 *pytorch* 的自定义虚拟环境，此外，还有随 Anaconda 一同安装的默认虚拟环境 *base*。在选择解释器时，您只需从下拉列表中点击相应环境名称即可完成选择。

2.4.2 PyCharm 安装和配置

PyCharm 是一款功能强大的集成开发环境（IDE），特别适合 Python 语言的开发，为机器学习项目提供了丰富的工具支持。假设你已经按照前面的内容安装了 Anaconda，以下是 PyCharm 的安装和配置步骤：

首先保证你本地网络可以正常访问，然后可以通过以下方式进行安装：

1. 访问 PyCharm 官方网站 (<https://www.jetbrains.com/pycharm/>)，根据操作系统选择合适的版本进行下载。
2. 下载完成后，运行安装程序。在安装过程中，按照向导提示进行操作，可选择安装路径等选项。
3. 安装完成后，启动 PyCharm，进行初始设置，如选择主题、配置等。

然后在安装完成后，在正式写编码之前，你需要构建一个项目文件，如下图2.2左边所示，具体步骤如下：

1. 打开 PyCharm，点击“File”->“New Project”。
2. 在弹出的对话框中，选择项目类型和设置，如项目位置等。

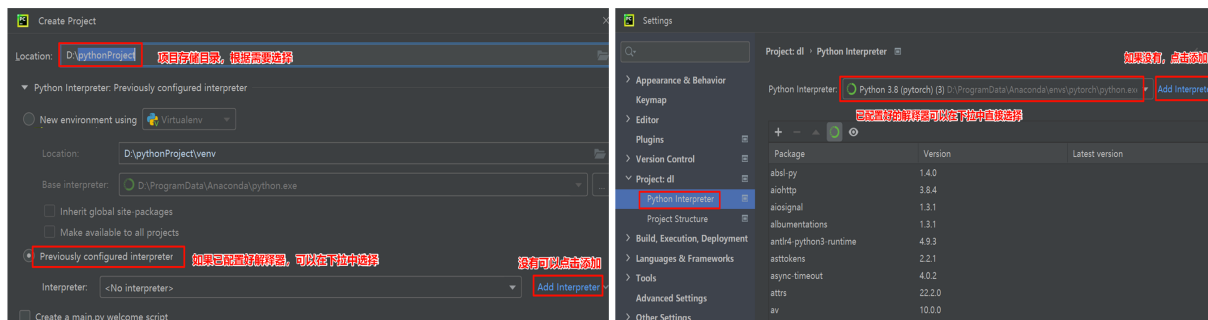


图 2.2: 在 pycharm 中新建项目和配置 Python 解释器

3. 项目创建完成后，可以在项目目录中创建新的 Python 文件，开始编写代码。

最后，假设你已经在 Pycharm 中打开了一个工程文件，参考图2.2右边所示，然后执行以下步骤：

1. 在 PyCharm 中，进入 “File” -> “Settings”（macOS 上是 “PyCharm” -> “Preferences”）。
2. 在左侧导航栏中选择 “Project: [项目名称]” -> “Python Interpreter”。
3. 点击右上角的齿轮图标，选择 “Add Interpreter” -> “Add Local Interpreter”。
4. 选择 Anaconda 安装的 Python 解释器路径，通常默认路径为 “Anaconda3\python.exe”（Windows）或 “anaconda3/bin/python”（macOS/Linux）。
5. 点击 “OK” 完成解释器添加。

2.5 免费在线开发资源

随着机器学习技术的普及，越来越多的在线平台为开发者和学习者提供了免费的云端开发环境、数据集、计算资源及协作工具。这些资源极大地降低了机器学习项目的门槛，尤其适合高校学生、研究者和初学者快速入门或开展项目。本节将详细介绍几个主流免费在线开发平台的使用方法，包括 **Kaggle**、**阿里天池**、**Google Colab** 和 **AWS SageMaker Studio**（基础版），并提供注册、配置和资源使用的详细指南。

2.5.1 Kaggle

Kaggle 是由谷歌收购的全球知名数据科学竞赛和社区平台，提供以下核心资源：

- **免费云端笔记本环境**：支持 Python、R 等语言，集成 Jupyter Notebook 和 GPU/TPU 资源（需符合使用规则）。
- **海量数据集**：覆盖计算机视觉、自然语言处理、生物信息学等领域的公开数据集。
- **竞赛与项目**：通过参与竞赛可获得实战经验，部分竞赛提供奖金或就业机会。
- **社区与学习资源**：提供教程、课程和用户分享的解决方案。

1. kaggle 注册

读者如果想使用 kaggle 平台提供的免费资源的话，需要首先在平台上进行注册。访问 <https://www.kaggle.com/>，点击右上角的 “Sign Up”。在对应界面上填写邮箱、用户名、密码，选择 “Create Account”。平台一般会有一个邮箱验证，读者需要验证邮箱后才能完成注册。如果需要使用 GPU

资源，读者需要参与至少一个竞赛并提交结果（例如“Titanic: Machine Learning from Disaster”）。通过 Kaggle 的信任度评估后，即可在 Notebook 中选择 GPU/TPU 资源。

2. kaggle 使用

完成注册后，就可以使用 kaggle 网站提供的免费计算资源，无需再安装 Python 编译环境和 IDE。首先，使用注册的用户名登录后，进入任意竞赛或数据集页面，点击“Notebook”标签。然后，选择“New Notebook”，配置环境（Python 3、GPU 等）。其次，在浏览器中编写代码，保存为 .ipynb 文件。并使用 `!pip install` 命令安装依赖库（受限于平台权限）。最后，如果使用数据集的话，需要将数据集先“下载”到个人空间，再挂载到 Notebook 中才能使用。

值得注意的是，GPU 的使用时长通常被限制为每次 6 小时，因此在使用过程中，您需要及时保存代码，避免因时间限制而导致工作丢失。此外，请务必遵守相关规定，禁止上传或共享任何敏感数据。

2.5.2 阿里天池

阿里天池是阿里巴巴集团旗下的数据科学竞赛与开发者平台，提供以下特色服务：

- **产业级竞赛：**与企业合作的真实业务问题，例如物流优化、图像识别等。
- **免费云端算力：**通过“数据魔方”提供 GPU/TPU 资源（按需分配，需申请）。
- **数据集与工具：**覆盖电商、金融、医疗等领域的脱敏数据。
- **学习社区：**提供机器学习课程和案例解析。

1. 阿里天池注册

首先，需要访问天池大数据众智平台（<https://tianchi.aliyun.com/>）。在页面的右上角找到并点击“注册”按钮以开始注册流程。您可以选择通过支付宝账户或电子邮箱进行注册。注册完成后，为了能够使用部分高级功能，您需要完成实名认证。

如果您的计算任务需要 GPU 支持，您还需要申请相应的算力资源。为此，请转至“数据魔方”页面，并提交您的资源申请，其中需包含资源用途说明及预计使用时长等详细信息。一旦您的申请通过审核，系统将为您分配相应的 GPU 资源（例如：Tesla V100）。

通过这一系列的步骤，读者将能够充分利用阿里天池平台的强大功能，为您的项目或研究提供必要的计算支持。

2. 阿里天池使用

注册成功后，就可以通过在线方式使用免费提供的 jupyter notebook 进行项目开发，具体环节如下：

1) 开发环境配置：

- 首页中点击“天池学习”中的“天池 notebook”，点击“新建 notebook”，就可以进入到编程环境中，环境中已经提供了 python 常用的库。
- 如果在编写程序中需要安装新的依赖库，需通过命令 `conda install` 或 `pip install`（部分库需管理员权限）。

2) 数据集管理：在平台上，用户可以使用自己上传的数据，也可以利用平台已有的数据。如果用户选择使用平台提供的竞赛数据，只需在页面中点击“挂载数据”按钮，并从列表中选择所需数据集。一旦数据集被成功挂载，用户便可在其 Jupyter Notebook 中直接访问该数据集，进而进行数据处理和分析。

3) 提交代码与结果：如果在参与天池平台组织的竞赛，通常需要在完成代码编写后，将代码或预测结果上传至指定平台，由自动评估系统进行评分。

在使用免费资源时，需注意资源有限，建议合理规划计算任务，以确保高效利用。同时，务必遵守阿里云的数据使用协议，禁止将资源用于商业用途。

2.5.3 AWS SageMaker Studio（基础版）

AWS SageMaker Studio 是亚马逊的机器学习开发平台，同样提供免费使用的编程环境（Free tier），主要特色如下：

- **免费资源：**750 小时/月的 `ml.t3.medium` 实例（CPU），有效期 12 个月。
- **全栈工具：**支持数据预处理、模型训练、部署和监控。
- **集成生态：**与 AWS S3、Lambda 等服务无缝对接。

1. AWS 注册

为了使用 AWS SageMaker Studio，同样需要完成 AWS 账户的注册并激活免费层级服务。首先，访问 AWS 官方网站 <https://aws.amazon.com/> 并点击页面上的“创建账户”或“Sign Up”按钮。在注册过程中，您需要提供有效的电子邮件地址、设置账户密码，并输入支付信息以完成身份验证。注册成功后，您将需要激活 AWS 的免费层级（Free Tier），该层级提供了一定量的免费服务使用额度，适合新用户进行初步体验。激活免费层级后，登录 AWS 管理控制台，在控制台的搜索栏中搜索“SageMaker”，并从搜索结果中选择 SageMaker 服务。在 SageMaker 服务页面中，点击“Start SageMaker Studio”按钮以启动 SageMaker Studio。请注意，AWS 的服务条款和免费层级政策可能会有所变动，因此在注册和使用服务前，请仔细阅读相关条款，并确保了解任何可能适用的费用，以避免在试用期结束后产生意外的账单。通过这一流程，您将能够利用 AWS SageMaker Studio 这一集成开发环境，高效地进行机器学习模型的开发、训练和部署。

2. AWS 使用

注册后，使用账号进行登录，就能够使用 AWS SageMaker Studio 了，大概流程如下：

1) 创建 Notebook 实例：在 SageMaker 控制台选择“Notebook instances”，配置实例类型为免费层支持的类型，例如 `ml.t3.medium`。

2) 开发与训练：使用 JupyterLab 环境编写代码，通过 `conda` 管理环境。注意，模型训练需指定免费层支持的实例类型（如 `ml.m5.large`，否则会产生计费。

在使用过程中，需特别注意免费层级资源有限，建议定期检查使用情况，以防产生额外计费。此外，S3 存储和网络流量也可能导致额外费用的产生。具体收费情况可以参考<https://aws.amazon.com/sagemaker/pricing/>。

2.5.4 选择与使用建议

通过合理选择和组合这些工具，读者可以高效完成从数据探索、模型训练到部署的全流程开发，同时减少本地硬件投入。如表2.1所示，建议读者根据具体需求（如竞赛、学习或产业项目）选择平台，并关注各平台的更新动态以优化资源利用。

表 2.1: 主流平台对比			
平台	适用场景	优势	局限性
Kaggle 阿里天池	竞赛、快速实验 产业项目、团队协作	大量数据集、社区活跃 企业级数据、算力申请灵活	GPU 使用时间限制 需实名认证
AWS SageMaker	完整 ML 流程（训练/部署）	企业级工具链、免费层足够学习	需熟悉 AWS 生态

在资源规划方面，需提前了解平台的配额和限制以避免任务中断；代码版本管理建议使用 Git 或平台自带工具（如 Kaggle 的“Version History”）进行追踪；数据安全方面，敏感数据需通过加密或脱敏处理防止泄露。

2.6 实践练习

通过本实践活动，学习并掌握机器学习中常用的几个 Python 库：numpy、pandas、matplotlib 和 sklearn。这些库是构建和评估机器学习模型的基础工具。在正式练习之前，确保你电脑上已经安装：

- Python 环境（建议使用 Anaconda 发行版）
- Jupyter Notebook 或任何 Python 支持的 IDE
- 安装 numpy、pandas、matplotlib 和 sklearn 库（如果未安装，请使用 `pip install numpy pandas matplotlib scikit-learn` 命令安装）

2.6.1 Numpy 练习

Numpy 是 Python 中用于科学计算的基础库，提供了大量的数学函数，可以轻松进行矩阵之间的运算，其基础知识点包括：高效多维数组对象 ndarray（支持动态属性如形状、数据类型），数组创建方法（如 `np.array()`, `np.zeros()`, `np.arange()`, `np.linspace()`），索引与切片操作（包括多维数组元素提取和切片规则），数学运算与统计功能（如逐元素运算、矩阵乘法、`mean()`, `std()`, `sum()` 等统计函数），广播机制（允许不同形状数组间的运算），数据类型控制（如 `astype()` 转换类型），以及文件输入输出（`np.save()/np.load()`）。此外，还包括布尔索引（按条件筛选数据）、形状变换（`reshape()`, `transpose()`）和随机数生成（`np.random` 模块），这些功能共同构成了高效数值计算和数据处理的基础工具集。以下为简单的示例练习，演示这些基础操作。

1. 创建一维数组并计算统计量：

```
1 import numpy as np
2
3 arr = np.array([15, 25, 35, 45, 55])
4 print("最大值:", arr.max())           # 输出: 55
5 print("累加和:", arr.sum())           # 输出: 175
6 print("标准化:", (arr - arr.mean())/arr.std()) # 标准化计算
7
```

2. 二维数组索引与切片：

```
1 matrix = np.array([[10, 20, 30], [40, 50, 60], [70, 80, 90]])
2 print("第二行:", matrix[1])          # 输出: [40 50 60]
3 print("右上角元素:", matrix[0, 2])   # 输出: 30
4 print("对角线:", matrix.diagonal())   # 输出: [10 50 90]
5
```

3. 布尔索引筛选与替换：

```
1 data = np.array([5, 12, 8, 19, 3, 22])
2 mask = data > 10
3 print("大于10的元素:", data[mask])    # 输出: [12, 19, 22]
4 data[data % 2 == 1] = -1              # 将奇数替换为-1
5 print("替换后的数组:", data)          # 输出: [5, 12, 8, -1, 3, 22]
6
```

4. 矩阵运算与广播:

```

1  A = np.array([[2, 3], [4, 5]])
2  B = np.array([[1, 0], [1, 1]])
3  print("逐元素乘法:", A * B)          # 输出: [[2 0], [4 5]]
4  print("矩阵转置:\n", A.T)           # 输出转置矩阵
5  print("行求和:", A.sum(axis=1))      # 输出: [5, 9]
6

```

5. 数据生成与形状操作:

```

1  # 生成线性间隔数组
2  linear = np.linspace(0, 2, 5)
3  print("线性间隔:", linear)          # 输出: [0.  0.5 1.  1.5 2. ]
4
5  # 生成随机矩阵并重塑
6  rand_mat = np.random.rand(2, 3)
7  print("转置后形状:", rand_mat.T.shape) # 输出: (3, 2)
8

```

2.6.2 Pandas 练习

Pandas 是基于 Numpy 的一个库，它提供了 DataFrame 和 Series 两个非常强大的数据结构，特别适合于表格数据和时间序列数据的处理。其核心功能包括：创建和操作 DataFrame（通过 `pd.DataFrame()` 构建数据结构，使用 `describe()` 快速生成统计摘要）；数据清洗（处理缺失值如 `fillna()`、`dropna()`，删除重复行 `drop_duplicates()` 并重置索引 `reset_index()`）；数据选择与过滤（通过列名选择子集、条件筛选如 `df[df['col']>value]`，以及基于位置的 `iloc` 索引）；数据合并（使用 `merge()` 按列关联不同 DataFrame）；分组与聚合（通过 `groupby()` 分组后应用聚合函数如 `mean()` 计算分组统计）；排序与导出（用 `sort_values()` 排序数据，通过 `to_csv()` 导出结果）。这些操作覆盖了数据处理的全流程，包括数据加载、清洗、转换、分析及存储，是数据科学与数据分析的核心工具。

1. 创建 DataFrame 并进行基本操作:

```

1  import pandas as pd
2
3  data = {
4      'Name': ['John', 'Anna', 'Peter', 'Linda'],
5      'Age': [28, 23, 34, 29],
6      'City': ['New York', 'Paris', 'Berlin', 'London']
7  }
8  df = pd.DataFrame(data)
9  print(df.describe()) # 显示数值列的统计信息（如年龄）

```

2. 数据清洗（处理缺失值）:

```

1  # 填充缺失值（假设Age列有NaN）
2  df['Age'].fillna(df['Age'].mean(), inplace=True)
3  # 删除包含缺失值的行
4  df = df.dropna()

```

3. 数据选择与过滤:

```

1  # 选择特定列
2  names = df[['Name', 'City']]
3  # 筛选年龄大于30的行
4  filtered = df[df['Age'] > 30]
5  # 使用iloc选择前两行和前两列
6  subset = df.iloc[:2, :2]

```


4. 数据合并与连接:

```

1  # 创建第二个DataFrame
2  extra_data = pd.DataFrame({
3      'Name': ['Anna', 'Linda'],
4      'Salary': [50000, 60000]
5  })
6  # 按Name列合并
7  merged = pd.merge(df, extra_data, on='Name')

```

5. 分组与聚合:

```

1  # 按城市分组, 计算平均年龄
2  grouped = df.groupby('City')['Age'].mean()
3  print(grouped)
4  # 输出示例:
5  # City
6  # Berlin      34.0
7  # London      29.0
8  # New York    28.0
9  # Paris       23.0

```

6. 排序与导出:

```

1  # 按年龄降序排序
2  sorted_df = df.sort_values(by='Age', ascending=False)
3  # 导出为CSV文件
4  df.to_csv('output.csv', index=False)

```

2.6.3 Matplotlib 练习

Matplotlib 是一个 Python 绘图库, 通过简单的命令就可以创建高质量的图表。其基础操作涵盖从基础图表绘制到复杂样式定制的全流程: 通过 `plt.plot()`、`plt.bar()`、`plt.scatter()` 等函数可快速生成折线图、柱状图、散点图等基础图表; 支持通过 `title()`、`xlabel()`、`ylabel()` 设置图表标题和坐标轴标签, 使用 `grid()` 添加网格, 以及通过颜色、标记样式 (如 `color='red'`、`marker='o'`) 和线型 (如 `linestyle='--'`) 自定义视觉效果; 通过 `plt.subplots()` 可创建多子图布局, 并利用 `tight_layout()` 自动调整间距; 还可与 Pandas 结合, 通过 `df.plot()` 直接基于 DataFrame 生成图表; 最后, 使用 `plt.savefig()` 可将图表保存为图像文件 (如 PNG、PDF), 并设置分辨率 (dpi) 和裁剪参数 (`bbox_inches`)。这些功能共同构成了数据可视化的核心工具集, 适用于数据分析、结果呈现及科学计算场景。

1. 绘制基本折线图:

```

1  import matplotlib.pyplot as plt
2
3  x = [1, 2, 3, 4]
4  y = [10, 20, 15, 25]
5  plt.plot(x, y, marker='o', linestyle='--', color='g')
6  plt.title("简单折线图")
7  plt.xlabel("X轴")
8  plt.ylabel("Y轴")
9  plt.show()

```

2. 绘制柱状图:

```

1  categories = ['A', 'B', 'C', 'D']
2  values = [23, 45, 12, 34]
3  plt.bar(categories, values, color=['blue', 'orange', 'green', 'red'])
4  plt.title("分类柱状图")
5  plt.grid(axis='y', linestyle='--', alpha=0.7)
6  plt.show()

```

3. 散点图与标签:

```

1  x_scatter = [5, 7, 3, 9, 4]
2  y_scatter = [15, 12, 20, 8, 18]
3  plt.scatter(x_scatter, y_scatter, c='purple', s=100, alpha=0.6)
4  for i, txt in enumerate(categories):
5  plt.text(x_scatter[i]+0.2, y_scatter[i], categories[i])
6  plt.show()

```

4. 子图与样式:

```

1  fig, axs = plt.subplots(1, 2, figsize=(10, 4))
2
3  axs[0].plot([1, 2, 3], [4, 5, 1])
4  axs[0].set_title("左图")
5
6  axs[1].bar(['X', 'Y'], [10, 20], color='skyblue')
7  axs[1].set_title("右图")
8
9  plt.tight_layout()
10 plt.show()

```

5. 数据可视化与 Pandas 结合:

```

1  import pandas as pd
2
3  df = pd.DataFrame({
4      'Quarter': [1, 2, 3, 4],
5      'Sales': [200, 300, 250, 400]
6  })
7  df.plot(x='Quarter', y='Sales', kind='line', marker='s')
8  plt.title("季度销售额")
9  plt.show()

```

6. 保存与样式设置:

```

1  plt.figure(figsize=(8, 5))
2  plt.plot([1, 2, 3], [5, 3, 7], 'r--^')
3  plt.title("保存示例", fontsize=14)
4  plt.savefig('plot.png', dpi=300, bbox_inches='tight')

```

2.6.4 Scikit-learn 练习

Scikit-learn 是一个简单高效的机器学习库，提供了许多常用的机器学习算法。其基础操作包括：数据加载（如 `load_iris()` 加载内置数据集），数据预处理（通过 `train_test_split` 分割数据集、`StandardScaler` 标准化特征），模型训练（如 `LogisticRegression` 的 `fit()` 方法训练分类模型或 `LinearRegression` 进行回归），预测与评估（使用 `predict()` 生成预测结果，并通过 `accuracy_score`、`confusion_matrix` 等指标评估性能），以及管道（`Pipeline`）的集成（通过 `make_pipeline` 自动串联预处理和模型步骤）。这些步骤覆盖了从数据准备、模型构建到评估的完整流程，支持分类、回归、聚类等任务，并通过统一的 API 设计简化了机器学习 workflow。

1. 加载数据集并预览:

```

1  from sklearn.datasets import load_iris
2  import pandas as pd
3
4  iris = load_iris()
5  df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
6  df['target'] = iris.target
7  print(df.head()) # 查看前几行数据

```

2. 数据预处理与分割:

```
1 from sklearn.model_selection import train_test_split
2 from sklearn.preprocessing import StandardScaler
3
4 X = df.drop('target', axis=1)
5 y = df['target']
6
7 # 分割训练集和测试集
8 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
9
10 # 特征标准化
11 scaler = StandardScaler()
12 X_train_scaled = scaler.fit_transform(X_train)
13 X_test_scaled = scaler.transform(X_test)
```

3. 训练分类模型:

```
1 from sklearn.linear_model import LogisticRegression
2
3 model = LogisticRegression(max_iter=200)
4 model.fit(X_train_scaled, y_train)
5 predictions = model.predict(X_test_scaled)
6 print("预测结果:", predictions[:5]) # 输出前5个预测值
```

4. 模型评估:

```
1 from sklearn.metrics import accuracy_score, confusion_matrix
2
3 accuracy = accuracy_score(y_test, predictions)
4 print("准确率:", accuracy)
5 print("混淆矩阵:\n", confusion_matrix(y_test, predictions))
```

5. 管道 (Pipeline) 集成:

```
1 from sklearn.pipeline import make_pipeline
2
3 pipeline = make_pipeline(StandardScaler(), LogisticRegression())
4 pipeline.fit(X_train, y_train)
5 score = pipeline.score(X_test, y_test)
6 print("Pipeline 准确率:", score)
```

2.6.5 综合练习

为了帮助大家更好地理解 and 掌握 NumPy、Pandas、Matplotlib 等机器学习相关库的用法，精心设计了一个综合性的练习。该练习涵盖了 NumPy、Pandas、Matplotlib 和 Scikit-learn 的核心功能，具体包括：使用 Pandas 加载并预览数据集，借助 NumPy 进行统计计算（例如计算均值），利用 Matplotlib 可视化数据分布；通过 Scikit-learn 完成数据标准化、训练集与测试集的划分、随机森林分类模型的训练及评估；最终通过混淆矩阵的可视化展示模型性能。整个流程贯穿了数据加载、清洗、可视化、预处理、模型训练与评估的完整机器学习工作流，充分体现了数值计算、数据处理、可视化和机器学习的综合应用。

1. 加载数据并预览:

```
1 import pandas as pd
2 from sklearn.datasets import load_iris
3
4 iris = load_iris()
```

```

5 df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
6 df['target'] = iris.target
7 print(df.head()) # 查看前5行数据

```

2. 数据清洗与统计分析:

```

1 # 删除重复数据
2 df = df.drop_duplicates()
3 # 计算数值列的描述性统计
4 print("统计信息: \n", df.describe())
5 # 使用NumPy计算花瓣长度的均值
6 mean_petal_length = np.mean(df['petal length (cm)'])
7 print("花瓣长度均值: ", mean_petal_length)

```

3. 数据可视化:

```

1 import matplotlib.pyplot as plt
2
3 # 绘制花瓣长度与宽度的散点图
4 plt.scatter(df['petal length (cm)'], df['petal width (cm)'], c=df['target'], cmap='
    viridis')
5 plt.xlabel('Petal Length (cm)')
6 plt.ylabel('Petal Width (cm)')
7 plt.title('Iris Species Distribution')
8 plt.colorbar(label='Target Class')
9 plt.show()

```

4. 数据预处理与划分:

```

1 from sklearn.model_selection import train_test_split
2 from sklearn.preprocessing import StandardScaler
3
4 X = df.drop('target', axis=1)
5 y = df['target']
6
7 # 分割训练集和测试集
8 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state
    =42)
9
10 # 标准化特征
11 scaler = StandardScaler()
12 X_train_scaled = scaler.fit_transform(X_train)
13 X_test_scaled = scaler.transform(X_test)

```

5. 训练分类模型:

```

1 from sklearn.ensemble import RandomForestClassifier
2
3 model = RandomForestClassifier(n_estimators=100, random_state=42)
4 model.fit(X_train_scaled, y_train)
5 predictions = model.predict(X_test_scaled)
6 print("预测结果前5个: ", predictions[:5])

```

6. 模型评估与可视化:

```

1 from sklearn.metrics import accuracy_score, confusion_matrix
2
3 accuracy = accuracy_score(y_test, predictions)
4 print("准确率: ", accuracy)
5
6 # 绘制混淆矩阵

```

```
7 plt.figure(figsize=(6, 4))
8 sns.heatmap(confusion_matrix(y_test, predictions), annot=True, cmap='Blues')
9 plt.xlabel('Predicted')
10 plt.ylabel('True')
11 plt.title('Confusion Matrix')
12 plt.show()
```

通过本实践活动，初步了解了 numpy、pandas、matplotlib 和 sklearn 库的核心使用方法。这些库为数据预处理、分析、可视化和建模提供了强大的工具，是任何希望进入数据科学和机器学习领域的人的必备技能。

2.7 习题

为了帮助读者更好地掌握机器学习开发中的程序运行环境相关知识，本章提供了以下习题，包括简答题和计算题。每道题目均附有详细答案。

2.7.1 简答题

1. 为什么 Python 是机器学习开发中最常用的编程语言？它有哪些优势？
2. 在机器学习项目中，如何选择合适的运行环境（如本地环境、云端平台或分布式计算框架）？
3. 常见的机器学习库（如 NumPy、Pandas、Scikit-learn、TensorFlow、PyTorch）分别适用于哪些任务？请举例说明。
4. 解释什么是依赖冲突，并描述一种解决依赖冲突的方法。
5. 如何优化程序运行环境以提高代码执行效率？请列举至少三种方法。
6. 什么是交叉验证，它为什么重要？
7. 如何判断训练好的模型是过拟合还是欠拟合？
8. 如何防止模型过拟合或者欠拟合？
9. 模型评估中，如何区分模型的泛化能力和复杂度？
10. 现在要训练一个模型，通过该模型可以预测房价，请设计训练数据对应的特征？

2.7.2 计算题

1. 假设一个机器学习任务需要处理的数据集大小为 10GB，而你的本地计算机内存只有 8GB。请设计一种解决方案，确保任务能够顺利完成。
2. 给定以下 Python 代码片段，分析其可能存在的性能瓶颈，并提出优化建议：

```
1 import numpy as np
2
3 # 创建一个大数组
4 data = np.random.rand(10000, 10000)
5
6 # 计算每列的平均值
7 col_means = [np.mean(data[:, i]) for i in range(data.shape[1])]
```


第三章 模型评估

在机器学习与人工智能的广阔领域中，模型评估占据着举足轻重的地位。随着数据科学的快速发展，构建高效、准确的预测模型已成为各行各业追求的核心目标之一。然而，一个模型的优劣并非仅凭其复杂度和训练过程中的表现就能全面衡量，更重要的是其在未知数据上的泛化能力和预测准确性。因此，模型评估作为连接模型训练与实际应用的关键桥梁，其重要性不言而喻。模型评估不仅是对模型性能的量化考核，更是指导模型优化与选择的重要依据。通过科学的评估方法，能够深入了解模型在不同数据集上的表现差异，识别出模型的强项与弱项，进而采取针对性的优化措施。无论是调整模型结构、优化参数配置，还是进行特征选择与工程，都离不开模型评估的反馈与指导。此外，模型评估还促进了机器学习技术的公平性与可解释性。在追求高性能的同时，也需要关注模型的鲁棒性、公平性和伦理考量。模型评估能够帮助识别并纠正潜在的偏见与歧视，确保技术应用的公正性与透明度。因此，深入理解和掌握模型评估的理论与方法，对于提升机器学习模型的实用性、可靠性和社会接受度具有重要意义。本章节将系统介绍模型评估的基本概念、常用指标、评估流程以及优化策略，旨在为读者构建一个全面而深入的模型评估知识体系，助力其在机器学习领域取得更加卓越的成果。

学习目标

本章系统介绍机器学习模型评估与选择的核心内容。理解模型评估在保障性能、泛化能力与稳定性中的关键作用，掌握分类与回归任务中常用评估指标（如准确率、F1 分数、MSE、 R^2 等）及其适用场景。通过交叉验证（特别是 K 折交叉验证）提升模型泛化能力，并能正确解读验证结果。掌握网格搜索、随机搜索和贝叶斯优化等超参数调优方法，理解其优劣与适用条件。最终能够在不同模型间进行科学比较，识别潜在偏见，做出合理选择，为构建高效可靠的机器学习系统奠定基础。

3.1 分类问题评估指标

学生石榴在学习机器学习时，遇到了分类问题，她希望通过几个具体的评估指标来衡量她的模型性能。以下是石榴在学习过程中遇到的几个关键评估指标，包括这些指标的基本定义和结合案例的解释说明。

3.1.1 基本术语定义

在机器学习和统计学中，混淆矩阵（Confusion Matrix）是一种特别有用的工具，用于评估分类模型的性能。它通过展示模型预测结果与实际标签之间的关系，帮助理解模型在不同类别上的表现。以下是对混淆矩阵中四个关键指标的详细解释和丰富说明：

真正例（True Positives, TP）：实际为正类，且被预测为正类的样本数。这是模型预测正确的情况之一，表明模型成功地识别出了正类样本。在医学诊断中，真正例可以类比为正确诊断出患者患有某种疾病的情况。

真反例 (True Negatives, TN): 实际为负类, 且被预测为负类的样本数。这同样是模型预测正确的情况, 表明模型成功地排除了负类样本。在安全系统中, 真反例可以类比为正确识别出非威胁行为的情况。

假正例 (False Positives, FP): 实际为负类, 但被错误地预测为正类的样本数。这是模型预测错误的情况之一, 可能会导致不必要的警报或资源浪费。在法律领域, 假正例可能类比为错误地指控某人无罪的情况。

假反例 (False Negatives, FN): 实际为正类, 但被错误地预测为负类的样本数。这是另一种模型预测错误的情况, 可能会导致严重的后果, 如错过重要的诊断或干预机会。在金融欺诈检测中, 假反例可能类比为未能识别出欺诈交易的情况。

通过上面的四个指标, 就可以得到下面的混淆矩阵:

	预测为正类	预测为负类
真实为正类	TP (真正率)	FN (假反率)
真实为负类	FP (假正率)	TN (真反率)

同样通过这四个指标, 可以计算出多种评估分类模型性能的指标, 如准确率 (Accuracy)、精确率 (Precision)、召回率 (Recall) 和 F1 分数 (F1 Score) 等。这些指标有助于从不同角度理解模型的优缺点, 从而进行相应的优化和调整。

3.1.2 准确率 (Accuracy)

准确率是所有正确预测的样本数占总样本数的比例, 其公式定义如下:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (3.1)$$

石榴使用她的模型对 100 张图片进行了分类, 其中 80 张图片被正确分类 (40 张正类被正确识别为 TP, 40 张负类被正确识别为 TN)。因此, 石榴的模型准确率为:

$$\text{Accuracy} = \frac{40 + 40}{100} = 0.8 \quad (\text{或 } 80\%) \quad (3.2)$$

3.1.3 精确率 (Precision) 与召回率 (Recall)

精确率定义为在所有被预测为正类的样本中, 真正为正类的比例, 公式如下:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (3.3)$$

召回率定义为在所有真正为正类的样本中, 被预测为正类的比例 (也称为真正率 TPR), 公式如下:

$$\text{Recall} = \text{TPR} = \frac{TP}{TP + FN} \quad (3.4)$$

石榴的模型在预测为正类的 50 张图片中, 有 45 张确实是正类图片 (TP=45, FP=5)。同时, 在所有真正的正类图片中 (共 50 张), 模型预测出了 45 张 (FN=5)。因此, 精确率和召回率分别为:

$$\begin{aligned} \text{Precision} &= \frac{45}{50} = 0.9 \quad (\text{或 } 90\%), \\ \text{Recall} &= \frac{45}{50} = 0.9 \quad (\text{或 } 90\%). \end{aligned} \quad (3.5)$$

3.1.4 F1 分数 (F1 Score)

F1 分数是精确率和召回率的调和平均数，用于综合评估模型的性能。公式表示为：

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (3.6)$$

结合上面的精确率和召回率，石榴的模型 F1 分数为：

$$\text{F1 Score} = 2 \times \frac{0.9 \times 0.9}{0.9 + 0.9} = 0.9 \quad (\text{或 } 90\%) \quad (3.7)$$

3.1.5 混淆矩阵 (Confusion Matrix)

利用上面得到的数据，可以使用混淆矩阵描述分类模型的性能，展示了 TP、TN、FP、FN 的数量。例如下面为一个案例矩阵：

	预测为正类	预测为负类
真实为正类	45	5
真实为负类	5	45

混淆矩阵直观地展示了石榴模型在各个分类上的表现。通过这些评估指标和案例说明，石榴能够更全面地了解她的模型性能，并据此进行进一步的优化和改进。

3.2 回归问题评估指标

在机器学习中，回归问题是指预测连续值输出的问题。为了评估回归模型的性能，通常使用一些特定的指标。本小节将介绍几个常用的回归评估指标，并结合案例进行详细说明。

3.2.1 均方误差 (Mean Squared Error, MSE)

均方误差是预测值与实际值之差的平方的平均值。它衡量的是预测值与实际值之间的平均误差平方。

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (3.8)$$

其中， n 是样本数量， y_i 是第 i 个样本的实际值， $\hat{y}_i$ 是第 i 个样本的预测值。

假设学生石榴有一组房价数据，包含 5 个样本，预测值和实际值如下表所示：

表 3.1: 房价预测案例数据集

样本编号	实际值 (y_i)	预测值 ($\hat{y}_i$)
1	200	220
2	300	280
3	150	160
4	400	380
5	250	245

根据公式 3.8, MSE 的计算过程如下:

$$\begin{aligned} \text{MSE} &= \frac{1}{5} [(200 - 220)^2 + (300 - 280)^2 + (150 - 160)^2 + (400 - 380)^2 + (250 - 245)^2] \\ \text{MSE} &= \frac{1}{5} [400 + 400 + 100 + 400 + 25] = \frac{1325}{5} = 265 \end{aligned} \quad (3.9)$$

3.2.2 均方根误差 (Root Mean Squared Error, RMSE)

均方根误差是均方误差的平方根, 它与 MSE 有相同的量纲, 更容易解释。

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (3.10)$$

使用上述案例中的数据, 可以计算 RMSE:

$$\begin{aligned} \text{RMSE} &= \sqrt{\frac{1}{5} [(200 - 220)^2 + (300 - 280)^2 + (150 - 160)^2 + (400 - 380)^2 + (250 - 245)^2]} \\ &= \sqrt{\frac{1325}{5}} \\ &= \sqrt{265} \\ &\approx 16.28 \end{aligned} \quad (3.11)$$

3.2.3 平均绝对误差 (Mean Absolute Error, MAE)

平均绝对误差是预测值与实际值之差的绝对值的平均值。

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (3.12)$$

使用上述案例中的数据, 可以计算 MAE:

$$\begin{aligned} \text{MAE} &= \frac{1}{5} [|200 - 220| + |300 - 280| + |150 - 160| + |400 - 380| + |250 - 245|] \\ \text{MAE} &= \frac{1}{5} [20 + 20 + 10 + 20 + 5] = \frac{75}{5} = 15 \end{aligned} \quad (3.13)$$

3.2.4 R 平方 (R-Squared)

R 平方是衡量模型解释变异性的一个指标, 其值介于 0 到 1 之间。R 平方越接近 1, 表示模型的解释能力越强。

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (3.14)$$

其中, $\bar{y}$ 是所有实际值的平均值。使用上述案例中的数据, 首先计算 $\bar{y}$:

$$\bar{y} = \frac{1}{5} (200 + 300 + 150 + 400 + 250) = \frac{1300}{5} = 260 \quad (3.15)$$

然后计算 R 平方:

$$\begin{aligned}
 R^2 &= 1 - \frac{1325}{\sum_{i=1}^5 (y_i - 260)^2} \\
 \sum_{i=1}^5 (y_i - 260)^2 &= (200 - 260)^2 + (300 - 260)^2 + (150 - 260)^2 + (400 - 260)^2 + (250 - 260)^2 \\
 \sum_{i=1}^5 (y_i - 260)^2 &= 3600 + 1600 + 1600 + 19600 + 100 = 23500 \\
 R^2 &= 1 - \frac{1325}{23500} \approx 1 - 0.0564 = 0.9436
 \end{aligned} \tag{3.16}$$

通过上述计算可以看到，石榴同学的模型在该案例中具有较高的解释能力。

3.3 性能度量方法

在机器学习中，交叉评估是一种重要的模型验证方法，它通过将数据集分成多个子集，反复训练和验证模型，以评估模型的泛化能力。本节将介绍交叉评估的基本概念、常见方法以及如何在实际案例中应用。

交叉评估（Cross-validation）是一种统计分析方法，用于验证分类器的性能。它的基本思想是将原始数据集分为训练集和验证集，使用训练集对分类器进行训练，然后利用验证集来测试训练得到的模型，以此来评价分类器的性能指标。

交叉评估的核心原则是确保模型在未见过的数据上也能有良好的表现，即评估模型的泛化能力。在实际应用中，交叉评估可以减少过拟合的风险，并从有限的数据中获取尽可能多的有效信息。交叉评估主要有以下几种方法：

3.3.1 留出法（Holdout Method）

这是最简单的交叉验证方法。将数据集随机分为两部分：训练集和测试集。通常，测试集占总数据集的 20%-30%，其余作为训练集。这种方法操作简单，但可能由于数据分布不均匀而导致模型评估不准确。

3.3.2 K 折交叉验证（K-Fold CV）

K 折交叉验证是将数据集分为 K 个子集，每次选择其中一个子集作为测试集，其余 K-1 个子集作为训练集。这个过程重复 K 次，每次选择不同的子集作为测试集，最终得到 K 个模型性能评估结果，取平均值作为最终评估结果。这种方法可以更全面地评估模型性能，但计算成本较高。

3.3.3 留一法交叉验证（Leave One out CV）

留一法交叉验证是 K 折交叉验证的特例，其中 K 等于数据集中的样本数。每次只留下一个样本作为测试集，其余所有样本作为训练集。这种方法可以最大程度地利用数据，但计算成本极高，通常不推荐在大数据集上使用。

3.3.4 自助法（Bootstrap Methods）

自助法交叉验证是一种基于重采样的方法。通过对原始数据集进行多次有放回的抽样，每次抽样得到一个新的数据集，然后使用这些数据集进行模型训练和评估。这种方法适用于小样本数据集，

可以提供更稳健的模型性能估计。

3.3.5 交叉验证示例

本小节通过一个具体的案例来说明如何应用交叉评估。假设学生石榴正在使用线性回归模型来预测房价，数据集包含 100 个样本。

为了更直观地理解交叉评估的过程，可以使用流程图来表示。以下是 5 折交叉验证的流程图：

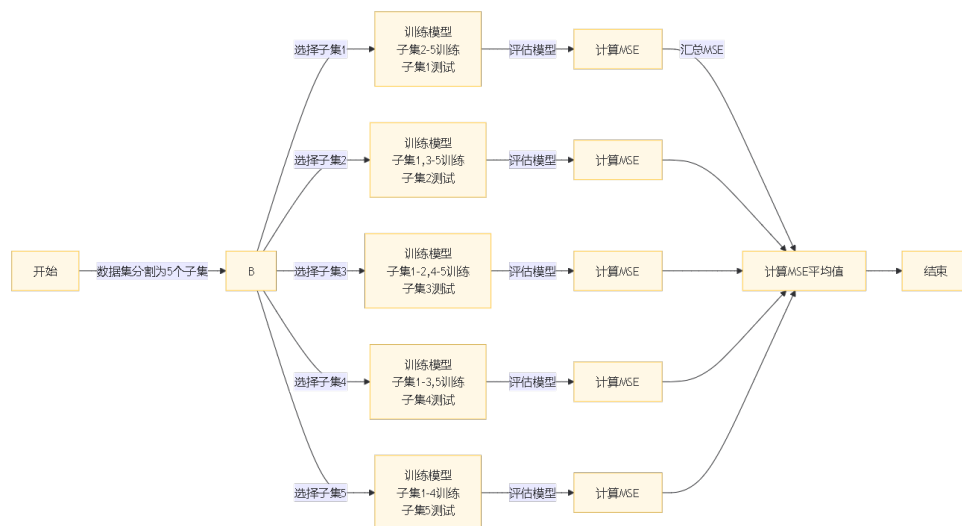


图 3.1: 交叉验证案例流程图

石榴首先将数据集分为特征矩阵 X 和目标向量 y ，其中 X 包含房屋的大小、位置等特征， y 是房价。然后选择使用 5 折交叉验证来评估模型。具体步骤如下：

1. 将 100 个样本随机分为 5 个子集，每个子集包含 20 个样本。
2. 进行 5 次训练和验证：
 - 使用第 1-4 个子集（80 个样本）训练模型，第 5 个子集（20 个样本）作为测试集。
 - 在测试集上计算模型的性能指标，如均方误差（MSE）。
 - 重复上述步骤，每次选择不同的子集作为测试集。
3. 计算 5 次验证的 MSE 平均值，作为模型的最终评估结果。

通过上述步骤，石榴同学得到了一个稳定且可靠的模型性能评估结果。这个过程不仅评估了模型的准确性，还提高了模型的泛化能力。

3.3.6 超参数调优

在机器学习的探索之旅中，石榴，一位对数据科学充满热情的学生，正面临着一个挑战：如何优化她的模型的超参数。石榴知道，超参数是在学习过程开始之前设置的参数，对模型的性能有着决定性的影响。与模型参数不同，模型参数是通过训练数据学习得到的。合适的超参数设置可以显著提升模型的性能。超参数调优是模型训练过程中的重要环节，它直接影响模型的泛化能力和训练效率。石榴探索了几种流行的超参数调优方法：网格搜索、随机搜索和贝叶斯优化。

1. 网格搜索 (Grid Search)

表 3.2: 支持向量机优化参数

C 值	Gamma 值
1	0.1
1	1
10	0.1
10	1

网格搜索是一种穷举搜索方法，它通过遍历超参数的所有可能组合来寻找最优超参数。这种方法的优点是全面，但计算成本高，特别是当超参数数量和候选值较多时。

2. 随机搜索 (Random Search)

随机搜索是一种随机化的搜索方法，它通过随机采样超参数的组合来寻找最优超参数。与网格搜索相比，随机搜索的计算量较小，能够在有限的时间内快速找到较好的超参数组合。

3. 贝叶斯优化 (Bayesian Optimization)

贝叶斯优化是一种基于概率模型的全局优化方法，它通过构建目标函数的概率模型来寻找最优超参数。贝叶斯优化利用已有的观测数据训练一个高斯过程模型，然后利用该模型预测未观测点的目标函数值和不确定性，从而找到最优超参数。下图展示了贝叶斯优化的整体流程：



图 3.2: 贝叶斯优化流程图

3.3.7 超参数调优示例

石榴有一组包含 100 个样本的数据集，每个样本都有 10 个特征。她决定使用这三种方法来调优她的 SVM 模型。

1. 网格搜索

石榴定义了一个超参数网格，包括不同的 C（正则化参数）和 gamma（RBF 核的参数）值：

对于每一组超参数，石榴都训练了她的 SVM 模型，并使用交叉验证来评估模型的性能。她计算了每个模型的准确率，并记录了结果。

2. 随机搜索

石榴在相同的超参数空间内随机选取了 10 组超参数组合进行评估。这种方法比网格搜索更快，但也更具随机性。

3. 贝叶斯优化

石榴使用贝叶斯优化来寻找最优的超参数组合。她首先定义了超参数的搜索空间，然后利用贝叶斯优化库进行优化。贝叶斯优化的过程如下：

$$\text{Best Parameters} = \operatorname{argmax}_{\text{params}} (\text{Accuracy}(\text{params})) \quad (3.17)$$

在尝试了所有方法后，石榴发现贝叶斯优化在她的案例中表现最好，因为它在较少的迭代次数内找到了最优的超参数组合。

3.3.8 模型对比与选择

在机器学习的探索之旅中，石榴同学正面临着另一个挑战：从多个模型中选择最佳模型，选择合适的模型对于解决特定的问题至关重要，该问题也称为模型对比。模型对比是机器学习中的一个重要步骤，它涉及到评估不同模型的性能，以便选择最适合特定问题需求的模型。模型选择直接影响到解决方案的准确性、效率和可扩展性。常用的模型对比方法：交叉验证、误差分析和模型融合。

1. 交叉验证 (Cross-Validation)

交叉验证是一种评估模型泛化能力的方法，它通过将数据集分成多个子集，反复训练和验证模型，以评估模型的性能。

2. 误差分析 (Error Analysis)

误差分析是一种分析模型预测错误的类型和原因的方法，它有助于理解模型在哪些方面表现不佳，从而指导模型的改进。

3. 模型融合 (Model Ensemble)

模型融合是一种结合多个模型预测结果的方法，它可以通过提高模型的稳定性和准确性来提高整体性能。

为了清晰且专业地展示模型对比的意义，石榴选用了包含 100 个样本的数据集，其中每个样本都具备 10 个特征。她计划采用决策树、随机森林和支持向量机这三种方法，来系统地比较不同模型的性能表现。石榴首先使用 5 折交叉验证来评估每个模型的性能。她计算了每个模型的准确率，并记录了结果。

表 3.3: 模型性能对比

模型	准确率	训练时间	测试时间
决策树	85%	0.5 秒	0.1 秒
随机森林	90%	1.5 秒	0.2 秒
支持向量机	88%	2.0 秒	0.3 秒

表3.3展示了三个模型的性能对比。从表中可以看出，随机森林模型具有最高的准确率，但训练时间较长。决策树模型的训练和测试时间最短，但准确率较低。

其次，石榴进行了误差分析，以了解每个模型在哪些类型的样本上表现不佳。她发现决策树和支持向量机在处理边缘样本时容易出错，而随机森林在处理噪声数据时表现较好。

再次，基于上述分析，石榴决定尝试模型融合，将三个模型的预测结果结合起来，以提高整体性能。她使用简单的平均规则来融合预测结果，并发现融合后的模型在测试集上的准确率提高到了 92%。

最后，综合考虑模型的性能、训练时间和应用场景，石榴最终选择了随机森林模型作为她的解决方案。虽然它的训练时间较长，但其高准确率和鲁棒性使其成为最佳选择。

3.4 实践练习

3.4.1 示例：评估指标计算

通过本实践活动，学习并掌握机器学习中几种重要的评估指标：准确率、AUC、F1 分数和混淆矩阵。这些评估指标是评价模型性能的关键工具。

Python 环境（建议使用 Anaconda 发行版），Jupyter Notebook 或任何 Python 支持的 IDE，scikit-learn 库（如果未安装，请使用 `pip install scikit-learn` 命令安装）。

1. 准确率 (Accuracy)

准确率是最直观的评估指标，表示模型预测正确的样本数占总样本数的比例。

1) 导入必要的库：

```
1 from sklearn.metrics import accuracy_score
2 from sklearn.model_selection import train_test_split
3 from sklearn.ensemble import RandomForestClassifier
4 from sklearn.datasets import load_iris
```

2) 加载数据集并划分训练集和测试集：

```
1 iris = load_iris()
2 X, y = iris.data, iris.target
3 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

3) 训练模型并计算准确率：

```
1 model = RandomForestClassifier(random_state=42)
2 model.fit(X_train, y_train)
3 y_pred = model.predict(X_test)
4 accuracy = accuracy_score(y_test, y_pred)
5 print(f"Accuracy: {accuracy:.2f}")
```

2. AUC (Area Under the Curve)

AUC 是评估二分类问题中模型性能的一个重要指标，表示模型对样本的正负类别区分能力的度量，可以使用以下程序进行计算：

```
1 from sklearn.metrics import roc_auc_score
2 y_pred_proba = model.predict_proba(X_test)[:, 1]
3 auc = roc_auc_score(y_test, y_pred_proba)
4 print(f"AUC: {auc:.2f}")
```

3. F1 分数 (F1 Score)

F1 分数是精确率和召回率的调和平均数，用于评估模型在二分类问题中的平衡性能，具体计算方式如下：

```
1 from sklearn.metrics import f1_score
2 f1 = f1_score(y_test, y_pred)
3 print(f"F1 Score: {f1:.2f}")
```

4. 混淆矩阵 (Confusion Matrix)

混淆矩阵是一个表格，用于描述分类模型的性能。它显示了实际类别和模型预测类别之间的关系，具体计算方法如下：

```
1 from sklearn.metrics import confusion_matrix
2 cm = confusion_matrix(y_test, y_pred)
3 print("Confusion Matrix:")
4 print(cm)
```

通过本实践活动，了解了准确率、AUC、F1 分数和混淆矩阵这四个重要的评估指标。这些指标有助于全面评价模型的性能，特别是在分类问题中。掌握这些评估指标的计算方法对于构建和优化机器学习模型至关重要。

3.4.2 示例：模型调优

通过本实践活动，学习并掌握模型调优的流程，包括超参数选择、交叉验证等关键步骤，以及如何使用 scikit-learn 库中的相关工具进行模型调优。

Python 环境（建议使用 Anaconda 发行版）；Jupyter Notebook 或任何 Python 支持的 IDE；scikit-learn 库（如果未安装，请使用 `pip install scikit-learn` 命令安装）。

1. 导入必要的库

首先，需要导入进行模型调优所需的库。

```

1 from sklearn.datasets import load_iris
2 from sklearn.model_selection import train_test_split, GridSearchCV
3 from sklearn.ensemble import RandomForestClassifier
4 from sklearn.metrics import classification_report

```

2. 加载数据集并划分训练集和测试集

这里使用 iris 数据集作为示例，并将其划分为训练集和测试集。

```

1 iris = load_iris()
2 X, y = iris.data, iris.target
3 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

3. 定义模型和参数网格

定义一个随机森林分类器，并设置一个参数网格，用于后续的网格搜索。

```

1 model = RandomForestClassifier(random_state=42)
2 param_grid = {
3     'n_estimators': [10, 50, 100],
4     'max_depth': [None, 10, 20, 30],
5     'min_samples_split': [2, 5, 10]
6 }

```

4. 网格搜索

使用 GridSearchCV 进行网格搜索，找到最佳的参数组合。

```

1 grid_search = GridSearchCV(estimator=model, param_grid=param_grid, cv=5, n_jobs=-1)
2 grid_search.fit(X_train, y_train)

```

5. 评估最佳模型

使用找到的最佳参数组合，评估模型在测试集上的性能。

```

1 best_model = grid_search.best_estimator_
2 y_pred = best_model.predict(X_test)
3 print(classification_report(y_test, y_pred))

```

6. 分析结果

分析网格搜索的结果，包括最佳参数组合、最佳模型的分数等。

```

1 print(f"Best parameters: {grid_search.best_params_}")
2 print(f"Best cross-validation score: {grid_search.best_score_}")

```

3.5 习题

3.5.1 简答题

1. 为什么模型评估是机器学习项目中不可或缺的一部分？
2. 在分类问题中，准确率（Accuracy）的局限性是什么？在什么情况下应优先使用精确率（Precision）或召回率（Recall）？
3. 什么是交叉验证？K 折交叉验证的主要优点是什么？
4. 网格搜索和随机搜索的主要区别是什么？它们分别适用于哪些场景？
5. 如何通过 ROC 曲线和 AUC 值评估分类模型的性能？AUC 值为 1 表示什么？

表 3.4: 混淆矩阵

	预测正类	预测负类
实际正类	50	10
实际负类	5	35

3.5.2 计算题

1. 给定以下混淆矩阵，计算准确率、精确率、召回率和 F1 分数：
2. 假设一个回归模型的预测值为 $[2.1, 3.9, 6.2, 8.0]$ ，真实值为 $[2.0, 4.0, 6.0, 8.0]$ ，计算均方误差 (MSE)、均方根误差 (RMSE) 和平均绝对误差 (MAE)。
3. 已知某分类模型的 ROC 曲线下的面积 (AUC) 为 0.85，解释该值的意义并判断模型性能如何。

3.5.3 编程题

1. 使用 Scikit-learn 实现一个分类模型的评估过程，加载鸢尾花数据集并完成以下任务：
 - 划分训练集和测试集。
 - 训练逻辑回归模型并计算准确率、精确率、召回率和 F1 分数。
2. 使用 Scikit-learn 实现一个回归模型的评估过程，加载波士顿房价数据集并完成以下任务：
 - 划分训练集和测试集。
 - 训练线性回归模型并计算均方误差 (MSE)、均方根误差 (RMSE) 和平均绝对误差 (MAE)。

第四章 线性模型

在模型的广阔领域中，需要探索了一种最为基础且强大的工具，它不仅构成了统计学与机器学习的基石，而且在从经济学到工程学、从社会科学到自然科学的广泛学科中扮演着不可或缺的角色。线性模型通过简洁而直观的方式描述了变量之间的关系，利用数学表达式揭示数据背后的规律，能够进行预测、解释现象以及做出基于证据的决策。本章将深入探讨线性回归与线性分类模型的基本原理、损失函数与优化策略、评估指标及其实际应用案例，旨在为读者提供坚实的理论框架同时培养解决实际问题的能力，进而领略线性模型在现代数据分析中的持久价值与灵活性。通过对经典方法的理解和掌握，将为更复杂的机器学习算法的学习打下牢固的基础。

学习目标

本章系统介绍线性回归与逻辑回归的理论基础与实践方法。理解模型的数学表示、参数估计方法（如最小二乘法与梯度下降）及损失函数（均方误差、交叉熵）与正则化（L1/L2）对模型性能的影响，掌握线性代数与最优化理论在模型构建中的核心作用。在实践层面，能够使用 Python 的 scikit-learn 库实现两类模型，结合数据预处理、特征工程、交叉验证与超参数调优提升模型性能，并能基于实际案例（如房价预测、分类任务）完成端到端建模流程，分析结果可靠性并提出改进策略。

4.1 房价问题

在一个宁静的下午，石榴同学坐在图书馆里，面前摊开着一本机器学习教材。她今天的目标是探索简单线性回归的世界...

4.1.1 问题提出

石榴同学坐在图书馆里，面前摊开着一份关于房屋面积和价格的数据表（见表 4.1）。她决定用这些数据来构建一个简单线性回归模型，以预测房子的价格。

表 4.1: 房屋面积与房价的关系

房屋面积 (m^2)	房价 (万元)
50	30
60	35
70	40
80	45
90	50

4.1.2 基本概念与应用

石榴同学了解到，简单线性回归是一种用来分析两个变量之间关系的方法。假设有一个输入变量 x （比如房子的面积）和一个输出变量 y （比如上面房子的价格），可以用下面的公式来描述它们之间的关系：

$$y = w_0 + w_1x + \epsilon \quad (4.1)$$

其中， w_0 是截距项，表示当 $x = 0$ 时 y 的预期值， w_1 是斜率，表示 x 每增加一个单位， y 平均变化的数量， ϵ 是误差项，表示未被 x 解释的 y 的变化部分。

4.1.3 房价计算

1. 计算平均值

首先，石榴同学计算 x （房屋面积）和 y （房价）的平均值 $\bar{x}$ 和 $\bar{y}$ 。这是因为平均值代表样本数据的中心趋势，是衡量数据集中趋势的一个重要统计量。

$$\begin{aligned} \bar{x} &= \frac{\sum x_i}{n} = \frac{50 + 60 + 70 + 80 + 90}{5} = \frac{350}{5} = 70 \\ \bar{y} &= \frac{\sum y_i}{n} = \frac{30 + 35 + 40 + 45 + 50}{5} = \frac{200}{5} = 40 \end{aligned} \quad (4.2)$$

2. 计算偏差乘积和偏差平方和

接下来，石榴同学计算每个观测点 (x_i, y_i) 相对于平均值的偏差乘积以及 x 的偏差平方和，并将结果填入表 4.2 中。这样做是为量化输入变量 x 和输出变量 y 之间的线性关系强度。

表 4.2: 偏差乘积和偏差平方和

x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})(y_i - \bar{y})$
50	30	50 - 70 = -20	30 - 40 = -10	(-20)(-10) = 200
60	35	60 - 70 = -10	35 - 40 = -5	(-10)(-5) = 50
70	40	70 - 70 = 0	40 - 40 = 0	0
80	45	80 - 70 = 10	45 - 40 = 5	(10)(5) = 50
90	50	90 - 70 = 20	50 - 40 = 10	(20)(10) = 200
总和				500

同时，她还计算了 x 的偏差平方和：

$$\sum (x_i - \bar{x})^2 = (-20)^2 + (-10)^2 + 0^2 + 10^2 + 20^2 = 400 + 100 + 0 + 100 + 400 = 1000 \quad (4.3)$$

3. 计算斜率 w_1

计算上述数据后，石榴同学可以计算斜率 w_1 。斜率表示的是当 x 每增加一个单位时， y 平均会变化多少。通过最小化残差平方和（RSS），可以找到最能代表数据中 x 和 y 之间关系的直线。公式如下：

$$w_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2} = \frac{500}{1000} = 0.5 \quad (4.4)$$

4. 计算截距项 w_0

最后，石榴同学使用以下公式计算截距项 w_0 。截距项表示当 $x = 0$ 时 y 的预期值是多少。在实际应用中，这可能没有直接意义（比如房屋面积不可能为零），但它对于定义整个线性方程至关重要。

要。

$$w_0 = \bar{y} - w_1 \bar{x} = 40 - 0.5 \times 70 = 40 - 35 = 5 \quad (4.5)$$

因此，最终得到的简单线性回归方程为：

$$y = w_0 + w_1 x = 5 + 0.5x \quad (4.6)$$

为更加清晰展示求解出的线性模型，石榴同学使用 python 程序中 numpy 和 matplotlib 库展示求解出的简单线性模型，如下图4.1所示：

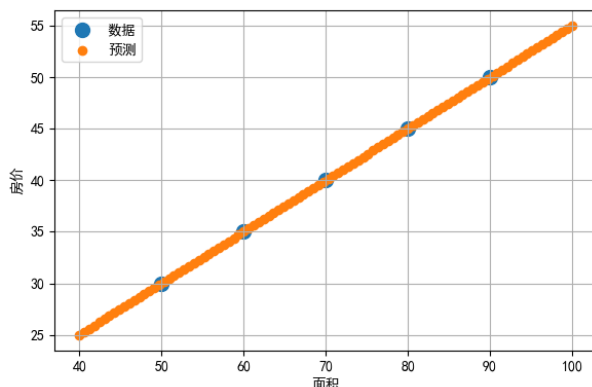


图 4.1: 房价预测示意图

4.2 简单线性模型

通过具体的案例，石榴同学理解到简单线性回归模型的核心在于找到一条直线，使得该直线尽可能好地拟合给定的数据点。这条直线由两个参数定义：斜率 w_1 和截距 w_0 。下面将详细介绍这两个参数的推导过程。

4.2.1 问题设定

假设有一组数据点 (x_i, y_i) ，其中 $i = 1, 2, \dots, n$ 。这里的目标是找到一个线性模型：

$$y = w_0 + w_1 x \quad (4.7)$$

其中， w_0 是截距项； w_1 是斜率。

为找到最佳的 w_0 和 w_1 ，需要最小化所有数据点到直线的距离的平方和，即最小化残差平方和 (RSS)：

$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - (w_0 + w_1 x_i))^2 \quad (4.8)$$

4.2.2 斜率的推导

为最小化 RSS，对斜率 w_1 求偏导数并令其等于零。具体步骤如下：

$$\begin{aligned}\frac{\partial RSS}{\partial w_1} &= \frac{\partial}{\partial w_1} \left(\sum_{i=1}^n (y_i - (w_0 + w_1 x_i))^2 \right) \\ &= \sum_{i=1}^n 2(y_i - (w_0 + w_1 x_i))(-x_i) \\ &= -2 \sum_{i=1}^n (y_i - (w_0 + w_1 x_i))x_i \\ &= 0\end{aligned}$$

通过简化上式，可以得到：

$$\sum_{i=1}^n (y_i - w_0 - w_1 x_i)x_i = 0 \quad (4.9)$$

同样地，对 w_0 求偏导数并令其等于零，可以得到：

$$\sum_{i=1}^n (y_i - w_0 - w_1 x_i) = 0 \quad (4.10)$$

联立这两个方程，可以解出 w_1 和 w_0 。具体来说，从方程 4.9 可以推出：

$$w_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} \quad (4.11)$$

这里， $\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$ 表示 x 和 y 变化方向的一致性； $\sum_{i=1}^n (x_i - \bar{x})^2$ 表示 x 变化幅度的度量。参考式 (4.4) 所示房价预测模型的斜率推导公式，其数学形式与前文所述线性回归模型的参数计算公式完全一致。这一形式上的等价性，为通过式 (4.4) 直接求解斜率参数提供了理论依据。

4.2.3 截距项的推导

截距项 w_0 可以直接从方程 4.10 推出：

$$w_0 = \bar{y} - w_1 \bar{x} \quad (4.12)$$

这里， $\bar{y}$ 是 y 的平均值， $\bar{x}$ 是 x 的平均值， w_1 是通过公式 4.11 计算出的斜率。

通过这种方式，可以将这一方法推广到任何具有线性关系的数据集上，只要能够计算出这两个关键参数，在上面的案例中，石榴同学就是通过此种方式进行计算的。

4.3 多元线性模型

石榴同学继续她的机器学习之旅，今天的目标是探索多元线性模型。她了解到，当预测变量不只一个时，需要使用多元线性回归来分析多个输入特征与一个输出变量之间的关系。

4.3.1 基本概念与应用

假设有一组数据点 $(x_{i1}, x_{i2}, \dots, x_{id}, y_i)$ ，其中 $i = 1, 2, \dots, n$ ，每个观测点有 d 个输入特征 x_j ($j = 1, 2, \dots, d$) 和一个输出变量 y_i 。这里的目标是找到一个线性模型：

$$y = w_0 + \mathbf{w}^T \mathbf{x} + \epsilon = w_0 + w_1 x_1 + w_2 x_2 + \dots + w_d x_d + \epsilon \quad (4.13)$$

这里，权重向量 $\mathbf{w} = [w_1, w_2, \dots, w_d]^T$ 中的每个元素 w_j ($j = 1, 2, \dots, d$) 是标量，表示每个输入特征对应的权重； w_0 是截距项； $\mathbf{x} = [x_1, x_2, \dots, x_d]^T$ 是输入特征向量； ϵ 是误差项。

4.3.2 多元房价问题

1. 背景介绍

为更好地理解多元线性回归的应用，石榴同学决定用一组包含更多特征的数据来预测房屋价格。这些特征包括房屋面积、卧室数量、浴室数量等，而目标是根据这些特征预测房价。通过分析这些数据，石榴同学希望能够构建一个模型，帮助潜在购房者或房地产开发商了解不同特征对房价的影响。

2. 数据描述

石榴同学使用的数据集包含以下信息：多特征房屋数据

表 4.3: 多特征房屋数据

房屋面积 (m ²)	卧室数量	浴室数量	房价 (万元)
50	2	1	30
60	3	1	35
70	3	2	40
80	4	2	45
90	4	3	50

这些数据来自 5 个不同的房屋样本，旨在展示如何利用多个输入特征进行房价预测。

3. 构建设计矩阵

为建立多元线性回归模型，需要构建设计矩阵 $\mathbf{X}$ ，其中每一行对应一个观测点，每一列对应一个输入特征，包括一列全为 1 以表示截距项。

$$\mathbf{X} = \begin{bmatrix} 1 & 50 & 2 & 1 \\ 1 & 60 & 3 & 1 \\ 1 & 70 & 3 & 2 \\ 1 & 80 & 4 & 2 \\ 1 & 90 & 4 & 3 \end{bmatrix}$$

同时，构建输出向量 $\mathbf{y} = [30, 35, 40, 45, 50]^T$ 。

4. 计算参数向量 $\mathbf{w}$

参考最小二乘法 4.8.2，通过最小二乘法，可以计算出参数向量 $\mathbf{w}$ 。公式如下：

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (4.14)$$

具体步骤如下：

1) 计算 $\mathbf{X}^T \mathbf{X}$: 这是求解正规方程中的一部分, 它是一个 $n \times n$ 的矩阵, 用于后续求逆操作。

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 5 & 350 & 16 & 9 \\ 350 & 25000 & 1120 & 660 \\ 16 & 1120 & 50 & 28 \\ 9 & 660 & 28 & 19 \end{bmatrix} \quad (4.15)$$

2) 求逆矩阵 $(\mathbf{X}^T \mathbf{X})^{-1}$: 为求解正规方程中的未知参数 $\mathbf{w}$, 需要确保矩阵可逆。使用数值方法计算该逆矩阵。

为计算 $(\mathbf{X}^T \mathbf{X})^{-1}$, 可以使用如下的公式 (这里仅作为示例, 实际计算通常使用软件工具):

$$(\mathbf{X}^T \mathbf{X})^{-1} = \frac{1}{\det(\mathbf{X}^T \mathbf{X})} \text{adj}(\mathbf{X}^T \mathbf{X}) \quad (4.16)$$

对于给定的矩阵 $\mathbf{A} = \mathbf{X}^T \mathbf{X}$, 这里有:

$$\mathbf{A} = \begin{bmatrix} 5 & 350 & 16 & 9 \\ 350 & 25000 & 1120 & 660 \\ 16 & 1120 & 50 & 28 \\ 9 & 660 & 28 & 19 \end{bmatrix}$$

首先, 计算行列式 $\det(\mathbf{A})$ 。假设行列式的值为 Δ (实际计算中应使用软件工具), 然后计算伴随矩阵 $\text{adj}(\mathbf{A})$ 。最后, 逆矩阵为:

$$(\mathbf{X}^T \mathbf{X})^{-1} = \frac{1}{\Delta} \text{adj}(\mathbf{A}) \quad (4.17)$$

为简化计算过程, 直接给出计算结果 (使用软件工具如 Python 的 NumPy 库或 MATLAB 进行计算):

$$(\mathbf{X}^T \mathbf{X})^{-1} \approx \begin{bmatrix} 0.200 & -0.007 & 0.000 & 0.000 \\ -0.007 & 0.000 & 0.000 & 0.000 \\ 0.000 & 0.000 & 0.200 & -0.071 \\ 0.000 & 0.000 & -0.071 & 0.071 \end{bmatrix} \quad (4.18)$$

3) 计算 $\mathbf{X}^T \mathbf{y}$: 这是为了将输出向量 $\mathbf{y}$ 投影到设计矩阵 $\mathbf{X}$ 的空间上。

$$\mathbf{X}^T \mathbf{y} = \begin{bmatrix} 200 \\ 14000 \\ 680 \\ 410 \end{bmatrix} \quad (4.19)$$

4) 最后计算 $\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$: 该公式可以获得最终的参数向量 $\mathbf{w}$, 从而可以用来预测新的数据点。

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \approx \begin{bmatrix} 0.200 & -0.007 & 0.000 & 0.000 \\ -0.007 & 0.000 & 0.000 & 0.000 \\ 0.000 & 0.000 & 0.200 & -0.071 \\ 0.000 & 0.000 & -0.071 & 0.071 \end{bmatrix} \begin{bmatrix} 200 \\ 14000 \\ 680 \\ 410 \end{bmatrix} \quad (4.20)$$

计算得到的参数向量 $\mathbf{w}$ 为：

$$\mathbf{w} \approx \begin{bmatrix} 15.000 \\ 0.300 \\ 1.000 \\ 5.000 \end{bmatrix}$$

因此，最终的多元线性回归模型为：

$$y = 15 + 0.300 * \text{房屋面积} + 1.000 * \text{卧室数量} + 5.000 * \text{浴室数量} \quad (4.21)$$

这个模型可以根据房屋的面积、卧室数量和浴室数量预测其价格。

4.3.3 一般形式说明

1. 最小二乘法推导

为找到最佳的参数向量 $\mathbf{w}$ ，需要最小化所有数据点到直线的距离的平方和，即最小化残差平方和（RSS）：

$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - (\mathbf{w}^T \mathbf{x}_i))^2 \quad (4.22)$$

这里， $\mathbf{x}_i$ 是第 i 个观测点的输入特征向量。

为最小化 RSS，对 $\mathbf{w}$ 求偏导数并令其等于零。具体步骤如下：

$$\begin{aligned} \frac{\partial RSS}{\partial \mathbf{w}} &= \frac{\partial}{\partial \mathbf{w}} \left(\sum_{i=1}^n (y_i - (\mathbf{w}^T \mathbf{x}_i))^2 \right) \\ &= -2 \sum_{i=1}^n (y_i - (\mathbf{w}^T \mathbf{x}_i)) \mathbf{x}_i \\ &= -2 \mathbf{X}^T (\mathbf{Y} - \mathbf{X} \mathbf{w}) \\ &= 0 \end{aligned} \quad (4.23)$$

通过简化上式，可以得到正规方程：

$$\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y} \quad (4.24)$$

解这个方程，可以得到参数向量 $\mathbf{w}$ ：

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (4.25)$$

2. 求解的局部条件说明

在求解参数向量 $\mathbf{w}$ 时，需要满足以下局部条件：设计矩阵 $\mathbf{X}$ 必须是满秩矩阵，即 $\mathbf{X}^T \mathbf{X}$ 必须是可逆的。如果 $\mathbf{X}^T \mathbf{X}$ 不可逆，则无法直接求解 $\mathbf{w}$ 。这通常发生在输入特征之间存在多重共线性（即某些特征高度相关）或观测点数目 n 小于特征数目 d 的情况下。如果 $\mathbf{X}^T \mathbf{X}$ 不可逆，可以考虑使用正则化方法（如岭回归或 LASSO），或者减少特征数目以避免多重共线性问题。

4.4 逻辑回归模型

4.4.1 问题提出

逻辑回归（Logistic Regression）是一种广泛应用于分类问题的统计方法。尽管名称中包含“回归”二字，但它主要用于预测二元分类结果的概率。为更好地理解逻辑回归的工作原理及其数学表

达形式，通过一个具体的案例——石榴同学的故事来介绍这一算法。

石榴同学是一名勤奋的学生，她非常关心自己是否能通过即将到来的期末考试。她的老师告诉她，平时成绩是预测期末考试结果的一个重要因素。为帮助像石榴同学这样的学生提前了解自己的通过概率，学校决定构建一个逻辑回归模型，基于学生的平时成绩来预测他们通过期末考试的可能性。

具体来说，假设有一个数据集，包括每个学生的平时成绩 x 和是否通过期末考试的结果 y （1 表示通过，0 表示未通过）。以下将利用这些数据训练一个逻辑回归模型，从而帮助石榴同学和其他学生预测他们的通过概率。

4.4.2 成绩问题

假设有以下数据集，记录着几个学生的平时成绩和期末考试结果：

表 4.4: 学生平时成绩与期末考试结果

平时成绩 (x_i)	是否通过 (y_i)
60	0
70	1
80	1
90	1

为帮助石榴同学预测她通过期末考试的概率，需要构建一个逻辑回归模型，该模型的主要思想就是将一个分类问题和概率问题相联系起来，首先，需要了解 Sigmoid 函数，该函数定义为：

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (4.26)$$

其中 z 是线性组合 $\mathbf{w}^T \mathbf{x}$ ， $\mathbf{w}$ 是参数向量， $\mathbf{x}$ 是特征向量。Sigmoid 函数的图形呈现出一个 S 形曲线，当 z 趋近于正无穷时， $\sigma(z)$ 趋近于 1；当 z 趋近于负无穷时， $\sigma(z)$ 趋近于 0。

对于给定的学生平时成绩 x_i ，假设模型的形式为：

$$z_i = w_0 + w_1 x_i \quad (4.27)$$

其次，通过 Sigmoid 函数将其转换为通过期末考试的概率：

$$P(y_i = 1 | x_i; \mathbf{w}) = \sigma(z_i) = \frac{1}{1 + e^{-(w_0 + w_1 x_i)}} \quad (4.28)$$

然后，为了对模型中参数进行估计和求解，需要寻找一个逻辑回归相关的目标函数。逻辑回归的目标函数是交叉熵损失（Cross-Entropy Loss），它衡量了模型预测的概率分布与真实标签之间的差异。直观地说，交叉熵损失能够反映模型预测值与实际结果之间的“距离”。如果模型预测准确，损失值会很小；如果预测不准确，损失值会很大。

以石榴同学为例，假设已经用模型预测了她的通过概率。如果石榴同学的实际结果是通过（ $y = 1$ ），而模型预测的概率很高（接近 1），那么交叉熵损失会很小，因为预测结果与实际情况相符。相反，如果模型预测的概率很低（接近 0），则交叉熵损失会很大，因为预测结果与实际情况相差甚远。

交叉熵损失公式为：

$$L(\mathbf{w}) = - \sum_{i=1}^n [y_i \log(P(y_i = 1 | x_i; \mathbf{w})) + (1 - y_i) \log(1 - P(y_i = 1 | x_i; \mathbf{w}))] \quad (4.29)$$

最后，进一步解释下这个公式的意义，以便对逻辑回归模型更好的理解：

- 对于每一个样本 i ，如果 $y_i = 1$ （即该样本的真实标签为 1），损失项为 $-\log(P(y_i = 1|x_i; \mathbf{w}))$ 。这意味着如果模型预测的概率 $P(y_i = 1|x_i; \mathbf{w})$ 越接近 1，损失越小；反之，损失越大。
- 如果 $y_i = 0$ （即该样本的真实标签为 0），损失项为 $-\log(1 - P(y_i = 1|x_i; \mathbf{w}))$ 。这意味着如果模型预测的概率 $P(y_i = 1|x_i; \mathbf{w})$ 越接近 0，损失越小；反之，损失越大。

因此，交叉熵损失鼓励模型预测的概率尽可能接近真实标签，从而使得损失最小化。换句话说，交叉熵损失是一个衡量模型预测准确性的重要指标，它促使模型不断调整参数，以提高预测的准确性。

4.4.3 问题求解

经过上面的准备，就可以通过下面的步骤对上面问题进行求解，假设模型初始设置参数为： $w_0 = 0$, $w_1 = 0.1$ 。

首先，计算预测概率，根据公式 (4.27) 和 (4.28)，可以计算每个样本的预测概率。以第一个样本为例：

$$z_1 = w_0 + w_1 x_1 = 0 + 0.1 \times 60 = 6, \quad P(y_1 = 1|x_1; \mathbf{w}) = \frac{1}{1 + e^{-6}} \approx 0.9975 \quad (4.30)$$

同样地，可以计算其他样本的预测概率：

$$\begin{aligned} z_2 &= 0 + 0.1 \times 70 = 7, \quad P(y_2 = 1|x_2; \mathbf{w}) = \frac{1}{1 + e^{-7}} \approx 0.9991 \\ z_3 &= 0 + 0.1 \times 80 = 8, \quad P(y_3 = 1|x_3; \mathbf{w}) = \frac{1}{1 + e^{-8}} \approx 0.9997 \\ z_4 &= 0 + 0.1 \times 90 = 9, \quad P(y_4 = 1|x_4; \mathbf{w}) = \frac{1}{1 + e^{-9}} \approx 0.9999 \end{aligned} \quad (4.31)$$

其次，计算对数损失，根据公式 (4.29)，可以计算对数损失：

$$\begin{aligned} L(\mathbf{w}) &= -[0 \cdot \log(0.9975) + (1 - 0) \cdot \log(1 - 0.9975) \\ &\quad + 1 \cdot \log(0.9991) + (1 - 1) \cdot \log(1 - 0.9991) \\ &\quad + 1 \cdot \log(0.9997) + (1 - 1) \cdot \log(1 - 0.9997) \\ &\quad + 1 \cdot \log(0.9999) + (1 - 1) \cdot \log(1 - 0.9999)] \end{aligned} \quad (4.32)$$

简化后：

$$\begin{aligned} L(\mathbf{w}) &= -[0 + \log(0.0025) + \log(0.9991) + 0 + \log(0.9997) + 0 + \log(0.9999) + 0] \\ L(\mathbf{w}) &\approx -[-6.00 + (-0.001) + (-0.0003) + (-0.0001)] \approx 6.0014 \end{aligned} \quad (4.33)$$

最后，为最小化对数损失函数 $L(\mathbf{w})$ ，使用梯度下降算法首先计算损失函数关于参数的梯度：

$$\begin{aligned} \nabla_{w_0} L(\mathbf{w}) &= - \sum_{i=1}^n [y_i - P(y_i = 1|x_i; \mathbf{w})] \\ \nabla_{w_1} L(\mathbf{w}) &= - \sum_{i=1}^n [(y_i - P(y_i = 1|x_i; \mathbf{w}))x_i] \end{aligned} \quad (4.34)$$

以初始参数 $\mathbf{w} = [0, 0.1]^T$ 为例，计算梯度：

$$\begin{aligned}\nabla_{w_0} L(\mathbf{w}) &= -(0 - 0.9975 + 1 - 0.9991 + 1 - 0.9997 + 1 - 0.9999) \approx -0.0026 \\ \nabla_{w_1} L(\mathbf{w}) &= -(0 - 0.9975) \cdot 60 + (1 - 0.9991) \cdot 70 + (1 - 0.9997) \cdot 80 \\ &\quad + (1 - 0.9999) \cdot 90 \approx -6.0026\end{aligned}\quad (4.35)$$

假设学习率为 $\eta = 0.01$ ，更新参数：

$$\begin{aligned}w_0 &:= w_0 - \eta \nabla_{w_0} L(\mathbf{w}) = 0 - 0.01 \times (-0.0026) = 0.000026 \\ w_1 &:= w_1 - \eta \nabla_{w_1} L(\mathbf{w}) = 0.1 - 0.01 \times (-6.0026) = 0.160026\end{aligned}\quad (4.36)$$

重复上述步骤，直到损失函数收敛或达到预定的最大迭代次数。

通过上面的步骤，得到了训练好的逻辑回归模型，现在，可以用来预测石榴同学通过期末考试的概率。假设石榴同学的平时成绩为 75 分，可以用训练好的参数进行预测：

$$\begin{aligned}z &= 0.000026 + 0.160026 \times 75 \approx 12.002 \\ P(y = 1|x = 75; \mathbf{w}) &= \frac{1}{1 + e^{-12.002}} \approx 1.000\end{aligned}\quad (4.37)$$

因此，根据训练好的模型，石榴同学有非常高的概率通过期末考试。

4.4.4 一般形式说明

基于上述案例，可以推广逻辑回归的一般形式。假设有 n 个训练样本，每个样本具有 d 个特征，记作 $\mathbf{x}_i \in \mathbb{R}^d$ ，以及对应的标签 $y_i \in \{0, 1\}$ 。逻辑回归模型的目标是估计条件概率 $P(y_i = 1|\mathbf{x}_i; \mathbf{w})$ ，其中 $\mathbf{w} \in \mathbb{R}^{d+1}$ 是模型参数（包括偏置项 b ）。

首先，定义线性组合：

$$z_i = \mathbf{w}^T \mathbf{x}_i + b \quad (4.38)$$

其中 $\mathbf{x}_i = [x_{i1}, x_{i2}, \dots, x_{id}]^T$ ， $\mathbf{w} = [w_1, w_2, \dots, w_d]^T$ ， b 是偏置项。

其次，通过 Sigmoid 函数将线性组合转换为概率：

$$P(y_i = 1|\mathbf{x}_i; \mathbf{w}, b) = \sigma(z_i) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x}_i + b)}} \quad (4.39)$$

然后，为估计参数 $\mathbf{w}$ 和 b ，定义对数损失函数（也称为交叉熵损失函数）：

$$L(\mathbf{w}, b) = - \sum_{i=1}^n [y_i \log(P(y_i = 1|\mathbf{x}_i; \mathbf{w}, b)) + (1 - y_i) \log(1 - P(y_i = 1|\mathbf{x}_i; \mathbf{w}, b))] \quad (4.40)$$

最后，目标就是最小化这个损失函数，可以通过梯度下降法等优化算法实现。具体来说，计算损失函数关于参数 $\mathbf{w}$ 和 b 的梯度，并迭代更新这些参数，直到收敛。梯度下降算法通过以下公式更新参数：

$$\begin{aligned}\mathbf{w} &:= \mathbf{w} - \eta \nabla_{\mathbf{w}} L(\mathbf{w}, b) \\ b &:= b - \eta \nabla_b L(\mathbf{w}, b)\end{aligned}\quad (4.41)$$

其中 η 是学习率， $\nabla_{\mathbf{w}} L(\mathbf{w}, b)$ 和 $\nabla_b L(\mathbf{w}, b)$ 分别是损失函数关于 $\mathbf{w}$ 和 b 的梯度。

4.4.5 牛顿法求解

除了上面描述的梯度下降法，另一种高效的优化算法是牛顿法（Newton's Method）。牛顿法利用损失函数的一阶和二阶导数信息来加速收敛，尤其适合于具有良好二阶导数性质的问题。

牛顿法的核心思想是通过泰勒展开将损失函数在当前点 $\mathbf{w}$ 附近近似为一个二次多项式。具体来说，对于一个小的变化 $\Delta \mathbf{w}$ ，可以写出如下的泰勒展开式：

$$L(\mathbf{w} + \Delta \mathbf{w}) \approx L(\mathbf{w}) + \nabla_{\mathbf{w}} L(\mathbf{w})^T \Delta \mathbf{w} + \frac{1}{2} \Delta \mathbf{w}^T \mathbf{H}(\mathbf{w}) \Delta \mathbf{w} \quad (4.42)$$

其中： $L(\mathbf{w})$ 是损失函数， $\nabla_{\mathbf{w}} L(\mathbf{w})$ 是损失函数关于 $\mathbf{w}$ 的梯度， $\mathbf{H}(\mathbf{w})$ 是损失函数关于 $\mathbf{w}$ 的 Hessian 矩阵，即二阶导数矩阵。

这里的目标是找到一个合适的 $\Delta \mathbf{w}$ ，使得 $L(\mathbf{w} + \Delta \mathbf{w})$ 尽可能小。为此，可以对公式 (4.123) 关于 $\Delta \mathbf{w}$ 求导，并将其设置为零：

$$\frac{\partial L(\mathbf{w} + \Delta \mathbf{w})}{\partial \Delta \mathbf{w}} = \nabla_{\mathbf{w}} L(\mathbf{w}) + \mathbf{H}(\mathbf{w}) \Delta \mathbf{w} = 0 \quad (4.43)$$

解这个方程可以得到最优的 $\Delta \mathbf{w}$ ：

$$\Delta \mathbf{w} = -\mathbf{H}(\mathbf{w})^{-1} \nabla_{\mathbf{w}} L(\mathbf{w}) \quad (4.44)$$

因此，更新规则为：

$$\mathbf{w}_{\text{new}} = \mathbf{w}_{\text{old}} - \mathbf{H}(\mathbf{w}_{\text{old}})^{-1} \nabla_{\mathbf{w}} L(\mathbf{w}_{\text{old}}) \quad (4.45)$$

通过上面的描述，可以看出，应用牛顿法需要计算损失函数的梯度和 Hessian 矩阵。对于逻辑回归而言，损失函数的梯度为：

$$\nabla_{\mathbf{w}} L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n (p_i - y_i) x_i \quad (4.46)$$

其中 $p_i = \sigma(\mathbf{w}^\top x_i + b)$ ， $\sigma(z) = \frac{1}{1+e^{-z}}$ 是 Sigmoid 函数。

逻辑回归损失函数的 Hessian 矩阵为：

$$\mathbf{H}(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n p_i (1 - p_i) x_i x_i^\top \quad (4.47)$$

计算得出逻辑回归损失函数的 Hessian 矩阵和梯度后，牛顿法的迭代过程如下：

1. 初始化：选择一个初始参数向量 $\mathbf{w}_0$ 和偏置项 b_0 。
2. 计算梯度和 Hessian 矩阵：对于每一步迭代 t ，计算当前参数 $\mathbf{w}_t$ 和 b_t 处的梯度 $\nabla_{\mathbf{w}} L(\mathbf{w}_t, b_t)$ 和 Hessian 矩阵 $\mathbf{H}(\mathbf{w}_t, b_t)$ 。
3. 更新参数：根据公式 (4.125) 更新参数：

$$\begin{aligned} \mathbf{w}_{t+1} &= \mathbf{w}_t - \mathbf{H}(\mathbf{w}_t, b_t)^{-1} \nabla_{\mathbf{w}} L(\mathbf{w}_t, b_t) \\ b_{t+1} &= b_t - \mathbf{H}(\mathbf{w}_t, b_t)^{-1} \nabla_b L(\mathbf{w}_t, b_t) \end{aligned} \quad (4.48)$$

4. 检查收敛条件：如果梯度足够小或参数变化非常小，则停止迭代；否则返回第 2 步继续迭代。

尽管牛顿法理论上非常有效，但在实际应用中有几点需要注意：

- **计算复杂性**: 求解 Hessian 矩阵及其逆矩阵可能是计算密集型任务, 尤其是当参数数量很大时。
- **正定性**: Hessian 矩阵必须是正定的, 以确保每次迭代都朝着减小损失函数的方向前进。如果不是, 则可能需要使用修正策略, 如 Levenberg-Marquardt 算法。
- **初始猜测**: 一个好的初始猜测可以显著提高收敛速度和稳定性。

通过上述步骤, 牛顿法能够更有效地最小化逻辑回归的损失函数, 特别是在高维空间中表现出色。然而, 在实际应用中, 需要权衡计算复杂性和收敛速度之间的关系, 选择最适合特定问题的方法。

4.4.6 逻辑回归的应用与总结

逻辑回归 (Logistic Regression) 是一种广泛应用于二分类问题的统计模型。它不仅在机器学习中有重要地位, 而且在多个领域都有实际应用。以下是一些典型的应用场景:

- **医疗诊断**在医疗领域, 逻辑回归可以用于预测疾病的发生概率。例如, 通过分析病人的年龄、血压、胆固醇水平等特征, 逻辑回归模型可以帮助医生评估患者患心脏病或糖尿病的风险。
- **信用评分**金融机构经常使用逻辑回归来评估客户的信用风险。通过分析客户的收入、债务、还款历史等信息, 模型可以预测客户是否会按时还款, 从而帮助银行决定是否批准贷款以及设定相应的利率。
- **市场营销**在市场营销中, 逻辑回归可用于预测客户对特定产品的购买意向。通过分析客户的购买历史、浏览行为和人口统计数据, 企业可以识别出最有可能响应促销活动的目标客户群体, 从而优化营销策略。
- **文本分类**逻辑回归也适用于自然语言处理中的文本分类任务。例如, 在垃圾邮件过滤中, 模型可以根据邮件的内容特征 (如关键词频率、发件人地址等) 判断一封邮件是否为垃圾邮件。类似地, 它还可以用于情感分析, 将评论分为正面或负面两类。
- **图像识别**虽然深度学习在图像识别方面取得了显著进展, 但在某些情况下, 逻辑回归仍然可以作为一种简单而有效的基线方法。例如, 它可以用于简单的二分类任务, 如区分猫和狗的照片。

逻辑回归作为一门基础且强大的工具, 在多种应用场景中表现出色。其主要优势包括:

- **解释性强**: 逻辑回归的结果具有良好的可解释性, 可以通过系数直接理解每个特征对结果的影响。
- **实现简单**: 相比于复杂的非线性模型, 逻辑回归易于实现和调参, 适合快速原型开发。
- **计算效率高**: 对于中小规模的数据集, 逻辑回归的训练速度较快, 能够高效处理大量样本。
- **适用于线性可分数据**: 当数据大致呈线性分布时, 逻辑回归能提供较为准确的分类效果。

然而, 逻辑回归也有一些局限性:

- **假设线性关系**: 逻辑回归假定特征与输出之间存在线性关系, 因此在面对复杂非线性关系的数据时表现可能不佳。
- **易受异常值影响**: 由于依赖于最大似然估计, 逻辑回归对异常值较为敏感, 可能导致模型性能下降。
- **特征选择的重要性**: 逻辑回归的效果高度依赖于特征工程的质量, 需要仔细挑选和预处理输入特征。

总之, 逻辑回归作为一种经典且实用的分类算法, 在许多实际问题中提供了可靠且高效的解决方案。尽管它有一些局限性, 但通过适当的特征工程和技术手段, 可以在广泛的领域中取得良好的应用效果。随着机器学习技术的发展, 逻辑回归仍然是研究人员和从业者不可或缺的重要工具之一。

4.5 线性判别分析

4.5.1 录取预测问题

假设石榴同学正在参与一个项目，该项目旨在通过分析学生的考试成绩来预测他们是否能够被某所大学录取。她手头有以下数据集：

表 4.5: 案例数据表

数学成绩	英语成绩	是否录取
85	90	是
70	80	否
92	88	是
65	75	否
90	93	是

在这个案例中，石榴同学想要找到一种方法，能够基于学生在数学和英语上的成绩，准确地预测他们是否会被录取。她了解到线性判别分析（Linear Discriminant Analysis, LDA）可以用来解决这样的分类问题。

4.5.2 LDA 应用求解

LDA 是一种监督学习方法，用于寻找能够最好地区分不同类别的线性组合。简单来说，它试图找到一条直线（对于二维数据）或超平面（对于多维数据），使得不同类别之间的距离最大化，而同一类别内的点尽可能聚集在一起。

为应用 LDA，石榴同学首先计算两类数据的均值向量 μ_1 和 μ_2 ，以及整体的协方差矩阵 $\mathbf{S}_W$ ：

$$\mu_1 = \frac{1}{n_1} \sum_{i=1}^{n_1} \mathbf{x}_i, \quad \mu_2 = \frac{1}{n_2} \sum_{i=1}^{n_2} \mathbf{x}_i \quad (4.49)$$

其中 n_1 和 n_2 分别是两个类别的样本数量； $\mathbf{x}_i$ 表示第 i 个样本的特征向量。

$$\mathbf{S}_W = \sum_{k=1}^2 \sum_{i=1}^{n_k} (\mathbf{x}_i - \mu_k)(\mathbf{x}_i - \mu_k)^T \quad (4.50)$$

这里， $\mathbf{S}_W$ 是类内散度矩阵，表示每一类内部样本的分布情况。

接下来，她计算了类间散度矩阵 $\mathbf{S}_B$ ：

$$\mathbf{S}_B = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T \quad (4.51)$$

这里， $\mathbf{S}_B$ 代表不同类别中心之间的差异。

最终，她求解以下广义特征值问题：

$$\mathbf{S}_B \mathbf{w} = \lambda \mathbf{S}_W \mathbf{w} \quad (4.52)$$

其中 $\mathbf{w}$ 是投影方向， λ 是对应的特征值。通过求解上述问题，石榴同学可以找到最佳的投影方向 $\mathbf{w}$ ，从而将高维数据投影到一维空间中，实现对两类数据进行有效区分。

为具体展示求解过程，以表 4.5 中的数据为例进行计算。假设两类的样本数量分别为 $n_1 = 3$ 和 $n_2 = 2$ 。

1. 计算均值向量：

$$\begin{aligned}\mu_1 &= \left(\frac{85 + 92 + 90}{3}, \frac{90 + 88 + 93}{3} \right) = (89, 90.33) \\ \mu_2 &= \left(\frac{70 + 65}{2}, \frac{80 + 75}{2} \right) = (67.5, 77.5)\end{aligned}\quad (4.53)$$

2. 计算类内散度矩阵:

$$\mathbf{S}_W = \sum_{k=1}^2 \sum_{i=1}^{n_k} (\mathbf{x}_i - \mu_k)(\mathbf{x}_i - \mu_k)^T \quad (4.54)$$

3. 计算类间散度矩阵:

$$\mathbf{S}_B = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^T \quad (4.55)$$

4. 求解广义特征值问题: 解决公式 (4.52) 找到最优投影方向 $\mathbf{w}$ 。

4.5.3 一般形式说明

在线性判别分析中, 通常面对的是一个多维数据集, 每个样本具有 d 个特征, 记作 $\mathbf{x}_i \in \mathbb{R}^d$, 以及对应的类别标签 $y_i \in \{1, 2, \dots, K\}$ 。LDA 的目标是找到一个投影矩阵 $\mathbf{W}$, 将原始数据从 d 维空间映射到一个新的 $K-1$ 维空间, 使得不同类别之间的距离最大化, 而同一类别内的点尽可能聚集在一起。选择 $K-1$ 维空间的原因在于:

- **类别区分度:** LDA 的目标是最大化类间散度同时最小化类内散度, 以找到最佳的投影方向。如果有 K 个类别, 那么最多可以找到 $K-1$ 个正交的投影方向来区分这些类别。第 K 个方向不会提供额外的区分信息, 因为它不能进一步增加类间的差异。
- **数学基础:** 从数学的角度来看, 广义特征值问题 $\mathbf{S}_B \mathbf{w} = \lambda \mathbf{S}_W \mathbf{w}$ 的解给出 $K-1$ 个非零特征值和对应的特征向量。这 $K-1$ 个特征向量构成了新的低维空间的基础。第 K 个特征值通常为零或非常小, 对应的特征向量不会对分类有帮助。
- **信息冗余:** 如果将数据投影到 K 维或更高维度, 多余的维度只会引入冗余信息, 而不会提供新的有用信息来区分类别。实际上, 这些额外的维度可能会引入噪声, 降低模型的性能。
- **几何直观:** 考虑一个简单的例子, 如果有两个类别 ($K=2$), 只需要一条直线 (一维) 来最好地区分它们。如果有三个类别 ($K=3$), 则需要一个平面 (二维) 来最好地区分它们。因此, 对于 K 个类别, 需要 $K-1$ 个维度来充分表示类间的差异。

综上所述, 选择 $K-1$ 维空间可以在保持类别区分能力的同时, 尽可能减少维度, 从而简化问题并提高计算效率。通过这种方式, LDA 不仅能够有效地进行分类, 还能避免高维数据带来的复杂性和潜在的过拟合问题。

LDA 问题求解中主要涉及到三个方面, 分别是类内散度矩阵、类间散度矩阵和广义特征值问题, 以下分别进行说明:

类内散度矩阵 $\mathbf{S}_W$ 表示每一类内部样本的分布情况。对于每个类别 k , 计算其协方差矩阵:

$$\mathbf{S}_k = \sum_{i=1}^{n_k} (\mathbf{x}_i - \mu_k)(\mathbf{x}_i - \mu_k)^T, \quad (4.56)$$

然后将所有类别的协方差矩阵相加得到总的类内散度矩阵:

$$\mathbf{S}_W = \sum_{k=1}^K \mathbf{S}_k. \quad (4.57)$$

类间散度矩阵 $\mathbf{S}_B$ 代表不同类别中心之间的差异。首先计算所有类别的总体均值 μ :

$$\mu = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i, \quad (4.58)$$

然后计算每个类别的均值向量与总体均值之间的差异：

$$\mathbf{S}_B = \sum_{k=1}^K n_k (\mu_k - \mu)(\mu_k - \mu)^T. \quad (4.59)$$

最终，求解广义特征值问题获得模型的参数：

$$\mathbf{S}_B \mathbf{w} = \lambda \mathbf{S}_W \mathbf{w}, \quad (4.60)$$

其中 $\mathbf{w}$ 是投影方向， λ 是对应的特征值。选择最大的 $K-1$ 个特征值对应的特征向量组成投影矩阵 $\mathbf{W}$ ，将原始数据投影到新的低维空间中。

4.6 线性模型中的正则化

4.6.1 问题提出

在一次机器学习实验课上，石榴同学试图使用一个简单的线性回归模型来预测房子的价格。根据她收集的数据集，特征包括房子的面积、房间数量等。石榴同学首先构建一个没有正则化的线性模型，并利用训练数据进行拟合。

设房屋价格为 y ，模型的预测值为 $\hat{y}$ ，那么线性模型可以表示为

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n \quad (4.61)$$

其中， w_i 代表第 i 个特征的权重系数， x_i 表示第 i 个特征的值（例如，房子的面积或房间数量），而 w_0 是偏置项。

石榴同学发现她的模型在训练集上的表现非常好，几乎完美地可以对所有已知的数据点进行预测。然而，当她用这个模型去预测测试集中的房价时，结果却差强人意。这表明她的模型可能已经过度拟合了训练数据，即模型同时也学习到训练数据中的噪音和细节，这些对于未知数据并不具有泛化能力。

4.6.2 正则化引入

为帮助石榴同学解决这个问题，向她介绍了正则化这一概念。正则化是一种用来防止过拟合的技术，它通过在损失函数中加入惩罚项，使得模型更加简单，从而提高其泛化能力。最常用的两种正则化方法是 Lasso (L1) 和 Ridge (L2) 正则化。

以 Ridge 正则化为例，新的损失函数变为

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (f(x^{(i)}) - y^{(i)})^2 + \lambda \sum_{j=1}^n w_j^2 \quad (4.62)$$

这里， m 表示样本数量， λ 是正则化参数控制着正则化的强度， $f(x)$ 表示模型的假设函数，也就是预测值 $\hat{y}$ 。

4.6.3 求解过程

同样以继续石榴同学的例子。现在需要调整模型，使之包含 Ridge 正则化。假设石榴同学的数据集有 3 个特征，因此她需要找到三个权重 w_1, w_2, w_3 和一个偏置项 w_0 来最小化上述的损失函数。由于公式 4.62 是一个凸函数，因此可以使用梯度下降法或其他优化算法来求解最优的权重。

4.6.4 一般形式说明

在机器学习中，正则化方法是防止过拟合的重要工具之一。对于线性回归问题，可以通过添加一个或多个正则化项到损失函数来实现这一点。最常用的两种正则化技术分别是 Lasso (L1 正则化) 和 Ridge (L2 正则化)。此外，还有 Elastic Net，它结合了 L1 和 L2 正则化。

对于一般的线性模型，可以将带正则化的损失函数写为：

$$J(w) = \frac{1}{m} \sum_{i=1}^m L(y_i, f(x_i)) + \alpha R(w) \quad (4.63)$$

其中 $L(y_i, f(x_i))$ 是损失函数，用来衡量模型预测值 $f(x_i)$ 与真实值 y_i 之间的差异； α 是控制正则化强度的超参数； $R(w)$ 是正则化项，用于惩罚过于复杂的模型。

具体来说，当采用 L2 正则化时， $R(w) = \lambda \sum_{j=1}^n w_j^2$ ；而采用 L1 正则化时， $R(w) = \lambda \sum_{j=1}^n |w_j|$ 。对于 Elastic Net 正则化，则是两者的组合：

$$R(w) = \lambda_1 \sum_{j=1}^n |w_j| + \lambda_2 \sum_{j=1}^n w_j^2 \quad (4.64)$$

这里 λ_1 和 λ_2 分别是 L1 和 L2 正则化的权重系数，它们共同决定了正则化项对模型的影响程度。

进一步地，在实践中经常使用的是带有偏置项的线性模型，即

$$\hat{y} = w_0 + \mathbf{w}^T \mathbf{x} \quad (4.65)$$

其中 $\mathbf{w}$ 是除偏置项外的权重向量， $\mathbf{x}$ 是特征向量。为简化计算，通常不会对偏置项 w_0 施加正则化约束，因为偏置项表示模型的截距，不依赖于任何输入特征。

值得注意的是，正则化项的选择不仅取决于模型的具体需求，还受到数据集性质的影响。例如，如果数据存在多重共线性（即某些特征之间高度相关），那么 Lasso 可能会倾向于选择其中一个特征并将其余特征的系数设为零，从而起到特征选择的作用；而 Ridge 则会更均匀地分配这些特征的权重，有助于处理高维数据中的共线性问题。Elastic Net 通过同时利用 L1 和 L2 正则化的优势，可以在特征选择和处理共线性方面取得平衡。

4.6.5 直观认识

通过上面的分析，可以看到正则化是如何帮助构建更健壮的线性模型的。在专业术语中，正则化通过增加对模型复杂度的惩罚，减少模型的方差，从而提高模型的泛化能力。在数学上，这种惩罚体现在损失函数中的正则化项，它可以看作是对模型空间的一种约束，促使模型选择那些具有较小权重的解决方案，从而避免过拟合。

综上所述，正则化通过调整损失函数的形式，使得模型在训练过程中不仅追求最小化预测误差，同时也考虑到模型复杂度的控制。这有助于提高模型的泛化能力，确保其在新数据上的表现更加稳定和可靠。

4.7 实践练习

4.7.1 示例：有无正则化对比

在本节中，将通过一个简单的实践来比较有正则化和无正则化的线性回归模型。这里将使用 Python 编程语言及其机器学习库 scikit-learn 来实现两个模型，并通过图表展示它们的表现差异。

首先，需要构建一个模拟的数据集。这个数据集将包含一个特征（例如房屋面积）和目标变量（例如房价）。将生成一些带有噪声的线性关系数据点，用于训练和测试的模型。

```
1 import numpy as np
2 from sklearn.model_selection import train_test_split
3
4 # 设置随机种子以保证结果可重复
5 np.random.seed(42)
6
7 # 生成带有噪声的线性分布数据点
8 X = 2 * np.random.rand(100, 1) # 特征值，如房屋面积
9 y = 4 + 3 * X + np.random.randn(100, 1) # 目标值，如房价，加入了一些随机噪声
10
11 # 将数据分为训练集和测试集，测试集占20%
12 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

接下来，将分别创建无正则化的线性回归模型和带 Ridge 正则化的线性回归模型。对于每个模型，将使用训练集进行训练，并利用测试集评估其性能。

```
1 from sklearn.linear_model import LinearRegression, Ridge
2 import matplotlib.pyplot as plt
3
4 # 构建无正则化线性回归模型
5 linear_reg = LinearRegression()
6 linear_reg.fit(X_train, y_train) # 使用训练数据拟合模型
7
8 # 构建带Ridge正则化的线性回归模型
9 ridge_reg = Ridge(alpha=1.0) # alpha是正则化强度参数
10 ridge_reg.fit(X_train, y_train) # 使用训练数据拟合模型
11
12 # 绘制模型预测结果，以便直观比较两模型的差异
13 # 设置支持中文
14 plt.rcParams['font.sans-serif'] = ['SimHei']
15 # 设置支持负号
16 plt.rcParams['axes.unicode_minus'] = False
17 plt.figure(figsize=(12, 3))
18 plt.subplot(1, 2, 1)
19 plt.scatter(X_train, y_train, color='blue', label='训练数据') # 绘制训练数据点
20 plt.plot(X_train, linear_reg.predict(X_train), color='red', label='线性回归') # 线性回归预测曲线
21 plt.plot(X_train, ridge_reg.predict(X_train), color='green', label='岭回归') # 带正则化的回归预测曲线
22 plt.xlabel('特征 (如面积)') # x轴标签，中文
23 plt.ylabel('目标 (如价格)') # y轴标签，中文
24 plt.title('线性回归模型与岭回归模型的预测效果对比') # 图表标题，中文
25 plt.legend() # 显示图例，自动使用中文标签
26 plt.grid()
```

现在已经训练好两个模型，下一步是用测试集来评估它们的性能。这里计算均方误差（MSE），以量化模型预测值与真实值之间的差异。

```
1 from sklearn.metrics import mean_squared_error
2
3 # 使用测试集评估无正则化线性回归模型
4 linear_predictions = linear_reg.predict(X_test) # 预测测试集的目标值
```

```

5 linear_mse = mean_squared_error(y_test, linear_predictions) # 计算MSE
6
7 # 使用测试集评估带Ridge正则化的线性回归模型
8 ridge_predictions = ridge_reg.predict(X_test) # 预测测试集的目标值
9 ridge_mse = mean_squared_error(y_test, ridge_predictions) # 计算MSE
10
11 print(f'线性回归 MSE: {linear_mse}') # 打印线性回归的MSE, 中文标签
12 print(f'岭回归 MSE: {ridge_mse}') # 打印Ridge回归的MSE, 中文标签
13
14 # 绘制两个模型预测值之间的差异
15 plt.subplot(1, 2, 2)
16 plt.bar(range(len(X_test)), np.abs(linear_predictions - y_test)[: ,0],
17         bottom=1, color='blue', label='线性回归预测差异', alpha=0.5)
18 plt.bar(range(len(X_test)), np.abs(ridge_predictions - y_test)[: ,0],
19         bottom=1, color='green', label='岭回归预测差异', alpha=0.5)
20 plt.xlabel('样本索引') # x轴标签, 中文
21 plt.ylabel('预测差异') # y轴标签, 中文
22 plt.title('预测差异对比') # 图表标题, 中文
23 plt.legend() # 显示图例, 自动使用中文标签
24 plt.grid()

```

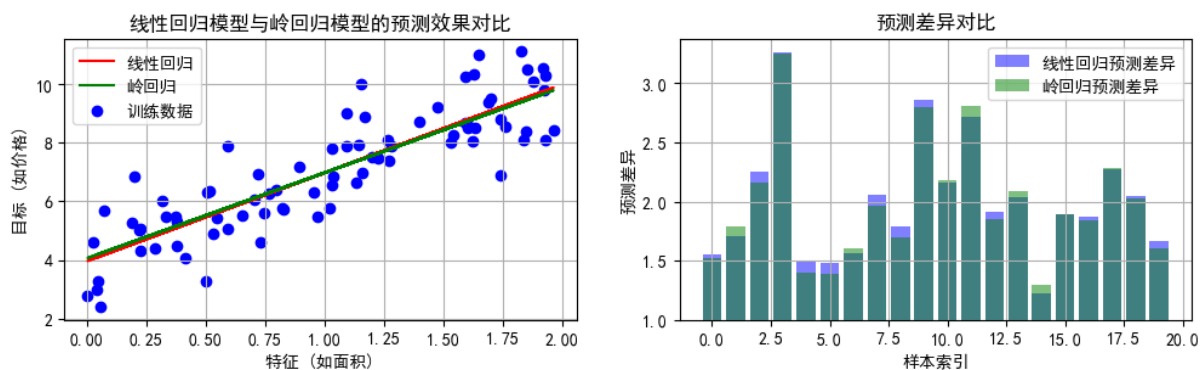


图 4.2: 线性回归模型与 Ridge 回归模型在训练数据上的预测效果对比

通过上述实践,可以直观地看到正则化对线性模型的影响。从图 4.2中可以看出,无正则化的线性回归模型可能会更加贴近训练数据,但这可能导致过拟合;而带 Ridge 正则化的模型则更平滑,它倾向于避免过度拟合训练数据中的随机噪音。因此,在面对新数据时,带正则化的模型通常会表现出更好的泛化能力。

4.7.2 示例: 房价预测

在本节中,将通过一个实际问题来展示如何使用线性模型解决现实中的问题。假设有一个房地产公司,想要预测房屋的售价。假设手头有一份包含多个特征的数据集,如房屋面积、卧室数量、浴室数量等。

在应用线性回归之前,需要对数据进行一些预处理,以确保模型的有效性和准确性。常见的预处理步骤包括:

1. 缺失值处理: 检查并处理数据中的缺失值。
2. 标准化/归一化: 将不同量纲的特征缩放到相同的范围内,例如标准化为均值为 0、方差为 1。
3. 特征选择: 选择对目标变量影响较大的特征,去除无关或冗余特征。

表 4.6: 房屋数据表

样本编号	房屋面积 (m ²)	卧室数量	浴室数量	售价 (\$)
1	120	3	2	500,000
2	90	2	1	300,000
3	150	4	3	700,000
4	80	2	1	250,000
5	180	5	4	900,000

接下来, 将使用 Python 中的 Scikit-learn 库来训练线性回归模型, 并评估其性能。首先, 需要导入必要的库, 代码如下:

```
1 import numpy as np
2 import pandas as pd
3 from sklearn.model_selection import train_test_split
4 from sklearn.linear_model import LinearRegression
5 from sklearn.metrics import mean_squared_error
6 from sklearn.preprocessing import StandardScaler
7 import matplotlib.pyplot as plt
```

其次, 加载和预处理数据:

```
1 # 加载数据
2 data = pd.read_csv('house_prices.csv')
3 # 检查并处理缺失值
4 data.dropna(inplace=True)
5 # 特征选择
6 cols = ['RM', 'LSTAT', 'PTRATIO', 'TAX', 'DIS', 'RAD'] # 假设这些特征
7 features = data[cols]
8 target = data['MEDV']
9 # 标准化特征
10 scaler = StandardScaler()
11 features_scaled = scaler.fit_transform(features)
```

其次, 需要将整个数据集划分训练集和测试集, 以备后面训练和测试使用:

```
1 X_train, X_test, y_train, y_test = train_test_split(
2     features_scaled, target, test_size=0.2, random_state=42
3 )
```

再次, 基于划分的训练数据训练线性回归模型:

```
1 model = LinearRegression()
2 model.fit(X_train, y_train)
```

最后, 使用训练出来的线性模型在测试数据集上进行评估:

```
1 # 预测
2 y_pred = model.predict(X_test)
3
4 # 计算均方误差
5 mse = mean_squared_error(y_test, y_pred)
6 print(f'Mean Squared Error: {mse}')
7
8 # 设置支持中文
9 plt.rcParams['font.sans-serif'] = ['SimHei']
10 # 设置支持负号
11 plt.rcParams['axes.unicode_minus'] = False
12 # 可视化预测结果
13 plt.figure(figsize=(6, 3))
14 x_range = np.linspace(y_test.min(), y_test.max(), len(y_test))
15 plt.scatter(x_range, y_pred)
```

```

16 plt.scatter(x_range, y_test, color='red', linewidth=1)
17 plt.xlabel('真实售价')
18 plt.ylabel('预测售价')
19 plt.title('真实售价 vs 预测售价')
20 plt.tight_layout()
21 plt.grid()
22 plt.show()

```

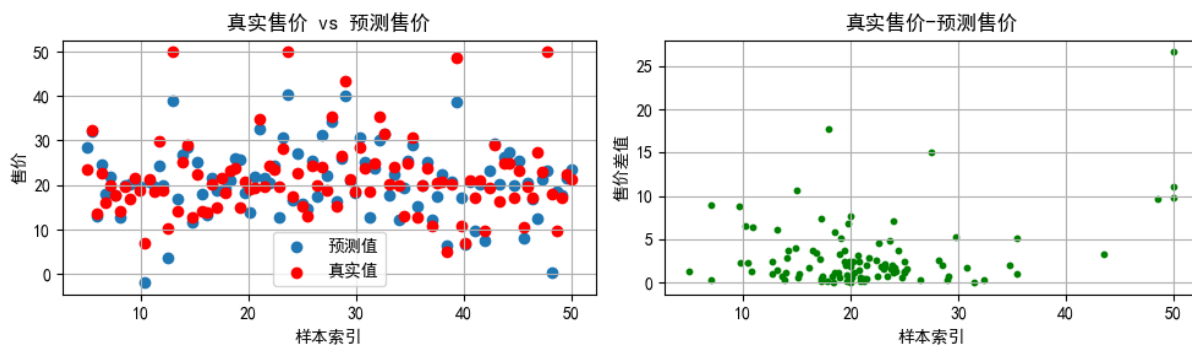


图 4.3: 真实售价 vs 预测售价

通过上述步骤，成功地使用线性回归模型预测了房屋售价。从结果可以看出，模型在大多数情况下都能给出较为准确的预测。然而，由于线性回归模型假设特征与目标变量之间存在线性关系，因此它可能无法捕捉到更复杂的数据模式。对于更复杂的任务，可以考虑使用其他更高级的机器学习模型，如决策树、随机森林或神经网络。

综上所述，线性回归作为一种简单而有效的工具，在许多实际问题中能够提供可靠且易于理解的解决方案。理解其原理和应用场景有助于更好地掌握和应用这一重要方法。

4.8 补充知识

为帮助读者更好地理解多元线性回归模型及其背后的数学原理，本节将详细解释在线性模型中使用到的关键数学概念和方法。

4.8.1 矩阵逆、行列式和伴随矩阵

矩阵逆、行列式和伴随矩阵是线性代数中的重要概念，在求解线性方程组、优化问题以及多元线性回归中广泛应用。这些概念有助于理解和操作矩阵，特别是在处理逆矩阵时尤为重要。

1. 矩阵逆

对于一个 $n \times n$ 的方阵 $\mathbf{A}$ ，如果存在一个 $n \times n$ 的矩阵 $\mathbf{B}$ 使得：

$$\mathbf{AB} = \mathbf{BA} = \mathbf{I} \quad (4.66)$$

其中 $\mathbf{I}$ 是单位矩阵，则称 $\mathbf{B}$ 为 $\mathbf{A}$ 的逆矩阵，记作 $\mathbf{A}^{-1}$ 。逆矩阵的存在要求原矩阵是非奇异的（即行列式不为零）。具体来说：

- 1) 如果 $\mathbf{A}$ 是一个非奇异矩阵，则其逆矩阵 $\mathbf{A}^{-1}$ 唯一存在。
- 2) 逆矩阵的性质包括：

- $(\mathbf{A}^{-1})^{-1} = \mathbf{A}$
- $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$

- $(\mathbf{A}^T)^{-1} = (\mathbf{A}^{-1})^T$

3) 计算逆矩阵的方法包括高斯消元法、LU 分解等数值方法，但在理论上，逆矩阵可以通过伴随矩阵和行列式计算得到：

$$\mathbf{A}^{-1} = \frac{1}{\det(\mathbf{A})} \text{adj}(\mathbf{A}) \quad (4.67)$$

考虑一个 2×2 矩阵： $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 4 & 3 \end{pmatrix}$ ，为找到 $\mathbf{A}$ 的逆矩阵，首先计算行列式 $\det(\mathbf{A})$ ：

$$\det(\mathbf{A}) = (2 \cdot 3) - (1 \cdot 4) = 6 - 4 = 2 \quad (4.68)$$

然后计算伴随矩阵 $\text{adj}(\mathbf{A})$ ：

$$\text{adj}(\mathbf{A}) = \begin{pmatrix} 3 & -1 \\ -4 & 2 \end{pmatrix} \quad (4.69)$$

因此，逆矩阵为：

$$\mathbf{A}^{-1} = \frac{1}{2} \begin{pmatrix} 3 & -1 \\ -4 & 2 \end{pmatrix} = \begin{pmatrix} 1.5 & -0.5 \\ -2 & 1 \end{pmatrix} \quad (4.70)$$

2. 行列式

行列式是一个标量值，表示矩阵的一些特性。对于一个 $n \times n$ 的方阵 $\mathbf{A}$ ，其行列式 $\det(\mathbf{A})$ 可以通过多种方式计算，例如通过余子式展开或高斯消元法。行列式为零意味着矩阵是奇异的，即不可逆。具体来说：

1) 行列式的定义：对于一个 2×2 矩阵 $\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ ，其行列式为：

$$\det(\mathbf{A}) = ad - bc \quad (4.71)$$

对于更高阶的矩阵，可以使用递归定义或展开公式进行计算。

2) 行列式的性质包括：

- $\det(\mathbf{I}) = 1$
- $\det(\mathbf{A}^T) = \det(\mathbf{A})$
- $\det(\mathbf{AB}) = \det(\mathbf{A}) \det(\mathbf{B})$
- 如果 $\mathbf{A}$ 有一个零行或零列，则 $\det(\mathbf{A}) = 0$

3) 行列式在判断矩阵是否可逆方面非常重要。如果 $\det(\mathbf{A}) \neq 0$ ，则 $\mathbf{A}$ 是非奇异矩阵，存在逆矩阵；如果 $\det(\mathbf{A}) = 0$ ，则 $\mathbf{A}$ 是奇异矩阵，不存在逆矩阵。

考虑一个 3×3 矩阵： $\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$ 可以使用行列式展开公式计算其行列式：

$$\det(\mathbf{A}) = 1 \cdot \det \begin{pmatrix} 5 & 6 \\ 8 & 9 \end{pmatrix} - 2 \cdot \det \begin{pmatrix} 4 & 6 \\ 7 & 9 \end{pmatrix} + 3 \cdot \det \begin{pmatrix} 4 & 5 \\ 7 & 8 \end{pmatrix} \quad (4.72)$$

分别计算每个 2×2 矩阵的行列式：

$$\begin{aligned}
\det \begin{pmatrix} 5 & 6 \\ 8 & 9 \end{pmatrix} &= 5 \cdot 9 - 6 \cdot 8 = 45 - 48 = -3 \\
\det \begin{pmatrix} 4 & 6 \\ 7 & 9 \end{pmatrix} &= 4 \cdot 9 - 6 \cdot 7 = 36 - 42 = -6 \\
\det \begin{pmatrix} 4 & 5 \\ 7 & 8 \end{pmatrix} &= 4 \cdot 8 - 5 \cdot 7 = 32 - 35 = -3
\end{aligned} \tag{4.73}$$

因此:

$$\det(\mathbf{A}) = 1 \cdot (-3) - 2 \cdot (-6) + 3 \cdot (-3) = -3 + 12 - 9 = 0 \tag{4.74}$$

由于行列式为零, 矩阵 $\mathbf{A}$ 是奇异矩阵, 没有逆矩阵。

3. 伴随矩阵

伴随矩阵是由矩阵的余子式组成的矩阵, 用于计算逆矩阵。对于一个 $n \times n$ 的方阵 $\mathbf{A}$, 其伴随矩阵 $\text{adj}(\mathbf{A})$ 定义为:

$$\text{adj}(\mathbf{A}) = [C_{ji}]^T \tag{4.75}$$

其中 C_{ij} 是矩阵 $\mathbf{A}$ 中元素 a_{ij} 的代数余子式。具体来说:

1) 余子式: 对于一个 $n \times n$ 的方阵 $\mathbf{A}$, 元素 a_{ij} 的余子式 M_{ij} 是去掉第 i 行和第 j 列后形成的 $(n-1) \times (n-1)$ 矩阵的行列式。

2) 代数余子式: 代数余子式 C_{ij} 定义为:

$$C_{ij} = (-1)^{i+j} M_{ij} \tag{4.76}$$

3) 伴随矩阵的性质包括:

- $\mathbf{A} \text{adj}(\mathbf{A}) = \text{adj}(\mathbf{A}) \mathbf{A} = \det(\mathbf{A}) \mathbf{I}$
- 如果 $\mathbf{A}$ 是非奇异矩阵, 则 $\mathbf{A}^{-1} = \frac{1}{\det(\mathbf{A})} \text{adj}(\mathbf{A})$

4) 伴随矩阵在计算逆矩阵时起到重要作用。对于一个非奇异矩阵 $\mathbf{A}$, 其逆矩阵可以通过以下公式计算:

$$\mathbf{A}^{-1} = \frac{1}{\det(\mathbf{A})} \text{adj}(\mathbf{A}) \tag{4.77}$$

考虑一个 2×2 矩阵: $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 4 & 3 \end{pmatrix}$, 为计算伴随矩阵 $\text{adj}(\mathbf{A})$, 首先计算每个元素的代数余子式:

$$\begin{aligned}
C_{11} &= (-1)^{1+1} \det(3) = 3, \\
C_{12} &= (-1)^{1+2} \det(4) = -4, \\
C_{21} &= (-1)^{2+1} \det(1) = -1, \\
C_{22} &= (-1)^{2+2} \det(2) = 2.
\end{aligned} \tag{4.78}$$

因此, 伴随矩阵为: $\text{adj}(\mathbf{A}) = \begin{pmatrix} 3 & -1 \\ -4 & 2 \end{pmatrix}$

4.8.2 最小二乘法原理及其应用

最小二乘法 (Least Squares Method, LSM) 是一种常用的统计方法, 用于估计线性回归模型中的参数。其基本思想是最小化观测值与预测值之间差异的平方和, 即残差平方和 (RSS)。通过求导并令其等于零, 可以得到正规方程。最小二乘法不仅可以提供直观的几何解释, 还能够保证在某些假设条件下 (如误差项服从正态分布) 估计的最优性。

最小二乘法的核心是找到一组参数, 使得模型的预测值与实际观测值之间的偏差尽可能小。具体来说, 对于一个给定的数据集 $(\mathbf{x}_i, y_i)$, 其中 $\mathbf{x}_i$ 是特征向量, y_i 是目标变量, 希望找到一个线性模型:

$$\hat{y}_i = \mathbf{w}^T \mathbf{x}_i \quad (4.79)$$

使得预测值 $\hat{y}_i$ 尽可能接近真实值 y_i 。为量化这种差距, 定义残差平方和 (RSS) 为:

$$RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2 = \sum_{i=1}^n (y_i - (\mathbf{w}^T \mathbf{x}_i))^2 \quad (4.80)$$

最小二乘法的目标就是找到参数向量 $\mathbf{w}$, 使得 RSS 最小化。从几何上讲, 这相当于寻找一个超平面, 使得所有数据点到该超平面的距离平方和最小。

1. 推导过程

为找到使 RSS 最小化的参数向量 $\mathbf{w}$, 对 $\mathbf{w}$ 求偏导数并令其等于零。具体步骤如下:

1) 定义目标函数:

$$RSS(\mathbf{w}) = \sum_{i=1}^n (y_i - (\mathbf{w}^T \mathbf{x}_i))^2 \quad (4.81)$$

2) 对 $\mathbf{w}$ 求偏导数:

$$\begin{aligned} \frac{\partial RSS}{\partial \mathbf{w}} &= \frac{\partial}{\partial \mathbf{w}} \left(\sum_{i=1}^n (y_i - (\mathbf{w}^T \mathbf{x}_i))^2 \right) \\ &= -2 \sum_{i=1}^n (y_i - (\mathbf{w}^T \mathbf{x}_i)) \mathbf{x}_i \\ &= -2 \mathbf{X}^T (\mathbf{Y} - \mathbf{X} \mathbf{w}) \end{aligned} \quad (4.82)$$

其中 $\mathbf{X}$ 是设计矩阵, 每一行是一个样本的特征向量; $\mathbf{Y}$ 是目标变量的列向量。

3) 令导数等于零:

$$\frac{\partial RSS}{\partial \mathbf{w}} = 0 \implies \mathbf{X}^T (\mathbf{Y} - \mathbf{X} \mathbf{w}) = 0 \quad (4.83)$$

4) 简化方程:

$$\mathbf{X}^T \mathbf{Y} = \mathbf{X}^T \mathbf{X} \mathbf{w} \quad (4.84)$$

5) 得到正规方程:

$$\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y} \quad (4.85)$$

6) 解正规方程:

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (4.86)$$

通过上述推导，得到了用于估计参数向量 $\mathbf{w}$ 的公式。这个公式表明，最小二乘法通过矩阵运算提供了一种简洁而有效的方法来估计线性回归模型的参数。

2. 具体应用

最小二乘法在多元线性回归中有广泛的应用。具体来说：

1) 参数估计：通过求解正规方程，可以获得最优的参数向量 $\mathbf{w}$ ，从而构建出拟合数据的最佳线性模型。

2) 模型评估：通过计算残差平方和 (RSS)，可以评估模型的拟合优度。RSS 越小，表示模型拟合数据的效果越好。

3) 预测：使用估计的参数向量 $\mathbf{w}$ ，可以对新的输入数据进行预测。对于一个新的特征向量 $\mathbf{x}_{\text{new}}$ ，预测值 $\hat{y}_{\text{new}}$ 可以通过以下公式计算：

$$\hat{y}_{\text{new}} = \mathbf{w}^T \mathbf{x}_{\text{new}} \quad (4.87)$$

4) 假设检验：在最小二乘法框架下，可以进行各种假设检验，例如检验某个特征是否显著影响目标变量。常用的检验方法包括 F 检验和 t 检验。

综上所述，最小二乘法不仅是多元线性回归的基础工具，而且在许多其他领域也有广泛应用。通过掌握最小二乘法的原理和应用，读者可以更好地理解如何构建和评估线性回归模型，从而做出更加准确的预测和决策。

4.8.3 矩阵求导及其应用

矩阵求导是处理多变量函数的一种有效工具，尤其在优化问题、机器学习和统计建模中广泛应用。常见的矩阵求导规则包括迹的导数、二次型的导数、矩阵乘积的导数以及标量函数对矩阵的导数等。这些规则有助于推导出复杂的优化问题中的梯度和 Hessian 矩阵，从而找到最优解。

1. 迹的导数

对于一个 $n \times n$ 的方阵 $\mathbf{A}$ ，其迹 $\text{tr}(\mathbf{A})$ 定义为对角线元素之和。迹的导数可以表示为：

$$\frac{\partial \text{tr}(\mathbf{A})}{\partial \mathbf{A}} = \mathbf{I} \quad (4.88)$$

考虑一个 2×2 矩阵： $\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}$ ，则其迹为：

$$\text{tr}(\mathbf{A}) = a_{11} + a_{22} \quad (4.89)$$

因此，迹关于 $\mathbf{A}$ 的导数为单位矩阵 $\mathbf{I}$ ：

$$\frac{\partial \text{tr}(\mathbf{A})}{\partial \mathbf{A}} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad (4.90)$$

2. 二次型的导数

对于一个向量 $\mathbf{x} \in \mathbb{R}^n$ 和一个对称矩阵 $\mathbf{A} \in \mathbb{R}^{n \times n}$ ，二次型 $\mathbf{x}^T \mathbf{A} \mathbf{x}$ 的导数可以表示为：

$$\frac{\partial \mathbf{x}^T \mathbf{A} \mathbf{x}}{\partial \mathbf{x}} = 2\mathbf{A}\mathbf{x} \quad (4.91)$$

考虑一个 2×2 对称矩阵 $\mathbf{A}$ 和一个 2×1 向量 $\mathbf{x}$ ：

$$\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \quad (4.92)$$

二次型为:

$$\mathbf{x}^T \mathbf{A} \mathbf{x} = 2x_1^2 + 2x_1x_2 + 3x_2^2 \quad (4.93)$$

其导数为:

$$\frac{\partial \mathbf{x}^T \mathbf{A} \mathbf{x}}{\partial \mathbf{x}} = \begin{pmatrix} 4x_1 + 2x_2 \\ 2x_1 + 6x_2 \end{pmatrix} = 2 \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = 2\mathbf{A}\mathbf{x} \quad (4.94)$$

3. 矩阵乘积的导数

对于两个矩阵 $\mathbf{A} \in \mathbb{R}^{m \times n}$ 和 $\mathbf{B} \in \mathbb{R}^{n \times p}$, 以及一个向量 $\mathbf{x} \in \mathbb{R}^n$, 矩阵乘积 $\mathbf{A}\mathbf{x}\mathbf{B}^T$ 的导数可以表示为:

$$\frac{\partial(\mathbf{A}\mathbf{x}\mathbf{B}^T)}{\partial \mathbf{x}} = \mathbf{A}^T \mathbf{B} \quad (4.95)$$

考虑一个 2×2 矩阵 $\mathbf{A}$, 一个 2×1 向量 $\mathbf{x}$, 和一个 2×2 矩阵 $\mathbf{B}$:

$$\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} \quad (4.96)$$

矩阵乘积为:

$$\mathbf{A}\mathbf{x}\mathbf{B}^T = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} \begin{pmatrix} 5 & 7 \\ 6 & 8 \end{pmatrix} \quad (4.97)$$

其导数为:

$$\frac{\partial(\mathbf{A}\mathbf{x}\mathbf{B}^T)}{\partial \mathbf{x}} = \mathbf{A}^T \mathbf{B} = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix} \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix} = \begin{pmatrix} 22 & 26 \\ 34 & 40 \end{pmatrix} \quad (4.98)$$

4. 标量函数对矩阵的导数

对于一个标量函数 $f(\mathbf{A})$, 其对矩阵 $\mathbf{A}$ 的导数可以表示为:

$$\frac{\partial f(\mathbf{A})}{\partial \mathbf{A}} = \left[\frac{\partial f}{\partial a_{ij}} \right] \quad (4.99)$$

考虑一个标量函数 $f(\mathbf{A}) = \text{tr}(\mathbf{A}^2)$, 其中 $\mathbf{A}$ 是一个 2×2 矩阵:

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} \quad (4.100)$$

则:

$$\begin{aligned} f(\mathbf{A}) &= \text{tr}(\mathbf{A}^2) = \text{tr} \left(\begin{pmatrix} a_{11}^2 + a_{12}a_{21} & a_{11}a_{12} + a_{12}a_{22} \\ a_{21}a_{11} + a_{22}a_{21} & a_{12}a_{21} + a_{22}^2 \end{pmatrix} \right) \\ &= a_{11}^2 + a_{12}a_{21} + a_{21}a_{12} + a_{22}^2 \end{aligned} \quad (4.101)$$

其导数为:

$$\frac{\partial f(\mathbf{A})}{\partial \mathbf{A}} = \begin{pmatrix} 2a_{11} & 2a_{12} \\ 2a_{21} & 2a_{22} \end{pmatrix} = 2\mathbf{A} \quad (4.102)$$

5. 矩阵求导的应用

矩阵求导在多元线性回归、最小二乘法和其他优化问题中有广泛的应用。具体来说：

1) 参数估计：在最小二乘法中，通过矩阵求导可以推导出用于估计参数向量 $\mathbf{w}$ 的正规方程。

对于目标函数：

$$RSS(\mathbf{w}) = \sum_{i=1}^n (y_i - (\mathbf{w}^T \mathbf{x}_i))^2 \quad (4.103)$$

可以通过求偏导数并令其等于零来得到正规方程：

$$\mathbf{X}^T \mathbf{X} \mathbf{w} = \mathbf{X}^T \mathbf{y} \quad (4.104)$$

进而求得参数向量 $\mathbf{w}$ ：

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} \quad (4.105)$$

2) 梯度下降算法：在梯度下降算法中，需要计算损失函数关于参数的梯度。例如，在线性回归中，损失函数 $L(\mathbf{w})$ 可以表示为：

$$L(\mathbf{w}) = \frac{1}{2} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|^2 \quad (4.106)$$

其梯度为：

$$\nabla L(\mathbf{w}) = -\mathbf{X}^T (\mathbf{y} - \mathbf{X}\mathbf{w}) \quad (4.107)$$

梯度下降算法通过迭代更新参数 $\mathbf{w}$ 来最小化损失函数。

4.8.3 残差及其应用

残差是指观测值与模型预测值之间的差异，用于评估模型的拟合优度。通过分析残差图，可以判断模型是否合适，是否存在系统性的偏差，以及是否需要改进模型结构或引入额外的特征。残差分析在统计建模和机器学习中具有重要意义，有助于理解数据的分布特性并提高模型的准确性。

对于一个给定的数据集 $(\mathbf{x}_i, y_i)$ ，其中 $\mathbf{x}_i$ 是特征向量， y_i 是目标变量，线性回归模型的预测值为：

$$\hat{y}_i = \mathbf{w}^T \mathbf{x}_i \quad (4.108)$$

残差定义为观测值与预测值之间的差异：

$$\epsilon_i = y_i - \hat{y}_i \quad (4.109)$$

考虑一个简单的线性回归问题，假设有以下数据点：

$$(\mathbf{x}_1, y_1) = (1, 2), (\mathbf{x}_2, y_2) = (2, 3), (\mathbf{x}_3, y_3) = (3, 5) \quad (4.110)$$

假设得到的线性回归模型为：

$$\hat{y} = 0.8x + 1.4 \quad (4.111)$$

计算各点的残差：

$$\begin{aligned}\epsilon_1 &= 2 - (0.8 \cdot 1 + 1.4) = 2 - 2.2 = -0.2, \\ \epsilon_2 &= 3 - (0.8 \cdot 2 + 1.4) = 3 - 3.0 = 0, \\ \epsilon_3 &= 5 - (0.8 \cdot 3 + 1.4) = 5 - 3.8 = 1.2.\end{aligned}\tag{4.112}$$

1. 残差平方和 (RSS)

残差平方和 (Residual Sum of Squares, RSS) 是衡量模型拟合效果的一个重要指标，定义为所有残差的平方和：

$$RSS = \sum_{i=1}^n \epsilon_i^2 = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \tag{4.113}$$

继续上面的例子，计算残差平方和：

$$RSS = (-0.2)^2 + 0^2 + 1.2^2 = 0.04 + 0 + 1.44 = 1.48 \tag{4.114}$$

较小的 RSS 表明模型拟合较好，而较大的 RSS 则表示模型可能需要改进。

2. 残差图

残差图是将残差绘制为观测值或预测值的函数，用于直观地评估模型的拟合效果。理想的残差图应该显示随机散布的点，表明模型能够很好地捕捉数据的趋势。如果残差图显示出明显的模式（如曲线），则可能意味着模型存在系统性的偏差，需要改进模型结构或引入额外的特征。

考虑一个残差图，横轴为预测值 $\hat{y}$ ，纵轴为残差 ϵ 。如果残差图呈现出随机散布的点，则表明模型拟合良好；如果残差图显示出某种趋势（如逐渐增加或减少），则表明模型可能存在系统性误差。

3. 标准化残差

标准化残差是对原始残差进行标准化处理的结果，通常用于检测异常值或离群点。标准化残差的计算公式为：

$$\text{Standardized Residual}_i = \frac{\epsilon_i}{\sqrt{\text{MSE}}} \tag{4.115}$$

其中 MSE 是均方误差 (Mean Squared Error)，定义为：

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n \epsilon_i^2 \tag{4.116}$$

继续上面的例子，首先计算均方误差：

$$\text{MSE} = \frac{1}{3} \times 1.48 \approx 0.493 \tag{4.117}$$

然后计算标准化残差：

$$\begin{aligned}\text{Standardized Residual}_1 &= \frac{-0.2}{\sqrt{0.493}} \approx -0.285, \\ \text{Standardized Residual}_2 &= \frac{0}{\sqrt{0.493}} = 0, \\ \text{Standardized Residual}_3 &= \frac{1.2}{\sqrt{0.493}} \approx 1.716.\end{aligned}\tag{4.118}$$

4. 残差的应用

残差分析在统计建模和机器学习中有广泛的应用，具体包括：

1) 模型评估：通过计算残差平方和（RSS）和其他相关指标，可以评估模型的拟合优度。RSS 越小，表示模型拟合数据的效果越好。

2) 异常值检测：标准化残差可以帮助识别数据中的异常值或离群点，这些点可能对模型的拟合效果产生显著影响。

3) 模型诊断：通过分析残差图，可以判断模型是否合适，是否存在系统性的偏差，以及是否需要改进模型结构或引入额外的特征。

4) 预测区间：残差分析还可以用于构建预测区间的估计，从而提供更加可靠的预测结果。

4.8.4 梯度

梯度（Gradient）是一个向量，它描述了函数在某一点处变化最快的方向和速率，其是多变量函数在某一点的导数的推广。对于单变量函数，导数表示函数在某一点的切线斜率，即函数在该点的变化率。对于多变量函数，梯度则是一个向量，它描述了函数在某一点沿各个自变量方向的变化率。具体来说，对于一个多元函数 $f(\mathbf{x})$ ，其梯度 $\nabla f(\mathbf{x})$ 是一个向量，其中每个元素是函数对相应变量的一阶偏导数。例如，对于二元函数 $f(x, y)$ ，梯度为：

$$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]^T \quad (4.119)$$

梯度的模（即梯度向量的长度）表示该点处的最大变化率，而梯度的方向则指向函数值增加最快的方向。因此，如果希望最小化一个损失函数 $L(\mathbf{w})$ ，可以通过沿着负梯度方向更新参数来实现。

为更直观地理解梯度，可以将其与物理世界中的地形图进行类比。假设你站在一座山上的某个位置，想要尽快到达山脚。此时，你可以观察周围的地形，找到最陡峭的下坡方向，并沿着这个方向前进。在这个过程中，你实际上是在沿着梯度的负方向移动，因为梯度指向的是山上最陡的方向，而你想去的是山脚。

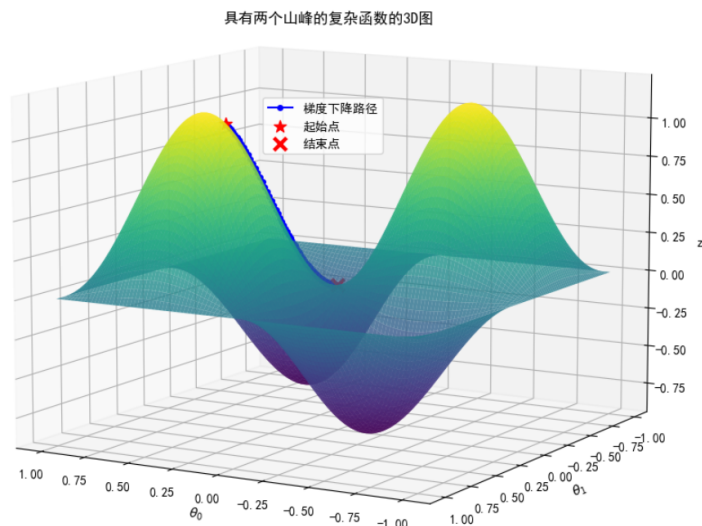


图 4.4: 地形图类比：梯度指向最陡峭的上坡方向

图 4.4 给出一个地形图的例子。假设你在图中标记的位置，你的目标是最快速地下降到山谷底部。根据地形图上的等高线，你可以确定当前点最陡峭的下坡方向，这相当于计算该点处的梯度并取其反方向。通过不断重复这一过程，你可以逐步接近最低点。

1. 图形化解释

除了物理类比，还可以通过图形化的方式进一步解释梯度。考虑一个简单的二元函数 $f(x, y) =$

$x^2 + y^2$ ，这是一个圆锥形的表面，顶点位于原点 $(0, 0)$ 。在任意点 (x_0, y_0) 处，梯度向量指向远离原点的方向，并且其长度与到原点的距离成正比。

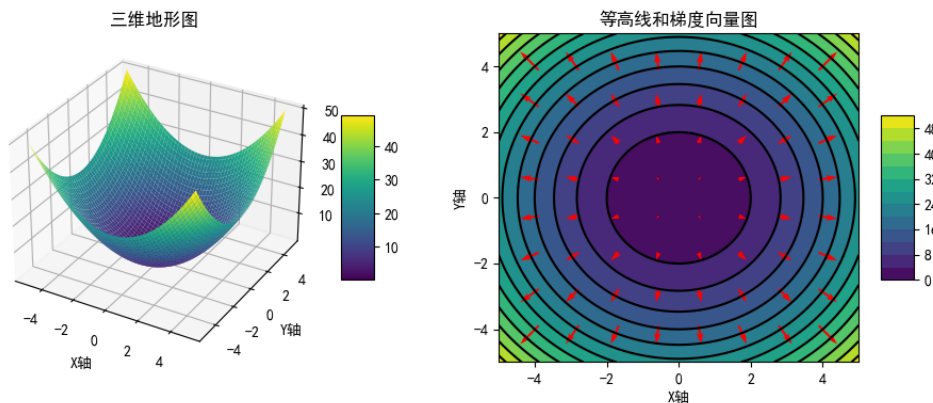


图 4.5: 函数 $f(x, y) = x^2 + y^2$ 的地形图和等高线图及梯度向量

图 4.5 展示了该函数的地形图和等高线图及其梯度向量。等高线图是通过在地形图上标识出海拔高度相同的点并将其连接成线来制作的，这些线条称为等高线，它们直观描绘地形的起伏，其中线条越密集表示坡度越陡，线条越稀疏则表示坡度越缓。通过这种方法，可以在二维平面上有效地表达三维地形的特征。可以看到，梯度向量总是垂直于等高线，并指向函数值增加最快的方向。如果你希望最小化这个函数，你应该沿着梯度的负方向移动，这样可以最快地降低函数值。

2. 梯度下降法的应用

在机器学习中，梯度下降法是一种常用的优化算法，用于最小化损失函数。以逻辑回归为例，假设有一个损失函数 $L(\mathbf{w})$ ，需要找到使得 $L(\mathbf{w})$ 最小化的参数 $\mathbf{w}$ 。为此，从初始参数 $\mathbf{w}_0$ 开始，按照以下规则迭代更新参数：

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} L(\mathbf{w}_t) \quad (4.120)$$

其中 η 是学习率，控制每次更新的步长。通过不断调整参数 $\mathbf{w}$ ，可以逐渐逼近最优解，使损失函数 $L(\mathbf{w})$ 达到最小值。

综上所述，梯度不仅是一个数学概念，它还具有直观的物理意义。通过物理类比和图形化解释，可以更好地理解梯度的作用和应用。在优化问题中，梯度为寻找最优解提供了方向和速度，是许多优化算法的基础。

4.8.5 牛顿法的基础数学知识

1. 泰勒展开

泰勒展开 (Taylor Expansion) 是一种用多项式来近似函数的方法，它在微积分中占有重要地位。对于一个可导函数 $f(x)$ ，在其某一点 x_0 附近的值可以用以下形式近似：

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0) + \frac{1}{2}f''(x_0)(x - x_0)^2 + \dots \quad (4.121)$$

其中 $f'(x_0)$ 和 $f''(x_0)$ 分别表示 $f(x)$ 在 x_0 处的一阶和二阶导数。当只考虑一阶和二阶项时，得到的是二次近似，这对于优化问题特别有用，因为它可以捕捉到局部曲率信息。

2. Hessian 矩阵

梯度 (Gradient) 是函数在某一点处变化最快的方向。对于一个多元函数 $f(\mathbf{x})$ ，其梯度是一个向量，表示为 $\nabla f(\mathbf{x})$ ，其中每个元素是函数对相应变量的一阶偏导数。Hessian 矩阵 (Hessian Matrix) 则包含函数的二阶偏导数，它是一个方阵，描述函数在某一点处的曲率。对于二元函数 $f(x, y)$ ，Hessian

矩阵为:

$$\mathbf{H}(f) = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix} \quad (4.122)$$

对于高维函数 $f(\mathbf{x})$, Hessian 矩阵是一个 $n \times n$ 的矩阵, 其中 n 是输入变量的数量。Hessian 矩阵的对角线元素表示各变量的二阶导数, 非对角线元素表示不同变量之间的混合偏导数。

3. 牛顿法的原理

牛顿法利用泰勒展开和 Hessian 矩阵来快速找到目标函数的极小值点。假设有一个凸优化问题, 目标是找到使损失函数 $L(\mathbf{w})$ 最小化的参数 $\mathbf{w}$ 。牛顿法的基本思想是通过泰勒展开来近似损失函数, 并找到使得近似函数最小化的参数更新规则。

对于逻辑回归的交叉熵损失函数 $L(\mathbf{w})$, 可以写出其二阶泰勒展开:

$$L(\mathbf{w} + \Delta \mathbf{w}) \approx L(\mathbf{w}) + \nabla_{\mathbf{w}} L(\mathbf{w})^T \Delta \mathbf{w} + \frac{1}{2} \Delta \mathbf{w}^T \mathbf{H}(\mathbf{w}) \Delta \mathbf{w} \quad (4.123)$$

为使 $L(\mathbf{w} + \Delta \mathbf{w})$ 最小化, 令一阶导数等于零:

$$\nabla_{\mathbf{w}} L(\mathbf{w}) + \mathbf{H}(\mathbf{w}) \Delta \mathbf{w} = 0 \quad (4.124)$$

解上述方程, 得到参数更新规则:

$$\Delta \mathbf{w} = -\mathbf{H}(\mathbf{w})^{-1} \nabla_{\mathbf{w}} L(\mathbf{w}) \quad (4.125)$$

因此, 牛顿法的参数更新公式为:

$$\mathbf{w} := \mathbf{w} - \mathbf{H}(\mathbf{w})^{-1} \nabla_{\mathbf{w}} L(\mathbf{w}) \quad (4.126)$$

为更好地理解牛顿法的实际应用, 以一个简单的二元分类问题为例, 使用牛顿法求解逻辑回归模型。

假设有以下数据集, 记录了几个学生的平时成绩和期末考试结果:

表 4.7: 学生平时成绩与期末考试结果

平时成绩 (x_i)	是否通过 (y_i)
60	0
70	1
80	1
90	1

这里的目标是训练一个逻辑回归模型, 预测学生通过期末考试的概率。根据之前的推导, 可以知道逻辑回归的损失函数 $L(\mathbf{w})$ 及其梯度和 Hessian 矩阵分别为:

$$\nabla_{\mathbf{w}} L(\mathbf{w}) = - \sum_{i=1}^n (y_i - P(y_i = 1 | \mathbf{x}_i; \mathbf{w})) \mathbf{x}_i \quad (4.127)$$

$$\mathbf{H}(\mathbf{w}) = \sum_{i=1}^n P(y_i = 1 | \mathbf{x}_i; \mathbf{w}) (1 - P(y_i = 1 | \mathbf{x}_i; \mathbf{w})) \mathbf{x}_i \mathbf{x}_i^T \quad (4.128)$$

假设初始设置参数为: $w_0 = 0, w_1 = 0.1$ 。

按照牛顿法的参数更新公式 (4.126) 进行迭代, 直到损失函数收敛或达到预定的最大迭代次数。每次迭代中, 计算当前参数下的梯度和 Hessian 矩阵, 然后根据公式更新参数。

4. 步骤说明

- 1) 初始化参数：设定初始参数 $\mathbf{w}_0 = [0, 0.1]^T$ 。
- 2) 计算梯度和 *Hessian* 矩阵：根据公式 (4.127) 和 (4.128) 计算当前参数下的梯度和 *Hessian* 矩阵。
- 3) 更新参数：根据公式 (4.126) 更新参数 $\mathbf{w}$ 。
- 4) 检查收敛条件：如果损失函数的变化小于某个阈值或达到最大迭代次数，则停止迭代；否则，返回步骤 2 继续迭代。

通过这个过程，可以有效地找到最优参数，从而提高模型的预测性能。牛顿法的优势在于它能够利用二阶导数信息，通常比梯度下降法更快地收敛到最优解，尤其是在接近最优点时。

4.8.6 协方差矩阵

协方差矩阵是统计学和机器学习中用于描述一组随机变量之间线性关系的重要工具。它不仅能够反映每个变量自身的波动情况（即方差），还能揭示不同变量之间的相关性。

对于一个 d 维随机向量 $\mathbf{x} = [x_1, x_2, \dots, x_d]^T$ ，其协方差矩阵 $\mathbf{S}$ 定义为：

$$\mathbf{S} = E[(\mathbf{x} - \mu)(\mathbf{x} - \mu)^T] \quad (4.129)$$

其中 $\mu = E[\mathbf{x}]$ 是均值向量， $E[\cdot]$ 表示期望值。矩阵 $\mathbf{S}$ 的第 (i, j) 个元素表示第 i 个特征和第 j 个特征之间的协方差：

$$S_{ij} = \text{Cov}(x_i, x_j) = E[(x_i - \mu_i)(x_j - \mu_j)] \quad (4.130)$$

当 $i = j$ 时， S_{ii} 就是特征 x_i 的方差，表示该特征自身的波动程度；当 $i \neq j$ 时， S_{ij} 则反映了两个特征之间的线性关系。

1. 计算方法

假设有一个数据集，包含 n 个样本，每个样本有 d 个特征，记作 $\mathbf{x}_i = [x_{i1}, x_{i2}, \dots, x_{id}]^T$ 。这里可以按照以下步骤计算协方差矩阵：

- 1) 计算均值向量：首先计算每个特征的平均值，得到均值向量 μ ：

$$\mu = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \quad (4.131)$$

- 2) 中心化数据：对每个样本减去均值向量，得到中心化的数据矩阵 $\mathbf{X}_{\text{centered}}$ ：

$$\mathbf{X}_{\text{centered}} = [\mathbf{x}_1 - \mu, \mathbf{x}_2 - \mu, \dots, \mathbf{x}_n - \mu] \quad (4.132)$$

- 3) 计算协方差矩阵：使用中心化的数据矩阵计算协方差矩阵 $\mathbf{S}$ ：

$$\mathbf{S} = \frac{1}{n-1} \mathbf{X}_{\text{centered}} \mathbf{X}_{\text{centered}}^T \quad (4.133)$$

2. 具体示例

为更直观地理解协方差矩阵的计算过程，考虑一个简单的例子。假设有以下数据集，包含三个样本，每个样本有两个特征（数学成绩和英语成绩）：

- 1) 计算均值向量：

$$\mu = \left(\frac{85 + 70 + 92}{3}, \frac{90 + 80 + 88}{3} \right) = (82.33, 86) \quad (4.134)$$

表 4.8: 示例数据表

样本编号	数学成绩	英语成绩
1	85	90
2	70	80
3	92	88

2) 中心化数据:

$$\mathbf{X}_{\text{centered}} = \begin{bmatrix} 85 - 82.33 & 90 - 86 \\ 70 - 82.33 & 80 - 86 \\ 92 - 82.33 & 88 - 86 \end{bmatrix} = \begin{bmatrix} 2.67 & 4 \\ -12.33 & -6 \\ 9.67 & 2 \end{bmatrix} \quad (4.135)$$

3) 计算协方差矩阵:

$$\begin{aligned} \mathbf{S} &= \frac{1}{2} \mathbf{X}_{\text{centered}} \mathbf{X}_{\text{centered}}^T = \frac{1}{2} \begin{bmatrix} 2.67 & -12.33 & 9.67 \\ 4 & -6 & 2 \end{bmatrix} \begin{bmatrix} 2.67 & 4 \\ -12.33 & -6 \\ 9.67 & 2 \end{bmatrix} \\ &= \begin{bmatrix} 133.33 & 60 \\ 60 & 28 \end{bmatrix} \end{aligned} \quad (4.136)$$

通过上述计算, 得到协方差矩阵 $\mathbf{S}$, 其中对角线元素分别表示数学成绩和英语成绩的方差, 非对角线元素则表示这两个特征之间的协方差。

在 LDA 中, 协方差矩阵用于计算类内散度矩阵 $\mathbf{S}_W$ 和类间散度矩阵 $\mathbf{S}_B$ 。具体来说:

- 类内散度矩阵: 表示每一类内部样本的分布情况。对于每个类别 k , 计算其协方差矩阵 $\mathbf{S}_k$, 然后将所有类别的协方差矩阵相加得到总的类内散度矩阵 $\mathbf{S}_W$ 。
- 类间散度矩阵: 代表不同类别中心之间的差异。首先计算所有类别的总体均值 μ , 然后计算每个类别的均值向量与总体均值之间的差异, 形成类间散度矩阵 $\mathbf{S}_B$ 。

通过这些矩阵, LDA 可以找到最佳的投影方向, 使得不同类别之间的距离最大化, 而同一类别内的点尽可能聚集在一起。

4.8.7 广义特征值问题

广义特征值问题是线性代数中的一个重要概念, 尤其在机器学习和统计分析中有着广泛的应用。它涉及到两个矩阵 $\mathbf{A}$ 和 $\mathbf{B}$, 形式为:

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{B}\mathbf{v}, \quad (4.137)$$

其中 $\mathbf{A}$ 和 $\mathbf{B}$ 是 $n \times n$ 的方阵, $\mathbf{v}$ 是特征向量, λ 是对应的广义特征值。

广义特征值问题的核心在于找到满足上述等式的特征向量 $\mathbf{v}$ 和特征值 λ 。当 $\mathbf{B}$ 是单位矩阵时, 该问题退化为标准的特征值问题。

1. 计算方法

求解广义特征值问题通常涉及以下步骤:

1) 检查矩阵性质: 首先确保矩阵 $\mathbf{B}$ 是对称正定或至少是可逆的。如果 $\mathbf{B}$ 不是对称正定的, 可能需要使用其他方法或进行预处理。

2) 求解特征值和特征向量：使用数值方法（如 QR 算法、Arnoldi 迭代等）来求解广义特征值问题。常见的数学软件包（如 MATLAB、NumPy、SciPy）都提供内置函数来解决这类问题。

3) 选择合适的特征值和特征向量：在某些应用中，只关心最大的几个特征值及其对应的特征向量。例如，在 LDA 中，通常选择最大的 $K - 1$ 个特征值对应的特征向量。

2. 具体示例

为更直观地理解广义特征值问题的求解过程，这里考虑一个简单的例子。假设有两个矩阵 $\mathbf{A} = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}$ 和 $\mathbf{B} = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}$ ，需要找到满足 $\mathbf{A}\mathbf{v} = \lambda\mathbf{B}\mathbf{v}$ 的特征值 λ 和特征向量 $\mathbf{v}$ 。

1) 构造广义特征值问题：将问题写成 $(\mathbf{A} - \lambda\mathbf{B})\mathbf{v} = \mathbf{0}$ 的形式：

$$\begin{bmatrix} 4 - 2\lambda & 1 \\ 1 & 3 - \lambda \end{bmatrix} \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad (4.138)$$

2) 求解特征多项式：求解行列式等于零的条件：

$$\det(\mathbf{A} - \lambda\mathbf{B}) = \begin{vmatrix} 4 - 2\lambda & 1 \\ 1 & 3 - \lambda \end{vmatrix} = (4 - 2\lambda)(3 - \lambda) - 1 = 0 \quad (4.139)$$

解这个二次方程得到特征值：

$$\begin{aligned} (4 - 2\lambda)(3 - \lambda) - 1 = 0 &\implies 6 - 4\lambda - 2\lambda + 2\lambda^2 - 1 = 0 \\ &\implies 2\lambda^2 - 6\lambda + 5 = 0 \end{aligned} \quad (4.140)$$

解得：

$$\begin{aligned} \lambda_1 &= \frac{3 + \sqrt{(-6)^2 - 4 \cdot 2 \cdot 5}}{2 \cdot 2} = \frac{3 + \sqrt{9 - 40}}{4} = \frac{3 + \sqrt{-31}}{4} \\ \lambda_2 &= \frac{3 - \sqrt{(-6)^2 - 4 \cdot 2 \cdot 5}}{2 \cdot 2} = \frac{3 - \sqrt{9 - 40}}{4} = \frac{3 - \sqrt{-31}}{4} \end{aligned} \quad (4.141)$$

由于这两个解是复数，可以进一步简化为： $\lambda_1 = \frac{3}{4} + i\frac{\sqrt{31}}{4}$, $\lambda_2 = \frac{3}{4} - i\frac{\sqrt{31}}{4}$

3) 求解特征向量：对每个特征值分别求解特征向量。以 λ_1 为例，根据 $(\mathbf{A} - \lambda_1\mathbf{B})\mathbf{v} = \mathbf{0}$ ，则：

$$\left(\begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix} - \left(\frac{3}{4} + i\frac{\sqrt{31}}{4} \right) \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix} \right) \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad (4.142)$$

这是一个复数线性系统，可以通过高斯消元法或其他数值方法求解。

3. 在 LDA 中的应用

在 LDA 中，广义特征值问题用于找到最佳的投影方向。具体来说，需要求解以下广义特征值问题：

$$\mathbf{S}_B \mathbf{w} = \lambda \mathbf{S}_W \mathbf{w}, \quad (4.143)$$

其中 $\mathbf{S}_B$ 是类间散度矩阵， $\mathbf{S}_W$ 是类内散度矩阵， $\mathbf{w}$ 是投影方向， λ 是对应的特征值。

通过求解上述问题，可以找到最优的投影方向 $\mathbf{w}$ ，使得不同类别之间的距离最大化，而同一类别内的点尽可能聚集在一起。选择最大的 $K - 1$ 个特征值对应的特征向量组成投影矩阵 $\mathbf{W}$ ，将原始数据投影到新的低维空间中。

4.9 习题

4.9.1 简答题

1. 什么是线性回归？它与逻辑回归的主要区别是什么？
2. 在线性模型中，L1 正则化和 L2 正则化的作用分别是什么？它们如何影响模型的参数？
3. 什么是梯度下降算法？它在线性模型中的主要作用是什么？
4. 线性模型的假设有哪些？这些假设对模型性能有何影响？
5. 什么是多重共线性问题？如何检测 and 解决多重共线性？

4.9.2 计算题

1. 给定一个简单的线性回归模型 $y = w_1x_1 + w_2x_2 + b$ ，其中 $w_1 = 2, w_2 = -1, b = 3$ 。如果输入数据为 $(x_1, x_2) = (4, 5)$ ，求预测值 y 。
2. 假设一个线性回归模型的目标是最小化均方误差（MSE），给定以下数据集：使用最小二乘法

表 4.9: 线性回归数据集

输入 x	输出 y
1	2.1
2	3.9
3	6.2
4	8.0

计算模型参数 w 和 b 。

3. 给定一个逻辑回归模型，其决策边界为 $z = 2x_1 - 3x_2 + 1$ 。如果输入数据为 $(x_1, x_2) = (1, 1)$ ，求该样本属于正类的概率（假设使用 Sigmoid 函数 $\sigma(z) = \frac{1}{1+e^{-z}}$ ）。

4.9.3 编程题

1. 使用 Scikit-learn 实现一个线性回归模型，加载波士顿房价数据集并完成以下任务：
 - 划分训练集和测试集。
 - 训练线性回归模型并评估其性能（使用均方误差和决定系数 R^2 ）。
2. 使用 Scikit-learn 实现一个逻辑回归模型，加载鸢尾花数据集并完成以下任务：
 - 划分训练集和测试集。
 - 训练逻辑回归模型并绘制决策边界。

第五章 决策树模型

决策树是一种直观且强大的机器学习算法，它通过一系列规则将数据逐步分类，类似于人类在做决策时根据条件层层筛选的过程。这种基于规则的分类方法不仅易于理解和可视化，还能广泛应用于分类和回归任务中。从医疗诊断到金融风控，决策树以其简单性和高效性成为解决实际问题的工具。

学习目标

本章系统介绍决策树的基本原理与应用方法。理解决策树在分类与回归任务中的核心思想，掌握基于信息增益、基尼指数等准则的数据划分机制，认识其可解释性强、无需特征缩放、能处理非线性关系等优势，并了解其易过拟合、对噪声敏感等局限性。深入分析剪枝技术（预剪枝与后剪枝）在提升泛化能力中的关键作用。在实践方面，能够使用 `scikit-learn` 库构建决策树模型，掌握关键超参数（如 `max_depth`、`min_samples_split`）的调优策略，并通过可视化工具分析模型结构，结合实际任务评估与优化模型性能。

5.1 活动计划问题

决策树是一种基于规则的模型，其核心思想是通过递归地划分数据集，将复杂的分类问题逐步简化为一系列简单的判断。为更好地理解这一过程，这里从一个与石榴同学相关的故事开始。

5.1.1 周末计划问题

周末到了，石榴同学正在思考如何安排自己的时间。她列出几个可能的活动选项：去公园散步、在家看电影、去图书馆学习，或者和朋友一起喝咖啡。然而，她的选择会受到一些条件的影响，比如天气状况（晴天、阴天、雨天）、是否有朋友约她（是/否）以及她的心情（开心、一般、疲惫）。为帮助自己做出决定，石榴同学决定画一棵“决策树”。

她首先问自己：“今天天气怎么样？”如果是晴天，她可能会倾向于去公园散步；如果是雨天，她更愿意选择在家看电影或去图书馆学习。接着，她又考虑了其他因素，比如“是否有朋友约我？”和“我现在的心情如何？”。通过一步步提问，石榴同学最终得到了一个清晰的行动计划。

这个过程其实就是一个典型的决策树构建过程！每个问题对应一个节点，而每个答案则对应一条分支。最终，所有的分支都会指向一个具体的决策结果（叶子节点）。接下来，将从石榴同学的故事出发，深入探讨决策树的核心原理及其在机器学习中的应用。

5.1.2 活动决策树的构建

决策树的构建过程可以看作是一个“分而治之”的策略，即将复杂的问题逐步拆解为简单的小问题。为帮助大家更直观地理解这一过程，继续以石榴同学的故事为例，并结合具体的数学概念进行讲解。

1. 选择最佳特征

在构建决策树时，首先需要确定哪个特征最适合作为根节点（即第一个分裂点）。这一步的核心是找到一个能够最大程度区分数据的特征。例如，在石榴同学的例子中，“天气”可能是最重要的特征，因为它直接影响了她的活动选择。如果天气是晴天，她可能倾向于去公园散步；如果是雨天，她可能会选择在家看电影或学习。

那么，如何量化一个特征的“区分能力”呢？这里引入两个常用的指标：信息增益和基尼指数。

- **信息增益**：衡量某个特征对数据集纯度的提升程度。举个例子，假设石榴同学的数据集中有两类活动：“户外活动”和“室内活动”。如果某个特征（如“天气”）能够将数据分成两类，且每类中的样本都属于同一活动类型，那么这个特征的信息增益就会很高。
- **基尼指数**：衡量数据集的不纯度。基尼指数越小，数据集的纯度越高。与信息增益类似，基尼指数也可以用来评估特征的区分能力。

通过计算每个特征的信息增益或基尼指数，可以选择一个最优特征作为当前节点的分裂依据。

2. 递归分裂

一旦选择最佳特征并完成第一次分裂，接下来需要对每个子节点重复上述过程。例如，在石榴同学的例子中，如果“天气”被选为根节点，那么数据会被分为三类：晴天、阴天和雨天。对于每一类，再选择下一个最佳特征进行分裂。比如，在“晴天”分支下，可能需要考虑“是否有朋友约我？”这个特征。

递归分裂的过程会一直持续，直到满足以下条件之一：

- 节点中的样本数小于预设阈值（如 5 个样本）。
- 节点的纯度达到一定标准（如所有样本都属于同一类别）。
- 树的深度达到最大限制。

3. 生成叶子节点

当递归分裂结束时，每个最终的节点（即没有进一步分裂的节点）被称为叶子节点。叶子节点对应于具体的分类结果。例如，在石榴同学的例子中，叶子节点可能是“去公园散步”、“在家看电影”或“去图书馆学习”。

为更好地直观理解决策树的构建过程，可以将其想象成一棵真实的树。根节点是树的起点，代表整个数据集；内部节点是树的分支，代表根据某个特征进行的分裂；叶子节点是树的末端，代表最终的分类结果。

回到石榴同学的例子，假设已经完成决策树的构建，得到如下规则：

- 如果天气是晴天，且有朋友约我，则选择“去公园散步”。
- 如果天气是雨天，且心情一般，则选择“在家看电影”。
- 如果天气是阴天，且没有朋友约我，则选择“去图书馆学习”。

这些规则实际上就是决策树的具体表现形式，它们清晰地展示了如何根据输入条件一步步得出结论。

为更直观地展示这一过程，下图是一个基于石榴同学周末计划的决策树示意图：

如图所示，决策树的每个节点表示一个判断条件（如“天气”或“朋友？”），每条分支表示一个可能的答案（如“晴天”或“否”），而叶子节点则表示最终的决策结果（如“去公园散步”）。通过这种树形结构，复杂的分类问题被逐步简化为一系列简单的判断规则。

综上所述，决策树的构建过程可以概括为三个关键步骤：选择最佳特征、递归分裂和生成叶子节点。通过这一过程，复杂的分类问题被逐步简化为一系列简单的判断规则。无论是石榴同学的周末计划，还是机器学习中的分类任务，决策树都能提供一种直观且高效的方法来解决问题。

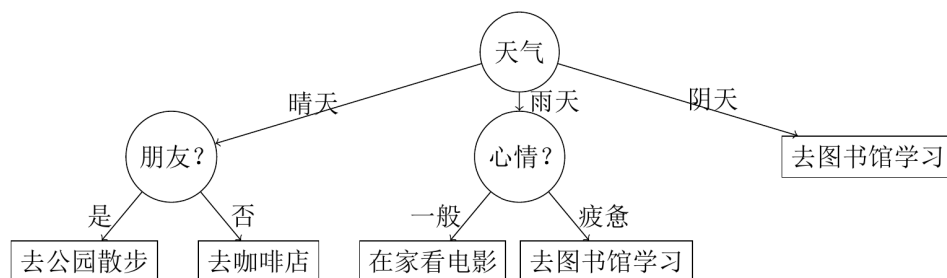


图 5.1: 基于石榴同学周末计划的决策树示例

5.2 决策树构建

在构建决策树的过程中，如何选择最佳特征进行分裂是关键问题。为解决这个问题，需要引入几个核心概念：数据集纯度、信息熵、信息增益和基尼指数。这些概念共同构成了决策树分裂准则的理论基础。接下来，将通过石榴同学的故事，逐步理解决策树构建的核心思想。

5.2.1 数据集纯度的概念

在决策树算法中，“数据集纯度”是一个非常重要的概念，它用于衡量数据集中样本的类别分布是否均匀。简单来说，数据集的纯度越高，说明数据集中样本的类别越单一；反之，纯度越低，说明数据集中样本的类别分布越混乱。

为更直观地理解这一概念，可以通过一个生活化的例子来说明。假设石榴同学正在整理她的书架，她将书籍分为两类：“小说”和“教科书”。如果书架上的所有书籍都是“小说”，那么这个书架的数据集纯度非常高，因为所有样本都属于同一类别。相反，如果书架上有一半是“小说”，另一半是“教科书”，那么这个书架的数据集纯度就很低，因为样本的类别分布非常均匀。

决策树的目标是通过选择最佳特征，尽可能提高数据集的纯度，从而简化分类任务。接下来，将介绍两种常用的量化数据集纯度的方法：信息熵和基尼指数。

5.2.2 信息熵：衡量不确定性

信息熵是衡量数据集纯度的一种常用方法，它通过计算数据集中每个类别的概率分布来反映数据的混乱程度。公式如下：

$$H(D) = - \sum_{k=1}^K p_k \log_2(p_k) \quad (5.1)$$

其中， p_k 是类别 k 在数据集 D 中的比例。公式的含义是：对于每个类别 k ，计算其出现的概率 p_k 乘以 $\log_2(p_k)$ 的负值，然后将所有类别的结果相加。

使用石榴同学的例子来理解信息熵。假设她的数据集中有两类活动：“户外活动”和“室内活动”。如果两类活动的比例是 50% 对 50%，那么信息熵达到最大值 1，表示数据完全混乱；如果所有样本都属于“户外活动”，那么信息熵为 0，表示数据完全纯净。

信息熵越低，说明数据的纯度越高，分类任务越容易完成。因此，在构建决策树时，目标是通过选择最佳特征，尽可能降低数据集的信息熵。

5.2.3 信息增益：减少不确定性

信息增益的核心思想是“减少不确定性”。想象一下，石榴同学正在决定周末活动，但她对天气状况一无所知。在这种情况下，她的选择充满不确定性（即熵较高）。如果她得知今天是晴天，那么

她的选择范围会大大缩小，不确定性也随之降低。这种不确定性的减少就是信息增益。

具体来说，信息增益衡量的是在使用某个特征进行分裂后，数据集的混乱程度（熵）能够减少多少。公式如下：

$$\text{Gain}(D, A) = H(D) - \sum_{v=1}^V \frac{|D_v|}{|D|} H(D_v) \quad (5.2)$$

其中， $H(D)$ 是原始数据集的熵，表示数据的不确定性； D_v 是根据特征 A 的取值划分后的子集； $\frac{|D_v|}{|D|}$ 是子集 D_v 在整个数据集中所占的比例。

举个例子，假设石榴同学的数据集中有两类活动：“户外活动”和“室内活动”。如果某个特征（如“天气”）能够将数据分成两类，且每类中的样本都属于同一活动类型，那么这个特征的信息增益就会很高。换句话说，信息增益越大，说明该特征越能有效地将数据分类。

5.2.4 基尼指数：衡量不纯度

基尼指数则是另一种衡量数据集“混乱程度”的方法。它的核心思想是“随机抽取两个样本，它们属于不同类别的概率”。如果一个数据集完全纯净（即所有样本都属于同一类别），那么基尼指数为 0；如果数据集非常混乱（即样本均匀分布在多个类别中），那么基尼指数会接近 1。

基尼指数的计算公式如下：

$$\text{Gini}(D) = 1 - \sum_{k=1}^K p_k^2 \quad (5.3)$$

其中， p_k 是类别 k 在数据集 D 中的比例。

继续用石榴同学的例子来理解基尼指数。如果某个分支中的样本全部属于“户外活动”，那么基尼指数为 0，表示该分支完全纯净；如果样本均匀分布在两类活动中，那么基尼指数会接近 0.5，表示该分支非常混乱。

无论是信息熵还是基尼指数，它们的本质都是帮助量化数据集的纯度，并指导大家选择最佳特征进行分裂。通过这些分裂准则，可以构建一棵高效的决策树，从而解决复杂的分类问题。正如石榴同学通过一系列简单的判断规则做出周末计划一样，决策树提供了一种直观且高效的方法来处理分类任务。

5.2.5 信息增益和基尼指数计算

为更好地理解信息增益和基尼指数的计算过程，通过一个具体的例子进行演示。假设石榴同学正在分析一组关于天气和活动的数据，目标是预测某一天是否会有人选择“户外活动”。数据如下表所示：

表 5.1: 天气与活动数据集

天气	活动
晴天	户外活动
晴天	户外活动
雨天	室内活动
雨天	室内活动
阴天	户外活动
阴天	室内活动

接下来，将分别计算信息增益和基尼指数，并比较它们的结果。

1. 计算原始数据集的信息熵

首先，需要计算整个数据集的熵 $H(D)$ 。数据集中共有 6 个样本，其中“户外活动”有 3 个，“室内活动”也有 3 个。因此，两类活动的概率分别为：

$$p_{\text{户外}} = \frac{3}{6} = 0.5, \quad p_{\text{室内}} = \frac{3}{6} = 0.5 \quad (5.4)$$

根据信息熵公式：

$$H(D) = - \sum_{k=1}^K p_k \log_2(p_k) \quad (5.5)$$

可以计算出：

$$H(D) = -(0.5 \cdot \log_2(0.5) + 0.5 \cdot \log_2(0.5)) = -(0.5 \cdot (-1) + 0.5 \cdot (-1)) = 1 \quad (5.6)$$

因此，原始数据集的熵为 1，表示数据完全混乱。

2. 计算基于“天气”特征的信息增益

选择“天气”作为分裂特征，并计算每个子集的熵。根据数据集，“天气”分为三类：晴天、雨天和阴天。

- **晴天**：共有 2 个样本，全部属于“户外活动”。因此，子集的熵为：

$$H(D_{\text{晴天}}) = -(1 \cdot \log_2(1) + 0 \cdot \log_2(0)) = 0 \quad (5.7)$$

- **雨天**：共有 2 个样本，全部属于“室内活动”。因此，子集的熵为：

$$H(D_{\text{雨天}}) = -(0 \cdot \log_2(0) + 1 \cdot \log_2(1)) = 0 \quad (5.8)$$

- **阴天**：共有 2 个样本，其中 1 个属于“户外活动”，1 个属于“室内活动”。因此，子集的熵为：

$$H(D_{\text{阴天}}) = -(0.5 \cdot \log_2(0.5) + 0.5 \cdot \log_2(0.5)) = 1 \quad (5.9)$$

然后，计算加权平均熵：

$$H(D_{\text{天气}}) = \frac{|D_{\text{晴天}}|}{|D|} H(D_{\text{晴天}}) + \frac{|D_{\text{雨天}}|}{|D|} H(D_{\text{雨天}}) + \frac{|D_{\text{阴天}}|}{|D|} H(D_{\text{阴天}}) \quad (5.10)$$

代入数据：

$$H(D_{\text{天气}}) = \frac{2}{6} \cdot 0 + \frac{2}{6} \cdot 0 + \frac{2}{6} \cdot 1 = \frac{2}{6} = 0.333 \quad (5.11)$$

最后，计算信息增益：

$$\text{Gain}(D, \text{天气}) = H(D) - H(D_{\text{天气}}) = 1 - 0.333 = 0.667 \quad (5.12)$$

因此，基于“天气”特征的信息增益为 0.667。

3. 计算基于“天气”特征的基尼指数

根据基尼指数公式，计算基尼指数：

$$\text{Gini}(D) = 1 - \sum_{k=1}^K p_k^2 \quad (5.13)$$

对于原始数据集：

$$Gini(D) = 1 - (0.5^2 + 0.5^2) = 1 - (0.25 + 0.25) = 0.5 \quad (5.14)$$

然后，分别计算每个子集的基尼指数：

- 晴天：所有样本都属于“户外活动”，因此：

$$Gini(D_{\text{晴天}}) = 1 - (1^2 + 0^2) = 0 \quad (5.15)$$

- 雨天：所有样本都属于“室内活动”，因此：

$$Gini(D_{\text{雨天}}) = 1 - (0^2 + 1^2) = 0 \quad (5.16)$$

- 阴天：样本均匀分布，因此：

$$Gini(D_{\text{阴天}}) = 1 - (0.5^2 + 0.5^2) = 0.5 \quad (5.17)$$

接着，计算加权平均基尼指数：

$$Gini(D_{\text{天气}}) = \frac{|D_{\text{晴天}}|}{|D|} Gini(D_{\text{晴天}}) + \frac{|D_{\text{雨天}}|}{|D|} Gini(D_{\text{雨天}}) + \frac{|D_{\text{阴天}}|}{|D|} Gini(D_{\text{阴天}}) \quad (5.18)$$

代入数据：

$$Gini(D_{\text{天气}}) = \frac{2}{6} \cdot 0 + \frac{2}{6} \cdot 0 + \frac{2}{6} \cdot 0.5 = \frac{1}{6} = 0.167 \quad (5.19)$$

因此，基于“天气”特征的基尼指数为 0.167。通过上述计算，得到了以下结果：

- 基于“天气”特征的信息增益为 0.667。
- 基于“天气”特征的基尼指数为 0.167。

可以看出，“天气”是一个非常有效的分裂特征，因为它能够显著降低数据集的混乱程度（无论是通过信息增益还是基尼指数衡量）。在实际应用中，会选择信息增益或基尼指数较高的特征作为分裂点，从而构建一棵高效的决策树。

5.3 剪枝操作

在决策树模型中，剪枝是一种重要的技术，用于防止过拟合并提高模型的泛化能力。本章将详细介绍预剪枝和后剪枝的过程，并通过具体案例和手动计算说明其原理。

5.3.1 剪枝的必要性

为理解剪枝的重要性，来看一个简单的例子。假设使用决策树对以下数据集进行分类：

如果不对树的深度进行限制，决策树可能会生成一棵非常复杂的树，如下图5.2所示：

这棵树虽然能够完美地拟合训练数据，但在测试集上的表现可能很差，因为它过于复杂，可以捕捉训练数据中的噪声和细节。这种现象称为**过拟合**。为避免过拟合，需要对决策树进行剪枝操作，简化树的结构，从而提高模型的泛化能力。

表 5.2: 二分类问题数据集

特征 A	特征 B	类别
1	2	正类
1	3	正类
0	4	负类
0	5	负类
1	6	负类

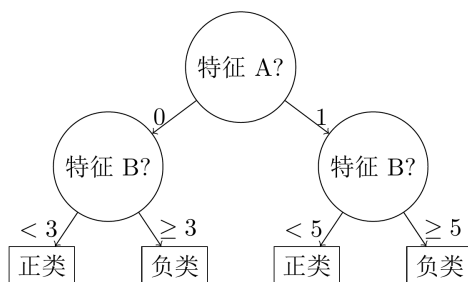


图 5.2: 未剪枝的决策树示例

5.3.2 预剪枝

预剪枝是指在构建决策树的过程中，通过设置一些限制条件（如最大深度、最小样本数等）提前停止分裂。这种方法的优点是简单高效，但可能导致欠拟合。

继续使用上述数据集，并通过限制树的最大深度为 2 来实现预剪枝。以下是手动计算过程：

1. 计算根节点的信息熵

原始数据集中共有 5 个样本，其中“正类”有 2 个，“负类”有 3 个。因此，两类的概率分别为：

$$p_{\text{正类}} = \frac{2}{5}, \quad p_{\text{负类}} = \frac{3}{5} \quad (5.20)$$

信息熵公式为：

$$H(D) = - \sum_{k=1}^K p_k \log_2(p_k) \quad (5.21)$$

代入数据：

$$H(D) = - \left(\frac{2}{5} \log_2 \frac{2}{5} + \frac{3}{5} \log_2 \frac{3}{5} \right) \approx 0.971 \quad (5.22)$$

2. 选择最佳特征进行分裂

根据信息增益公式，计算每个特征的分裂效果。

• 特征 A：

- 子集 1（特征 A=0）：包含 2 个样本，全部为“负类”，熵为 0。
- 子集 2（特征 A=1）：包含 3 个样本，其中“正类”有 2 个，“负类”有 1 个，熵为：

$$H(D_{A=1}) = - \left(\frac{2}{3} \log_2 \frac{2}{3} + \frac{1}{3} \log_2 \frac{1}{3} \right) \approx 0.918 \quad (5.23)$$

加权平均熵为：

$$H(D_{\text{特征 A}}) = \frac{2}{5} \cdot 0 + \frac{3}{5} \cdot 0.918 \approx 0.551 \quad (5.24)$$

信息增益为：

$$\text{Gain}(D, \text{特征 A}) = H(D) - H(D_{\text{特征 A}}) = 0.971 - 0.551 = 0.420 \quad (5.25)$$

• 特征 B:

- 子集 1 (特征 $B < 5$): 包含 3 个样本, 其中“正类”有 2 个, “负类”有 1 个, 熵为:

$$H(D_{B < 5}) = - \left(\frac{2}{3} \log_2 \frac{2}{3} + \frac{1}{3} \log_2 \frac{1}{3} \right) \approx 0.918 \quad (5.26)$$

- 子集 2 (特征 $B \geq 5$): 包含 2 个样本, 全部为“负类”, 熵为 0。

加权平均熵为:

$$H(D_{\text{特征 B}}) = \frac{3}{5} \cdot 0.918 + \frac{2}{5} \cdot 0 \approx 0.551 \quad (5.27)$$

信息增益为:

$$\text{Gain}(D, \text{特征 B}) = H(D) - H(D_{\text{特征 B}}) = 0.971 - 0.551 = 0.420 \quad (5.28)$$

由于两个特征的信息增益相同, 可以任选一个作为分裂特征 (例如特征 A)。

3. 限制最大深度

在预剪枝中, 限制树的最大深度为 2。因此, 在第二层分裂时, 不再进一步分裂子节点。最终生成的决策树如下图所示:

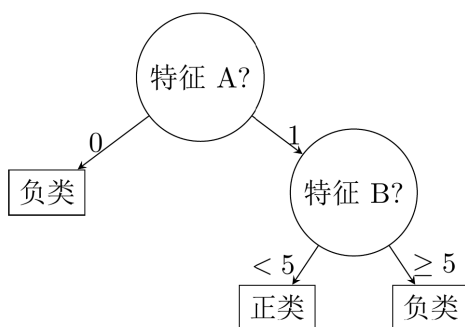


图 5.3: 预剪枝后的决策树

这里使用一个模拟数据集, 并通过限制树的最大深度为 2 来实现预剪枝, 进一步方便读者理解。以下是代码实现:

```

1  from sklearn.datasets import make_classification
2  from sklearn.tree import DecisionTreeClassifier, plot_tree
3  from sklearn.model_selection import train_test_split
4  import matplotlib.pyplot as plt
5
6  # 生成数据集
7  X, y = make_classification(n_samples=100, n_features=2, n_informative=2, n_redundant=0,
8                             random_state=42)
9  X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
10
11 # 构建决策树模型 (限制最大深度为2)
12 clf = DecisionTreeClassifier(max_depth=2, random_state=42)
13 clf.fit(X_train, y_train)
14
15 # 可视化决策树

```

```

15 plt.figure(figsize=(10, 6))
16 plot_tree(clf, filled=True, feature_names=["特征 A", "特征 B"], class_names=["负类", "正类"])
17 plt.title("预剪枝后的决策树")
18 plt.show()

```

运行上述代码后，可以得到一棵深度为 2 的决策树。相比未剪枝的树，这棵树的结构更简单，泛化能力更强。

5.3.3 后剪枝

后剪枝是指先生成一棵完整的决策树，然后通过移除不必要的分支来简化树的结构。这种方法通常比预剪枝更灵活，但计算成本更高。继续使用上述数据集，并通过验证集评估剪枝的效果。以下是手动计算过程：

1. **生成完整决策树。**首先，生成一棵未剪枝的完整决策树（见图 5.2）。
2. **计算每个节点的误差。**使用验证集计算每个节点的误差。假设验证集如下表所示：

表 5.3: 验证集数据		
特征 A	特征 B	类别
1	2	正类
0	5	负类
1	6	负类

对于每个节点，计算其预测误差。例如，对于根节点，如果直接将其标记为“负类”，则误差为：

$$\text{误差} = \frac{\text{错误分类样本数}}{\text{总样本数}} = \frac{1}{3} \quad (5.29)$$

类似地，计算其他节点的误差。

3. **剪枝决策。**根据误差和复杂度参数 α ，决定是否剪枝某个节点。具体公式为：

$$R_{\alpha}(T) = R(T) + \alpha|T| \quad (5.30)$$

其中， $R(T)$ 是树的误差， $|T|$ 是树的叶节点数， α 是复杂度参数。通过调整 α 的值，可以找到最优的剪枝点。

4. **生成剪枝后的决策树。**最终生成的剪枝后的决策树如下图5.4所示：

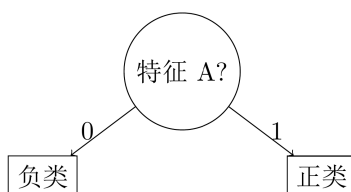


图 5.4: 后剪枝后的决策树

为进一步帮助理解，可以通过 sklearn 的 `cost_complexity_pruning_path` 方法实现后剪枝。以下是代码实现：

```

1 from sklearn.tree import DecisionTreeClassifier, plot_tree
2 from sklearn.model_selection import train_test_split
3 from sklearn.datasets import make_classification
4 import matplotlib.pyplot as plt
5

```

```

6  # 生成数据集
7  X, y = make_classification(n_samples=100, n_features=2, n_informative=2, n_redundant=0,
    random_state=42)
8  X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
9
10 # 构建完整决策树
11 clf = DecisionTreeClassifier(random_state=42)
12 path = clf.cost_complexity_pruning_path(X_train, y_train)
13 ccp_alphas = path.ccp_alphas
14
15 # 选择最优的 ccp_alpha 值
16 best_alpha = None
17 best_score = 0
18 for alpha in ccp_alphas:
19     clf = DecisionTreeClassifier(ccp_alpha=alpha, random_state=42)
20     clf.fit(X_train, y_train)
21     score = clf.score(X_test, y_test)
22     if score > best_score:
23         best_score = score
24         best_alpha = alpha
25
26 # 使用最优的 ccp_alpha 值重新训练模型
27 final_clf = DecisionTreeClassifier(ccp_alpha=best_alpha, random_state=42)
28 final_clf.fit(X_train, y_train)
29
30 # 可视化剪枝后的决策树
31 plt.figure(figsize=(10, 6))
32 plot_tree(final_clf, filled=True, feature_names=["特征A", "特征B"], class_names=["负类", "正类"]
    )
33 plt.title("后剪枝后的决策树")
34 plt.show()

```

运行上述代码后，可以得到一棵经过后剪枝的决策树。通过调整 `ccp_alpha` 参数，可以在模型复杂度和泛化能力之间找到平衡。

预剪枝和后剪枝是两种常用的剪枝方法，各有优缺点。预剪枝通过提前停止分裂来简化树的结构，计算效率高，但可能导致欠拟合；后剪枝通过移除不必要的分支来优化树的性能，灵活性更强，但计算成本较高。在实际应用中，可以根据任务需求选择合适的剪枝方法，并结合验证集或交叉验证评估模型的性能。

5.4 连续属性与缺失值处理

5.4.1 连续属性的节点划分

决策树在处理连续属性时，需通过离散化将其转化为有限区间。以下通过石榴同学的案例说明具体步骤。回忆本章开头，石榴同学思考如何安排自己的周末时间。她列出几个可能的活动选项：去公园散步、在家看电影、去图书馆学习，或者和朋友一起喝咖啡。然而，她的选择会受到一些条件的影响，比如天气状况（晴天、阴天、雨天）、是否有朋友约她（是/否）以及温度（连续值）。为帮助自己做出决定，石榴同学决定画一棵“决策树”。假设石榴同学使用“温度”作为连续属性来决定是否去公园。数据如下：

1. 排序与候选分割点生成

将温度升序排序为 18, 22, 25, 28, 30，生成候选分割点为相邻值的中点：

$$\text{候选点} = \frac{18+22}{2} = 20, \frac{22+25}{2} = 23.5, \frac{25+28}{2} = 26.5, \frac{28+30}{2} = 29 \quad (5.31)$$

表 5.4: 温度与活动数据

温度	是否去公园
18	否
22	是
25	是
28	否
30	否

2. 计算信息增益选择最佳分割点

为解说方便，以分割点 23.5 为例，称温度 ≤ 23.5 的所有样本组成的集合为左子集，其包括样本 18, 22，类别分别一“是”，一个“否”，所以熵为：

$$H_{\text{左}} = -\left(\frac{1}{2} \log_2 \frac{1}{2} + \frac{1}{2} \log_2 \frac{1}{2}\right) = 1 \quad (5.32)$$

温度 > 23.5 的样本组成的集合称为右子集，包含样本 25, 28, 30，其中“是”1 个、“否”2 个，所以信息熵为：

$$H = -\left(\frac{1}{3} \log_2 \frac{1}{3} + \frac{2}{3} \log_2 \frac{2}{3}\right) \approx 0.918 \quad (5.33)$$

加权熵为：

$$H_{\text{split}} = \frac{2}{5} \cdot 1 + \frac{3}{5} \cdot 0.918 \approx 0.951 \quad (5.34)$$

所以，最后的信息增益为：

$$\begin{aligned} H_{\text{原始}} &= -\left(\frac{2}{5} \log_2 \frac{2}{5} + \frac{3}{5} \log_2 \frac{3}{5}\right) \\ \text{Gain}(23.5) &= H_{\text{原始}} - H_{\text{split}} = 0.971 - 0.951 = 0.02 \end{aligned} \quad (5.35)$$

遍历所有候选点后，发现当节点选择 26.5 的时候，信息增益最大，所以该节点作为最佳分割阈值，规则为：

$$\text{温度} \leq 26.5 \rightarrow \text{去公园}, \quad \text{温度} > 26.5 \rightarrow \text{不去公园} \quad (5.36)$$

综上，对于一般的连续属性，决策树的节点划分主要遵循以下步骤：

- 1) 对连续属性的所有取值按升序排列，计算相邻两个样本值的中点作为候选分割点。
- 2) 遍历所有候选分割点，计算每个分割点的信息增益或基尼指数，选择使信息增益最大（或基尼指数最小）的分割点作为最佳划分阈值。
- 3) 在确定最佳分割点后，将数据集划分为左右子集，并对每个子集重复上述过程。
- 4) 当递归分裂结束时，每个最终节点称为叶子节点，对应具体的分类结果。

对于高维数据，可采用分箱（Binning）策略对连续属性进行预处理，通过减少候选分割点的数量来降低计算复杂度；在算法选择方面，CART 算法通常采用二分法（Binary Split），而 C4.5 算法则支持多路划分，需根据具体任务需求（如是否需要多类别分裂或计算效率优先）选择合适的算法。

5.4.2 缺失值的处理方法

决策树可以处理训练数据缺失的情况，其主要通过概率分配方式处理缺失值。以下通过石榴同学的案例说明。数据集中“家长是否同意”存在缺失值：

表 5.5: 课外活动参与数据

天气	家长同意（缺失）	是否参加
晴天	未知	是
雨天	否	否
晴天	是	是

1. **统计非缺失样本分布。**对于特征“家长同意”来说，非缺失样本共 2 个：1 个“是”→ 参加（晴天，是 → 是），1 个“否”→ 不参加（雨天，否 → 否），各子节点的概率：

$$p_{\text{是}} = \frac{1}{2}, \quad p_{\text{否}} = \frac{1}{2} \quad (5.37)$$

2. **分配缺失样本到子节点。**依据上面计算的各个分支点概率，将缺失样本（晴天，未知 → 是）按上面比例进行分配： $\frac{1}{2}$ 的权重分配到“是”分支， $\frac{1}{2}$ 的权重分配到“否”分支。

3. **加权计算子节点熵。**对每个子节点 v ，加权熵公式为：

$$H_{\text{加权}} = \sum_{v=1}^V \frac{|D_v| + w_v \cdot |D_{\text{缺失}}|}{|D_{\text{总}}|} \cdot H(D_v) \quad (5.38)$$

其中， $|D_v|$ 是非缺失样本的子集大小， w_v 是分配权重， $|D_{\text{缺失}}|$ 是缺失样本数量。在案例中， $|D_v| = 2$ ，有两个非缺失样本， $|D_{\text{缺失}}| = 1$ 有一个缺失样本。对于“是”分支，加权样本数为 $2 + 1 \cdot 0.5 = 2.5$ ，“是”分支对应的熵为 $H_{\text{是}} = 0$ （所有样本为“是”）；对于“否”分支，分子为 $1 + 1 \cdot 0.5 = 1.5$ ，熵 $H_{\text{否}} = 0$ （所有样本为“否”）。所以最后的加权熵为：

$$H_{\text{split}} = \frac{2.5}{3} \cdot 0 + \frac{1.5}{3} \cdot 0 = 0 \quad (5.39)$$

4. **调整信息增益。**假设原始熵 $H_{\text{原始}} = 1$ （两类样本均等分布），则修正后的信息增益为：

$$\text{Gain}_{\text{修正}} = H_{\text{原始}} - H_{\text{split}} = 1 - 0 = 1 \quad (5.40)$$

因此，“家长是否同意”特征被选为分裂点。综上，对于缺失值的处理，概率分配的方式一般可以采用以下的步骤进行：

1. **统计非缺失样本分布。**对当前特征的非缺失样本，计算每个子节点（分裂后类别）的概率 p_v ：

$$p_v = \frac{|D_v|}{|D_{\text{非缺失}}|} \quad (5.41)$$

其中， D_v 是非缺失样本中属于子节点 v 的数量， $D_{\text{非缺失}}$ 是非缺失样本总数。

2. **分配缺失样本权重。**将缺失样本 $D_{\text{缺失}}$ 的权重按概率 p_v 分配到各个子节点，子节点 v 的权重为 $w_v = p_v \cdot |D_{\text{缺失}}|$ 。

3. **计算加权熵。**对每个子节点 v ，计算加权后的熵 $H_{\text{加权}}(v)$ ：

$$H_{\text{加权}}(v) = \frac{|D_v| + w_v}{|D_{\text{总}}|} \cdot H(D_v) \quad (5.42)$$

其中， $|D_{\text{总}}| = |D_{\text{非缺失}}| + |D_{\text{缺失}}|$ ，为所有样本的个数。

4. 计算加权平均熵。根据下面的公式计算加权后的平均熵：

$$H_{\text{split}} = \sum_{v=1}^V H_{\text{加权}}(v) \quad (5.43)$$

5. 修正信息增益。计算划分后的信息增益：

$$\text{Gain}_{\text{修正}} = H(D) - H_{\text{split}} \quad (5.44)$$

最后，选择使 $\text{Gain}_{\text{修正}}$ 最大的特征作为分裂点。

除上述提及的概率分配方法外，还存在默认方向的处理方法，其具体实现方式可参考 Xgboost 的相关研究 [10]。对于一般的缺失值问题，决策树能够提供多种灵活的处理方式，具体方法如下：

- 1) 在预处理阶段，低缺失率（<10%）可以直接删除或填充均值/众数；高缺失率（>30%）使用概率分配或模型内置方法（如 XGBoost）。
- 2) 在划分阶段，忽略缺失样本仅使用非缺失样本计算信息增益或基尼指数，或通过概率分配法将缺失样本按比例分配到子节点。
- 3) 改进算法的处理：XGBoost 和 LightGBM 等现代算法内置有缺失值处理机制，能够自动学习缺失值的分裂方向。

无论是连续属性的划分，还是缺失值的处理，决策树的核心思想均在于借助规则化方法简化复杂问题。通过上述分析可知，决策树算法不仅适用于石榴同学的周末计划，还在分类与回归任务中得到了广泛应用。合理选择分裂准则、处理连续属性及缺失值，能够显著提升模型的性能与泛化能力。

5.5 实践练习

在本章中，将通过多个实例展示如何使用决策树模型解决实际问题。每个实例都包含背景介绍、数据描述、代码实现和结果分析，帮助读者全面掌握决策树的应用方法。

5.5.1 示例：鸢尾花分类问题

鸢尾花数据集是机器学习领域最常用的数据集之一，包含 150 个样本，每个样本有 4 个特征（花萼长度、花萼宽度、花瓣长度、花瓣宽度）和一个目标标签（鸢尾花的种类）。目标是根据这些特征预测鸢尾花的类别。数据集中的类别包括：Setosa、Versicolor、Virginica。

下面将使用决策树模型对数据集进行分类，并可视化生成的决策树。该示例中，使用的是鸢尾花数据集，其具体信息如下：

表 5.6: 鸢尾花数据集的特征描述

特征	描述	单位
花萼长度	花萼的长度	厘米
花萼宽度	花萼的宽度	厘米
花瓣长度	花瓣的长度	厘米
花瓣宽度	花瓣的宽度	厘米

目标变量是一个三分类问题，表示鸢尾花的种类。以下是完整的代码实现过程，分为数据加载、模型训练、模型评估和可视化四个部分。

1. 数据加载与预处理。首先，需要加载鸢尾花数据集并进行简单的预处理。scikit-learn 已提供内置的鸢尾花数据集，可以直接使用。

```
1 from sklearn.datasets import load_iris
2 from sklearn.model_selection import train_test_split
3 # 加载数据集
4 iris = load_iris()
5 X, y = iris.data, iris.target
6 # 将数据集划分为训练集和测试集
7 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
8 print("训练集大小:", X_train.shape)
9 print("测试集大小:", X_test.shape)
```

2. 构建与训练决策树模型。接下来，使用 DecisionTreeClassifier 构建决策树模型，并在训练集上进行训练。

```
1 from sklearn.tree import DecisionTreeClassifier
2 # 初始化决策树模型
3 clf = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=42)
4 # 训练模型
5 clf.fit(X_train, y_train)
6 print("模型训练完成")
```

3. 模型评估。为评估模型的性能，在测试集上计算准确率，并输出分类报告。

```
1 from sklearn.metrics import accuracy_score, classification_report
2 # 在测试集上进行预测
3 y_pred = clf.predict(X_test)
4 # 计算准确率
5 accuracy = accuracy_score(y_test, y_pred)
6 print("测试集准确率:", accuracy)
7 # 输出分类报告
8 print("分类报告:\n", classification_report(y_test, y_pred, target_names=iris.target_names))
```

运行上面的代码，可以得到以下所示的结果：

测试集准确率：0.9777777777777777

分类包括：

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	19
versicolor	1.00	0.92	0.96	13
virginica	0.93	1.00	0.96	13
accuracy			0.98	45
macro avg	0.98	0.97	0.97	45
weighted avg	0.98	0.98	0.98	45

4. 可视化决策树。最后，可以将生成的决策树可视化，以便更直观地理解模型的分裂规则。

```
1 from sklearn.tree import export_text, plot_tree
2 import matplotlib.pyplot as plt
3 # 设置支持中文
4 plt.rcParams['font.sans-serif'] = ['SimHei']
5 # 设置支持负号
6 plt.rcParams['axes.unicode_minus'] = False
7
8 # 打印决策树的文本规则
9 tree_rules = export_text(clf, feature_names=iris.feature_names)
10 print("决策树规则:\n", tree_rules)
11
12 # 绘制决策树图形
```

```

13 plt.figure(figsize=(12, 8))
14 plot_tree(clf, feature_names=iris.feature_names, class_names=iris.target_names, filled=True)
15 # plt.title("决策树可视化")
16 plt.tight_layout()
17 plt.show()

```

通过上面的代码，可以得到类似下面的决策树图，如图5.5所示：

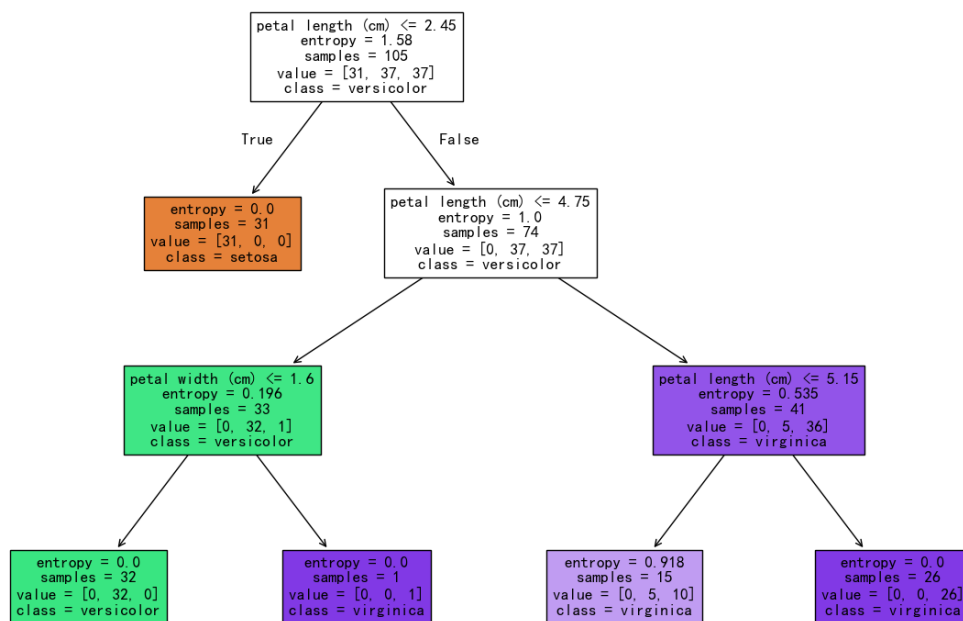


图 5.5: 决策树可视化

从上述代码运行结果来看，可以得到以下结论：

- 测试集的分类准确率通常在 95% 以上，表明决策树模型在鸢尾花数据集上表现良好。
- 分类报告能够显示每个类别的精确率、召回率和 F1 分数，进一步验证了模型的性能。
- 决策树的可视化可以展示模型的分裂规则。例如，模型可能首先根据“花瓣长度”进行分裂，因为这一特征对区分不同种类的鸢尾花具有较高的区分度。

通过本案例，能够学习如何使用决策树模型解决分类问题。具体步骤包括数据加载、模型训练、性能评估和结果可视化。决策树因其直观性和可解释性，在许多实际应用中都表现出色。然而，也需要注意过拟合问题，可以通过调整树的最大深度或使用剪枝技术来优化模型性能。

5.5.2 示例：加利福尼亚房价预测问题

加利福尼亚房价数据集包含 20,640 个样本，每个样本有 8 个特征，描述了加利福尼亚地区不同街区的房屋信息。目标变量是该街区的房价中位数（单位：美元）。这是一个典型的回归问题，目标变量为连续值（房价中位数）。下面将使用决策树模型对数据集进行建模，预测房价中位数，并评估其性能。加利福尼亚房价数据集的具体信息如表5.7所示：

表 5.7: 加利福尼亚房价数据集的特征描述

特征	描述	类型
MedInc	街区收入中位数	连续
HouseAge	房屋年龄中位数	连续
AveRooms	每户平均房间数	连续
AveBedrms	每户平均卧室数	连续
Population	街区人口数	连续
AveOccup	每户平均居住人数	连续
Latitude	纬度	连续
Longitude	经度	连续

以下是完整的代码实现过程，同样分为数据加载、模型训练、模型评估和可视化四个部分。

1. 数据加载与预处理。首先，需要加载加利福尼亚房价数据集并进行简单的预处理。scikit-learn 已提供内置的加利福尼亚房价数据集，可以直接使用。

```

1 from sklearn.datasets import fetch_california_housing
2 from sklearn.model_selection import train_test_split
3 # 加载数据集
4 california = fetch_california_housing()
5 X, y = california.data, california.target
6 # 将数据集划分为训练集和测试集
7 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
8 print("训练集大小:", X_train.shape)
9 print("测试集大小:", X_test.shape)

```

2. 构建与训练决策树模型。接下来，使用 DecisionTreeRegressor 构建决策树回归模型，并在训练集上进行训练。

```

1 from sklearn.tree import DecisionTreeRegressor
2 # 初始化决策树回归模型
3 reg = DecisionTreeRegressor(max_depth=5, random_state=42)
4 # 训练模型
5 reg.fit(X_train, y_train)
6 print("模型训练完成")

```

3. 模型评估。为评估模型的性能，在测试集上计算均方误差（MSE）和决定系数（ R^2 ），并输出结果。

```

1 from sklearn.metrics import mean_squared_error, r2_score
2 # 在测试集上进行预测
3 y_pred = reg.predict(X_test)
4 # 计算均方误差和决定系数
5 mse = mean_squared_error(y_test, y_pred)
6 r2 = r2_score(y_test, y_pred)
7 print("测试集均方误差 (MSE):", mse)
8 print("测试集决定系数 (R^2):", r2)

```

4. 可视化特征重要性。为理解哪些特征对预测房价最为重要，可以绘制特征重要性图。

```

1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # 获取特征重要性
5 feature_importances = reg.feature_importances_
6
7 # 绘制特征重要性
8 plt.barh(range(len(feature_importances)), feature_importances, align='center')
9 plt.yticks(np.arange(len(california.feature_names)), california.feature_names)
10 plt.xlabel("特征重要性")

```

```

11 plt.title("决策树特征重要性")
12 plt.show()

```

从上面运行结果来看，可以得到以下结论：

- 测试集的均方误差（MSE）通常在 0.5-1.0 之间，表明模型对房价的预测具有一定的准确性。
- 决定系数（ R^2 ）通常在 0.5-0.7 之间，说明模型能够解释大部分房价的变化。
- 特征重要性图能够展示哪些特征对预测房价最为关键。例如，“街区收入中位数（MedInc）”通常是最重要的特征。

通过本案例，可以学习如何使用决策树模型解决回归问题。具体步骤包括数据加载与预处理、模型训练、性能评估和特征重要性分析。决策树因其直观性和可解释性，在回归问题中同样表现出色。然而，也要注意过拟合问题，可以通过调整树的最大深度或使用剪枝技术来优化模型性能。此外，特征重要性分析能够提供对数据的深入理解，能够识别出对目标变量影响最大的特征。

5.5.3 示例：特征空间的划分

决策树的核心思想是通过对特征空间进行递归划分，将数据点分配到不同的类别或回归值中。为更直观地理解这一过程，将使用一个二维合成数据集，其中每个样本有两个特征（ x_1 和 x_2 ），以及一个二分类目标变量（0 或 1）。示例的目标是训练一个决策树模型，绘制决策树的划分边界，并观察它是如何一步步将空间分割成不同区域的。

随机生成一个简单的二维数据集，其中，特征 x_1 和 x_2 是连续值，目标变量是一个二分类问题，表示样本属于类别 0 或类别 1。为便于可视化，将数据点分布在二维平面上，并用颜色区分不同的类别。

以下是完整的代码实现过程，分为数据生成、模型训练和可视化三个部分。首先，使用 scikit-learn 的 `make_moons` 函数生成一个非线性可分的二维数据集。

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import make_moons
4 from sklearn.model_selection import train_test_split
5 # 生成二维数据集
6 X, y = make_moons(n_samples=300, noise=0.2, random_state=42)
7 # 将数据集划分为训练集和测试集
8 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
9 # 可视化数据分布
10 plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.RdBu, edgecolor='k')
11 plt.title("原始数据分布")
12 plt.xlabel("特征 x1")
13 plt.ylabel("特征 x2")
14 plt.show()

```

运行上面的代码，可以看到模拟的 moon 数据：

接下来，使用 `DecisionTreeClassifier` 构建决策树模型，并在训练集上进行训练。

```

1 from sklearn.tree import DecisionTreeClassifier
2 # 初始化决策树模型
3 clf = DecisionTreeClassifier(criterion='entropy', max_depth=3, random_state=42)
4 # 训练模型
5 clf.fit(X_train, y_train)
6 print("模型训练完成")

```

为展示决策树如何划分特征空间，可以在二维平面上绘制决策边界。

```

1 from matplotlib.colors import ListedColormap
2 # 定义绘图函数
3 def plot_decision_boundary(clf, X, y):

```

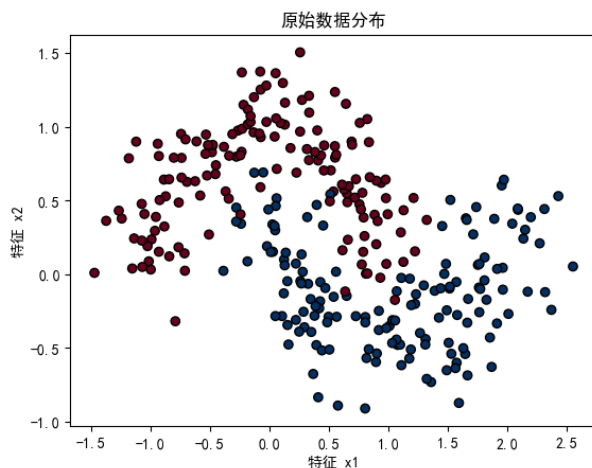


图 5.6: Moon 数据可视化

```

4  x_min, x_max = X[:, 0].min() - 0.5, X[:, 0].max() + 0.5
5  y_min, y_max = X[:, 1].min() - 0.5, X[:, 1].max() + 0.5
6  xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02),
7                        np.arange(y_min, y_max, 0.02))
8  Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
9  Z = Z.reshape(xx.shape)
10 # 绘制决策边界
11 plt.contourf(xx, yy, Z, alpha=0.4, cmap=plt.cm.RdBu)
12 plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.RdBu, edgecolor='k')
13 plt.title("决策树的特征空间划分")
14 plt.xlabel("特征 x1")
15 plt.ylabel("特征 x2")
16 plt.show()
17 # 绘制决策边界
18 plot_decision_boundary(clf, X, y)

```

运行上面的代码，可以得到如下的决策树的分割边界：

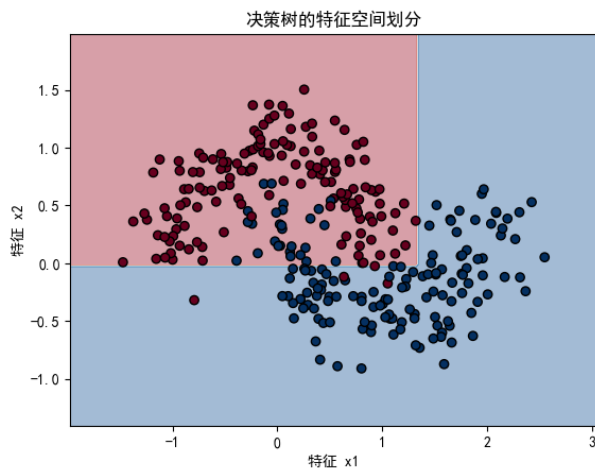


图 5.7: 决策树可视化

从上述代码运行结果来看，可以得到以下结论：

- 原始数据分布图可以展示两类数据点在二维平面上的分布情况，类别之间存在一定的重叠。
- 决策边界图能够展示决策树如何通过一系列垂直或水平的直线将特征空间划分为不同的区域。

每个区域对应一个类别（0 或 1）。

- 决策树的分裂规则会优先选择能够最大程度区分类别的特征，并通过递归划分逐步细化空间。

通过本案例，能够学习如何通过可视化方式深入理解决策树对特征空间的划分过程。具体步骤包括数据生成、模型训练和决策边界的绘制。这种直观的方法有助于更好地理解决策树的工作原理，尤其是其基于规则的递归划分特性。此外，通过调整决策树的参数（如最大深度），可以观察到划分边界的复杂度变化。例如，较深的树会产生更复杂的边界，但也可能导致过拟合；而较浅的树则会产生更简单的边界，但可能欠拟合。

5.6 习题

5.6.1 简答题

- 什么是决策树？它在分类和回归任务中的主要区别是什么？
- 决策树的构建过程中，如何选择最佳特征进行分裂？常用的分裂准则有哪些？
- 信息增益和基尼指数的主要区别是什么？它们分别适用于哪些场景？
- 什么是过拟合问题？在决策树中如何通过剪枝技术缓解过拟合？
- 决策树的优点和缺点分别是什么？在实际应用中需要注意哪些问题？

5.6.2 计算题

- 给定以下数据集，计算基于“天气”特征的信息增益和基尼指数。

表 5.8: 天气与活动数据集

天气	活动
晴天	户外活动
晴天	户外活动
雨天	室内活动
雨天	室内活动
阴天	户外活动
阴天	室内活动

- 假设一个二分类问题的数据集如下表所示，计算基于“特征 A”的信息增益。

表 5.9: 二分类问题数据集

特征 A	特征 B	类别
1	2	正类
1	3	正类
0	4	负类
0	5	负类
1	6	负类

- 在一个回归问题中，假设目标变量的真实值为 [3.0, 5.0, 7.0]，预测值为 [2.8, 5.2, 6.9]。计算均方误差（MSE）和决定系数（ R^2 ）。

5.6.3 编程题

1. 使用 scikit-learn 的 `DecisionTreeClassifier` 对鸢尾花数据集进行分类，并完成以下任务：
 - 加载数据集并划分为训练集和测试集。
 - 构建决策树模型并训练。
 - 计算测试集的分类准确率，并绘制决策树的可视化图。
2. 使用 scikit-learn 的 `DecisionTreeRegressor` 对加利福尼亚房价数据集进行回归分析，并完成以下任务：
 - 加载数据集并划分为训练集和测试集。
 - 构建决策树回归模型并训练。
 - 计算测试集的均方误差（MSE）和决定系数（ R^2 ），并绘制特征重要性图。
3. 编写一个函数，手动实现基于信息增益的特征选择算法。给定一个数据集，函数应返回每个特征的信息增益值，并选择信息增益最大的特征作为分裂点。

第六章 支持向量机

在机器学习领域，分类问题是一个核心任务，它涉及到将数据集划分为不同的类别。随着数据量的增长和复杂性的增加，传统的分类方法逐渐显示出其局限性。在这样的背景下，支持向量机（Support Vector Machine, SVM）应运而生，它是一种强大的分类模型，以其在高维空间中寻找最佳决策边界的能力而闻名。

支持向量机的概念最早可以追溯到 1963 年 Vapnik 和 Chervonenkis 提出的线性分类器理论。然而，直到 1990 年代，Vapnik 和其同事在统计学习理论的基础上进一步发展了这一理论，提出 SVM 的概念，并将其应用于非线性分类问题。SVM 的核心思想是在特征空间中找到一个超平面，这个超平面能够最大化地分开不同类别的数据点，即最大化两类数据之间的间隔。

学习目标

本章系统介绍支持向量机（SVM）的基本理论及其线性与非线性应用。理解 SVM 作为分类器的核心原理，掌握支持向量、间隔最大化、硬间隔与软间隔等关键概念，能够推导线性 SVM 的对偶问题并掌握其优化思想。进一步地，理解如何通过核函数将 SVM 扩展至非线性情形，熟悉多项式核、径向基函数核等常见核函数的性质，掌握线性不可分情况下的数学建模方法，为构建高效分类模型奠定坚实的理论基础。

6.1 水果分类问题

在机器学习的世界里，石榴同学遇到一个挑战。她正在参加一场由著名数据科学家组织的比赛，比赛的任务是根据一组水果样本进行分类，假设给定包含重量（g）和果皮红度值（0-100）两个特征的水果数据集，其中苹果 50 个（类别 0），橙子 50 个（类别 1）。图6.1中黑黄两个颜色的圆点展示两类样本的分布情况，其中存在一个重量异常的苹果样本（230g）。当采用传统分类方法（如逻辑回归）时，石榴发现那个重达 230 克的苹果样本像块磁铁，把逻辑回归的 S 型决策边界“吸”向苹果阵营（如图6.1左边蓝色直线所示），这个异常样本显著影响决策边界形态。这种“贴合式”分界面在测试集上易出现过拟合现象，因此，需要寻找一种能保持分类边界稳定性的方法。

石榴通过观察图6.1，思考到：“如果让这两个点群各自后退三步，中间留出无人区会怎样？就像操场上的拔河比赛，绳子中间的标记区域越大，越不容易被偶然的力量波动影响”。如何找到一条最优的线性决策边界，使得两类水果能够被尽可能准确地分开？这正是支持向量机（Support Vector Machine, SVM）要解决的问题，如图6.1右边所示，SVM 通过寻找位于间隔（上下紫色虚线）中间的紫色分界线作为分类面，以实现水果分类。

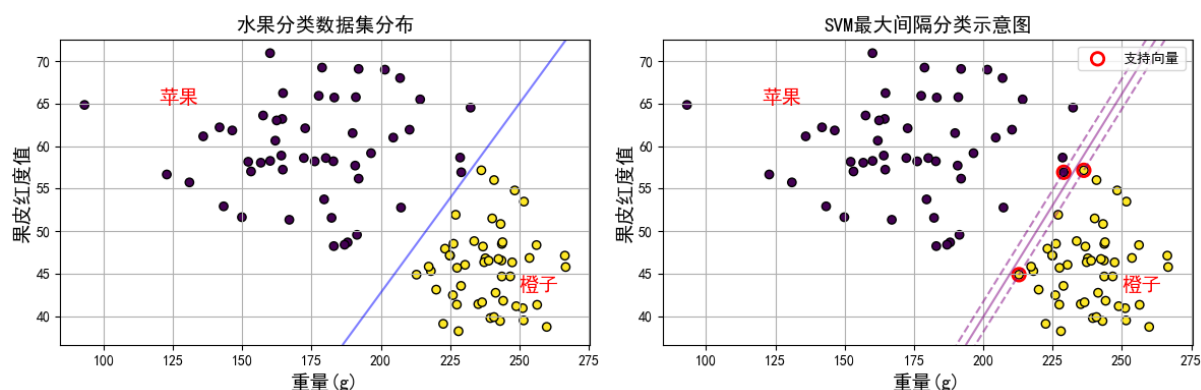


图 6.1: 水果分类问题示意图

6.2 SVM 基本概念

6.2.1 SVM 的目标

从上面水果分类的问题可以看出，支持向量机的主要目标是找到一个最优的超平面，将不同类别的数据点分开，同时最大化这个超平面两侧的间隔。如下图所示，假设现在有两类样本，黄色表示负样本，蓝色表示正样本，SVM 通过寻找一个分类面，将两类样本分开。而这样的分类存在无数个，例如下图6.2 (a) 所示，圆圈表示负样本，三角表示正样本，蓝色③、绿色②和橙色①三个分类面，都可以将正负样本分隔开，但 SVM 方法以获得橙色分类面①为目标。

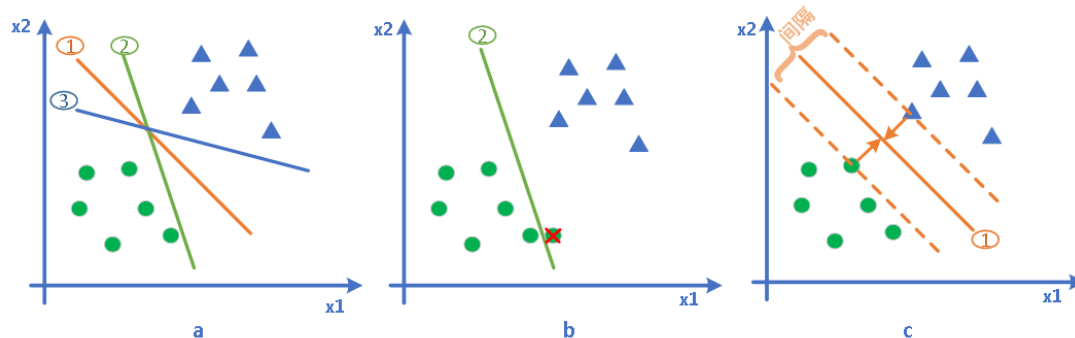


图 6.2: SVM 模型的目标

在支持向量机 (Support Vector Machine, SVM) 中，有几个核心概念对于理解其工作原理至关重要。本节将详细介绍这些概念，包括间隔、硬间隔、软间隔、核函数、支持向量和对偶问题。

6.2.2 间隔

间隔 (Margin) 是指在两类数据之间找到的最大距离。SVM 的目标是在这两类数据之间找到一个最佳的分界线 (称为超平面)，使得这个分界线两侧的最近数据点之间的距离最大。图6.2 (c) 中的橙色实线①为 SVM 的最佳分类面，两条橙色虚线之间的距离为间隔，通过最大化这个距离，SVM 可以提高模型的泛化能力，减少过拟合的风险。图6.2 (b) 中给出一个反例，如果寻找到的不是训练数据上的最大间隔的分类面，则在进行测试时，出现临近某类样本边缘的样本很容易被分错，大大降低分类面的泛化性能。

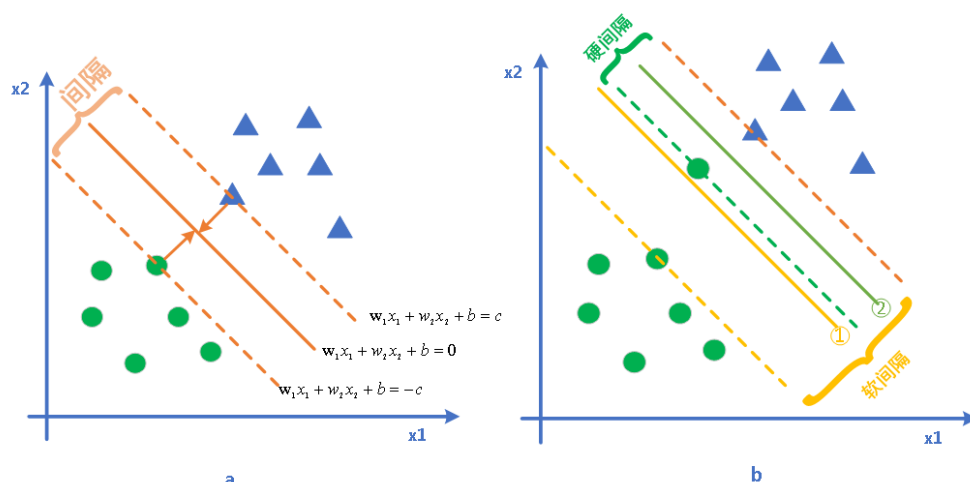


图 6.3: SVM 模型的软硬间隔

6.2.3 硬间隔

硬间隔 (Hard Margin) 是指在数据完全线性可分的情况下, 找到一个超平面, 使得所有数据点都严格位于超平面的正确一侧。SVM 通过最小化权重向量的长度来实现这一点, 同时确保每个数据点都满足一定的条件, 即每个数据点到超平面的距离至少为一个固定的值。图6.3 (a) 中给出一个实例, 实线 $w_1x_1 + w_2x_2 + b = 0$ 为 SVM 的决策分类面, 上面的虚线为 $w_1x_1 + w_2x_2 + b = c$ 称为正超平面, 下面的虚线 $w_1x_1 + w_2x_2 + b = -c$ 称为负超平面, 分类决策面位于正负超平面的中心。正负超平面之间就是所谓的硬间隔, 间隔中没有任何样本, 形成一个分类的缓冲区, 间隔越大, 缓冲区则越大, 决策面分类的效果越好。

6.2.4 软间隔

在实际应用中, 数据往往不是完全线性可分的, 可能存在噪声或异常值。软间隔允许某些数据点位于决策边界错误的一侧, 通过引入松弛变量来衡量这些“违规”数据点的程度。SVM 通过平衡间隔最大化和错误容忍度来找到一个较为合理的解决方案。这样做可以使得模型更加鲁棒, 不会因为少数异常值而过度拟合。图6.3 (b) 展示出一个软间隔实例。一个负样本 (圆圈) 出现在分类决策面①的上面, 表明该负样本被分类错误, 所以最外层上下虚线形成的间隔不是硬间隔, 是软间隔 (Soft Margin)。

6.2.5 核函数

核函数 (Kernel Function) 用于将数据从原始空间映射到一个更高维的空间, 使得在高维空间中数据更容易线性可分。常见的核函数包括:

- **线性核:** 适用于数据本身线性可分的情况。
- **多项式核:** 可以捕捉数据中的非线性关系。
- **高斯径向基核 (RBF 核):** 适用于数据分布复杂的情况, 能够映射到无限维空间。

通过使用核函数, SVM 可以在不显式计算高维空间中的坐标的情况下, 有效地处理非线性分类问题。

6.2.6 支持向量

支持向量 (Support Vectors) 是那些对超平面位置和方向有直接影响的训练数据点。在 SVM 中, 只有支持向量对最终的决策函数有贡献, 这意味着模型的复杂度主要取决于支持向量的数量, 而不是整个训练集的大小。这一特性使得 SVM 在处理大规模数据集时仍然具有较高的效率。图6.4 (a) 中, 标记为小太阳符号的样本就是支持向量, 这些支持向量能够决定 SVM 中决策分类面的位置, 而其他样本对于决策分类面没有影响, 因此, 在计算决策面时只需要使用支持向量即可, 大大降低模型的计算量。

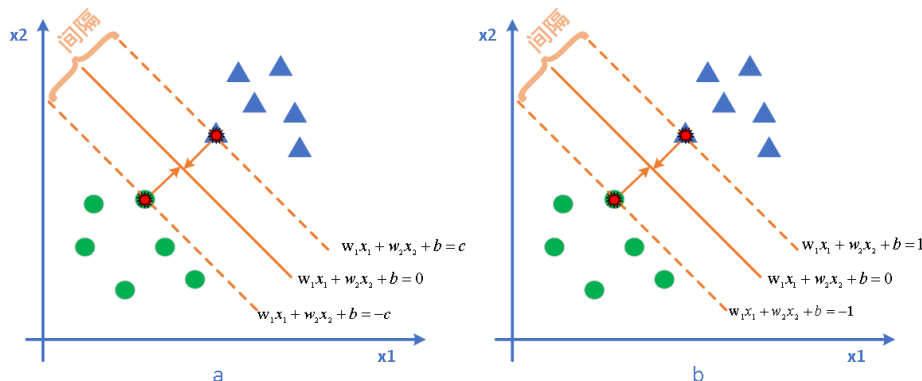


图 6.4: 支持向量和间隔示意图

6.2.7 对偶问题

对偶问题 (Dual Problem) 是 SVM 优化过程中的关键步骤。SVM 的优化目标是最大化间隔, 同时最小化分类错误。这个问题可以通过对偶问题来解决, 即将原始问题转换为一个更容易求解的形式。通过对偶问题, SVM 可以找到最优的权重向量和偏置项, 从而确定超平面的位置。决策函数的形式是基于支持向量的, 通过计算支持向量与新数据点之间的相似度 (由核函数决定), 来预测新数据点的类别。

6.3 硬间隔 SVM

本节将从 SVM 基础概念出发, 逐步引出 SVM 的目标函数, 通过对目标函数的求解, 进一步引出对偶问题及相关的 KKT 条件。

6.3.1 基础概念

SVM 的目标是根据已知的训练数据集 $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, 其中 $\mathbf{x}_i \in \mathbb{R}^d$ 是输入特征向量, $y_i \in \{-1, 1\}$ 是对应的标签, 构建一个分类器 $f(\mathbf{x})$, 使得对于新的输入 $\mathbf{x}$, 能够准确预测其标签 y 。

为进一步从数学的角度对 SVM 进行描述, 对图6.4 (a) 中正负超平面的表达式进行进一步简化。因为间隔不为 0, 所以 $c \neq 0$, 可以在正负超平面等式的两边同时都除以 c , 就变为:

$$\frac{\mathbf{w}^T}{c} \mathbf{x} + \frac{b}{c} = \pm 1 \quad (6.1)$$

因为 $\frac{\mathbf{w}}{c}$ 和 $\frac{b}{c}$ 本质就是变量，所以简单起见，使用 $\mathbf{w}$ 和 b 代替，正负超平面表达式可以表示为：

$$\mathbf{w}^T \mathbf{x} + b = \pm 1 \quad (6.2)$$

6.3.2 硬间隔 SVM 优化目标

SVM 的优化目标包括两部分，分别为目标函数和约束条件，首先来看下如何得到目标函数。

如图6.5中（a）所示，首先在正负超平面上任意取两个点，一个记为 x_m ，另一个记为 x_n ，由于两个点分别位于正负超平面上，所以满足正负超平面的方程，即：

$$\mathbf{w}^T \mathbf{x}_m + b = 1 \quad (6.3)$$

$$\mathbf{w}^T \mathbf{x}_n + b = -1 \quad (6.4)$$

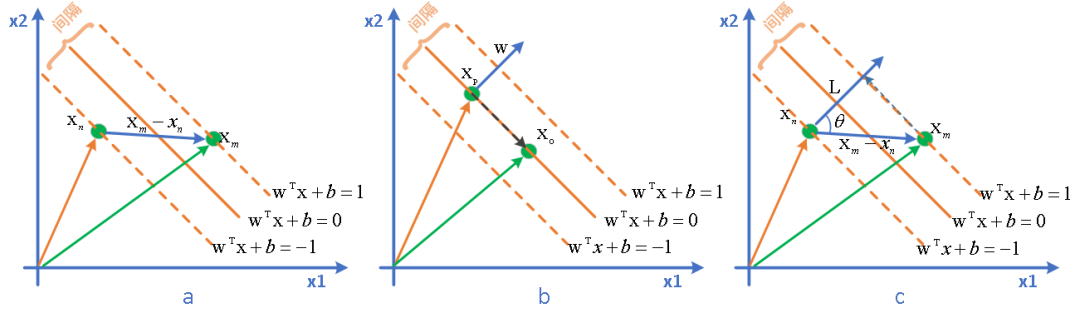


图 6.5: SVM 模型的目标函数计算

公式6.3减去公式6.4，可以得到以下表示：

$$\mathbf{w}^T (\mathbf{x}_m - \mathbf{x}_n) = 2 \quad (6.5)$$

注意，这里 $\mathbf{x}_m - \mathbf{x}_n$ 是一个向量，方向从向量 $\mathbf{x}_n$ 指向向量 $\mathbf{x}_m$ ，如图6.5（a）中所示。

同理，如图6.5（b）图所示，在分类决策面上任意选择两个不同的点 $\mathbf{x}_p$ 和 $\mathbf{x}_o$ ，由于两个点位于决策面上，所以满足决策面方程：

$$\mathbf{w}^T \mathbf{x}_p + b = 0 \quad (6.6)$$

$$\mathbf{w}^T \mathbf{x}_o + b = 0 \quad (6.7)$$

公式6.6减去公式6.7，可以得到以下表达式：

$$\mathbf{w}^T (\mathbf{x}_o - \mathbf{x}_p) = 0 \quad (6.8)$$

上面等式右边是一个内积，其中 $\mathbf{w} = (w_1; w_2; \dots; w_d)$, $\mathbf{x} = (x_1; x_2; \dots; x_d)$ ，而内积可以表示为：

$$\mathbf{w}^T (\mathbf{x}_o - \mathbf{x}_p) = \|\mathbf{w}\| \times \|\mathbf{x}_o - \mathbf{x}_p\| \times \cos \theta = 0 \quad (6.9)$$

因为 $\|\mathbf{w}\|$ 和 $\|\mathbf{x}_o - \mathbf{x}_p\|$ 为向量的模，两者都大于 0，所以 $\cos \theta = 0$ ， $\|\mathbf{w}\|$ 和 $\|\mathbf{x}_o - \mathbf{x}_p\|$ 相互垂直，即 $\|\mathbf{w}\|$ 为超平面的法向量。

同理，根据内积的定义，公式6.5重新整理为：

$$\mathbf{w}^T(\mathbf{x}_m - \mathbf{x}_n) = \|\mathbf{w}\| \times \|\mathbf{x}_m - \mathbf{x}_n\| \times \cos \theta = 2 \quad (6.10)$$

另一方面，如图6.5 (c) 所示， $\|\mathbf{x}_m - \mathbf{x}_n\| \times \cos \theta$ 就是向量 $\mathbf{x}_m - \mathbf{x}_n$ 在决策面法向量上的投影，也就是 SVM 目标中的间隔，如果令间隔为 $L = \|\mathbf{x}_m - \mathbf{x}_n\| \times \cos \theta$ ，则上式可以表示为：

$$\|\mathbf{w}\| \times \|\mathbf{x}_m - \mathbf{x}_n\| \times \cos \theta = \|\mathbf{w}\| \times L = 2 \quad (6.11)$$

所以，间隔 L 可以表示为：

$$L = \frac{2}{\|\mathbf{w}\|} \quad (6.12)$$

根据上面的分析，间隔越大，SVM 的分类能力越强，因为，SVM 的目标函数就是最大化间隔 L ：

$$\max L = \max \frac{2}{\|\mathbf{w}\|} \quad (6.13)$$

显然，预最大化间隔，等价于最小化 $\|\mathbf{w}\|$ ，上述公式可以重写为：

$$\max L \equiv \min \|\mathbf{w}\| \equiv \min \frac{1}{2} \|\mathbf{w}\|^2 \quad (6.14)$$

其中， $\|\mathbf{w}\|$ 是向量的 L2 范式（具体算法可以参考后面的补充知识章节6.14.2），有平方根存在，后面求导计算不方便，所以对其进行平方处理。前面的 $\frac{1}{2}$ 是为后面求导出现 2 倍关系正好消去使用，范式平方和 $\frac{1}{2}$ 都不会妨碍后面优化问题的求解。

接下来看下如何都得到约束条件，如图6.3所示，SVM 的目标是找到一个最优的超平面，将不同类别的数据点分开，同时最大化这个超平面两侧的间隔。如果样本 $\mathbf{x}_i$ 是正样本，则该样本应该位于正超平面的上方，则应满足：

$$\mathbf{w}^T \mathbf{x}_i + b \geq 1 \quad (6.15)$$

如果 $\mathbf{x}_i$ 是负样本，则负样本应该位于负超平面的下方，则满足：

$$\mathbf{w}^T \mathbf{x}_i + b \leq -1 \quad (6.16)$$

如果 $y_i = 1$ 表示该样本为正样本，满足公式6.15，显然 $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$ 。如果 $y_i = -1$ 表示该样本为负样本，满足公式6.16，同样有 $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$ 。所以，将这两个条件合并，可以得到：

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad i = 1, 2, \dots, n \quad (6.17)$$

最后，合并 SVM 的目标函数和约束条件，可以得到 SVM 优化目标为：

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{s.t.} \quad & y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1, \quad i = 1, 2, \dots, n \end{aligned} \quad (6.18)$$

6.4 凸优化直接求解

根据凸优化理论（参考补充知识中凸优化理论6.14.3），SVM 的目标函数（公式6.12）是凸函数，并且约束条件形成的定义域为凸集，所以 SVM 的优化目标是一个凸优化问题，而凸优化问题可以通过以下的方法进行求解。

6.4.1 梯度下降法

梯度下降法是一种常用的优化方法，通过迭代更新参数来最小化目标函数。每一步中，计算目标函数的梯度，并根据梯度的方向和步长（学习率 η ）更新参数。在每一步更新后，需要检查约束条件是否满足，如果不满足，则调整参数以满足约束条件。这种方法简单且易于实现，但可能需要较多的迭代次数才能收敛。

表 6.1: 梯度下降法

步骤	操作
1. 初始化	初始化 $\mathbf{w}$ 和 b
2. 计算梯度	$\nabla_{\mathbf{w}}f(\mathbf{w}, b) = \mathbf{w}$
3. 更新参数	$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla_{\mathbf{w}}f(\mathbf{w}, b)$ $b \leftarrow b - \eta \nabla_b f(\mathbf{w}, b)$
4. 检查约束条件	如果约束条件不满足，调整参数以满足约束条件
5. 重复步骤	重复上述步骤直到收敛

其中， η 为学习率， $\nabla_{\mathbf{w}}f(\mathbf{w}, b)$ 为目标函数关于 $\mathbf{w}$ 的梯度， $\nabla_b f(\mathbf{w}, b)$ 表示目标函数关于 b 的梯度。

6.4.2 牛顿法

牛顿法是一种二阶优化方法，通过利用目标函数的二阶导数（Hessian 矩阵）来加速收敛。每一步中，计算目标函数的梯度和 Hessian 矩阵，并根据这些信息更新参数。牛顿法通常比梯度下降法收敛更快，但计算 Hessian 矩阵及其逆矩阵可能会增加计算复杂度。

表 6.2: 牛顿法

步骤	操作
1. 初始化	初始化 $\mathbf{w}$ 和 b
2. 计算梯度和 Hessian 矩阵	$\nabla_{\mathbf{w}}f(\mathbf{w}, b) = \mathbf{w}$ $\nabla_{\mathbf{w}}^2 f(\mathbf{w}, b) = I$
3. 更新参数	$\mathbf{w} \leftarrow \mathbf{w} - (\nabla_{\mathbf{w}}^2 f(\mathbf{w}, b))^{-1} \nabla_{\mathbf{w}}f(\mathbf{w}, b)$ $b \leftarrow b - \eta \nabla_b f(\mathbf{w}, b)$
4. 检查约束条件	如果约束条件不满足，调整参数以满足约束条件
5. 重复步骤	重复上述步骤直到收敛

其中， η 表示学习率， $\nabla_{\mathbf{w}}f(\mathbf{w}, b)$ 为目标函数关于 $\mathbf{w}$ 的梯度， $\nabla_b f(\mathbf{w}, b)$ 是目标函数关于 b 的梯度， $\nabla_{\mathbf{w}}^2 f(\mathbf{w}, b)$ 表示目标函数关于 $\mathbf{w}$ 的 Hessian 矩阵

6.4.3 内点法

内点法是一种处理带约束优化问题的有效方法，通过引入障碍函数来处理约束条件。每一步中，定义一个障碍函数，该函数在接近约束边界时趋于无穷大，从而引导优化过程远离不可行区域。通过逐步减小障碍参数 t ，使优化过程逐渐逼近原问题的解。内点法在处理大规模带约束优化问题时表现良好，但需要仔细选择障碍参数的更新策略。

其中， t 表示障碍参数， $B(\mathbf{w}, b, t)$ 表示障碍函数， $\mathbf{x}_i$ 是第 i 个样本的特征向量， y_i 为第 i 个样本的标签。

表 6.3: 内点法

步骤	操作
1. 定义障碍函数	$B(\mathbf{w}, b, t) = \frac{1}{2} \ \mathbf{w}\ ^2 - \frac{1}{t} \sum_{i=1}^n \log(y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1)$
2. 初始化	初始化 $\mathbf{w}$ 和 b , 选择初始障碍参数 t_0
3. 最小化障碍函数	通过梯度下降或牛顿法最小化障碍函数 $B(\mathbf{w}, b, t)$
4. 减小障碍参数	逐渐减小障碍参数 t , 重复上述步骤直到 t 足够小

通过上述方法, 可以直接通过凸优化理论求解 SVM 的原问题。虽然对偶求解方法在 SVM 中更为常用, 因为对偶问题往往更易于求解, 但直接使用凸优化方法也是一个有效的途径。特别是当数据集较小或约束条件较简单时, 直接求解原问题可能更加高效。

6.5 对偶问题

SVM 优化目标 (公式6.18) 是凸优化问题, 虽然可以直接通过凸优化求解工具进行求解, 但是考虑到计算效率及后面核函数的运用, 一般转化为对偶问题进行求解。

6.5.1 原问题到对偶问题

根据拉格朗日理论 (参考补充知识6.14.3), 为了将约束条件纳入目标函数中, 引入拉格朗日乘数 α_i , 构造拉格朗日函数:

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i [y_i(\mathbf{w} \cdot \mathbf{x}_i + b) - 1] \quad (6.19)$$

其中, $\boldsymbol{\alpha} = [\alpha_1, \alpha_2, \dots, \alpha_n]^T$ 是拉格朗日乘数向量。

1. 求解拉格朗日函数的极值

为求解拉格朗日函数的极值, 需要对 $\mathbf{w}$ 和 b 求偏导数, 并令其等于零:

$$\nabla_{\mathbf{w}} \mathcal{L} = \mathbf{w} - \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i = 0 \quad (6.20)$$

$$\nabla_b \mathcal{L} = - \sum_{i=1}^n \alpha_i y_i = 0 \quad (6.21)$$

从第一个方程中解得:

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \quad (6.22)$$

从第二个方程中解得:

$$\sum_{i=1}^n \alpha_i y_i = 0 \quad (6.23)$$

2. 对偶问题

将 $\mathbf{w}$ 的表达式代入拉格朗日函数 $\mathcal{L}$ 中, 得到对偶问题的目标函数:

$$\mathcal{L}(\boldsymbol{\alpha}) = \frac{1}{2} \left(\sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \right)^T \left(\sum_{j=1}^n \alpha_j y_j \mathbf{x}_j \right) - \sum_{i=1}^n \alpha_i \left[y_i \left(\sum_{j=1}^n \alpha_j y_j \mathbf{x}_j \cdot \mathbf{x}_i + b \right) - 1 \right] \quad (6.24)$$

简化上述表达式：

$$\mathcal{L}(\alpha) = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j - \sum_{i=1}^n \alpha_i \left[y_i \left(\sum_{j=1}^n \alpha_j y_j \mathbf{x}_j \cdot \mathbf{x}_i + b \right) - 1 \right] \quad (6.25)$$

进一步简化：

$$\mathcal{L}(\alpha) = \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j - \sum_{i=1}^n \alpha_i y_i \left(\sum_{j=1}^n \alpha_j y_j \mathbf{x}_j \cdot \mathbf{x}_i + b \right) + \sum_{i=1}^n \alpha_i \quad (6.26)$$

利用 $\sum_{i=1}^n \alpha_i y_i = 0$ ，可以消去中间项：

$$\mathcal{L}(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \quad (6.27)$$

因此，对偶问题可以表示为：

$$\max_{\alpha} W(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \quad (6.28)$$

同时满足以下约束条件：

$$\sum_{i=1}^n \alpha_i y_i = 0 \quad (6.29)$$

$$\alpha_i \geq 0, \quad i = 1, 2, \dots, n \quad (6.30)$$

6.5.2 KKT 条件

KKT (Karush-Kuhn-Tucker) 条件是一组必要条件，用于确保在带有约束条件的优化问题中找到的解是最优解。基于上面的推导过程，对于 SVM 的对偶问题，KKT 条件包括以下几部分：

1. 原问题的可行性条件确保原问题的约束条件得到满足，即

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1, \quad i = 1, 2, \dots, n \quad (6.31)$$

2. 对偶问题的可行性条件确保对偶问题的约束条件得到满足，即

$$\sum_{i=1}^n \alpha_i y_i = 0 \quad (6.32)$$

和

$$\alpha_i \geq 0, \quad i = 1, 2, \dots, n \quad (6.33)$$

3. 互补松弛条件确保每个约束条件要么是活跃的（即紧的），要么对应的拉格朗日乘数为零，即

$$\alpha_i [y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1] = 0, \quad i = 1, 2, \dots, n \quad (6.34)$$

这表明如果某个样本不在支持向量上（即 $y_i(\mathbf{w}^T \mathbf{x}_i + b) > 1$ ），那么对应的 α_i 必须为零；反之，如果 $\alpha_i > 0$ ，则该样本必须是一个支持向量，即 $y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1$ 。

4. 梯度条件确保目标函数的梯度为零，这是优化问题中找到极值的必要条件，即

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \quad (6.35)$$

KKT 条件是原问题最优解的必要条件，如果一个解是原问题的最优解，那么它必须满足 KKT 条件。通过验证 KKT 条件，可以确认解的正确性和有效性，特别是在凸优化问题中，KKT 条件既是必要条件也是充分条件。KKT 条件在 SVM 的对偶问题求解中起着关键作用，确保在求解对偶问题时找到的解 α 是原问题的最优解。

通过求解对偶问题，可以找到最优的 α_i 值。一旦得到 α_i ，可以通过以下公式计算 $\mathbf{w}$ 和 b ：

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \quad (6.36)$$

$$b = y_k - \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i^T \mathbf{x}_k \quad (6.37)$$

其中， k 是任意一个支持向量的索引，即 $\alpha_k > 0$ 。

6.6 SMO 算法求解对偶问题

正如本节开头所述，通过将原始问题转化为对偶问题，可以通过其他高效的方法对其进行求解，其中 SMO (Sequential Minimal Optimization) 算法就是一种高效求解 SVM 对偶问题的方法。SMO 算法的核心思想是将复杂的优化问题分解为一系列简单的子问题。每次选择两个拉格朗日乘子（对偶变量）进行优化，固定其他变量。这种策略不仅简化了计算，而且能够保证每一步都能有效地接近全局最优解。下面是 SMO 算法的详细求解过程：

1. 初始化

- 1) 初始化对偶变量：选择初始的 α_i 值，通常可以随机初始化或全部设为零。
- 2) 初始化误差缓存：计算每个样本点的预测误差 $E_i = f(\mathbf{x}_i) - y_i$ ，其中 $f(\mathbf{x}_i) = \sum_{j=1}^n \alpha_j y_j \mathbf{x}_j^T \mathbf{x}_i + b$ 。

2. 选择变量

- 1) 外层循环：遍历所有样本点，选择违反 KKT 条件最严重的样本点作为第一个变量 α_1 。
- 2) 内层循环：选择第二个变量 α_2 。选择标准通常是使 α_2 的变化最大，即选择使 $|E_1 - E_2|$ 最大的样本点。

3. 计算边界

计算 α_2 的上下界：如果 $y_1 \neq y_2$ ，则：

$$L = \max(0, \alpha_2 - \alpha_1), \quad H = \min(C, C + \alpha_2 - \alpha_1) \quad (6.38)$$

如果 $y_1 = y_2$ ，则：

$$L = \max(0, \alpha_2 + \alpha_1 - C), \quad H = \min(C, \alpha_2 + \alpha_1) \quad (6.39)$$

4. 更新变量

1) 计算 α_2 的新值:

$$\alpha_2^{\text{new, unc}} = \alpha_2 + \frac{y_2(E_1 - E_2)}{\eta} \quad (6.40)$$

其中 $\eta = K_{11} + K_{22} - 2K_{12}$, $K_{ij} = \mathbf{x}_i^T \mathbf{x}_j$ 。

2) 将 α_2 裁剪到边界:

$$\alpha_2^{\text{new}} = \begin{cases} H & \text{if } \alpha_2^{\text{new, unc}} > H \\ \alpha_2^{\text{new, unc}} & \text{if } L \leq \alpha_2^{\text{new, unc}} \leq H \\ L & \text{if } \alpha_2^{\text{new, unc}} < L \end{cases} \quad (6.41)$$

3) 更新 α_1 :

$$\alpha_1^{\text{new}} = \alpha_1 + y_1 y_2 (\alpha_2 - \alpha_2^{\text{new}}) \quad (6.42)$$

5. 更新偏置 b

1) 计算新的偏置 b_1 和 b_2 :

$$\begin{aligned} b_1 &= b - E_1 - y_1(\alpha_1^{\text{new}} - \alpha_1)K_{11} - y_2(\alpha_2^{\text{new}} - \alpha_2)K_{12} \\ b_2 &= b - E_2 - y_1(\alpha_1^{\text{new}} - \alpha_1)K_{12} - y_2(\alpha_2^{\text{new}} - \alpha_2)K_{22} \end{aligned} \quad (6.43)$$

2) 选择新的偏置 b :

$$b^{\text{new}} = \begin{cases} b_1 & \text{if } 0 < \alpha_1^{\text{new}} < C \\ b_2 & \text{if } 0 < \alpha_2^{\text{new}} < C \\ \frac{b_1 + b_2}{2} & \text{otherwise} \end{cases} \quad (6.44)$$

6. 检查收敛

1) 检查 KKT 条件: 如果所有 α_i 都满足 KKT 条件, 则算法终止。

2) 更新误差缓存: 更新所有样本点的预测误差 E_i 。

7. 重复外层和内层循环继续选择新的变量对进行优化, 直到所有变量都满足 KKT 条件。

6.7 软间隔 SVM

在实际应用中, 数据往往不是完全线性可分的, 可能存在噪声或异常值, 如下图6.6 (a) 所示, 图中共有两类样本, 正样本使用三角形表示, 负样本使用圆形表示, 其中一个负样本出现在间隔区域内, 另一个负样本出现在决策面的上方, 这种情况就属于软间隔, 这里的“软”表示只允许少量的样本犯错。

1. $L_{0/1}$ 损失

为处理这种情况, 一个简单的方法就是引入一个计数器, 该计数器记录犯错样本的个数, 只要样本不符合上述的要求, 就在错误计数器上加 1。参考公式 (6.17), 回忆一下, 如果样本被分类正确, 则有 $y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \geq 0$ 。与之相对, 如果一个样本犯错, 即该样本落入间隔之间, 或者被错误分类, 则有 $y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 < 0$, 所以目标函数在该样本上扣除 1 分, 在数学上表示就是:

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n L_{0/1}(y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1) \quad (6.45)$$

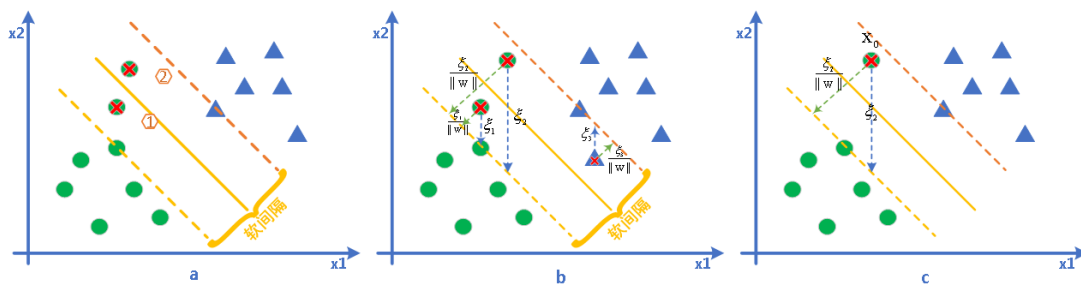


图 6.6: SVM 软间隔

这里的 $L_{0/1}$ 就是上面提到的错误样本的计数器:

$$L_{0/1} = \begin{cases} 1, & \text{if } z < 0; \\ 0, & \text{other} \end{cases} \quad (6.46)$$

从公式上看, 如果 $z < 0$, 则记错一次, 公式6.45中的 $z = y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1$, $z < 0$ 即 $z = y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 < 0$, $y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 < 0$ 表示样本犯错, 所以 $l_{0/1}$ 损失记录的只是犯错样本的个数, 就是前面所说的错误样本的计数器。

2. Hinge 损失

上述提到的方法简单, 并且容易理解, 但是缺点也明显。对于犯错样本, 无论样本犯错多还是犯错少, 都会被计数, 样本之间犯错的大小不会被区分对待, 这与直觉不太一致。另一方面, 从数学的角度来说, $L_{0/1}$ 损失函数不连续、非凸函数, 因此一般不会直接使用它, 而是使用它的“替代品”。

其中常用的一种方式就是使用 hinge 函数, 该函数的表达式如下:

$$L(z) = \max(0, 1 - z) \quad (6.47)$$

令上面公式 $z = y_i(\mathbf{w}^T \mathbf{x}_i + b)$, 如果公式6.47中 $L(z) = 0$, 则说明 $1 - z < 0$, 即 $1 - y_i(\mathbf{w}^T \mathbf{x}_i + b) < 0$, 表示样本 x_i 是被正确分类。反之, 如果 $L(z) > 0$, 则 $1 - z > 0$, 即 $1 - z = 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b) > 0$, 推出 $y_i(\mathbf{w}^T \mathbf{x}_i + b) < 1$, 样本被分类错误, 此时,

$$L(z) = 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b) \quad (6.48)$$

上式和 $L_{0/1}$ 中 $z < 0$ 的情况不同, 对于 $L_{0/1}$ 函数, 当 $z < 0$ 的时候, 永远是 1, 而 $1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)$ 会随着点 x_i 取值的不同而发生编号, 所以是一个可变的数据, 在一定程度上可以弥补 $L_{0/1}$ 函数中不区分犯错大小的缺点。另一方面, hinge 损失函数是一个连续函数, 性质要比 $L_{0/1}$ 函数好, 弥补了上述 $L_{0/1}$ 数学性质不好的另一个缺点。

6.8 SVM 直观理解

从直觉上看, 如图6.6 (b) 所示, 对于任意一个样本 x_i , 引入一个对应的 $\xi_i = \max(0, 1 - z)$, 如果该样本成功分类正确, 即正样本位于正超平面的上方, 负样本位于负超平面的下方, 则 $\xi_i = 0$, 反之, 如果样本不符合上述要求, 例如子图 (b) 的负样本 x_1 位于软间隔内容, x_2 位于分类决策面的上方, 则 ξ_i 的大小可以定义为该样本点到垂直到负超平面的距离, 其到负超平面的距离为 $\frac{\xi_i}{\|\mathbf{w}\|}$, 对于正样本 x_3 , 同样成立。要充分理解图6.6 (b) 中的含义, 需要了解以下几个知识点:

6.8.1 点到超平面的距离

假设点 $\mathbf{x}_0$ 到超平面 $\mathbf{w}^T \mathbf{x} + b = 0$ 之间的距离 d 可以用以下公式表示：

$$d = \frac{|\mathbf{w}^T \mathbf{x}_0 + b|}{\|\mathbf{w}\|} \quad (1)$$

其中， $\|\mathbf{w}\|$ 表示法向量 $\mathbf{w}$ 的欧几里得范数（L2 范数），是向量长度的一种衡量方式，计算公式具体参考补充知识6.14.2。

为方便理解，在此对上面的结论进行简单证明。首先，超平面 $\mathbf{w}^T \mathbf{x} + b = 0$ 的法向量是 $\mathbf{w}$ ，显然，法向量 $\mathbf{w}$ 与该超平面垂直。关于为何法向量和超平面垂直，可以参考前面公式（6.7）到公式（6.8）。

其次，假设 $\mathbf{x}_1$ 是超平面上的任意一点，则 $\mathbf{x}_1$ 满足：

$$\mathbf{w}^T \mathbf{x}_1 + b = 0 \quad (6.49)$$

从而：

$$\mathbf{w}^T \mathbf{x}_1 = -b \quad (6.50)$$

而从点 $\mathbf{x}_0$ 到点 $\mathbf{x}_1$ 的向量是 $\mathbf{x}_1 - \mathbf{x}_0$ 。

然后，点 $\mathbf{x}_0$ 到超平面的垂直距离 d 等于向量 $\mathbf{x}_1 - \mathbf{x}_0$ 在法向量 $\mathbf{w}$ 上的投影长度。这个投影长度可以表示为

$$d = \frac{|\mathbf{w}^T (\mathbf{x}_1 - \mathbf{x}_0)|}{\|\mathbf{w}\|} \quad (6.51)$$

最后，由于 $\mathbf{x}_1$ 在超平面上，有 $\mathbf{w}^T \mathbf{x}_1 = -b$ 。代入上式：

$$\mathbf{w}^T (\mathbf{x}_1 - \mathbf{x}_0) = \mathbf{w}^T \mathbf{x}_1 - \mathbf{w}^T \mathbf{x}_0 = -b - \mathbf{w}^T \mathbf{x}_0 \quad (6.52)$$

因此：

$$d = \frac{|-b - \mathbf{w}^T \mathbf{x}_0|}{\|\mathbf{w}\|} = \frac{|\mathbf{w}^T \mathbf{x}_0 + b|}{\|\mathbf{w}\|} \quad (6.53)$$

6.8.2 平行超平面之间的距离

在多维空间中，平行超平面之间的距离可以通过它们的法向量和截距来表示。假设有两个平行的超平面，它们的方程分别为 $\mathbf{w}^T \mathbf{x} + b_1 = 0$ 和 $\mathbf{w}^T \mathbf{x} + b_2 = 0$ ，则两者之间的距离为：

$$d = \frac{|b_1 - b_2|}{\|\mathbf{w}\|} \quad (6.54)$$

负超平面方程为 $\mathbf{w}^T \mathbf{x} + b = -1$ ，而正超平面表达式为 $\mathbf{w}^T \mathbf{x} + b = 1$ ，根据平行平面之间距离的公式，两者之间的距离为：

$$L = \frac{|-1 - 1|}{\|\mathbf{w}\|} = \frac{2}{\|\mathbf{w}\|} \quad (6.55)$$

显然，两者之间的距离就是上面所说的间隔，和前面公式6.12推导的结果一致。

6.8.3 点沿某维度到超平面距离

如图6.6（c）中 ξ 所示，假设超平面方程为 $\mathbf{w}^T \mathbf{x} + b = 0$ ，注意， b 是截距，表示超平面在维度 x_2 上距离原点的距离。超平面外的点坐标为 x_0 ，沿该点平行于负超平面画一个新的超平面，假设新

超平面的方程是

$$\mathbf{w}^T \mathbf{x} + \hat{b} = 0 \quad (6.56)$$

$\hat{b}$ 表示该新超平面在 x_2 截距。则两超平面在维度 x_2 上距离为

$$\xi = |b - \hat{b}| \quad (6.57)$$

由于点 x_0 位于新超平面上，所以满足新超平面的方程，即 $\mathbf{w}^T \mathbf{x}_0 + \hat{b} = 0$ ，所以

$$\hat{b} = -\mathbf{w}^T \mathbf{x}_0 \quad (6.58)$$

将公式6.58带入到公式6.57中，得到：

$$\xi = |b - \hat{b}| = |\mathbf{w}^T \mathbf{x}_0 + b| \quad (6.59)$$

以上就证明图6.6 (c) 中 ξ 是如何的来的。这里的 ξ 如果经过进一步的约束，就可以转化为 hinge 损失：

$$L(y_i, f(\mathbf{x}_i)) = \max(0, 1 - y_i f(\mathbf{x}_i)) = \max(0, 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)) \quad (6.60)$$

其中，铰链损失函数（Hinge 函数）实际上是对松弛变量 ξ_i 的一种隐式表示。具体来说，如果 $y_i(\mathbf{w}^T \mathbf{x}_i + b) \leq 1$ ，则铰链损失为 0，表示该样本点正确分类且在分类边界之外；如果 $y_i(\mathbf{w}^T \mathbf{x}_i + b) < 1$ ，则铰链损失为 $1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)$ ，表示该样本点在分类边界的一侧，且损失值为 $1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)$ 。注意这里损失值 $1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)$ 和上面公式（6.59）看上去好像不一样，其本质是一样的。

首先，负超平面方程为 $\mathbf{w}^T \mathbf{x} + b + 1 = 0$ ，假设存在一个负样本 $\mathbf{x}_i$ ，根据公式6.59，该样本 $\mathbf{x}_i$ 到负超平面之间的竖直距离为：

$$\xi_i = |\mathbf{w}^T \mathbf{x}_i + b + 1| \quad (6.61)$$

其次，在 Hinge 函数中，如果样本 x_i 为负样本，则 $y_i = -1$ ，带入损失值：

$$\xi_i = 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1 - (-1)(\mathbf{w}^T \mathbf{x}_i + b) = 1 + (\mathbf{w}^T \mathbf{x}_i + b) = |\mathbf{w}^T \mathbf{x}_i + b + 1| \quad (6.62)$$

同理，如果是正样本的话，也可以推出相同的结论，所以，在 Hinge 函数中，“犯错”样本的损失值可以使用该点到对应超平面在某个坐标轴的距离来表示，即负样本到负超平面的距离，正样本到正超平面的距离。该距离也就还有另外一个名字，就是“松弛因子”。更进一步，这里的距离其实和样本到对应超平面的垂直距离也有关系，垂直距离就是该距离在法向量方向的投影。假设“犯错”负样本为 $\mathbf{x}_i$ ，根据公式6.53，其到负超平面 $\mathbf{w}^T \mathbf{x} + b + 1 = 0$ 的距离为：

$$d = \frac{|\mathbf{w}^T \mathbf{x} + b + 1|}{\|\mathbf{w}\|} = \frac{\xi_i}{\|\mathbf{w}\|} \quad (6.63)$$

这和图6.6 (b) 和 (c) 中“犯错”样本到对应超平面的距离 $\frac{\xi_i}{\|\mathbf{w}\|}$ 是一致的。

6.9 软间隔 SVM 求解

在软间隔支持向量机 (Soft Margin Support Vector Machine) 中, 当数据集不是完全线性可分时, 允许一些样本点位于决策边界错误的一侧。原始优化问题可以表述为:

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t. } & y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \\ & \xi_i \geq 0, i = 1, 2, \dots, n, \end{aligned} \quad (6.64)$$

其中, $\mathbf{w}$ 是权重向量, b 是偏置项, ξ_i 是松弛变量, 用于衡量第 i 个样本点的容许错误程度, $C > 0$ 是正则化参数, 用来平衡间隔大小与分类错误之间的权衡, $y_i \in \{-1, 1\}$ 是第 i 个样本的标签, $\mathbf{x}_i$ 是第 i 个样本的特征向量。

1. 拉格朗日函数

类似硬间隔问题, 为将原始问题转化为对偶问题, 引入拉格朗日乘数 α_i 和 r_i , 定义拉格朗日函数为:

$$L(\mathbf{w}, b, \xi, \alpha, r) = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i - \sum_{i=1}^n \alpha_i [y_i(\mathbf{w}^T \mathbf{x}_i + b) - (1 - \xi_i)] - \sum_{i=1}^n r_i \xi_i, \quad (6.65)$$

其中, $\alpha_i \geq 0$ 是针对不等式约束 $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i$ 的拉格朗日乘数, $r_i \geq 0$ 是针对不等式约束 $\xi_i \geq 0$ 的拉格朗日乘数。

2. 对偶问题

通过对拉格朗日函数进行极小化操作, 可以得到对偶问题。首先, 对 $\mathbf{w}$ 、 b 和 ξ_i 进行偏导数并令其等于零, 得到:

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{w}} &= \mathbf{w} - \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i = 0 \implies \mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i, \\ \frac{\partial L}{\partial b} &= - \sum_{i=1}^n \alpha_i y_i = 0 \implies \sum_{i=1}^n \alpha_i y_i = 0, \\ \frac{\partial L}{\partial \xi_i} &= C - \alpha_i - r_i = 0 \implies \alpha_i + r_i = C. \end{aligned} \quad (6.66)$$

由于 $r_i \geq 0$, 因此 $0 \leq \alpha_i \leq C$ 。将 $\mathbf{w}$ 和 b 的表达式代入拉格朗日函数中, 得到对偶问题:

$$\begin{aligned} \max_{\alpha} & \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n y_i y_j \alpha_i \alpha_j \mathbf{x}_i^T \mathbf{x}_j, \\ \text{s.t. } & 0 \leq \alpha_i \leq C, \quad i = 1, 2, \dots, n, \\ & \sum_{i=1}^n y_i \alpha_i = 0. \end{aligned} \quad (6.67)$$

3. 计算 $\mathbf{w}$ 和 b

一旦求解出 α_i , 可以通过以下公式计算 $\mathbf{w}$ 和 b :

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i. \quad (6.68)$$

对于偏置 b ，通常选择一个或几个支持向量（即满足 $0 < \alpha_i < C$ 的样本点）来计算，具体为：

$$b = y_k - \mathbf{w}^T \mathbf{x}_k, \quad \text{for some } k \text{ such that } 0 < \alpha_k < C. \quad (6.69)$$

6.10 支持向量回归

支持向量回归（Support Vector Regression, SVR）是支持向量机的一种扩展，用于解决回归问题。与传统的回归方法不同，SVR 通过构造一个间隔带（epsilon-insensitive tube），使得落在间隔带内的预测误差不被惩罚，从而提高模型的泛化能力。

6.10.1 SVR 原始问题

在 SVR 中，给定一组训练样本 $(\mathbf{x}_i, y_i)$ ，其中 $\mathbf{x}_i \in \mathbb{R}^d$ 是输入特征向量， $y_i \in \mathbb{R}$ 是目标值，目标是找到一个函数 $f(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x}) + b$ ，使得预测值与真实值之间的误差尽可能小。这里的 $\phi(\mathbf{x})$ 是一个映射函数，将输入特征向量映射到高维空间。SVR 的核心思想是构造一个间隔带，使得落在间隔带内的预测误差不被惩罚，使用间隔带外的数据进行拟合。首先要构造出一个间隔带，在该间隔带的内部的样本点的损失应该为 0，在间隔带外样本才计算与目标值之间的差异，所以，SVR 使用的是 ϵ -不敏感损失函数（ ϵ -insensitive loss function）。这个损失函数定义了一个宽度为 2ϵ 的间隔带（tube），在这个间隔带内的预测误差被视为零，不会受到惩罚。具体来说，损失函数定义为：

$$L_\epsilon(y, f(\mathbf{x})) = \begin{cases} 0 & \text{if } |y - f(\mathbf{x})| \leq \epsilon, \\ |y - f(\mathbf{x})| - \epsilon & \text{otherwise.} \end{cases} \quad (6.70)$$

为在优化问题中明确表示正向和负向误差，SVR 引入两个松弛变量 ξ_i 和 ξ_i^* ：

- ξ_i 表示正向误差，即当预测值 $f(\mathbf{x}_i)$ 小于真实值 y_i 且超出 ϵ 时的误差。
- ξ_i^* 表示负向误差，即当预测值 $f(\mathbf{x}_i)$ 大于真实值 y_i 且超出 ϵ 时的误差。

通过引入这两个松弛变量，SVR 的优化问题可以更精确地表示预测值与真实值之间的误差。具体来说，优化问题的约束条件变为：

$$\begin{aligned} y_i - (\mathbf{w}^T \mathbf{x}_i + b) &\leq \epsilon + \xi_i, \\ (\mathbf{w}^T \mathbf{x}_i + b) - y_i &\leq \epsilon + \xi_i^*, \\ \xi_i &\geq 0, \quad \xi_i^* \geq 0. \end{aligned} \quad (6.71)$$

这些约束条件能够确保：

- 当预测值 $f(\mathbf{x}_i)$ 小于真实值 y_i 且超出 ϵ 时， ξ_i 会大于零，表示正向误差。
- 当预测值 $f(\mathbf{x}_i)$ 大于真实值 y_i 且超出 ϵ 时， ξ_i^* 会大于零，表示负向误差。

引入两个松弛变量 ξ_i 和 ξ_i^* 使得模型具有对称性和灵活性。这样，模型可以更灵活地处理不同方向的误差，而不仅仅是单一方向的误差。这有助于提高模型的鲁棒性和泛化能力。综上，SVR 的

原始优化问题可以表述为：

$$\begin{aligned} \min_{\mathbf{w}, b, \xi, \xi^*} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*) \\ \text{s.t.} \quad & \begin{cases} y_i - (\mathbf{w}^T \mathbf{x}_i + b) \leq \epsilon + \xi_i, \\ (\mathbf{w}^T \mathbf{x}_i + b) - y_i \leq \epsilon + \xi_i^*, \\ \xi_i \geq 0, \quad \xi_i^* \geq 0, \quad i = 1, 2, \dots, n, \end{cases} \end{aligned} \quad (6.72)$$

其中， $\mathbf{w}$ 是权重向量， b 是偏置项， ξ_i 和 ξ_i^* 是松弛变量，用于衡量第 i 个样本点的容许误差， $C > 0$ 表示正则化参数，用来平衡模型复杂度与训练误差之间的权衡， ϵ 是不敏感损失函数的参数，定义了间隔带的宽度。

6.10.2 SVR 对偶问题

通过拉格朗日乘数法，可以将原始问题转化为对偶问题。定义拉格朗日函数为：

$$\begin{aligned} L(\mathbf{w}, b, \xi, \xi^*, \alpha, \alpha^*, r, r^*) = & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*) \\ & - \sum_{i=1}^n \alpha_i [y_i - (\mathbf{w}^T \mathbf{x}_i + b) - \epsilon - \xi_i] \\ & - \sum_{i=1}^n \alpha_i^* [(\mathbf{w}^T \mathbf{x}_i + b) - y_i - \epsilon - \xi_i^*] \\ & - \sum_{i=1}^n r_i \xi_i - \sum_{i=1}^n r_i^* \xi_i^* \end{aligned} \quad (6.73)$$

其中， $\alpha_i \geq 0$ 和 $\alpha_i^* \geq 0$ 是针对不等式约束的拉格朗日乘数， $r_i \geq 0$ 和 $r_i^* \geq 0$ 是针对不等式约束的拉格朗日乘数。通过对拉格朗日函数进行极小化操作，可以得到对偶问题：

$$\begin{aligned} \max_{\alpha, \alpha^*} \quad & \sum_{i=1}^n y_i (\alpha_i - \alpha_i^*) - \epsilon \sum_{i=1}^n (\alpha_i + \alpha_i^*) - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n (\alpha_i - \alpha_i^*) (\alpha_j - \alpha_j^*) \mathbf{x}_i^T \mathbf{x}_j \\ \text{s.t.} \quad & 0 \leq \alpha_i, \alpha_i^* \leq C, \quad i = 1, 2, \dots, n, \\ & \sum_{i=1}^n (\alpha_i - \alpha_i^*) = 0, \end{aligned} \quad (6.74)$$

6.10.3 对偶问题求解

对偶问题是一个凸二次规划问题，可以使用各种优化算法求解，如 SMO 算法（参考本节后面的补充知识部分中的6.6）。具体步骤如下：

1. **初始化：** 选择初始的 α_i 和 α_i^* 值，通常可以随机初始化或全部设为零。
2. **选择变量：** 选择两个变量 α_i 和 α_i^* 进行优化，选择标准可以基于违反 KKT 条件的程度。
3. **计算边界：** 根据 $0 \leq \alpha_i, \alpha_i^* \leq C$ 和 $\sum_{i=1}^n (\alpha_i - \alpha_i^*) = 0$ 计算 α_i 和 α_i^* 的上下界。
4. **更新变量：** 通过解析求解的方式更新 α_i 和 α_i^* 。
5. **检查收敛：** 检查是否满足停止条件，如 α_i 和 α_i^* 的变化小于某个阈值。

6. **重复**: 重复步骤 2 到 5, 直到所有变量都满足 KKT 条件。

一旦求解出 α_i 和 α_i^* , 可以通过以下公式计算 $\mathbf{w}$ 和 b :

$$\mathbf{w} = \sum_{i=1}^n (\alpha_i - \alpha_i^*) \phi(\mathbf{x}_i). \quad (6.75)$$

对于偏置 b , 通常选择一个或几个支持向量 (即满足 $0 < \alpha_i < C$ 或 $0 < \alpha_i^* < C$ 的样本点) 来计算, 具体为:

$$b = y_k - \mathbf{w}^T \phi(\mathbf{x}_k) - \epsilon, \quad \text{for some } k \text{ such that } 0 < \alpha_k < C \text{ or } 0 < \alpha_k^* < C. \quad (6.76)$$

表 6.4: SVR 中的关键参数

符号	含义	描述
$\mathbf{w}$	权重向量	回归函数的权重
b	偏置项	回归函数的偏置
ξ_i	松弛变量	正向误差的容许程度
ξ_i^*	松弛变量	负向误差的容许程度
C	正则化参数	控制模型复杂度与训练误差的权衡
ϵ	不敏感损失参数	定义间隔带的宽度
α_i	对偶变量	拉格朗日乘子, 表示样本点的重要性
α_i^*	对偶变量	拉格朗日乘子, 表示样本点的重要性

为帮助理解, 这里通过一个简单例子进行说明。假设有一个简单的回归数据集, 包含三个样本点 $(\mathbf{x}_1, y_1) = ((1, 0), 1.1)$, $(\mathbf{x}_2, y_2) = ((0, 1), 0.9)$, 和 $(\mathbf{x}_3, y_3) = ((-1, 0), -1.1)$ 。使用 $\epsilon = 0.1$ 和 $C = 1$, 通过求解上述对偶问题, 可以找到最优的 α_i 和 α_i^* 值, 进而确定 $\mathbf{w}$ 和 b 。具体的计算过程包括设置初始值, 迭代更新 α_i 和 α_i^* 直到收敛, 并利用最终的 α_i 和 α_i^* 值计算 $\mathbf{w}$ 和 b 。

6.11 核函数

核函数 (Kernel Function) 是支持向量机 (SVM) 和支持向量回归 (SVR) 中的一项关键技术。通过核函数, 可以将输入数据从原始特征空间映射到高维特征空间, 从而使得在高维空间中更容易找到线性分离超平面或回归超平面。核函数不仅可以提高模型的表达能力, 还能够避免显式计算高维特征空间中的内积, 从而提高计算效率。

6.11.1 核函数的定义

核函数 $K(\mathbf{x}_i, \mathbf{x}_j)$ 是一个函数, 它将两个输入向量 $\mathbf{x}_i$ 和 $\mathbf{x}_j$ 映射到高维特征空间中的内积。形式上, 核函数可以表示为:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j), \quad (6.77)$$

其中 $\phi(\cdot)$ 是从输入空间到高维特征空间的映射函数。

回忆一下, 如果训练数据不是线性可分, SVM 需要通过对数据进行升维操作, 将训练数据从一个低维空间转化到一个高维特征, 在高维特征空间上再进行线性分类。此种方式有一个重要的问题就是维度转化后的特征可能具有很高维, 如果直接在高维空间上计算内积, 计算量巨大, 核函数另辟蹊径, 将高维空间的内积计算放到低维空间进行。所以, 核函数公式中的输入是两个低维空间的特征 $\mathbf{x}_i$ 和 $\mathbf{x}_j$, 输出却是高维特征空间的内积 $\phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$ 。

6.11.2 常用的核函数

1. 线性核函数 (Linear Kernel):

$$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j \quad (6.78)$$

线性核函数是最简单的核函数，适用于线性可分的数据集。

2. 多项式核函数 (Polynomial Kernel):

$$K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + c)^d \quad (6.79)$$

其中 c 是常数项， d 是多项式的次数。多项式核函数可以捕捉数据的非线性关系。

3. 高斯径向基核函数 (Gaussian Radial Basis Function, RBF Kernel):

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) \quad (6.80)$$

其中 $\gamma > 0$ 是参数，控制核函数的宽度。RBF 核函数是使用最广泛的核函数之一，适用于各种类型的非线性数据。

4. Sigmoid 核函数 (Sigmoid Kernel):

$$K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\kappa \mathbf{x}_i^T \mathbf{x}_j + \theta) \quad (6.81)$$

其中 κ 和 θ 是参数。Sigmoid 核函数类似于神经网络中的激活函数，适用于某些特定类型的数据。

6.11.3 核函数的选择

表6.5给出了常见核函数的表达式和对应参数，选择合适的核函数对于构建有效的 SVM 或 SVR 模型至关重要。选择核函数时，可以考虑以下因素：

- **数据特性：**根据数据的分布和特征选择合适的核函数。例如，如果数据线性可分，可以选择线性核函数；如果数据具有复杂的非线性关系，可以选择 RBF 核函数。
- **计算效率：**不同的核函数计算复杂度不同。线性核函数计算简单，适用于大规模数据集；RBF 核函数虽然计算复杂度较高，但适用范围更广。
- **模型性能：**通过交叉验证等方法评估不同核函数的性能，选择性能最佳的核函数。

表 6.5: 常用核函数及其参数

核函数	表达式	参数
线性核函数	$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$	无
多项式核函数	$K(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + c)^d$	c, d
高斯径向基核函数	$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \ \mathbf{x}_i - \mathbf{x}_j\ ^2)$	γ
Sigmoid 核函数	$K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\kappa \mathbf{x}_i^T \mathbf{x}_j + \theta)$	κ, θ

6.12 核技巧 (Kernel Trick)

核技巧是指通过核函数直接计算高维特征空间中的内积，而不需要显式地进行特征映射。这不仅可以简化计算，还能够避免高维特征空间中的维度灾难问题。核技巧是 SVM 和 SVR 能够高效处理非线性问题的关键技术。

6.12.1 核技巧直观理解

如下图6.7(a)所示,考虑如下两类在二维平面上的数据点,圆形数据点:\$(1, 1), (2, 2), (-1, -1), (-2, -2)\$, 三角形数据点: \$(1, -1), (2, -2), (-1, 1), (-2, 2)\$。在原始的二维空间中,很明显这些数据点无法通过一条直线进行线性分割,因为圆形数据点分布在直线 \$y = x\$ 上,三角形数据点分布在直线 \$y = -x\$ 上。

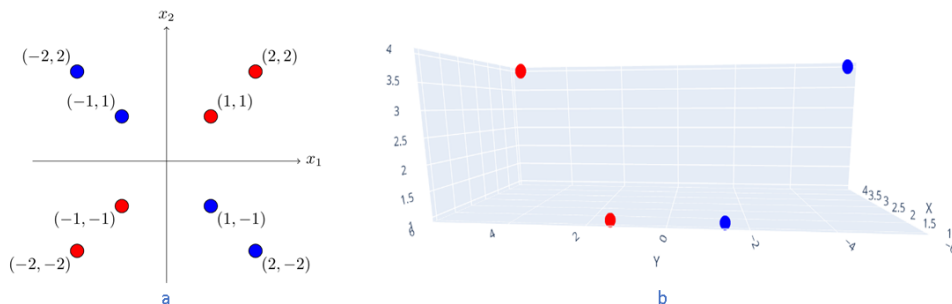


图 6.7: 二维平面上的数据点分布

现在考虑维度转化线性可分的方法。使用映射函数 $\varphi(x_1, x_2) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$ 将数据点映射到三维空间。

对于圆形数据点 \$(1, 1)\$, 映射后为 $\varphi(1, 1) = (1^2, \sqrt{2} \times 1 \times 1, 1^2) = (1, \sqrt{2}, 1)$ 。同理, \$(2, 2)\$ 映射后为 $\varphi(2, 2) = (4, 4\sqrt{2}, 4)$, \$(-1, -1)\$ 映射后为 $\varphi(-1, -1) = (1, \sqrt{2}, 1)$, \$(-2, -2)\$ 映射后为 $\varphi(-2, -2) = (4, 4\sqrt{2}, 4)$ 。

对于三角形数据点 \$(1, -1)\$, 映射后为 $\varphi(1, -1) = (1, -\sqrt{2}, 1)$ 。\$(2, -2)\$ 映射后为 $\varphi(2, -2) = (4, -4\sqrt{2}, 4)$, \$(-1, 1)\$ 映射后为 $\varphi(-1, 1) = (1, -\sqrt{2}, 1)$, \$(-2, 2)\$ 映射后为 $\varphi(-2, 2) = (4, -4\sqrt{2}, 4)$ 。

在这个三维空间中,可以找到一个平面 $w_1x_1 + w_2x_2 + w_3x_3 + b = 0$ 来将两类数据点线性分开。例如,取 $w_1 = 0, w_2 = 1, w_3 = 0, b = 0$, 则平面方程为 $x_2 = 0$, 圆形数据点的 x_2 坐标(即 $\sqrt{2}x_1x_2$ 中的 $\sqrt{2}x_1x_2$ 部分)均大于 0, 三角形数据点的 x_2 坐标均小于 0, 从而实现线性分割。经过维度转化的新特征如图6.7 (b) 所示,从图中可以看出,新的特征点可以很容易被线性分割。

接下来看核函数的使用。这里使用多项式核函数 $K(x, y) = (x^T y + 1)^2$ 为例。

对于圆形数据点 \$(1, 1)\$ 和 \$(2, 2)\$, 计算核函数值:

$$\begin{aligned} K((1, 1), (2, 2)) &= ((1 \times 2 + 1 \times 2 + 1)^2) \\ &= (5^2) \\ &= 25 \end{aligned} \quad (6.82)$$

其中 $x = (1, 1), y = (2, 2)$, 这里 $x_1 = 1, x_2 = 1, y_1 = 2, y_2 = 2$ 。

通过核函数,在原始二维空间中就能计算出类似于在高维空间中的内积结果,避免显式的高维映射带来的计算复杂度。

从这个案例可以看出,对于原始二维空间中不能线性分割的数据,维度转化线性可分需要将数据映射到高维空间并找到合适的分割平面,计算和存储成本较高;而核函数则在原始低维空间通过特定的计算来模拟高维空间的效果,相对更高效且能处理高维空间计算时可能遇到的维度灾难问题。

6.12.2 高斯径向基函数理解

高斯径向基函数是一种常用的核函数,在机器学习中广泛应用于支持向量机(SVM)、神经网络和其他非线性模型。它具有良好的平滑性和局部性,能够有效地处理非线性问题。高斯径向基函

数的形式如下：

$$k(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) \quad (6.83)$$

其中, $k(\mathbf{x}_i, \mathbf{x}_j)$ 为高斯径向基函数。 $\mathbf{x}_i, \mathbf{x}_j$ 为输入向量。 γ 表示核函数的带宽参数, 控制函数的宽度。

1. 泰勒展开

高斯径向基函数可以看作是将输入空间映射到一个无限维特征空间中的内积。为证明这一点, 首先考虑高斯径向基函数的泰勒展开。

对于任意两个输入向量 $\mathbf{x}_i$ 和 $\mathbf{x}_j$, 高斯径向基函数可以表示为:

$$k(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) \quad (6.84)$$

可以将其泰勒展开为:

$$\exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) = \sum_{n=0}^{\infty} \frac{1}{n!} (-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2)^n \quad (6.85)$$

进一步展开, 得到:

$$\exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) = \sum_{n=0}^{\infty} \frac{(-1)^n \gamma^n}{n!} \|\mathbf{x}_i - \mathbf{x}_j\|^{2n} \quad (6.86)$$

2. 特征映射

假设存在一个特征映射 $\phi(\mathbf{x})$, 使得:

$$k(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle \quad (6.87)$$

这里需要找到这样的 $\phi(\mathbf{x})$, 使得上述等式成立。通过比较泰勒展开的形式, 可以发现:

$$\phi(\mathbf{x}) = \left(1, \sqrt{2\gamma} \mathbf{x}, 2\gamma \mathbf{x} \otimes \mathbf{x}, \sqrt{\frac{(2\gamma)^2}{2!}} \mathbf{x} \otimes \mathbf{x} \otimes \mathbf{x}, \dots \right) \quad (6.88)$$

其中, $\otimes$ 表示张量积。从上述表达式可以看出, $\phi(\mathbf{x})$ 包含所有可能的多项式项, 因此它是无限维的。每个维度对应于不同阶的多项式项, 这表明高斯径向基函数实际上是在一个无限维的特征空间中进行计算。

在基于高斯径向基核的 SVM 中, 参数 γ 起着至关重要的作用, 它对模型的性能有着显著的影响。为深入探究 γ 的影响, 对高斯径向基函数在不同 γ 值下的行为进行对比分析, 如下表所示:

表 6.6: 不同 γ 值下的高斯径向基函数值

γ	$k(\mathbf{x}_i, \mathbf{x}_j)$ (当 $\ \mathbf{x}_i - \mathbf{x}_j\ = 1$)	描述
0.5	0.6065	宽高斯分布
1.0	0.3679	中等宽度高斯分布
2.0	0.1353	窄高斯分布

3. 不同 γ 的径向基函数

更进一步, 绘制不同 γ 的径向基函数的图像, 如图6.8所示。图6.8中给出四个不同 γ 值的 RBF 函数图像。随着 γ 值的增加, RBF 函数的峰值变得更加尖锐, 且在中心点以外的区域迅速衰减至零。这表明 γ 控制 RBF 函数的宽度或者说是“影响半径”。当 γ 较小时, RBF 函数更加平缓, 影响范围更大; 而当 γ 较大时, RBF 函数变得更为集中, 影响范围更小。

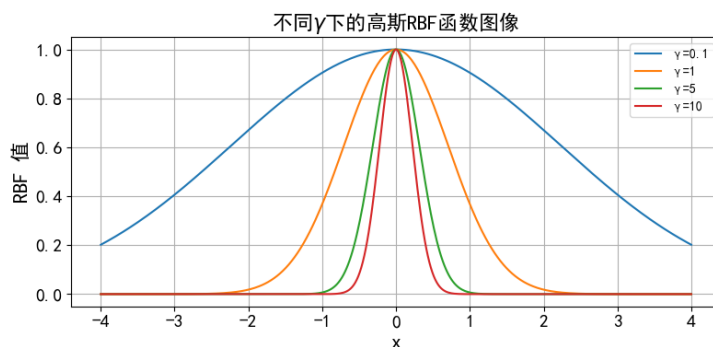


图 6.8: 高斯径向基函数的图形表示

通过这个直观的图形对比，可以看到 γ 参数是如何影响 RBF 函数形状的。这对于理解支持向量机中的核函数参数选择非常重要，因为 γ 值的选择直接影响到模型的复杂度和泛化能力。较大的 γ 值意味着模型会考虑更多的局部细节，可能导致过拟合；而较小的 γ 值则倾向于简化决策边界，可能造成欠拟合。因此，在实际应用中，通常需要通过交叉验证等方法来找到最佳的 γ 值。

4. 案例数据与详细计算步骤

最后，我通过一个简单的案例来进一步理解高斯径向基函数。假设有两个二维输入向量 $\mathbf{x}_i = [1, 2]$ 和 $\mathbf{x}_j = [3, 4]$ ，带宽参数 $\gamma = 1$ 。

(1) 计算欧氏距离：

$$\|\mathbf{x}_i - \mathbf{x}_j\| = \sqrt{(1-3)^2 + (2-4)^2} = \sqrt{4+4} = \sqrt{8} = 2\sqrt{2} \quad (6.89)$$

(2) 计算高斯径向基函数值：

$$k(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-1 \cdot (2\sqrt{2})^2\right) = \exp(-8) \approx 0.000335 \quad (6.90)$$

通过上述步骤，可以看到高斯径向基函数的具体计算过程。

6.13 实践练习

6.13.1 示例：支持向量可视化

在支持向量机中，支持向量是指那些最接近决策边界的样本点，它们对于模型的构建至关重要。通过可视化这些支持向量，可以更好地理解 SVM 是如何对数据进行分类的。本小节将展示如何使用 scikit-learn 的 inspection 模块来识别并可视化支持向量。

首先，需要导入必要的库，并准备一些示例数据。这里使用 `make_blobs` 函数生成一些简单的二分类数据。

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import make_blobs
4 from sklearn.svm import SVC
5 from sklearn.inspection import DecisionBoundaryDisplay
6 # 产生模拟数据
7 X, y = make_blobs(n_samples=50, centers=2, random_state=0, cluster_std=0.60)
8 # 建立模型并训练
9 svm = SVC(kernel='linear', C=1).fit(X, y)
```

接下来，将使用 `DecisionBoundaryDisplay` 来绘制决策边界，并且高亮显示支持向量。

```

1  # 支持向量可视化
2  disp = DecisionBoundaryDisplay.from_estimator(
3      svm,
4      X,
5      response_method="auto",
6      plot_method="contour",
7      xlabel='Feature 1',
8      ylabel='Feature 2',
9      levels=[-1, 0, 1],
10     linestyle=['--', '-', '--'],
11     alpha=0.5
12 )
13
14 # 绘制训练数据
15 disp.ax_.scatter(X[:, 0], X[:, 1], c=y, edgecolors='k', cmap=plt.cm.Paired)
16
17 # 绘制支持向量
18 support_vectors = svm.support_vectors_
19 disp.ax_.scatter(support_vectors[:, 0], support_vectors[:, 1], s=100,
20                 facecolors='none', edgecolors='k')
21
22 plt.title('SVM with Support Vectors Highlighted')
23 plt.show()

```

运行结果如图6.9所示，在这段代码中，`SVC` 被用来拟合数据，`DecisionBoundaryDisplay` 用于绘制决策边界。最后，通过 `svm.support_vectors_` 属性获取到的支持向量被特别标记出来，以帮助直观地看到哪些点是支持向量。

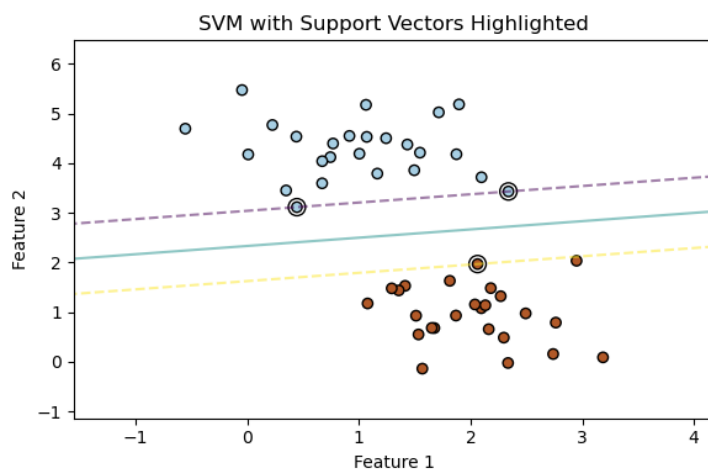


图 6.9: 高斯径向基函数的图形表示

6.13.2 示例：不同 γ 值高斯核函数可视化

在本节中，将演示 γ 参数对支持向量机（SVM）使用径向基函数（RBF）核函数时决策边界的影响。示例中使用 `sklearn.datasets` 模块中的 `make_blobs` 函数生成模拟数据，并使用 `sklearn.inspection` 中的 `DecisionBoundaryDisplay` 来可视化不同 γ 值下的决策边界。

首先，使用 `make_blobs` 函数生成模拟数据。该函数创建一组在二维空间中分组为簇的数据点。该示例将创建两个簇，每个簇包含 100 个样本。

```

1  from sklearn.datasets import make_blobs
2  X, y = make_blobs(n_samples=100, centers=2, random_state=42)

```

接下来，使用 RBF 核函数和三个不同的 γ 值（0.1、1 和 10）来训练 SVM 模型。训练完成后，使用 DecisionBoundaryDisplay 绘制决策边界和支持向量。

```

1  from sklearn.svm import SVC
2  from sklearn.inspection import DecisionBoundaryDisplay
3  import matplotlib.pyplot as plt
4  gammas = [0.1, 1, 10]
5  fig, axes = plt.subplots(1, len(gammas), figsize=(15, 5))
6
7  for i, gamma in enumerate(gammas):
8      svm = SVC(kernel='rbf', gamma=gamma) # 基于RBF的支持向量机
9      svm.fit(X, y)
10     disp = DecisionBoundaryDisplay.from_estimator(
11         svm,
12         X,
13         response_method="predict",
14         plot_method="pcolormesh", # 绘制面
15         xlabel='feature 1',
16         ylabel='feature 2',
17         alpha=0.5,
18         ax=axes[i],
19         shading='auto')
20     # 绘制支持向量
21     disp.ax_.scatter(X[:, 0], X[:, 1], c=y, edgecolor='k', s=50, cmap=plt.cm.Paired)
22     sv = svm.support_vectors_
23     disp.ax_.scatter(sv[:, 0], sv[:, 1], facecolors='none', edgecolors='r', s=100, label='support
        vector')
24     disp.ax_.set_title(f'$\gamma = {gamma}$')
25 plt.tight_layout()
26 plt.show()

```

生成的图像6.10展示出不同 γ 值下的决策边界变化。较小的 γ 值会导致更平滑的决策边界，而较大的 γ 值则会产生更复杂的边界，可能导致过拟合。支持向量，即定义决策边界的关键点，用红色标记。

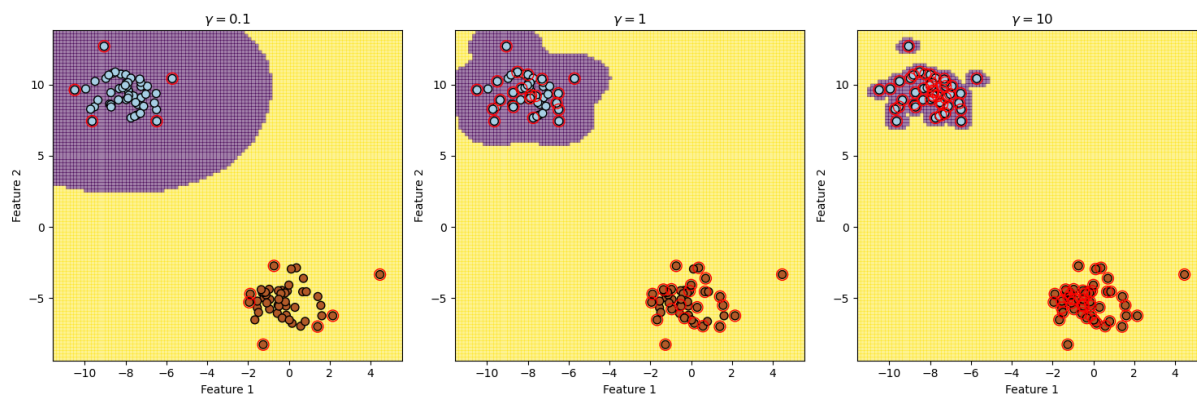


图 6.10: 高斯径向基函数的图形表示

6.13.3 示例：手写数字识别

在本小节中，将探讨如何利用支持向量机 (Support Vector Machine, SVM) 来实现手写数字识别。手写数字识别是模式识别领域的一个经典问题，它涉及到将手写的数字图像转换成机器可读的格式。由于其在实际场景中的广泛应用，如邮政编码识别、银行支票处理等，因此一直受到研究者的关注。对于每一个手写数字图像，首先需要将其转换为适合 SVM 模型输入的形式。在此案例中，直

接将每张图片的像素值展平成为一个 784 维 ($28 \times 28 = 784$) 的向量作为特征向量。此外, 考虑到 SVM 对输入规模敏感的特点, 还对所有特征进行归一化处理, 确保每个特征的值域位于 $[0,1]$ 之间。完成特征提取后, 可以通过以下步骤构建模型:

1. **选择核函数:** 针对非线性可分的情况, 这里采用 RBF (径向基函数) 作为 SVM 的核函数。RBF 核能够有效地映射原始低维空间到高维空间, 在那里数据可能变得线性可分。
2. **参数调优:** 通过交叉验证方法调整 C (正则化参数) 和 γ (RBF 核宽度), 以找到最佳模型参数组合。具体来说, 设置 $C=[0.1, 1, 10, 100]$, $\gamma=[0.001, 0.01, 0.1, 1]$, 并使用网格搜索法寻找最优解。
3. **训练过程:** 使用选定的最佳参数配置, 以及训练集数据来训练 SVM 分类器。整个训练过程中, 特别注意避免过拟合现象的发生。

以下 Python 代码给出对应的实现, 用于完成上述过程, 首先导入完成任务所需的库。

```
1 # 导入所需库
2 from sklearn import datasets, svm, metrics
3 from sklearn.model_selection import train_test_split, GridSearchCV
4 from sklearn.preprocessing import StandardScaler
5 import numpy as np
6 import matplotlib.pyplot as plt
```

其次, 加载数据集, 并进行可视化。

```
1 # 加载MNIST数据集
2 digits = datasets.load_digits()
3 X, y = digits.data, digits.target
4 # 数据预览
5 _, axes = plt.subplots(2, 5)
6 images_and_labels = list(zip(digits.images, digits.target))
7 for ax, (image, label) in zip(axes.ravel(), images_and_labels[:10]):
8     ax.set_axis_off()
9     ax.imshow(image, cmap=plt.cm.gray_r, interpolation='nearest')
10    ax.set_title(f'Training:{label}')
11 plt.show()
```

程序运行后, 将会看见如下图6.11结果, 由于图像的大小较小: 28×28 , 所以显示较为模糊。



图 6.11: 手写数字可视化

然后, 通过网格搜索的方法获得支持向量机的最佳超参数, 并打印最佳的参数。

```
1 # 特征提取与数据划分
2 X = X.reshape((len(X), -1))
3 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
4 # 标准化特征
5 scaler = StandardScaler()
6 X_train = scaler.fit_transform(X_train)
7 X_test = scaler.transform(X_test)
8 # 模型构建与训练, 通过网格搜索的方式找到最佳超参数
9 svc = svm.SVC(kernel='rbf')
```

```

10 param_grid = {'C': [0.1, 1, 10, 100], 'gamma': [0.001, 0.01, 0.1, 1]}
11 grid_search = GridSearchCV(svc, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
12 grid_search.fit(X_train, y_train)
13 print("Best parameters found: ", grid_search.best_params_)
14 best_svc = grid_search.best_estimator_

```

输出的结果类似: Best parameters found: 'C': 100, 'gamma': 0.01。最后, 通过混淆矩阵方式显示分类器的结果。

```

1 y_pred = best_svc.predict(X_test)
2 print(f"Classification report for classifier {best_svc}:\n"
3       f"{metrics.classification_report(y_test, y_pred)}\n")
4 disp = metrics.plot_confusion_matrix(best_svc, X_test, y_test)
5 disp.figure_.suptitle("Confusion Matrix")
6 plt.show()

```

通过对 MNIST 数据集的应用, 成功地实现基于 SVM 的手写数字识别系统。实验结果如图6.12所示, 通过适当的参数调优, SVM 能够有效地识别手写数字, 具有较高的准确性和实用性。

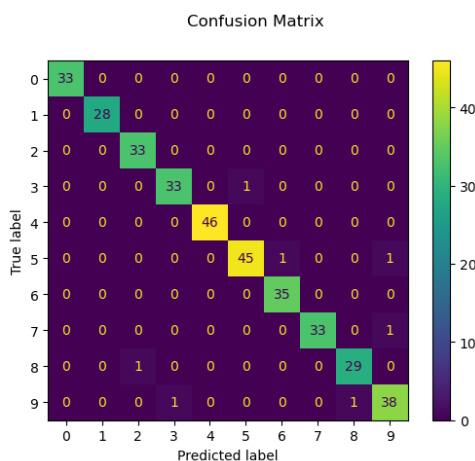


图 6.12: 手写数字识别混淆矩阵

6.14 补充知识

在支持向量机和其他机器学习算法中, 向量求导和向量范式是重要的数学工具。本节将详细介绍这两个概念, 并通过具体示例进行说明。

6.14.1 向量求导基础

向量求导涉及到对向量和矩阵的函数进行求导。常见的向量求导类型包括标量对向量的导数、向量对向量的导数等。以下是一些基本的向量求导规则和示例。

1. 标量对向量的导数

假设 $f: \mathbb{R}^n \rightarrow \mathbb{R}$ 是一个从 n -维向量到标量的函数, 记作 $f(\mathbf{x})$, 其中 $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$ 。标量 f 对向量 $\mathbf{x}$ 的导数是一个 n -维向量, 表示为:

$$\nabla_{\mathbf{x}} f = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]^T \quad (6.91)$$

拉格朗日函数进行求导的过程中 (参考公式6.9), 其中关键的一部是对 $\mathbf{w}$ 求偏导, 就是标量对向量的导数, 结果也是一个 d 维向量。以下是一个具体示例, 假设 $f(\mathbf{x}) = \mathbf{a}^T \mathbf{x}$, 其中 $\mathbf{a}$ 和 $\mathbf{x}$ 都是

n -维向量。求 f 对 $\mathbf{x}$ 的导数。

$$f(\mathbf{x}) = \mathbf{a}^T \mathbf{x} = \sum_{i=1}^n a_i x_i$$

$$\nabla_{\mathbf{x}} f = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right]^T = \mathbf{a} \quad (6.92)$$

2. 向量对向量的导数

假设 $\mathbf{f}: \mathbb{R}^n \rightarrow \mathbb{R}^m$ 是一个从 n -维向量到 m -维向量的函数, 记作 $\mathbf{f}(\mathbf{x}) = [f_1(\mathbf{x}), f_2(\mathbf{x}), \dots, f_m(\mathbf{x})]^T$ 。向量 $\mathbf{f}$ 对向量 $\mathbf{x}$ 的导数是一个 $m \times n$ 的雅可比矩阵 J , 表示为:

$$J = \nabla_{\mathbf{x}} \mathbf{f} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots & \frac{\partial f_2}{\partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \frac{\partial f_m}{\partial x_2} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} \quad (6.93)$$

以下为一个向量对向量求导的示例, 以帮助大家理解。假设 $\mathbf{f}(\mathbf{x}) = A\mathbf{x}$, 其中 A 是一个 $m \times n$ 的矩阵, $\mathbf{x}$ 是一个 n -维向量。求 $\mathbf{f}$ 对 $\mathbf{x}$ 的导数。

$$\mathbf{f}(\mathbf{x}) = A\mathbf{x} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \quad (6.94)$$

$$\nabla_{\mathbf{x}} \mathbf{f} = A$$

另一个示例为二次形式的导数, 假设 $f(\mathbf{x}) = \mathbf{x}^T A \mathbf{x}$, 其中 A 是一个 $n \times n$ 的对称矩阵, $\mathbf{x}$ 是一个 n -维向量。求 f 对 $\mathbf{x}$ 的导数。

$$f(\mathbf{x}) = \mathbf{x}^T A \mathbf{x} = \sum_{i=1}^n \sum_{j=1}^n a_{ij} x_i x_j$$

$$\frac{\partial f}{\partial x_k} = \sum_{i=1}^n a_{ik} x_i + \sum_{j=1}^n a_{kj} x_j \quad (6.95)$$

由于 A 是对称矩阵, 即 $a_{ij} = a_{ji}$, 所以:

$$\frac{\partial f}{\partial x_k} = 2 \sum_{i=1}^n a_{ik} x_i \quad (6.96)$$

因此, 导数为:

$$\nabla_{\mathbf{x}} f = 2A\mathbf{x} \quad (6.97)$$

6.14.2 向量范式基础知识

在机器学习和优化算法中, 向量的范式 (Norm) 是一个重要的概念, 用于衡量向量的大小或长度。本节将介绍向量范式的基本知识, 包括常见的范式类型及其性质。

1. 向量范式的定义和类型

向量的范式是一种衡量向量大小的方法。对于一个 n -维向量 $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$, 范式通常定

义为:

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p} \quad (6.98)$$

其中, p 是一个正实数。根据 p 的不同取值, 可以定义不同的范式类型。

2. L1 范式 (Manhattan Norm)

L1 范式也称为曼哈顿范式, 定义为向量各元素绝对值之和:

$$\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i| \quad (6.99)$$

L1 范式常用于稀疏表示和特征选择, 因为它倾向于产生稀疏解, 即许多元素为零, 具体细节可以参考后面特征选择章节中的嵌入法。

3. L2 范式 (Euclidean Norm)

L2 范式也称为欧几里得范式, 定义为向量各元素平方和的平方根:

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2} \quad (6.100)$$

L2 范式是最常用的范式, 用于衡量向量的长度或大小。在优化问题中, L2 范式常用于正则化, 以防止过拟合。SVM 中的目标函数就是超平面权重 w 的二范式的平方, 参考公式 (6.18)。

4. L-infinity 范式 (Maximum Norm)

L-infinity 范式也称为最大范式, 定义为向量各元素绝对值的最大值:

$$\|\mathbf{x}\|_\infty = \max_{i=1,2,\dots,n} |x_i| \quad (6.101)$$

L-infinity 范式用于衡量向量中最大的元素, 常用于某些最优化问题中。

5. L0 范式

虽然 L0 范式严格意义上不是一个范式, 但它经常被用来表示向量中非零元素的个数:

$$\|\mathbf{x}\|_0 = \text{number of non-zero elements in } \mathbf{x} \quad (6.102)$$

L0 范式在稀疏表示和特征选择中非常有用, 但由于其非凸性质, 优化问题通常难以直接求解, 一般通过使用 L1 范式来求解。

6. 向量范式的性质

向量范式具有一些重要的性质, 这些性质在理论分析和实际应用中非常有用:

1) 非负性: 对于任意向量 $\mathbf{x}$, 范式总是非负的:

$$\|\mathbf{x}\|_p \geq 0 \quad (6.103)$$

并且当且仅当 $\mathbf{x} = \mathbf{0}$ 时, 范式等于零。

2) 齐次性: 对于任意标量 c 和向量 $\mathbf{x}$, 范式满足:

$$\|c\mathbf{x}\|_p = |c| \|\mathbf{x}\|_p \quad (6.104)$$

3) 三角不等式: 对于任意两个向量 $\mathbf{x}$ 和 $\mathbf{y}$, 范式满足:

$$\|\mathbf{x} + \mathbf{y}\|_p \leq \|\mathbf{x}\|_p + \|\mathbf{y}\|_p \quad (6.105)$$

6.14.3 拉格朗日求解方法

拉格朗日求解方法是一种用于求解带有约束条件的优化问题的技术。它通过引入拉格朗日乘数将约束条件纳入目标函数中，从而将约束优化问题转化为无约束优化问题。本节将介绍拉格朗日乘数法在处理既有等式约束又有不等式约束的优化问题时的应用。

1. 拉格朗日乘数法的基本概念

假设要最小化一个目标函数 $f(\mathbf{x})$ ，同时满足一组等式约束 $g_i(\mathbf{x}) = 0$ 和一组不等式约束 $h_j(\mathbf{x}) \leq 0$ ，其中 $\mathbf{x} \in \mathbb{R}^n$ 。优化问题可以表示为：

$$\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}) \\ \text{s.t.} \quad & g_i(\mathbf{x}) = 0, \quad i = 1, 2, \dots, m \\ & h_j(\mathbf{x}) \leq 0, \quad j = 1, 2, \dots, p \end{aligned} \quad (6.106)$$

为将约束条件纳入目标函数中，引入拉格朗日乘数 λ_i 和 μ_j ，构造拉格朗日函数：

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}, \boldsymbol{\mu}) = f(\mathbf{x}) + \sum_{i=1}^m \lambda_i g_i(\mathbf{x}) + \sum_{j=1}^p \mu_j h_j(\mathbf{x}) \quad (6.107)$$

其中， $\boldsymbol{\lambda} = [\lambda_1, \lambda_2, \dots, \lambda_m]^T$ 是等式约束的拉格朗日乘数向量， $\boldsymbol{\mu} = [\mu_1, \mu_2, \dots, \mu_p]^T$ 是不等式约束的拉格朗日乘数向量。

拉格朗日乘数法的核心是求解以下方程组：

$$\begin{aligned} \nabla_{\mathbf{x}} \mathcal{L} &= 0 \\ g_i(\mathbf{x}) &= 0, \quad i = 1, 2, \dots, m \\ h_j(\mathbf{x}) &\leq 0, \quad j = 1, 2, \dots, p \\ \mu_j &\geq 0, \quad j = 1, 2, \dots, p \\ \mu_j h_j(\mathbf{x}) &= 0, \quad j = 1, 2, \dots, p \end{aligned} \quad (6.108)$$

最后一条条件称为互补松弛条件，确保不等式约束要么是活跃的（即 $h_j(\mathbf{x}) = 0$ ），要么对应的拉格朗日乘数为零（即 $\mu_j = 0$ ）。

2. 拉格朗日应用实例

考虑一个简单的优化问题：

$$\begin{aligned} \min_{x,y} \quad & f(x, y) = x^2 + y^2 \\ \text{s.t.} \quad & g(x, y) = x + y - 1 = 0 \\ & h(x, y) = x - 1 \leq 0 \end{aligned} \quad (6.109)$$

构造拉格朗日函数：

$$\mathcal{L}(x, y, \lambda, \mu) = x^2 + y^2 + \lambda(x + y - 1) + \mu(x - 1) \quad (6.110)$$

求解拉格朗日函数的梯度：

$$\begin{aligned}
 \nabla_x \mathcal{L} &= 2x + \lambda + \mu = 0 \\
 \nabla_y \mathcal{L} &= 2y + \lambda = 0 \\
 \nabla_\lambda \mathcal{L} &= x + y - 1 = 0 \\
 \nabla_\mu \mathcal{L} &= x - 1 \leq 0 \\
 \mu &\geq 0 \\
 \mu(x - 1) &= 0
 \end{aligned} \tag{6.111}$$

解上述方程组：

如果 $\mu = 0$ ，则 $x - 1 < 0$ ，即 $x < 1$ 。

$$\begin{aligned}
 2x + \lambda &= 0 \Rightarrow \lambda = -2x \\
 2y + \lambda &= 0 \Rightarrow \lambda = -2y \\
 x + y - 1 &= 0
 \end{aligned} \tag{6.112}$$

将 $\lambda = -2x$ 和 $\lambda = -2y$ 代入第三个方程：

$$\begin{aligned}
 x + y - 1 &= 0 \Rightarrow x + y = 1 \\
 -2x &= -2y \Rightarrow x = y
 \end{aligned} \tag{6.113}$$

联立解得 $x = y = \frac{1}{2}$ 。因此，最优解为 $(x, y) = (\frac{1}{2}, \frac{1}{2})$ ，对应的最小值为：

$$f\left(\frac{1}{2}, \frac{1}{2}\right) = \left(\frac{1}{2}\right)^2 + \left(\frac{1}{2}\right)^2 = \frac{1}{2} \tag{6.114}$$

如果 $\mu > 0$ ，则 $x - 1 = 0$ ，即 $x = 1$ 。

$$\begin{aligned}
 2x + \lambda + \mu &= 0 \Rightarrow 2(1) + \lambda + \mu = 0 \Rightarrow 2 + \lambda + \mu = 0 \\
 2y + \lambda &= 0 \Rightarrow \lambda = -2y \\
 x + y - 1 &= 0 \Rightarrow 1 + y - 1 = 0 \Rightarrow y = 0
 \end{aligned} \tag{6.115}$$

将 $y = 0$ 代入 $\lambda = -2y$ ，可以得到 $\lambda = 0$ ，代入 $2 + \lambda + \mu = 0$ ：

$$2 + 0 + \mu = 0 \Rightarrow \mu = -2 \tag{6.116}$$

但是 $\mu \geq 0$ ，因此这种情况不成立。综上所述，最优解为 $(x, y) = (\frac{1}{2}, \frac{1}{2})$ ，对应的最小值为 $\frac{1}{2}$ 。

6.14.4 凸优化问题

凸优化是一类特殊的数学优化问题，其中目标函数和约束条件均为凸函数。这类问题具有重要的性质，即任何局部最优解也是全局最优解，这使得凸优化问题比一般的非凸优化问题更容易解决。

1. 定义

一个标准形式的凸优化问题可以表示为：

$$\begin{aligned}
 \min \quad & f_0(x) \\
 \text{s.t.} \quad & f_i(x) \leq 0, \quad i = 1, \dots, m \\
 & Ax = b,
 \end{aligned} \tag{6.117}$$

其中 $x \in \mathbb{R}^n$ 是优化变量, $f_0: \mathbb{R}^n \rightarrow \mathbb{R}$ 是凸的目标函数, $f_i: \mathbb{R}^n \rightarrow \mathbb{R}$ ($i = 1, \dots, m$) 是凸的不等式约束函数, $A \in \mathbb{R}^{p \times n}$ 和 $b \in \mathbb{R}^p$ 定义了线性的等式约束。

2. 判断方法

为判断一个优化问题是否为凸优化问题, 需要检查以下几点:

- **目标函数:** $f_0(x)$ 必须是凸函数。如果 $f_0(x)$ 在其定义域上对任意两点 x_1, x_2 和任意 $\theta \in [0, 1]$ 满足: $f_0(\theta x_1 + (1 - \theta)x_2) \leq \theta f_0(x_1) + (1 - \theta)f_0(x_2)$, 则 $f_0(x)$ 是凸函数。
- **不等式约束函数:** $f_i(x)$ 必须是凸函数。同样地, 如果每个 $f_i(x)$ 在其定义域上对任意两点 x_1, x_2 和任意 $\theta \in [0, 1]$ 满足 $f_i(\theta x_1 + (1 - \theta)x_2) \leq \theta f_i(x_1) + (1 - \theta)f_i(x_2)$, 则 $f_i(x)$ 是凸函数。
- **等式约束:** 必须是仿射的, 即形如 $Ax = b$ 的线性方程组。

3. 常见的凸优化函数

在凸优化中, 常见的凸函数包括但不限于:

- **二次函数:** $f(x) = x^T Q x + c^T x + d$, 其中 Q 是半正定矩阵。
- **指数函数:** $f(x) = e^{ax+b}$, 其中 a 和 b 是常数。
- **对数函数:** $f(x) = -\log(x)$, 定义域为 $x > 0$ 。
- **绝对值函数:** $f(x) = |x|$ 。
- **范数函数:** $f(x) = \|x\|_p$, 其中 $p \geq 1$ 。
- **最大值函数:** $f(x) = \max\{x_1, x_2, \dots, x_n\}$ 。
- **对数障碍函数:** $f(x) = -\sum_{i=1}^m \log(b_i - a_i^T x)$, 其中 $a_i^T x < b_i$ 。

4. 求解方法

凸优化问题可以通过多种算法来求解, 包括但不限于梯度下降法、牛顿法、内点法等。这些算法利用凸性带来的便利, 如梯度或 Hessian 矩阵的信息, 来迭代地寻找最优解。

- **梯度下降法:** 当目标函数可微时, 通过沿着负梯度方向逐步更新变量来减少目标函数值。
- **牛顿法:** 利用目标函数的二阶导数 (Hessian 矩阵) 来加速收敛过程。
- **内点法:** 处理带有约束的问题, 通过引入障碍函数将原问题转换为一系列无约束优化问题来求解。

5. 凸优化实例

考虑一个简单的凸优化问题:

$$\begin{aligned} \min \quad & x^2 + y^2 \\ \text{s.t.} \quad & x + y \geq 1. \end{aligned} \tag{6.118}$$

这里, 目标函数 $f_0(x, y) = x^2 + y^2$ 是一个凸函数, 因为它的 Hessian 矩阵

$$\nabla^2 f_0(x, y) = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} \tag{6.119}$$

是正定的。约束条件 $x + y \geq 1$ 也是一个凸集表示, 因为它是线性的。因此, 这是一个凸优化问题。通过观察, 可以发现最优解位于直线 $x + y = 1$ 上, 且该点应满足目标函数最小化的要求。计算得出最优解为 $(x^*, y^*) = (0.5, 0.5)$, 此时目标函数值为 0.5。通过这个简单的例子, 能够看到凸优化问题不仅在理论上有良好的性质, 在实际应用中也易于求解。

6.14.5 KKT 条件

KKT (Karush-Kuhn-Tucker) 条件是优化问题中的一组必要条件, 用于确定最优解。对于 SVM 的对偶问题, KKT 条件包括以下几部分:

1. 原问题的可行性条件

$$y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, 2, \dots, n \quad (6.120)$$

2. 对偶问题的可行性条件

$$\sum_{i=1}^n \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C, \quad i = 1, 2, \dots, n \quad (6.121)$$

3. 互补松弛条件

$$\alpha_i (y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i) = 0, \quad \xi_i \geq 0, \quad i = 1, 2, \dots, n \quad (6.122)$$

4. 梯度条件

$$\mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \quad (6.123)$$

通过求解对偶问题并满足 KKT 条件, 可以找到最优的 α_i 值, 进而确定超平面的参数 $\mathbf{w}$ 和 b 。决策函数的形式为:

$$f(\mathbf{x}) = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i^T \mathbf{x} + b \quad (6.124)$$

其中, 只有支持向量的 α_i 不为零。

6.14.6 张量积

张量积 (Tensor Product) 是一种重要的数学运算, 广泛应用于线性代数、多线性代数、泛函分析和量子力学等领域。它能够两个或多个向量空间、矩阵或张量组合成一个新的张量, 从而在更高维度的空间中进行操作。张量积不仅在理论上具有重要地位, 而且在实际应用中也非常有用, 特别是在机器学习和信号处理等领域。

1. 定义

给定两个向量空间 V 和 W 以及它们的基 $\{\mathbf{v}_i\}$ 和 $\{\mathbf{w}_j\}$, 张量积 $V \otimes W$ 是一个新的向量空间, 其基为 $\{\mathbf{v}_i \otimes \mathbf{w}_j\}$ 。对于任意的 $\mathbf{v} \in V$ 和 $\mathbf{w} \in W$, 它们的张量积 $\mathbf{v} \otimes \mathbf{w}$ 是一个新向量, 属于 $V \otimes W$ 。

具体来说, 如果 $\mathbf{v} = \sum_i a_i \mathbf{v}_i$ 和 $\mathbf{w} = \sum_j b_j \mathbf{w}_j$, 那么:

$$\mathbf{v} \otimes \mathbf{w} = \left(\sum_i a_i \mathbf{v}_i \right) \otimes \left(\sum_j b_j \mathbf{w}_j \right) = \sum_{i,j} a_i b_j (\mathbf{v}_i \otimes \mathbf{w}_j) \quad (6.125)$$

2. 性质

张量积具有以下几个性质:

1) **双线性**: 张量积是双线性的, 即对于任何标量 c 和向量 $\mathbf{u}, \mathbf{v} \in V$ 以及 $\mathbf{x}, \mathbf{y} \in W$, 有:

$$\begin{aligned} (c\mathbf{u} + \mathbf{v}) \otimes \mathbf{x} &= c(\mathbf{u} \otimes \mathbf{x}) + \mathbf{v} \otimes \mathbf{x}, \\ \mathbf{u} \otimes (c\mathbf{x} + \mathbf{y}) &= c(\mathbf{u} \otimes \mathbf{x}) + \mathbf{u} \otimes \mathbf{y}. \end{aligned} \quad (6.126)$$

2) **结合律**: 对于三个向量空间 U, V, W , 有:

$$(U \otimes V) \otimes W \cong U \otimes (V \otimes W) \quad (6.127)$$

3) **分配律**: 对于向量空间 U, V, W , 有:

$$U \otimes (V \oplus W) \cong (U \otimes V) \oplus (U \otimes W) \quad (6.128)$$

3. 应用

张量积在许多领域都有广泛的应用, 包括但不限于:

1) **量子力学**: 在量子力学中, 张量积用于描述复合系统的状态。例如, 两个量子态 ψ_1 和 ψ_2 的张量积 $\psi_1 \otimes \psi_2$ 能够描述这两个系统的联合状态。

2) **机器学习**: 在机器学习中, 张量积用于特征映射, 特别是在核方法中。例如, 在高斯径向基函数中, 通过张量积可以生成高阶多项式项, 从而在无限维特征空间中表示输入向量。

(3) **信号处理**: 在信号处理中, 张量积用于表示多通道或多维信号。例如, 图像可以看作是一个二维张量, 视频可以看作是一个三维张量。

4. 高斯径向基核

在介绍核函数的时候, 重点介绍了高斯径向基核, 并证明高斯径向基核可以将特征从有限维映射到无限维, 这里给出一个简单的例子, 方便理解。假设有一个二维输入向量 $\mathbf{x} = [x_1, x_2]$, 可以构建以下特征映射 $\phi(\mathbf{x})$:

1) **零阶项**: 1

2) **一阶项**: $\sqrt{2\gamma}\mathbf{x} = \sqrt{2\gamma}[x_1, x_2]$

3) **二阶项**: $2\gamma\mathbf{x} \otimes \mathbf{x}$

具体计算如下:

$$\mathbf{x} \otimes \mathbf{x} = \begin{bmatrix} x_1^2 & x_1x_2 \\ x_2x_1 & x_2^2 \end{bmatrix} \quad (6.129)$$

因此, 二阶项为:

$$2\gamma\mathbf{x} \otimes \mathbf{x} = 2\gamma \begin{bmatrix} x_1^2 & x_1x_2 \\ x_2x_1 & x_2^2 \end{bmatrix} \quad (6.130)$$

4) **三阶项**:

$$\sqrt{\frac{(2\gamma)^2}{2!}}\mathbf{x} \otimes \mathbf{x} \otimes \mathbf{x} \quad (6.131)$$

具体计算如下:

$$\mathbf{x} \otimes \mathbf{x} \otimes \mathbf{x} = \begin{bmatrix} x_1^3 & x_1^2x_2 & x_1x_2^2 & x_2^3 \end{bmatrix} \quad (6.132)$$

因此, 三阶项为:

$$\sqrt{\frac{(2\gamma)^2}{2!}}\mathbf{x} \otimes \mathbf{x} \otimes \mathbf{x} = \sqrt{\frac{(2\gamma)^2}{2!}} \begin{bmatrix} x_1^3 & x_1^2x_2 & x_1x_2^2 & x_2^3 \end{bmatrix} \quad (6.133)$$

通过上述特征映射, 高斯径向基函数能够在无限维特征空间中有效地表示输入向量, 从而处理非线性问题。

6.15 习题

6.15.1 简答题

1. 支持向量机 (SVM) 的基本思想是什么? 它如何处理线性不可分的情况?
2. 什么是核函数? 常用的核函数有哪些?
3. SVM 中的软间隔 (Soft Margin) 有什么作用? 如何通过调整超参数 C 来控制模型的复杂度?

6.15.2 计算题

1. 给定二维平面上的两个类别样本点: 类别 1 为 $(1,1), (2,2)$, 类别 2 为 $(4,4), (5,5)$ 。假设使用线性核的 SVM, 请计算最优分类超平面的方程。
2. 如果使用高斯径向基核 (RBF), 且 $\gamma = 1$, 请计算以下两点之间的核函数值: $(0,0)$ 和 $(1,1)$ 。

6.15.3 编程题

1. 使用 Scikit-learn 实现一个基于 SVM 的二分类模型, 加载鸢尾花数据集并完成以下任务:
 - 划分训练集和测试集。
 - 使用线性核训练 SVM 模型。
 - 计算测试集上的分类准确率。
2. 编写一个 Python 函数, 接收一组二维数据点和标签作为输入, 使用高斯径向基核 (RBF) 训练 SVM 模型, 并绘制决策边界。

第七章 贝叶斯模型

在机器学习和统计学领域，贝叶斯模型是一类重要的概率模型。它们基于贝叶斯定理来更新关于未知参数的概率分布，当观察到新的数据时，可以用来预测未来的事件或解释已有的观测结果。本章将探讨贝叶斯理论的基础，以及如何使用它来构建强大的预测模型。

学习目标

本章系统介绍贝叶斯理论及其在机器学习中的应用。理解贝叶斯定理的核心思想，掌握先验、似然与后验概率的概念及其关系，能够结合先验知识与观测数据进行概率推断，并应用于医学诊断、垃圾邮件分类等实际问题。进一步地，深入掌握朴素贝叶斯分类器的基本原理，理解其特征条件独立性假设的意义，熟悉多项式、伯努利和高斯朴素贝叶斯的适用场景及特点，能够根据任务需求合理选择模型变体，并客观认识其计算效率高但对特征相关性敏感的特性。

7.1 信件问题

石榴同学收到一封没有署名的信，她想知道这封信来自她的哪位朋友——小明还是小红。石榴知道小明通常会用“嘿”作为问候语，而小红更喜欢说“嗨”。通过回顾以前收到的信件，石榴发现大约 60% 的小明的信以“嘿”开头，而 80% 的小红的信以“嗨”开始。现在，石榴面前的这封新信是以“嘿”开头的。那么，根据这个信息，这封信更有可能是来自谁呢？

在这个例子中，想要找出给定新证据（信件以“嘿”开头）下最可能的假设（信件来自小明或小红）。这就是贝叶斯推理的核心：结合先验知识（过去信件的习惯）与新证据（当前信件的内容），更新不同假设的信心程度。

7.2 贝叶斯公式

7.2.1 贝叶斯决策论

在做出决策时，通常希望选择能够最大化预期效用或最小化预期损失的行动。贝叶斯决策论提供了一个框架，它允许在不确定性的条件下通过使用概率来优化决策。在这个理论中，目标是找到一个决策规则 d ，使得对于所有可能的观测结果 x ，都能选择出最合适的类别 y 。

假设有一个分类问题，其中要决定输入 x 属于哪个类别 y 。每个类别 y 都有其对应的损失函数 $L(y, d(x))$ ，该函数定义了当采取决策 $d(x)$ 而真实类别为 y 时所遭受的损失。理想情况下，希望找到一个决策规则 d^* ，它能最小化平均风险（即预期损失）：

$$R(d) = \sum_y \int L(y, d(x)) p(x, y) dx \quad (7.1)$$

这里, $L(y, d(x))$ 表示损失函数, 它量化了采取决策 $d(x)$, 即根据观测 x 选择某个类别作为输出时, 而实际类别是 y 所导致的损失程度。损失函数可以有不同的形式, 例如 0-1 损失、平方损失等, 这取决于具体的应用场景。接着, $p(x, y)$ 代表联合概率分布, 表示同时观察到输入 x 和真实类别 y 的概率。该公式通过积分符号 $\int$ 对所有可能的输入 x 求和, 并且使用求和符号 $\sum_y$ 遍历所有可能的类别 y , 从而计算出整个输入空间上以及所有类别的加权平均损失。因此, 这个公式实际上是在衡量对于给定的决策规则 d , 在整个输入空间和所有类别上的预期损失大小, 最终得到的是一个标量值, 反映了决策规则 d 的好坏: 平均风险越小, 说明决策规则 d 越好。

7.2.2 贝叶斯公式推导

为从贝叶斯决策论的角度引出贝叶斯公式, 首先考虑全概率定律和条件概率的关系。设 X 为观察变量, Y 为类别标签, 则有:

$$P(X = x) = \sum_y P(X = x|Y = y)P(Y = y) \quad (7.2)$$

这里, $P(X = x)$ 是边缘概率, 表示在任何情况下观察 x 发生的概率。 $P(X = x|Y = y)$ 是似然, 即在类别 y 下观察到 x 的概率; $P(Y = y)$ 是先验概率, 代表在看到任何数据之前对类别 y 的先验 (信念)。

现在, 根据贝叶斯定理, 可以表达后验概率 $P(Y = y|X = x)$ 为:

$$P(Y = y|X = x) = \frac{P(X = x|Y = y)P(Y = y)}{P(X = x)} \quad (7.3)$$

将上面的全概率定律代入分母中, 得到贝叶斯公式的一般形式:

$$P(Y = y|X = x) = \frac{P(X = x|Y = y)P(Y = y)}{\sum_{y'} P(X = x|Y = y')P(Y = y')} \quad (7.4)$$

此公式为提供了一种方法, 基于观察到的数据 $X = x$ 来更新关于类别 $Y = y$ 的信念。它告诉我们如何结合新的证据与先前的知识来形成更准确的理解。

分子中的 $P(X = x|Y = y)$ 是似然函数, 描述了如果已知类别 y , 那么观察 x 的可能性有多大。 $P(Y = y)$ 是先验概率, 代表在没有观察 x 之前, 认为类别 y 出现的概率。分母是一个归一化常数, 确保后验概率之和等于 1, 它实际上就是所有可能类别的似然乘以先验的总和。

注意: 似然函数与概率函数虽然在数学形式上相似, 但它们的角色和应用背景却有所不同。概率函数是在给定参数的情况下, 用来描述随机变量取特定值的可能性; 而似然函数则是在观测数据已知的条件下, 用以衡量模型参数取不同值时产生这些观测数据的可能性大小。换句话说, 当讨论概率时假设参数固定且已知, 然后考虑不同数据出现的概率; 而在讨论似然性时, 是根据已观察到的数据, 来评估不同参数值的可能性, 目的是为找到最有可能生成观测数据的参数值。因此, 概率函数是关于可能观测的分布, 而似然函数则是关于参数空间中模型参数的函数。

7.2.3 贝叶斯决策规则

基于贝叶斯公式, 现在就可以构建一个决策规则 $d^*(x)$, 它选择那个使后验期望损失最小化的类别:

$$d^*(x) = \arg \min_y \sum_{y'} L(y', d(x))P(Y = y'|X = x) \quad (7.5)$$

换句话说, 对于每一个可能的类别 y' , 如果选择 $d(x) = y$ 作为决策所带来的预期损失, 并选择那个导致最小预期损失的类别作为最终决策。这就是贝叶斯决策规则的核心思想, 它确保在给定观测 x 的情况下, 所做的决策是理论上最优的。

$L(y', d(x))$ 是损失函数, 表示当真实类别为 y' , 但决策是 $d(x)$ 时所造成的损失。 $P(Y = y'|X = x)$ 是后验概率, 反映了在观察 x 之后对类别 y' 的新信念。整个求和项代表的是在给定 x 的情况下, 选择 $d(x)$ 为决策的期望损失。 $\arg \min_y$ 意味着寻找那个能够最小化上述期望损失的类别 y 作为最后的决策 $d^*(x)$ 。

通过这种方式, 贝叶斯决策论不仅有助于理解如何在不确定性中做出最佳决策, 还提供了处理新信息的方法, 即通过贝叶斯公式更新推理的信念。

7.3 0/1 损失最优贝叶斯

在机器学习的目标是根据输入 x 预测输出 y 。为做出这样的预测, 使用一个决策函数 $d(x)$ 来估计未知的输出 Y 。当使用 0/1 损失函数 $L(y', d(x))$ 时, 这意味着如果预测正确则损失为 0, 否则损失为 1, 即:

$$L(y', d(x)) = \begin{cases} 1 & \text{if } y' \neq d(x) \\ 0 & \text{if } y' = d(x) \end{cases} \quad (7.6)$$

7.3.1 最优贝叶斯分类器

本质上, $L(y', d(x))$ 可以视为一个决策错误的计数器, 如果预 $d(x)$ 决策正确就不计算分数, 如果 $d(x)$ 决策错误, 就计算 1 分, 直觉上, 希望决策错误的次数越少越好。最优决策的公式 (7.5) 可以重写为两部分之和:

$$\sum_{y' \neq y} 1 \cdot P(Y = y'|X = x) + \sum_{y' = y} 0 \cdot P(Y = y'|X = x) \quad (7.7)$$

第一项表示所有预测错误的情况的总和, 第二项表示所有预测正确的情况的总和。因为预测正确时损失为 0, 所以它对最小化没有贡献, 可以忽略第二项, 于是得到:

$$\sum_{y' \neq y} P(Y = y'|X = x) \quad (7.8)$$

这实际上等于所有预测错误情况的概率之和。但因为所有可能的输出 y' 的概率加起来等于 1, 所以可以用 1 减去预测正确的概率来表示预测错误的概率, 即:

$$\sum_{y' \neq y} P(Y = y'|X = x) = 1 - P(Y = y|X = x) \quad (7.9)$$

这里 $P(Y = y|X = x)$ 是预测正确的概率。因此, 公式 (7.5) 变为:

$$d^*(x) = \arg \min_y [1 - P(Y = y|X = x)] \quad (7.10)$$

要使这个表达式最小化, 需要最大化 $P(Y = y|X = x)$ 。换句话说, 应该选择使得后验概率最大的类别作为预测结果。因此, 可以得到以下结论:

$$d^*(x) = \arg \max_y P(Y = y|X = x) \quad (7.11)$$

这就是著名的贝叶斯最优分类器 (Bayesian Optimal Classifier)。它表明, 在 0/1 损失下, 最优的预测就是那个具有最高后验概率的类别。也就是说, 对于每一个输入 x , 应当选择最可能的类别 y 作为预测结果。

7.3.2 后验概率直接计算的挑战

在贝叶斯分类器中, 目标是找到给定输入 $X = x$ 时, 使得后验概率 $P(Y = y|X = x)$ 最大的类别 y 。然而, 直接计算这个后验概率通常不是一件容易的事情, 主要原因有以下几点:

1. 数据稀缺: 对于高维空间中的连续型特征, 每个可能的 x 值都对应着一个理论上不同的后验概率。在实践中, 几乎不可能有足够的训练样本覆盖所有可能的 x 值, 这导致对某些 x 值的后验概率难以估计。

2. 维度灾难: 随着特征数量的增加, 需要估计的参数数量呈指数增长, 这不仅增加了计算复杂度, 还可能导致过拟合的问题。

3. 联合概率分布: 要准确地计算 $P(Y = y|X = x)$, 需要知道联合概率分布 $P(X = x, Y = y)$, 而这个联合分布往往是非常复杂的, 并且在实际问题中很难获取或估计。

4. 边缘化: 为计算后验概率, 需要应用贝叶斯公式, 这涉及到计算证据 $P(X = x)$, 即所有类别的边缘概率之和。在高维情况下, 计算这个边缘概率可能是计算上不可行的。

7.3.3 贝叶斯定理引入

为解决上述难题, 可以采用一些实用的方法来间接求解最大后验概率对应的类别 y 。根据贝叶斯定理, 可以将后验概率转换为先验概率和似然函数的乘积再除以证据:

$$P(Y = y|X = x) = \frac{P(X = x|Y = y)P(Y = y)}{P(X = x)} \quad (7.12)$$

其中, $P(Y = y|X = x)$ 是后验概率, 即在观察到 x 之后类别 y 的概率。 $P(X = x|Y = y)$ 是似然函数, 也称为类条件概率, 表示当类别是 y 时, 观测到 x 的概率。 $P(Y = y)$ 是先验概率, 表示在没有看到任何数据之前类别 y 的概率。 $P(X = x)$ 是边缘概率, 表示观测到 x 的概率, 与类别无关。

所以根据贝叶斯公式 (7.12), 贝叶斯最优分类器 (7.11) 可以写为:

$$d^*(x) = \arg \max_y P(Y = y|X = x) = \arg \max_y \frac{P(X = x|Y = y)P(Y = y)}{P(X = x)} \quad (7.13)$$

根据公式 (7.13), 可以专注于最大化未归一化的后验概率 $P(Y = y)P(X = x|Y = y)$ 。通过分别求解先验概率和似然函数, 就可以间接求解最大后验概率。

7.3.4 贝叶斯定理引入的问题

通过引入贝叶斯定理, 虽然可以在实践中有效地解决分类问题。然而, 很多场景下, 直接计算最大化未归一化的后验概率 $P(Y = y)P(X = x|Y = y)$ 也不是一件容易事件。

在贝叶斯分类中, 最大化未归一化的后验概率 $P(Y = y)P(X = x|Y = y)$ 是识别给定输入 $X = x$ 时最可能的类别 y 的关键步骤。然而, 在实际应用中, 这个过程并非总是简单的, 以下是其中的一些挑战:

1. 先验概率 $P(Y = y)$ 的确定: 对于某些问题, 可能没有足够的领域知识来准确地估计每个类别的先验概率。此外, 如果数据集不是从总体中随机抽取的, 那么从数据集中估计的先验概率可能会有偏差。

2. **似然函数 $P(X = x|Y = y)$ 的复杂度**：当特征空间维度较高时，计算似然函数变得非常复杂。特别是对于连续型特征，通常需要假设一个特定的概率分布形式（如高斯分布），但这种假设可能并不总是合理的。

3. **数据稀疏性和过拟合**：当训练样本不足以覆盖所有可能的特征组合时，会导致模型对未见数据泛化能力差，即过拟合现象。这在高维特征空间中尤为明显。

4. **计算效率**：对于大型数据集或具有大量特征的数据点，直接计算 $P(X = x|Y = y)$ 可能是计算上不可行的，尤其是在必须考虑所有特征之间的交互作用时。

7.3.5 相关解决方案

为解决上面存在的问题，一般从以下几个方面进行入手：

- **假设条件独立**：通过假设特征之间条件独立（即给定类别后特征间相互独立），可以将复杂的联合概率分解成更易于管理的形式。例如，在朴素贝叶斯分类器中，可以利用这一假设来简化后验概率的计算。
- **参数模型**：当能够假设数据服从某种特定的概率分布（如高斯分布）时，可以使用该分布的参数形式来描述数据，并通过最大似然估计等方法来确定这些参数。这样可以大大简化后验概率的计算。
- **近似方法**：在一些情况下，可能会采用诸如蒙特卡洛方法、变分推断等技术来近似计算后验概率。

综上所述，虽然直接计算后验概率 $P(Y = y|X = x)$ 面临诸多挑战，但通过引入合理的假设、选择适当的模型以及使用有效的算法，依然可以在实践中有效地解决分类问题。

7.4 朴素贝叶斯方法

7.4.1 邮件分类问题

为说明朴素贝叶斯方法的使用，从邮件分类的案例出发，描述如何通过朴素贝叶斯的方法对邮件进行分类。考虑石榴同学正在构建一个邮件分类器，她有如下训练数据：

表 7.1: 训练数据集

邮件编号	特征（关键词）	类别
1	free, win, prize	垃圾邮件
2	meeting, office, schedule	非垃圾邮件
3	free, urgent, offer	垃圾邮件
4	project, deadline, update	非垃圾邮件
5	click, now, opportunity	垃圾邮件
6	team, discuss, ideas	非垃圾邮件

7.4.2 直接计算的挑战

如果尝试直接根据贝叶斯定理计算每封邮件属于垃圾邮件的概率，需要估计每个特征组合下的条件概率 $P(\text{特征}|\text{类别})$ 。以邮件 1 为例，其包含三个特征：“free”，“win”，和“prize”。为计算这封邮件是垃圾邮件的概率，实际上需要知道所有可能的特征组合出现在垃圾邮件中的概率。这不仅涉及到大量参数的估计，而且对于未在训练集中出现过的特征组合，根本无法得到准确的概率值。此外，随着特征数量的增加，计算复杂度呈指数级增长，使得这种方法变得不切实际。

7.4.3 朴素贝叶斯假设

为解决上述问题，可以做出一个简化假设：给定类别后，各个特征之间相互独立。这意味着，即使在同一封邮件中，“free”、“win”和“prize”这三个词是否同时出现并不影响它们各自作为垃圾邮件指示符的有效性。因此，原本复杂的联合概率分布可以分解为单个特征条件概率的乘积。

7.4.4 朴素贝叶斯解决邮件分类

现在使用朴素贝叶斯方法来判断一封新邮件是不是垃圾邮件。假设这封邮件含有两个特征：“free”和“urgent”。根据训练数据，可以统计出以下信息：

- $P(\text{垃圾邮件}) = \frac{3}{6}$, $P(\text{非垃圾邮件}) = \frac{3}{6}$
- $P(\text{free}|\text{垃圾邮件}) = \frac{2}{3}$, $P(\text{free}|\text{非垃圾邮件}) = 0$
- $P(\text{urgent}|\text{垃圾邮件}) = \frac{1}{3}$, $P(\text{urgent}|\text{非垃圾邮件}) = 0$

那么，该邮件是垃圾邮件的概率为：

$$\begin{aligned} P(\text{垃圾邮件}|\text{free, urgent}) &\propto P(\text{垃圾邮件}) \times P(\text{free}|\text{垃圾邮件}) \times P(\text{urgent}|\text{垃圾邮件}) \\ &= \frac{3}{6} \times \frac{2}{3} \times \frac{1}{3} \end{aligned} \quad (7.14)$$

而它不是垃圾邮件的概率则为零，因为在这组训练数据中，“free”和“urgent”从未同时出现在非垃圾邮件中。所以，根据朴素贝叶斯模型，这封邮件应该被分类为垃圾邮件。

7.4.5 朴素贝叶斯一般形式

在朴素贝叶斯分类器中，假设所有特征 X_i 都是条件独立的，即一个特征的存在不影响另一个特征的概率。因此，可以将联合条件概率 $P(X|Y)$ 简化为各个特征条件概率的乘积：

$$P(X = x|Y = y) = \prod_{i=1}^n P(x_i|Y = y) \quad (7.15)$$

这里 x_i 表示第 i 个特征， $\mathbf{x}$ 代表特征向量， Y 表示类别标签。为简化该问题，朴素贝叶斯假设每个特征 x_i 在给定类别 y 的情况下都是相互独立的。这种简化大大减少需要估计的参数数量，同时也降低了计算成本。

7.5 参数估计

为应用上述理论，需要从训练数据中估计出两个主要成分：先验概率 $P(Y = y)$ 和条件概率 $P(x_i|Y = y)$ 。先验概率可以通过计算训练集中每个类别的频率来获得；而条件概率则取决于特征的性质。对于离散型特征，可以直接统计各类别下每个特征值出现的次数；对于连续型特征，则通常假定它们服从某种分布（如高斯分布），并使用最大似然估计等方法来确定分布参数。

7.5.1 离散特征参数估计

当涉及到实际的数据集时，需要根据数据来估计上述公式的参数。对于离散特征，通常直接统计各类别下不同取值出现的频率。为更好地理解如何从数据中估计这些参数，将通过一个具体的例子来详细说明。

假设正在开发一个垃圾邮件过滤系统，使用朴素贝叶斯算法来预测一封邮件是否为垃圾邮件 (spam)。特征向量由一系列二进制变量组成，每个变量代表特定单词是否出现在邮件中。例如，如果单词“免费”出现在邮件里，则对应的 $x_{\text{免费}} = 1$ ；否则 $x_{\text{免费}} = 0$ 。

1. 先验概率的估计

首先，需要计算两类邮件——正常邮件和垃圾邮件——的先验概率 $P(Y = \text{正常})$ 和 $P(Y = \text{垃圾})$ 。这可以通过统计训练集中每种类别的邮件数量占总邮件数的比例来完成。

假定训练集包含 200 封邮件，其中 150 封是正常邮件，50 封是垃圾邮件。那么，对于类别“正常”的先验概率可以表示为：

$$P(Y = \text{正常}) = \frac{\text{正常邮件的数量}}{\text{总邮件数量}} = \frac{150}{200} = 0.75 \quad (7.16)$$

同理，对于类别“垃圾”的先验概率为：

$$P(Y = \text{垃圾}) = \frac{\text{垃圾邮件的数量}}{\text{总邮件数量}} = \frac{50}{200} = 0.25 \quad (7.17)$$

2. 条件概率的估计

接下来，需要估计给定类别下每个特征出现的条件概率。对于离散型特征（如二进制变量），只需要统计每个类别中该特征取值为 1 或 0 的频率。

以单词“免费”为例，如果发现所有垃圾邮件中有 40 封包含这个单词，而正常邮件中有 10 封包含它，那么可以得出以下条件概率：

$$\begin{aligned} P(x_{\text{免费}} = 1 | Y = \text{垃圾}) &= \frac{\text{垃圾邮件中包含“免费”的邮件数}}{\text{垃圾邮件总数}} = \frac{40}{50} = 0.8 \\ P(x_{\text{免费}} = 1 | Y = \text{正常}) &= \frac{\text{正常邮件中包含“免费”的邮件数}}{\text{正常邮件总数}} = \frac{10}{150} = \frac{1}{15} \end{aligned} \quad (7.18)$$

同样地，可以计算当 $x_{\text{免费}} = 0$ 时的条件概率。这里值得注意的是，即使某个特征在训练集中从未与某一类别共同出现过，也应该给予其一个小于零的概率，以避免零概率问题。这通常通过拉普拉斯平滑来实现，即在分子上加 1，在分母上加可能的特征值数目（对于二进制变量来说就是 2，具体参考下面的拉普拉斯方法 7.8.2）。

$$\begin{aligned} P(x_{\text{免费}} = 0 | Y = \text{垃圾}) &= \frac{\text{垃圾邮件中不含“免费”的邮件数} + 1}{\text{垃圾邮件总数} + 2} = \frac{10 + 1}{50 + 2} = \frac{11}{52} \\ P(x_{\text{免费}} = 0 | Y = \text{正常}) &= \frac{\text{正常邮件中不含“免费”的邮件数} + 1}{\text{正常邮件总数} + 2} = \frac{140 + 1}{150 + 2} = \frac{141}{152} \end{aligned} \quad (7.19)$$

7.5.2 连续特征

对于连续特征，不能像离散特征那样直接统计各类数量，为此，一般假设数据服从某种分布，通常假设它们服从正态分布，并利用训练样本估计均值 μ 和方差 σ^2 。

例如，如果特征是“重量”，分别计算苹果和橙子两组样本的平均重量以及各自的方差。然后，可以用这些统计量来定义条件概率密度函数，从而计算出任意重量出现在特定类别的概率。

$$P(X = x | Y = c) = \frac{1}{\sqrt{2\pi\sigma_c^2}} e^{-\frac{(x-\mu_c)^2}{2\sigma_c^2}} \quad (7.20)$$

其中， μ_c 和 σ_c^2 分别是类别 c 下特征 X 的均值和方差。这两个参数可以通过下面的最大似然估计 (Maximum Likelihood Estimation, MLE) 得到（参考 7.8.3）。

通过上述过程，不仅能够得到类别先验概率，还能获得给定类别下每个特征的条件概率。这些概率构成了朴素贝叶斯分类器的基础，能够在新的邮件到达时，基于其内容中的单词信息，准确地判断它是正常邮件还是垃圾邮件。此外，通过对实际数据进行分析，可以不断优化模型参数，提高分类准确性。

综上所述，朴素贝叶斯分类器通过结合贝叶斯定理和特征条件独立性的假设，提供了一种简单但有效的分类方法。尽管其“朴素”的假设有时并不完全符合实际情况，但在许多应用场景中，尤其是文本分类、垃圾邮件过滤等领域，它仍然表现出色。此外，由于其计算效率高，对缺失数据较为鲁棒，因此成为一种广泛使用的机器学习工具。

7.6 半朴素贝叶斯

7.6.1 半朴素贝叶斯的基本思想

半朴素贝叶斯是一种改进的贝叶斯分类器，它通过放松特征独立性假设，允许部分特征之间存在依赖关系，从而提升模型的表达能力。相比于朴素贝叶斯，半朴素贝叶斯更适合处理实际问题中特征间存在一定相关性的场景。

在教育领域，预测学生的学业表现是一个重要的研究方向。目标是希望通过学生的学习行为数据（如学习时间、作业完成率、课堂参与度等）预测其最终成绩是否达到优秀（例如，成绩是否大于等于 90 分）。在这个任务中，特征之间可能存在一定的相关性，例如：

- 学习时间和作业完成率可能高度相关；
- 课堂参与度和学习时间也可能存在一定的依赖关系。

在这种情况下，半朴素贝叶斯能够更好地捕捉特征之间的依赖关系，从而提高分类性能。

7.6.2 依赖关系引入

半朴素贝叶斯的核心思想是允许部分特征之间存在依赖关系，例如案例中的学习时间和作业完成率、课堂参与度和学习时间之间存在依赖关系。为引入依赖关系，引入贝叶斯定理，后验概率可以表示为：

$$P(Y = c | X_1 = x_1, X_2 = x_2, \dots, X_n = x_n) = \frac{P(Y = c) \cdot P(X_1 = x_1, X_2 = x_2, \dots, X_n = x_n | Y = c)}{P(X_1 = x_1, X_2 = x_2, \dots, X_n = x_n)} \quad (7.21)$$

这里，假设样本中的第 i 个特征为 x_i 。简单起见，上式表示为：

$$P(Y | x_1, x_2, \dots, x_n) = \frac{P(Y = c) \cdot P(x_1, x_2, \dots, x_n | Y = c)}{P(x_1, x_2, \dots, x_n)} \quad (7.22)$$

由于分母 $P(x_1, x_2, \dots, x_n)$ 对所有类别 c 相同，因此可以通过比例符号 $\propto$ 简化为：

$$P(Y = c | x_1, x_2, \dots, x_n) \propto P(Y = c) \cdot P(x_1, x_2, \dots, x_n | Y = c) \quad (7.23)$$

1. 条件概率的分解

为简化联合概率 $P(x_1, x_2, \dots, x_n|Y = c)$ ，利用条件概率的链式法则进行分解：

$$\begin{aligned} P(x_1, x_2, \dots, x_n|Y = c) &= P(x_1|Y = c) \\ &\cdot P(x_2|Y = c, x_1) \\ &\cdot P(x_3|Y = c, x_1, x_2) \cdots \end{aligned} \quad (7.24)$$

上述分解方式的核心在于逐步引入更多的条件变量来计算每个特征的概率。具体来说：第一个特征 x_1 的概率仅依赖于类别 y ，即 $P(x_1|Y = c)$ ；第二个特征 x_2 的概率不仅依赖于类别 y ，还依赖于第一个特征 x_1 ，即 $P(x_2|Y = c, x_1)$ ；第三个特征 x_3 的概率则进一步依赖于类别 y 和前两个特征 x_1 和 x_2 ，即 $P(x_3|Y = c, x_1, x_2)$ ；以此类推，第 n 个特征 x_n 的概率需要考虑类别 y 和所有前序特征 $x_1, x_2, \dots, x_{n-1}$ ，即 $P(x_n|Y = c, x_1, x_2, \dots, x_{n-1})$ 。随着特征数量 n 的增加，每个特征的概率计算都需要依赖越来越多的条件变量，导致模型的维度迅速增长。这种分解方式会引入复杂的高维依赖关系，导致计算复杂度急剧增加。

2. 半朴素贝叶斯假设

为缓解高维依赖带来的复杂性，半朴素贝叶斯引入一种折中的假设：允许特征之间存在有限依赖关系。其中最常用的策略是独依赖估计（One-Dependence Estimators (ODE)），即假设每个特征 x_j 在给定类别 $Y = c$ 的条件下最多依赖一个其他特征 x_i （称为父特征），即 x_j 在给定类别 $Y = c$ 的条件下独立于其他特征 x_k （只和 x_i 有关，和其他特征无关）：

$$P(X_j|Y = c, x_i, x_k) = P(X_j|Y = c, x_i) \quad (\text{对任意 } k \neq i, j) \quad (7.25)$$

所以 $P(X_j = x_j|Y = c, X_1 = x_1, X_2 = x_2, \dots, X_n = x_n)$ 可以表示为：

$$P(X_j = x_j|Y = c, X_1 = x_1, X_2 = x_2, \dots, X_n = x_n) = P(X_j = x_j|Y = c, X_i = x_i) \quad (7.26)$$

上面的公式只考虑一个特征 X_j ，如果考虑所有的特征，则条件类联合概率可以表达为

$$\begin{aligned} P(X_1 = x_1, X_2 = x_2, \dots, X_n = x_n|Y = c) &\approx \\ \sum_{i=1}^n P(X_i = x_i|Y = c) \prod_{j \neq i} P(X_j = x_j|Y = c, X_i = x_i) \end{aligned} \quad (7.27)$$

超父独依赖估计（Super-Parent One-Dependence Estimator, SPODE），平均独依赖估计（Averaged One-Dependence Estimators, AODE）和树增强朴素贝叶斯（Tree-Augmented Naive Bayes, TAN）是半朴素贝叶斯分类器中处理属性依赖关系的三种主要方法，它们在假设、实现方式和适用场景上存在显著差异。以下从区别、实现方式和案例分析三方面详细阐述。

7.6.3 SPODE 半朴素贝叶斯分类器

SPODE（Super-Parent One-Dependence Estimator）是一种经典的半朴素贝叶斯分类器，它通过引入超父特征来放松特征独立性假设，从而允许部分特征之间存在依赖关系。相比于朴素贝叶斯，SPODE 能够更好地捕捉特征之间的相关性，同时保持较低的计算复杂度。

SPODE 的核心思想是选择一个特定的特征作为超父特征，并假设其他所有特征在给定类别和超父特征的条件下是条件独立的。例如，在学生成绩预测任务中，如果选择学习时间作为超父特征，则可以假设作业完成率和课堂参与度在已知成绩类别和学习时间的条件下是独立的。下图7.1给出了以特征 x_j 为超父节点的示意图。

对联合概率 $P(x_1, x_2, \dots, x_n|Y = c)$ 进行分解。根据 SPODE 的假设，所有特征 x_j 在给定类别

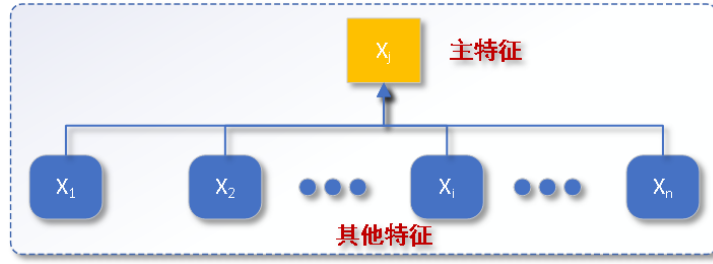


图 7.1: SPODE 方法示意图

$Y = c$ 和超父特征 x_p 的条件下是条件独立的。即：

$$P(x_j|Y = c, x_1, x_2, \dots, x_n) \approx P(x_j|Y = c, x_p) \quad (7.28)$$

从联合概率的链式法则出发，逐步应用 SPODE 的假设进行简化：首先，根据链式法则，联合概率可以写为：

$$\begin{aligned} P(x_1, x_2, \dots, x_n|Y = c) = & \\ & P(x_p|Y = c) \cdot P(x_1|Y = c, x_p) \cdot \\ & P(x_2|Y = c, x_p, x_1) \cdots \\ & P(x_n|Y = c, x_p, x_1, x_2, \dots, x_{n-1}) \end{aligned} \quad (7.29)$$

然后，应用 SPODE 的条件独立性假设，所有特征 x_j ($j \neq p$) 在给定 $Y = c$ 和 x_p 的条件下是独立的。因此，上述公式可以简化为：

$$P(x_1, x_2, \dots, x_n|Y = c) = P(x_p|Y = c) \prod_{j \neq p} P(x_j|Y = c, x_p) \quad (7.30)$$

为便于理解，以学生成绩预测为例，假设有以下特征：学习时间（小时），作业完成率（百分比），课堂参与度（评分范围为 1 到 5）。如果选择学习时间作为超父特征 x_p ，则 SPODE 假设：作业完成率和课堂参与度在已知成绩类别 $Y = c$ 和学习时间 x_p 的条件下是独立的。换句话说，SPODE 假设：

$$\begin{aligned} & P(\text{作业完成率}, \text{课堂参与度}|Y = c, \text{学习时间}) = \\ & P(\text{作业完成率}|Y = c, \text{学习时间}) \cdot P(\text{课堂参与度}|Y = c, \text{学习时间}) \end{aligned} \quad (7.31)$$

这种假设不仅可以简化模型的计算，同时能够保留特征之间的部分依赖关系。SPODE 的主要优势在于其计算效率高，适合处理特征数量较多的问题。然而，它的性能高度依赖于超父特征的选择。如果选择的超父特征与目标变量无关，则可能导致模型性能下降。

7.6.4 AODE 半朴素贝叶斯分类器

AODE (Averaged One-Dependence Estimators) 是一种改进的半朴素贝叶斯分类器，它通过对多个一对多依赖模型的结果进行加权平均，避免单一模型可能存在的偏差，从而提高预测的鲁棒性。AODE 的核心思想是构建多个子模型，每个子模型假设其他特征在给定类别和某个特定特征的条件下是独立的。然后，AODE 对这些子模型的结果进行加权平均，得到最终的预测结果。

1. 单个子模型的联合概率

对于每个特征 x_i ，构建一个子模型，假设其他特征 x_j 在给定类别 $Y = c$ 和 x_i 的条件下是条件独立的，例如就使用上面刚介绍的 SPODE，因此，联合概率可以表示为：

$$P(x_1, x_2, \dots, x_n | Y = c) \approx P(x_i | Y = c) \prod_{j \neq i} P(x_j | Y = c, x_i) \quad (7.32)$$

2. 多个子模型的加权平均

AODE 的核心思想是对所有可能的子模型结果进行加权平均。具体来说，可以将联合概率近似为所有子模型的联合概率的加权和：

$$P(x_1, x_2, \dots, x_n | Y = c) \approx \sum_{i=1}^n P(x_i | Y = c) \prod_{j \neq i} P(x_j | Y = c, x_i) \quad (7.33)$$

注意上面多个子模型的公式同单个子模型的联合概率相比，多了一个求和的操作。对于所有数据而言，每个样本有 n 个特征，需要考虑所有的特征属性，所以有 n 个这样的单个模型。求和操作相当于考虑所有的特征情况。

继续以学生成绩预测为例，有三个特征：学习时间、作业完成率和课堂参与度。AODE 会分别为每个特征构建一个子模型：

- 如果选择学习时间为主特征，则假设作业完成率和课堂参与度在已知成绩类别和学习时间的条件下是独立的。
- 如果选择作业完成率为主特征，则假设学习时间和课堂参与度在已知成绩类别和作业完成率的条件下是独立的。
- 如果选择课堂参与度为主特征，则假设学习时间和作业完成率在已知成绩类别和课堂参与度的条件下是独立的。

AODE 最终将这三个子模型的结果进行加权平均，得到最终的预测结果。AODE 的主要优势在于其鲁棒性强，能够避免单一模型可能存在的偏差。然而，由于需要构建多个子模型，AODE 的计算复杂度略高于 SPODE。

7.6.5 TAN 半朴素贝叶斯分类器

TAN (Tree-Augmented Naive Bayes) 是一种基于树结构的半朴素贝叶斯分类器，它通过构建特征之间的树状依赖关系图，进一步放松特征独立性假设，从而提升模型的表达能力。TAN 的核心思想是利用最大生成树算法构建特征之间的依赖关系图。在该图中，每个特征最多依赖于一个其他特征（除了根节点），从而形成一棵树状结构。这种设计既可以保留特征之间的部分依赖关系，又能够避免完全建模高维依赖带来的复杂性。

首先，计算根节点的概率。根节点 x_r 的条件概率仅依赖于类别 $Y = c$ ，即：

$$P(x_r | Y = c) \quad (7.34)$$

其次，计算非根节点的概率。对于每个非根节点特征 x_j ($j \neq r$)，其条件概率依赖于类别 $Y = c$ 和其父节点特征 $x_{\text{parent}(j)}$ ，即：

$$P(x_j | Y = c, x_{\text{parent}(j)}) \quad (7.35)$$

最后，对联合概率进行分解。将所有特征的概率相乘，得到联合概率的最终形式：

$$\begin{aligned}
 P(x_1, x_2, \dots, x_n | Y = c) &= P(x_1 | Y = c) \cdot P(x_2 | Y = c, x_1) \cdot \\
 &\quad P(x_3 | Y = c, x_1, x_2) \cdots \\
 &\quad P(x_n | Y = c, x_1, x_2, \dots, x_{n-1}) \\
 &= P(x_r | Y = c) \prod_{j \neq r} P(x_j | Y = c, x_{\text{parent}(j)})
 \end{aligned} \tag{7.36}$$

仍然以学生成绩预测为例，假设有三个特征：学习时间、作业完成率和课堂参与度。TAN 会构建一棵树状依赖关系图，例如：以学习时间作为根节点，作业完成率依赖于学习时间，课堂参与度依赖于学习时间。在这种情况下，联合概率可以分解为：

$$\begin{aligned}
 P(\text{学习时间}, \text{作业完成率}, \text{课堂参与度} | Y = c) &= \\
 P(\text{学习时间} | Y = c) \cdot P(\text{作业完成率} | Y = c, \text{学习时间}) \cdot \\
 P(\text{课堂参与度} | Y = c, \text{学习时间})
 \end{aligned} \tag{7.37}$$

TAN 的主要优势在于其能够灵活地捕捉特征之间的依赖关系，同时保持较低的计算复杂度。然而，TAN 的性能依赖于树状依赖关系图的构建质量。如果依赖关系图不准确，则可能导致模型性能下降。

7.7 实践练习

7.7.1 示例：电子邮件分类

考虑一个实际问题，即自动过滤垃圾邮件。将使用朴素贝叶斯分类器来判断一封电子邮件是否为垃圾邮件，区分垃圾邮件（spam）和正常邮件（ham）。以下是详细的代码及注释。

```

1  # 导入必要的库
2  import numpy as np
3  import pandas as pd
4  from sklearn.model_selection import train_test_split
5  from sklearn.feature_extraction.text import CountVectorizer
6  from sklearn.naive_bayes import MultinomialNB
7  from sklearn.metrics import accuracy_score, classification_report
8
9  # 1. 数据准备
10 """
11 我们使用一个模拟的电子邮件数据集，其中包含邮件内容和标签。
12 标签为 'spam' 表示垃圾邮件，'ham' 表示正常邮件。
13 """
14 data = {
15     "email": [
16         "Win money now!",
17         "Hey, can we meet tomorrow?",
18         "Free entry in a contest",
19         "Important meeting at 10 AM",
20         "Congratulations! You've won a prize",
21         "Please review the attached document"
22     ],
23     "label": ["spam", "ham", "spam", "ham", "spam", "ham"]
24 }
25
26 # 将数据转换为 DataFrame
27 df = pd.DataFrame(data)

```

```

28
29 # 查看数据集
30 print("原始数据集:")
31 print(df)
32
33 # 2. 特征提取
34 """
35 将文本数据转换为数值特征向量。我们使用 CountVectorizer 将邮件内容转换为词频矩阵。
36 """
37 vectorizer = CountVectorizer()
38 X = vectorizer.fit_transform(df["email"]) # 提取特征
39 y = df["label"] # 标签
40
41 # 查看特征矩阵
42 print("\n特征矩阵:")
43 print(X.toarray())
44
45 # 3. 数据集划分
46 """
47 将数据集划分为训练集和测试集，比例为 7:3。
48 """
49 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
50
51 # 4. 使用朴素贝叶斯进行训练
52 """
53 初始化并训练朴素贝叶斯分类器 (MultinomialNB)，适用于离散特征（如词频）。
54 """
55 model = MultinomialNB()
56 model.fit(X_train, y_train)
57
58 # 5. 在测试集上进行预测
59 """
60 在测试集上进行预测，并计算准确率和分类报告。
61 """
62 y_pred = model.predict(X_test)
63 print("\n预测结果:")
64 print(y_pred)
65
66 # 计算准确率
67 accuracy = accuracy_score(y_test, y_pred)
68 print(f"朴素贝叶斯分类器准确率: {accuracy:.2f}")
69
70 # 输出分类报告
71 report = classification_report(y_test, y_pred)
72 print("\n分类报告:")
73 print(report)
74
75 # 6. 测试新邮件
76 """
77 我们可以使用训练好的模型对新的邮件进行分类。
78 """
79 new_emails = ["You have won a free ticket", "Let's catch up over coffee"]
80 new_emails_transformed = vectorizer.transform(new_emails) # 转换为特征向量
81 new_predictions = model.predict(new_emails_transformed)
82
83 print("\n新邮件分类结果:")
84 for email, label in zip(new_emails, new_predictions):
85     print(f"邮件内容: '{email}' -> 分类: {label}")

```

运行上述代码后，可以得到以下输出：

原始数据集：

```
email label
0          Win money now!  spam
1  Hey, can we meet tomorrow?  ham
2      Free entry in a contest  spam
3  Important meeting at 10 AM  ham
4  Congratulations! You've won a prize  spam
5  Please review the attached document  ham
```

特征矩阵:

```
[[0 0 0 1 0 0 1 0 0 0]
 [0 1 0 0 0 0 0 0 0 1]
 [0 0 0 0 1 0 0 1 0 0]
 [0 0 1 0 0 1 0 0 0 0]
 [0 0 0 0 0 0 1 0 1 0]
 [1 0 0 0 0 0 0 0 0 0]]
```

预测结果:

```
['ham' 'spam']
```

朴素贝叶斯分类器准确率: 1.00

分类报告:

precision	recall	f1-score	support	
ham	1.00	1.00	1.00	1
spam	1.00	1.00	1.00	1
accuracy			1.00	2
macro avg	1.00	1.00	1.00	2
weighted avg	1.00	1.00	1.00	2

新邮件分类结果:

邮件内容: 'You have won a free ticket' -> 分类: spam

邮件内容: "Let's catch up over coffee" -> 分类: ham

从结果可以看出: 模型在测试集上的分类准确率为 100%; 新邮件的分类结果与预期一致。通过本次实践, 展示了如何使用朴素贝叶斯模型对电子邮件进行分类。朴素贝叶斯是一种简单而高效的分类算法, 特别适合处理文本分类任务 (如垃圾邮件检测)。在实际应用中, 可以通过引入更多的特征 (如 TF-IDF) 或优化预处理步骤进一步提升模型性能。

7.7.2 示例: 疾病诊断

疾病诊断是医疗领域中的重要问题。本节通过朴素贝叶斯分类器实现疾病预测, 利用贝叶斯定理计算患者患有某种疾病的概率。通过该实践, 掌握朴素贝叶斯的基本原理及其在实际问题中的应用。

首先, 准备数据。使用一个模拟的医疗数据集, 包含患者的症状特征 (如体温、血压、血糖水平等) 以及是否患病的标签。对数据进行预处理, 包括:

- 数据清洗：去除缺失值或异常值。
- 特征离散化：将连续特征（如体温）划分为离散区间。
- 数据划分：将数据集分为训练集和测试集。

其次，构建模型。基于朴素贝叶斯公式，构建分类器。假设每个特征条件独立，则后验概率可表示为：

$$P(\text{Disease}|\text{Symptoms}) = \frac{P(\text{Symptoms}|\text{Disease}) \cdot P(\text{Disease})}{P(\text{Symptoms})} \quad (7.38)$$

其中， $P(\text{Disease})$ 是先验概率，表示患病的概率。 $P(\text{Symptoms}|\text{Disease})$ 是条件概率，表示在已知患病的情况下出现特定症状的概率。 $P(\text{Symptoms})$ 是归一化常数，用于确保概率总和为 1。

最后，对模型进行评估。使用测试集评估模型性能，计算准确率、召回率和 F1 分数等指标。下面是基于 Python 的朴素贝叶斯分类器实现代码。

```

1  # 导入必要的库
2  import numpy as np
3  from sklearn.model_selection import train_test_split
4  from sklearn.naive_bayes import CategoricalNB
5  from sklearn.metrics import accuracy_score, classification_report
6  from sklearn.preprocessing import OrdinalEncoder
7
8  # 数据集使用了模拟的医疗数据，包含患者的症状特征和是否患病的标签
9  # 数据格式：[体温(低/中/高), 血压(低/正常/高), 血糖(低/正常/高)]
10 data = [
11     ["低", "正常", "低", "无病"],
12     ["中", "正常", "正常", "无病"],
13     ["高", "高", "高", "有病"],
14     ["高", "正常", "高", "有病"],
15     ["低", "低", "正常", "无病"],
16     ["中", "高", "高", "有病"],
17     ["低", "正常", "低", "无病"],
18     ["高", "高", "正常", "有病"],
19     ["中", "正常", "正常", "无病"],
20     ["高", "低", "高", "有病"]
21 ]
22
23 # 将数据转换为特征和标签
24 X = np.array([row[:-1] for row in data]) # 特征 (症状)
25 y = np.array([row[-1] for row in data]) # 标签 (是否患病)
26
27 # 使用 OrdinalEncoder 将字符串特征编码为数值，便于模型处理
28 encoder = OrdinalEncoder()
29 X_encoded = encoder.fit_transform(X)
30
31 # 通过 train_test_split 将数据划分为训练集和测试集，确保模型评估的可靠性
32 X_train, X_test, y_train, y_test = train_test_split(X_encoded, y, test_size=0.3, random_state=42)
33
34 # 使用 CategoricalNB 实现朴素贝叶斯分类器，适用于离散特征
35 model = CategoricalNB()
36 model.fit(X_train, y_train)
37
38 # 在测试集上进行预测
39 y_pred = model.predict(X_test)
40
41 # 输出评估结果
42 print("预测结果:", y_pred)
43 print("真实标签:", y_test)
44 print("准确率:", accuracy_score(y_test, y_pred))
45 print("分类报告:\n", classification_report(y_test, y_pred))

```

```

46
47 # 示例：预测新患者是否患病，展示其在实际问题中的应用
48 new_patient = [["高", "正常", "高"]] # 新患者的症状
49 new_patient_encoded = encoder.transform(new_patient) # 编码新患者数据
50 prediction = model.predict(new_patient_encoded)
51 print("新患者预测结果:", prediction[0])

```

本实践展示了如何利用朴素贝叶斯分类器解决疾病诊断问题。通过简单的数据预处理和模型构建，能够快速实现一个有效的分类系统。朴素贝叶斯方法因其简单性和高效性，在小规模数据集上表现尤为突出。

7.7.3 示例：半朴素贝叶斯

本节通过一个完整的 Python 程序展示如何使用半朴素贝叶斯分类器（AODE）进行训练和预测，模拟学生成绩数据集，并利用 AODE 方法实现分类任务。以下是完整的代码及详细注释：

```

1 # 导入必要的库
2 import numpy as np
3 from sklearn.model_selection import train_test_split
4 from sklearn.naive_bayes import CategoricalNB
5 from sklearn.metrics import accuracy_score
6
7 # 定义 AODE 类
8 class AODE:
9     def __init__(self):
10         """
11         初始化 AODE 模型。
12         self.models 用于存储多个“一对多依赖”模型。
13         """
14         self.models = []
15
16     def fit(self, X, y):
17         """
18         构建多个“一对多依赖”模型。
19         对于每个特征 i，排除该特征后训练一个朴素贝叶斯模型。
20         """
21         n_features = X.shape[1] # 获取特征数量
22         for i in range(n_features):
23             model = CategoricalNB() # 使用 Scikit-learn 的朴素贝叶斯分类器
24             model.fit(np.delete(X, i, axis=1), y) # 排除第 i 个特征
25             self.models.append(model)
26
27     def predict(self, X):
28         """
29         对多个模型的结果进行加权平均。
30         每个模型基于排除某个特征后的数据进行预测，最终取所有模型结果的平均值。
31         """
32         predictions = np.zeros((X.shape[0], len(self.models))) # 初始化预测结果矩阵
33         for i, model in enumerate(self.models):
34             predictions[:, i] = model.predict(np.delete(X, i, axis=1)) # 排除第 i 个特征后预测
35         return np.round(predictions.mean(axis=1)).astype(int) # 取平均值并四舍五入为整数
36
37 # 模拟学生成绩数据集
38 """
39 数据格式：[学习时间(小时)，作业完成率(%)，课堂参与度(0-5评分)，成绩是否优秀(0/1)]
40 目标变量是成绩是否优秀（0 表示非优秀，1 表示优秀）。
41 """
42 data = [
43     [8, 90, 4, 1], # 优秀
44     [6, 70, 3, 0], # 非优秀
45     [10, 95, 5, 1], # 优秀

```

```

46 [5, 60, 2, 0], # 非优秀
47 [7, 80, 4, 1], # 优秀
48 [4, 50, 1, 0], # 非优秀
49 [9, 85, 5, 1], # 优秀
50 [3, 40, 2, 0], # 非优秀
51 ]
52
53 # 将数据转换为特征和标签
54 X = np.array([row[:-1] for row in data]) # 特征
55 y = np.array([row[-1] for row in data]) # 标签
56
57 # 数据集划分
58 """
59 将数据集划分为训练集和测试集，比例为 7:3。
60 random_state 固定随机种子以确保结果可复现。
61 """
62 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=55)
63
64 # 使用 AODE 进行训练和预测
65 """
66 初始化 AODE 模型并进行训练。
67 在测试集上进行预测，并计算准确率。
68 """
69 aode_model = AODE()
70 aode_model.fit(X_train, y_train) # 训练 AODE 模型
71 aode_pred = aode_model.predict(X_test) # 在测试集上进行预测
72
73 # 输出准确率和预测结果
74 print("半朴素贝叶斯 (AODE) 准确率:", accuracy_score(y_test, aode_pred))
75 print("半朴素贝叶斯 (AODE) 预测结果:", aode_pred)

```

运行上述代码后，可以得到 AODE 模型的准确率和预测结果。例如，输出可能如下：

半朴素贝叶斯 (AODE) 准确率: 1.0

半朴素贝叶斯 (AODE) 预测结果: [1 0 1]

这表明 AODE 模型在测试集上的分类准确率为 100%，并且预测结果与真实标签完全一致。本次实践演示了如何使用 AODE 方法实现半朴素贝叶斯分类器。相比于朴素贝叶斯，AODE 能够更好地捕捉特征之间的部分依赖关系，从而提升分类性能。在实际应用中，可以根据具体问题选择合适的半朴素贝叶斯方法（如 SPODE、AODE 或 TAN），以平衡模型的复杂度和性能。

7.8 补充知识

7.8.1 概率论的基础概念

1. 随机试验与样本空间

概率论研究的是随机现象的规律性。随机试验是指在相同条件下可以重复进行，且每次试验的结果具有不确定性的过程。例如，掷一枚硬币、掷一颗骰子或从一副扑克牌中抽取一张牌都属于随机试验。

随机试验的所有可能结果组成的集合称为样本空间（Sample Space），通常用符号 S 表示。例如：

- 掷一枚硬币的样本空间为 $S = \{\text{正面}, \text{反面}\}$ 。
- 掷一颗骰子的样本空间为 $S = \{1, 2, 3, 4, 5, 6\}$ 。

2. 事件与概率

样本空间的子集称为事件 (Event)。事件是随机试验中感兴趣的结果集合。例如，在掷一颗骰子的试验中，“出现偶数点”是一个事件，对应的集合为 $\{2, 4, 6\}$ 。

概率 (Probability) 是对事件发生可能性的量化度量，通常用 $P(A)$ 表示事件 A 的概率。概率满足以下基本性质：

- 非负性：对于任意事件 A ，有 $P(A) \geq 0$ 。
- 规范性：样本空间的概率为 1，即 $P(S) = 1$ 。
- 可加性：如果事件 $A_1, A_2, \dots$ 互不相交，则

$$P\left(\bigcup_{i=1}^{\infty} A_i\right) = \sum_{i=1}^{\infty} P(A_i) \quad (7.39)$$

3. 条件概率与独立性

条件概率 (Conditional Probability) 描述了在已知某个事件发生的条件下，另一个事件发生的概率。设 A 和 B 是两个事件，且 $P(B) > 0$ ，则 A 在 B 条件下的条件概率定义为：

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \quad (7.40)$$

如果事件 A 和 B 满足 $P(A \cap B) = P(A) \cdot P(B)$ ，则称 A 和 B 是相互独立的 (Independent Events)。此时，事件 A 的发生与否不会影响事件 B 的发生概率。

4. 全概率公式与贝叶斯定理

全概率公式 (Law of Total Probability) 用于计算复杂事件的概率。假设样本空间 S 被划分为若干个互不相交的事件 $B_1, B_2, \dots, B_n$ ，即 $S = \bigcup_{i=1}^n B_i$ ，则对于任意事件 A ，有：

$$P(A) = \sum_{i=1}^n P(A|B_i)P(B_i) \quad (7.41)$$

贝叶斯定理 (Bayes' Theorem) 是条件概率的一个重要推论，用于更新事件的概率估计。其公式为：

$$P(B|A) = \frac{P(A|B)P(B)}{P(A)} \quad (7.42)$$

其中， $P(B)$ 称为先验概率 (Prior Probability)， $P(B|A)$ 称为后验概率 (Posterior Probability)。

5. 全概率与贝叶斯公式理解

如下图7.2所示，图中包含两层变量：上层为变量 B ，下层为变量 A 。要使变量 A 发生，必须通过变量 B 的某一条路径。具体来说，变量 A 的发生可以通过路径 $B_1, B_2, \dots, B_n$ 中的任意一条实现。因此，计算随机变量 A 发生的概率时，可以利用加法定理。

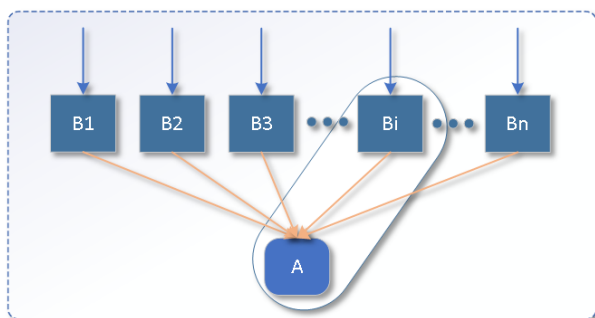


图 7.2: 全概率公式和贝叶斯公式理解示意图

对于路径 i ，其发生的概率分为两步：首先，随机变量 B_i 必须发生，其概率为 $P(B_i)$ ；然后，在 B_i 发生的前提下，变量 A 发生的概率为 $P(A|B_i)$ 。

因为路径 i 包括两步，所以路径 i 发生的话，需要使用乘法原理，即 $P(A|B_i)P(B_i)$ ，由于总共有 n 条路径可以导致 A 发生，随机变量 A 的总概率可表示为下式：

$$P(A) = \sum_{i=1}^n P(A|B_i)P(B_i) \quad (7.43)$$

可以看出上面的公式和前面的全概率公式 (7.41) 相同。接下来，如果想计算某条特定路径 i 导致 A 发生的条件概率 $P(B_i|A)$ 。具体而言，路径 i 的条件概率等于该路径发生的概率与所有路径总概率的比值：

$$P(B_i|A) = \frac{P(A|B_i)P(B_i)}{P(A)} \quad (7.44)$$

其中，分子部分 $P(A|B_i)P(B_i)$ 表示路径 i 发生的概率；分母部分 $P(A)$ 表示随机变量 A 发生的总概率，即所有路径的概率之和，即公式 (7.43)，将其代入上面公式，最终得到：

$$P(B_i|A) = \frac{P(A|B_i)P(B_i)}{\sum_{i=1}^n P(A|B_i)P(B_i)} \quad (7.45)$$

该公式等同于上面的公式 (7.44)，与公式 (7.42) 一致，体现了贝叶斯定理的核心思想：通过已知的条件概率 $P(A|B_i)$ 和先验概率 $P(B_i)$ ，计算后验概率。

6. 随机变量与分布

随机变量 (Random Variable) 是将样本空间中的每个结果映射到实数的函数。随机变量可以分为离散型和连续型两种：

- 离散型随机变量：取值为有限或可数无穷多个，例如掷骰子的点数。
- 连续型随机变量：取值为某个区间内的任意实数，例如测量某物体的长度。

随机变量的概率分布描述其取值的可能性。常见的概率分布包括：

- 离散型分布：如二项分布、泊松分布。
- 连续型分布：如正态分布、均匀分布。

7. 期望与方差

期望 (Expectation) 是随机变量的平均值，表示随机变量的中心趋势。对于离散型随机变量 X ，其期望定义为：

$$E(X) = \sum_x x \cdot P(X = x) \quad (7.46)$$

对于连续型随机变量 X ，其期望定义为：

$$E(X) = \int_{-\infty}^{\infty} x \cdot f(x) dx \quad (7.47)$$

其中 $f(x)$ 是 X 的概率密度函数。

方差 (Variance) 衡量随机变量与其期望之间的偏离程度，定义为：

$$\text{Var}(X) = E[(X - E(X))^2] = E(X^2) - [E(X)]^2 \quad (7.48)$$

7.8.2 拉普拉斯平滑

1. 背景与问题

在许多机器学习任务中（如朴素贝叶斯分类器），需要根据训练数据计算条件概率。例如：

$$P(x_i|y) = \frac{\text{Count}(x_i, y)}{\text{Count}(y)} \quad (7.49)$$

其中， $\text{Count}(x_i, y)$ 表示类别 y 中特征 x_i 出现的次数， $\text{Count}(y)$ 表示类别 y 的总样本数。

然而，如果某个特征值 x_i 在训练数据中从未出现过，则 $\text{Count}(x_i, y) = 0$ ，导致 $P(x_i|y) = 0$ 。这种零概率问题会影响模型的泛化能力，尤其是在处理未见过的数据时。

2. 拉普拉斯平滑的核心思想

拉普拉斯平滑通过在分子和分母上分别加上一个小的常数（通常为 1），避免零概率问题。其公式为：

$$P(x_i|y) = \frac{\text{Count}(x_i, y) + \alpha}{\text{Count}(y) + \alpha \cdot k} \quad (7.50)$$

其中， α 是平滑参数，通常取值为 1， k 是特征 x_i 可能取值的总数（即可能的类别数或状态数）。通过引入平滑参数 α ，即使 $\text{Count}(x_i, y) = 0$ ，概率也不会为零，而是被赋予一个较小的正值。

3. 示例

假设有一个文本分类任务，词汇表中有 3 个单词： $\{A, B, C\}$ 。在类别 y_1 中，单词 A 和 B 各出现 2 次，而单词 C 未出现。不使用平滑时，单词 C 的条件概率为：

$$P(C|y_1) = \frac{\text{Count}(C, y_1)}{\text{Count}(y_1)} = \frac{0}{4} = 0 \quad (7.51)$$

使用拉普拉斯平滑后：

$$P(C|y_1) = \frac{\text{Count}(C, y_1) + 1}{\text{Count}(y_1) + 1 \cdot 3} = \frac{0 + 1}{4 + 3} = \frac{1}{7} \quad (7.52)$$

4. 优点与应用场景

拉普拉斯平滑具有以下优点：

- 避免零概率问题，提高模型的鲁棒性。
- 简单易实现，适用于多种概率估计场景。

拉普拉斯平滑的应用场景包括：

- 朴素贝叶斯分类器中的条件概率估计。
- 语言模型中的 n-gram 概率计算。
- 其他涉及稀疏数据的统计建模任务。

7.8.3 最大似然估计正态分布的参数

1. 背景与目标

在机器学习中，正态分布常用于建模连续型数据。假设有一组样本 $X = \{x_1, x_2, \dots, x_n\}$ ，它们服从正态分布：

$$P(X = x|Y = c) = \frac{1}{\sqrt{2\pi\sigma_c^2}} e^{-\frac{(x-\mu_c)^2}{2\sigma_c^2}} \quad (7.53)$$

其中， μ_c 和 σ_c^2 分别表示类别 c 的均值和方差。这里的目标是通过最大似然估计（Maximum Likelihood Estimation, MLE）方法，从样本数据中估计出正态分布的参数 μ_c 和 σ_c^2 。

2. 数学推导

对于一组独立同分布（i.i.d.）的样本 $X = \{x_1, x_2, \dots, x_n\}$ ，其联合概率密度函数（即似然函数）为：

$$L(\mu_c, \sigma_c^2) = \prod_{i=1}^n P(x_i|\mu_c, \sigma_c^2) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma_c^2}} e^{-\frac{(x_i-\mu_c)^2}{2\sigma_c^2}} \quad (7.54)$$

为简化计算，取对数似然函数：

$$\ln L(\mu_c, \sigma_c^2) = -\frac{n}{2} \ln(2\pi) - \frac{n}{2} \ln(\sigma_c^2) - \frac{1}{2\sigma_c^2} \sum_{i=1}^n (x_i - \mu_c)^2 \quad (7.55)$$

接下来，分别对 μ_c 和 σ_c^2 求偏导，并令导数为零以最大化对数似然函数。

3. 对 μ_c 的估计

对 μ_c 求偏导：

$$\frac{\partial \ln L(\mu_c, \sigma_c^2)}{\partial \mu_c} = \frac{1}{\sigma_c^2} \sum_{i=1}^n (x_i - \mu_c) \quad (7.56)$$

令导数为零，得到：

$$\sum_{i=1}^n (x_i - \mu_c) = 0 \quad \Rightarrow \quad \mu_c = \frac{1}{n} \sum_{i=1}^n x_i \quad (7.57)$$

因此， μ_c 的最大似然估计为样本均值：

$$\hat{\mu}_c = \frac{1}{n} \sum_{i=1}^n x_i \quad (7.58)$$

4. 对 σ_c^2 的估计

对 σ_c^2 求偏导：

$$\frac{\partial \ln L(\mu_c, \sigma_c^2)}{\partial \sigma_c^2} = -\frac{n}{2\sigma_c^2} + \frac{1}{2(\sigma_c^2)^2} \sum_{i=1}^n (x_i - \mu_c)^2 \quad (7.59)$$

令导数为零，得到：

$$-\frac{n}{2\sigma_c^2} + \frac{1}{2(\sigma_c^2)^2} \sum_{i=1}^n (x_i - \mu_c)^2 = 0 \quad (7.60)$$

整理后可得：

$$\sigma_c^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu_c)^2 \quad (7.61)$$

因此, σ_c^2 的最大似然估计为样本方差:

$$\hat{\sigma}_c^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{\mu}_c)^2 \quad (7.62)$$

需要注意的是, 上述方差估计是有偏估计 (Bias)。如果需要无偏估计, 可以将分母改为 $n - 1$:

$$\hat{\sigma}_c^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \hat{\mu}_c)^2 \quad (7.63)$$

5. 总结

通过最大似然估计, 可以得到正态分布参数的估计公式:

- 均值: $\hat{\mu}_c = \frac{1}{n} \sum_{i=1}^n x_i$
- 方差: $\hat{\sigma}_c^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{\mu}_c)^2$ (有偏)
- 无偏方差: $\hat{\sigma}_c^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \hat{\mu}_c)^2$

6. Python 实现

以下是一个简单的 Python 代码示例, 用于计算正态分布的参数估计:

```
1 import numpy as np
2
3 # 样本数据
4 data = np.array([1.2, 2.3, 3.1, 4.0, 5.5])
5
6 # 计算均值
7 mu_hat = np.mean(data)
8
9 # 计算方差 (有偏)
10 sigma2_hat_biased = np.var(data, ddof=0)
11
12 # 计算方差 (无偏)
13 sigma2_hat_unbiased = np.var(data, ddof=1)
14
15 print("样本均值 (mu_hat):", mu_hat)
16 print("有偏方差 (sigma2_hat_biased):", sigma2_hat_biased)
17 print("无偏方差 (sigma2_hat_unbiased):", sigma2_hat_unbiased)
```

运行结果示例:

样本均值 (mu_hat): 3.22

有偏方差 (sigma2_hat_biased): 1.8376

无偏方差 (sigma2_hat_unbiased): 2.297

7.9 习题

7.9.1 简答题

1. 解释为什么朴素贝叶斯被称为“朴素”的原因是什么?
2. 在什么情况下朴素贝叶斯可能不是最佳选择? 给出至少两个例子。
3. 编写一段 Python 代码实现一个简单的文本分类任务, 使用朴素贝叶斯方法。

7.9.2 计算题

如果某疾病在人群中的患病率为 1%，而一项检测该疾病的测试有 99% 的准确性（即如果一个人患有此病，测试呈阳性的概率为 99%，且如果一个人不患此病，测试呈阴性的概率也为 99%）。如果一个人的测试结果是阳性，那么他实际上患病的概率是多少？

7.9.3 编程题

1. 使用提供的训练数据，构建一个朴素贝叶斯分类器来区分垃圾邮件与非垃圾邮件。
2. 编写一个 Python 函数，该函数接收一个文本字符串列表和对应的类别标签作为输入，使用朴素贝叶斯分类器训练模型，并返回模型的预测准确度。

第八章 聚 类

在机器学习领域，聚类是一种无监督学习方法，旨在将数据点分组为若干个簇（clusters），使得同一簇内的数据点彼此相似，而不同簇的数据点差异较大。聚类广泛应用于计算视觉中的图像分割、目标检测以及数据压缩等领域。例如，在自动驾驶场景中，通过聚类可以将道路标志与背景分离，从而帮助车辆识别关键信息。

本章将带领读者深入理解聚类的基本概念、常用算法及其在实际问题中的应用。同样以故事的形式引入核心思想，并结合理论、实践和案例分析，帮助读者全面掌握聚类技术。

学习目标

本章介绍聚类分析的基本理论与实践方法。理解聚类的概念、常见算法（如 K-Means 与 DBSCAN）的工作原理及其适用场景，掌握评估聚类结果质量的基本策略。在实践方面，能够使用 Python 实现主流聚类算法，并结合计算机视觉任务进行应用，通过可视化工具直观分析聚类效果，提升对无监督学习技术的理解与运用能力。

8.1 照片分类问题

石榴同学是一位热爱摄影的大学生。最近，她拍摄到一组风景照片，但面对数百张图片，她感到不知所措：“这些照片太杂乱了！能不能把它们按主题分类呢？”她的朋友建议使用聚类算法来自动分组照片。

于是，石榴同学开始研究聚类算法。她首先尝试 K-Means 算法，这是一种简单但高效的聚类方法。通过实验，她发现 K-Means 能够很好地将照片分为几类，比如“日出”、“森林”和“河流”。然而，当她尝试处理一些包含噪声的照片时，K-Means 的效果却不尽如人意。这时，她又接触到另一种算法——DBSCAN，它对噪声更加鲁棒，最终成功解决了问题。

接下来，将跟随石榴同学的脚步，一起探索聚类算法涉及到的两个重要问题：性能度量问题和距离问题。然后进一步了解原型聚类、密度聚类和层次聚类等经典算法。

8.2 聚类性能度量

聚类是一种无监督学习方法，其目标是将数据集划分为若干个簇，使得同一簇内的数据点彼此相似，而不同簇之间的数据点差异较大。然而，由于聚类结果通常没有真实标签作为参考，如何评估聚类算法的性能成为一个关键问题。本节将详细介绍聚类性能度量的主要方法及其适用场景。

8.2.1 内部指标

内部指标（Internal Metrics）仅依赖于数据本身，通过分析簇内紧密性和簇间分离性来评估聚类质量。这些指标不需要真实标签，适用于无监督场景。

1. 轮廓系数

轮廓系数 (Silhouette Coefficient) 衡量每个数据点的簇内紧密性和簇间分离性的平衡, 定义为:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \quad (8.1)$$

其中, $a(i)$: 数据点 i 到同一簇内其他点的平均距离 (簇内紧密性), $b(i)$: 数据点 i 到最近的其他簇的平均距离 (簇间分离性)。轮廓系数的范围为 $[-1, 1]$, 值越接近 1 表示聚类效果越好, 值越接近 -1 表示聚类效果越差。具体解释如下:

- 如果 $s(i)$ 接近 1, 则表明数据点 i 被正确分配到合适的簇。
- 如果 $s(i)$ 接近 0, 则表明数据点 i 位于两个簇的边界。
- 如果 $s(i)$ 接近 -1, 则表明数据点 i 可能被错误分配到不合适的簇。

2. 戴维森-鲍尔丁指数

戴维森-鲍尔丁指数 (Davies-Bouldin Index, DBI) 衡量簇间的分离性与簇内的紧密性之比, 定义为:

$$DB = \frac{1}{K} \sum_{i=1}^K \max_{j \neq i} \left(\frac{S_i + S_j}{d(\mu_i, \mu_j)} \right) \quad (8.2)$$

其中, S_i : 第 i 个簇的平均簇内距离, 定义为:

$$S_i = \frac{1}{|C_i|} \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \mu_i\| \quad (8.3)$$

这里, μ_i 是第 i 个簇的中心, C_i 是第 i 个簇的数据点集合, $d(\mu_i, \mu_j)$: 簇中心之间的距离, 通常采用欧氏距离。DBI 的值越小表示聚类效果越好。其核心思想是: 簇内距离越小、簇间距离越大, 则聚类效果越好。

3. Calinski-Harabasz 指数

Calinski-Harabasz 指数 (CHI) 通过簇间方差与簇内方差的比值来评估聚类质量, 定义为:

$$CH = \frac{\text{簇间方差}}{\text{簇内方差}} \cdot \frac{N - K}{K - 1} \quad (8.4)$$

其中, 簇间方差表示簇中心与全局中心的距离平方和, 定义为:

$$\text{簇间方差} = \sum_{i=1}^K |C_i| \cdot \|\mu_i - \mu\|^2 \quad (8.5)$$

这里, μ 是全局中心。簇内方差表示簇内数据点与簇中心的距离平方和, 定义为:

$$\text{簇内方差} = \sum_{i=1}^K \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \mu_i\|^2 \quad (8.6)$$

其中, N 为数据点总数, K 为簇的数量。CHI 的值越大表示聚类效果越好。其核心思想是: 簇间差异越大、簇内差异越小, 则聚类效果越好。

4. Dunn 指数

Dunn 指数衡量簇间的最小距离与簇内的最大直径之比, 定义为:

$$D = \min_{1 \leq i < j \leq K} \frac{d(C_i, C_j)}{\max_{1 \leq k \leq K} \delta(C_k)} \quad (8.7)$$

其中, $d(C_i, C_j)$ 表示簇 C_i 和簇 C_j 之间的最小距离, 定义为:

$$d(C_i, C_j) = \min_{\mathbf{x} \in C_i, \mathbf{y} \in C_j} \|\mathbf{x} - \mathbf{y}\| \quad (8.8)$$

$\delta(C_k)$ 表示簇 C_k 的直径, 定义为:

$$\delta(C_k) = \max_{\mathbf{x}, \mathbf{y} \in C_k} \|\mathbf{x} - \mathbf{y}\| \quad (8.9)$$

Dunn 指数的值越大表示聚类效果越好。其核心思想是: 簇间距离越大、簇内直径越小, 则聚类效果越好。

5. 小结

内部指标的核心在于利用数据本身的几何特性评估聚类质量。常见的内部指标包括:

- 轮廓系数: 综合考虑簇内紧密性和簇间分离性。
- 戴维森-鲍尔丁指数 (DBI): 通过簇内距离和簇间距离的比值评估聚类效果。
- Calinski-Harabasz 指数 (CHI): 通过簇间方差与簇内方差的比值评估聚类效果。
- Dunn 指数: 通过簇间最小距离与簇内最大直径的比值评估聚类效果。

选择合适的内部指标需根据数据分布特性和聚类目标来决定。

8.2.2 外部指标

外部指标 (External Metrics) 需要真实的标签信息, 通过比较聚类结果与真实标签的一致性来评估聚类质量。这些指标通常用于有监督场景, 例如在已知数据类别的情况下验证聚类算法的效果。

1. 调整兰德指数

调整兰德指数 (Adjusted Rand Index, ARI) 衡量聚类结果与真实标签之间的一致性, 同时考虑随机分配的影响。其定义为:

$$ARI = \frac{\text{实际一致对数} - \text{期望一致对数}}{\text{最大一致对数} - \text{期望一致对数}} \quad (8.10)$$

其中:

- 实际一致对数: 在聚类结果中, 属于同一簇且具有相同真实标签的点对数量。
- 期望一致对数: 在随机分配情况下, 属于同一簇且具有相同真实标签的点对数量的期望值。
- 最大一致对数: 所有可能的点对数量。

ARI 的范围为 $[-1, 1]$, 值越接近 1 表示一致性越高, 值接近 0 表示聚类结果与随机分配无异, 负值表示聚类结果比随机分配更差。

2. 互信息

互信息 (Mutual Information, MI) 衡量聚类结果与真实标签之间的信息共享程度, 反映了两个分布之间的依赖关系。其定义为:

$$MI(U, V) = \sum_{i=1}^{|U|} \sum_{j=1}^{|V|} p(i, j) \log \frac{p(i, j)}{p(i)p(j)} \quad (8.11)$$

其中, U 和 V 分别是聚类结果和真实标签, $p(i, j)$ 是同时属于簇 i 和标签 j 的概率, $p(i)$ 和 $p(j)$ 分别是属于簇 i 和标签 j 的边缘概率。MI 的值越大表示一致性越高。然而, MI 的绝对值难以

解释，因此通常使用标准化互信息（Normalized Mutual Information, NMI）进行归一化处理：

$$NMI(U, V) = \frac{MI(U, V)}{\sqrt{H(U)H(V)}} \quad (8.12)$$

其中， $H(U)$ 和 $H(V)$ 分别是聚类结果和真实标签的熵。NMI 的范围为 $[0, 1]$ ，值越接近 1 表示一致性越高。

3. 同质性

同质性（Homogeneity）衡量每个簇是否只包含一个类别的样本。其定义为：

$$H = 1 - \frac{H(C|K)}{H(C)} \quad (8.13)$$

其中， $H(C|K)$ 是给定聚类结果 K 时，真实标签 C 的条件熵， $H(C)$ 是真实标签 C 的熵。同质性的范围为 $[0, 1]$ ，值越接近 1 表示每个簇内的样本越纯。

4. 完整性

完整性（Completeness）衡量同一个类别的样本是否被分配到同一个簇。其定义为：

$$C = 1 - \frac{H(K|C)}{H(K)} \quad (8.14)$$

其中， $H(K|C)$ 是给定真实标签 C 时，聚类结果 K 的条件熵， $H(K)$ 是聚类结果 K 的熵。完整性的范围为 $[0, 1]$ ，值越接近 1 表示同一个类别的样本越集中。

5. V-Measure

V-Measure 综合同质性和完整性，采用调和平均的方式平衡两者的权重，定义为：

$$V = 2 \cdot \frac{\text{Homogeneity} \cdot \text{Completeness}}{\text{Homogeneity} + \text{Completeness}} \quad (8.15)$$

V-Measure 的范围为 $[0, 1]$ ，值越接近 1 表示聚类效果越好。

6. Fowlkes-Mallows 指数

Fowlkes-Mallows 指数（FMI）衡量聚类结果与真实标签之间的匹配程度，基于正确分类的点对比例。其定义为：

$$FMI = \sqrt{\frac{TP}{TP + FP} \cdot \frac{TP}{TP + FN}} \quad (8.16)$$

其中，真正例（True Positives），即在聚类结果和真实标签中都属于同一组的点对数量，假正例（False Positives），即在聚类结果中属于同一组但在真实标签中不属于同一组的点对数量，FN：假负例（False Negatives），即在真实标签中属于同一组但在聚类结果中不属于同一组的点对数量。FMI 的范围为 $[0, 1]$ ，值越接近 1 表示聚类效果越好。

7. Jaccard 系数

Jaccard 系数衡量聚类结果与真实标签之间的重叠程度，定义为：

$$J = \frac{TP}{TP + FP + FN} \quad (8.17)$$

Jaccard 系数的范围为 $[0, 1]$ ，值越接近 1 表示聚类结果与真实标签的重叠程度越高。

8. 小结

外部指标的核心在于利用真实标签信息评估聚类结果的质量。常见的外部指标包括：调整兰德指数（ARI）：综合考虑一致性与随机性；互信息（MI）及其标准化版本（NMI）：衡量信息共享程度；同质性、完整性与 V-Measure：分别评估簇内纯度和类别集中度；Fowlkes-Mallows 指数（FMI）：

基于正确分类点对的比例；Jaccard 系数：衡量重叠程度。选择合适的外部指标需根据具体任务需求和数据特性来决定。

8.2.3 可视化辅助度量

对于低维数据，可以通过可视化直接观察聚类效果：

- **散点图**：将数据点投影到二维或三维空间，观察簇的分布。
- **树状图 (Dendrogram)**：用于层次聚类，展示数据的层次化结构。
- **t-SNE 或 UMAP 降维**：将高维数据降维到二维或三维，观察聚类结果。

8.2.4 综合评价

在实际应用中，单一指标可能无法全面反映聚类效果，通常需要结合多个指标进行综合评价。例如：

- 使用**轮廓系数**评估簇内紧密性和簇间分离性；
- 使用 **ARI 或 NMI** 验证聚类结果与真实标签的一致性；
- 结合领域知识，通过可视化或业务指标进一步分析。

8.2.5 性能度量小结

聚类性能度量主要包括以下几类：

- **内部指标**：如轮廓系数、戴维森-鲍尔丁指数、Calinski-Harabasz 指数，适用于无监督场景。
- **外部指标**：如调整兰德指数、互信息、同质性与完整性，适用于有监督场景。
- **可视化辅助度量**：如散点图、树状图、t-SNE/UMAP 降维，适用于直观分析。

选择合适的度量方法需根据具体任务需求、数据特性和是否具备真实标签来决定。

8.3 聚类中的距离度量

在聚类分析中，距离度量是衡量数据点之间相似性或差异性的核心工具。不同的距离度量方法适用于不同类型的数据和任务，选择合适的距离度量对聚类效果至关重要。本节将详细介绍常见的距离度量方法及其特点与应用场景。

8.3.1 欧氏距离

欧氏距离 (Euclidean Distance) 是最常用的距离度量方法之一，用于衡量两点之间的直线距离。其定义为：

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (8.18)$$

其中， $\mathbf{x} = (x_1, x_2, \dots, x_n)$ 和 $\mathbf{y} = (y_1, y_2, \dots, y_n)$ 是两个 n 维向量。该距离的特点与应用场景如下：

- 适用于连续型数值数据，尤其在低维空间中表现良好。
- 对噪声和异常值较为敏感，因此在噪声较多的情况下需谨慎使用。
- 假设所有维度具有相同的权重，因此需要对数据进行归一化或标准化处理。
- 常用于 K-Means 聚类以及层次聚类中的平均链和全链方法。

8.3.2 曼哈顿距离

曼哈顿距离 (Manhattan Distance) 也称为城市街区距离, 用于衡量两点之间的路径距离 (沿坐标轴方向)。其定义为:

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n |x_i - y_i| \quad (8.19)$$

该距离的特点与应用场景如下:

- 对噪声和异常值的鲁棒性较强, 适合稀疏数据或高维数据。
- 计算简单, 适合大规模数据集。
- 常用于文本分类中的词频向量以及密度聚类 (如 DBSCAN) 中的邻域计算。

8.3.3 闵可夫斯基距离

闵可夫斯基距离 (Minkowski Distance) 是欧氏距离和曼哈顿距离的推广形式, 定义为:

$$d(\mathbf{x}, \mathbf{y}) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{1/p} \quad (8.20)$$

其中, p 是参数, 当 $p = 2$ 时, 退化为欧氏距离。当 $p = 1$ 时, 退化为曼哈顿距离。该距离的特点与应用场景如下:

- 参数 p 控制距离的敏感性, 灵活性较高。
- p 越大, 越关注维度间的最大差异, 适合需要强调极端差异的任务。
- 常用于需要灵活调整距离敏感性的场景。

8.3.4 余弦相似度

余弦相似度 (Cosine Similarity) 用于衡量两个向量之间的夹角余弦值, 常用于文本数据或高维稀疏数据。其定义为:

$$\text{cosine}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|} \quad (8.21)$$

其中, $\mathbf{x} \cdot \mathbf{y}$ 是向量内积, $\|\mathbf{x}\|$ 和 $\|\mathbf{y}\|$ 是向量的模。该距离的特点与应用场景如下:

- 不受向量长度的影响, 仅关注方向, 适合高维稀疏数据 (如 TF-IDF 向量)。
- 常用于文本聚类、推荐系统中的用户兴趣相似性计算。

8.3.5 马氏距离

马氏距离 (Mahalanobis Distance) 考虑了数据的协方差结构, 能够消除不同维度之间的相关性和尺度差异。其定义为:

$$d(\mathbf{x}, \mathbf{y}) = \sqrt{(\mathbf{x} - \mathbf{y})^T \mathbf{S}^{-1} (\mathbf{x} - \mathbf{y})} \quad (8.22)$$

其中, $\mathbf{S}$ 是数据的协方差矩阵。该距离的特点与应用场景如下:

- 能够处理多维数据的相关性, 适合分布存在异方差性的情况。
- 对数据分布假设较强 (需估计协方差矩阵), 计算复杂度较高。
- 常用于数据分布存在相关性或异方差性的场景。

8.3.6 汉明距离

汉明距离 (Hamming Distance) 用于衡量两个等长字符串之间的差异，定义为对应位置上不同字符的数量：

$$d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^n \mathbb{I}(x_i \neq y_i) \quad (8.23)$$

其中， $\mathbb{I}(\cdot)$ 是指示函数。该距离的特点与应用场景如下：

- 适用于离散型数据（如二进制向量），计算简单且直观。
- 常用于二进制数据聚类、错误检测与纠正。

8.3.7 杰卡德距离

杰卡德距离 (Jaccard Distance) 用于衡量两个集合之间的差异，定义为：

$$d(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|} \quad (8.24)$$

其中， A 和 B 是两个集合。该距离的特点与应用场景如下：

- 适用于集合型数据，关注集合的交集与并集关系。
- 常用于文本聚类中的词汇集合比较、图像分割中的区域相似性计算。

8.3.8 编辑距离

编辑距离 (Edit Distance) 用于衡量两个字符串之间的差异，定义为将一个字符串转换为另一个字符串所需的最少操作次数（插入、删除、替换）。其具体实现包括 Levenshtein 距离和 Damerau-Levenshtein 距离。其特点与应用场景如下：

- 适用于字符串数据，能够捕捉字符串的局部差异。
- 常用于拼写检查、生物信息学中的 DNA 序列比对。

8.3.9 小结

距离度量的选择直接影响聚类算法的效果。常见距离度量包括：欧氏距离：适用于连续型数值数据，尤其在低维空间中表现良好；曼哈顿距离：适用于稀疏数据或高维数据，对噪声较鲁棒；闵可夫斯基距离：适用于需要灵活调整敏感性的场景；余弦相似度：适用于高维稀疏数据，如文本聚类；马氏距离：适用于多维数据的相关性分析；汉明距离：适用于离散型数据，如二进制向量；杰卡德距离：适用于集合型数据，如词汇集合比较；编辑距离：适用于字符串数据，如拼写检查。根据数据类型和任务需求选择合适距离度量，可以显著提升聚类性能。

8.4 原型聚类

8.4.1 原型聚类的基本思想

原型聚类是一类以“原型”为核心思想的聚类方法。这里的“原型”通常指代簇的代表点，例如簇的中心点、某种概率分布的中心，或者模糊隶属度的权重。通过将数据点分配到与其最相似的原型所在的簇中，可以实现数据的分组。这种方法在计算视觉中被广泛应用于图像分割、特征提取等任务。

原型聚类的核心优势在于其直观性和高效性，特别适合处理大规模数据集。然而，不同原型聚类方法在具体实现和应用场景上存在差异，接下来将详细介绍几种经典的原型聚类方法。

8.4.2 K-Means

K-Means 是一种经典的划分聚类算法，广泛应用于机器学习和数据挖掘领域。其核心思想是将数据集划分为 K 个簇，使得每个数据点与其所属簇中心的距离最小化。K-Means 以其简单高效的特点成为聚类任务中最常用的算法之一。

1. K-Means 的基本思想

K-Means 的目标是将数据集中的 n 个数据点划分为 K 个簇 ($K \leq n$)，使得同一簇内的数据点彼此相似，而不同簇的数据点差异较大。具体而言，K-Means 通过优化以下目标函数实现这一目标：

$$J = \sum_{i=1}^K \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \boldsymbol{\mu}_i\|^2 \quad (8.25)$$

其中， C_i 表示第 i 个簇， $\mathbf{x}$ 是数据点， $\boldsymbol{\mu}_i$ 是第 i 个簇的中心， $\|\mathbf{x} - \boldsymbol{\mu}_i\|$ 表示数据点 $\mathbf{x}$ 与簇中心 $\boldsymbol{\mu}_i$ 之间的欧氏距离。目标函数 J 表示所有数据点到其所属簇中心的距离平方和。K-Means 试图通过迭代优化该目标函数，找到最优的簇划分。

K-Means 的实现过程可以分为以下几个步骤：

1) 初始化簇中心：随机选择 K 个数据点作为初始簇中心，或者使用改进方法（如 K-Means++）选择初始簇中心。

2) 分配数据点到最近的簇中心：对于每个数据点 $\mathbf{x}_j$ ，计算其到所有簇中心的距离，并将其分配到距离最近的簇：

$$C_i = \{\mathbf{x}_j \mid \|\mathbf{x}_j - \boldsymbol{\mu}_i\| \leq \|\mathbf{x}_j - \boldsymbol{\mu}_k\|, \forall k \neq i\} \quad (8.26)$$

3) 更新簇中心：对于每个簇 C_i ，重新计算其簇中心为簇内所有数据点的均值：

$$\boldsymbol{\mu}_i = \frac{1}{|C_i|} \sum_{\mathbf{x} \in C_i} \mathbf{x} \quad (8.27)$$

其中， $|C_i|$ 表示簇 C_i 中数据点的数量。

4) 重复步骤 2 和 3：重复上述分配和更新过程，直到满足以下条件之一：

- 簇中心不再发生显著变化（即收敛）。
- 达到最大迭代次数。

2. K-Means 的优点

K-Means 作为一种经典的聚类算法，具有以下优点：

- **简单高效**：算法实现简单，计算复杂度较低，适合处理大规模数据集。
- **可扩展性强**：可以通过分布式计算框架（如 Spark）扩展到超大规模数据集。
- **结果直观**：聚类结果易于解释，适合初步探索性数据分析。

3. K-Means 的局限性

尽管 K-Means 具有诸多优点，但其在实际应用中也存在一些局限性：

- **需要预先指定簇数 K** ：K-Means 要求用户事先指定簇的数量，但在许多实际场景中，簇数可能未知。
- **对初始簇中心敏感**：初始簇中心的选择可能影响最终的聚类结果，导致陷入局部最优解。

- **假设簇的形状为球形：**K-Means 假设簇的形状为球形，可能无法处理复杂的分布（如非凸形簇）。
- **对噪声和异常值敏感：**噪声和异常值可能显著影响簇中心的计算，从而降低聚类质量。

4. K-Means 的实际案例

为更好地理解 K-Means 的工作原理，通过一个简单的例子进行说明。假设有一个二维数据集，包含 6 个数据点：(1, 1), (1, 2), (2, 1), (8, 8), (8, 9), (9, 8)。需要将其分为两个簇。其大概都计算过程如下：

1) 初始化簇中心。假设初始簇中心为 (1, 1) 和 (8, 8)。

2) 分配数据点到最近的簇中心：

- 数据点 (1, 1), (1, 2), (2, 1) 距离 (1, 1) 更近，因此属于第一个簇。
- 数据点 (8, 8), (8, 9), (9, 8) 距离 (8, 8) 更近，因此属于第二个簇。

3) 更新簇中心：

$$\begin{aligned}\mu_1 &= \frac{(1, 1) + (1, 2) + (2, 1)}{3} = (1.33, 1.33) \\ \mu_2 &= \frac{(8, 8) + (8, 9) + (9, 8)}{3} = (8.33, 8.33)\end{aligned}\tag{8.28}$$

4) 重复分配和更新过程，直到簇中心不再变化。最终，簇中心收敛为 (1.33, 1.33) 和 (8.33, 8.33)。

5. K-Means 的变体与改进

为克服 K-Means 的局限性，研究者提出多种改进方法，包括：

- **K-Means++：**通过改进初始簇中心的选择策略，提高算法的稳定性和性能。
- **Mini-Batch K-Means：**通过随机采样小批量数据进行更新，减少计算开销。
- **Kernel K-Means：**通过核函数将数据映射到高维空间，处理非线性分布的数据。

K-Means 是一种简单高效的聚类算法，适用于各种规模的数据集。尽管其存在一些局限性，但通过结合改进方法（如 K-Means++）或与其他算法结合使用，可以显著提升其性能和适用范围。在实际应用中，K-Means 常用于图像分割、客户分群、文本聚类等任务。

8.4.3 K-Means++

K-Means 是一种经典的聚类算法，但其性能高度依赖于初始簇中心的选择。在传统的 K-Means 算法中，初始簇中心通常是随机选择的。这种方法存在以下问题：

- **局部最优：**随机初始化可能导致初始簇中心过于集中，使得算法陷入局部最优解。
- **收敛速度慢：**如果初始簇中心分布不合理，算法可能需要更多的迭代次数才能收敛。
- **结果不稳定：**由于初始簇中心的随机性，多次运行算法可能得到不同的结果。

1. K-Means++ 的原则

为解决这些问题，K-Means++ 提出一种改进的初始簇中心选择策略，通过概率采样使初始簇中心尽可能分散，从而提高算法的收敛速度和稳定性，显著提高了算法的性能。具体而言，K-Means++ 的初始化过程遵循以下原则：

- 第一个簇中心随机选择。
- 后续簇中心的选择概率与其到已有簇中心的距离成正比。距离越远的数据点被选为新簇中心的概率越高。

2. K-Means++ 的步骤

上述策略能够确保初始簇中心覆盖数据集的不同区域,从而减少局部最优的可能性。K-Means++ 的初始化过程可以分为以下几个步骤:

- 1) **随机选择第一个簇中心:** 从数据集中随机选择一个数据点作为第一个簇中心 μ_1 。
- 2) **计算每个数据点到最近簇中心的距离:** 对于每个数据点 $\mathbf{x}_j$, 计算其到已有簇中心的最短距离:

$$d(\mathbf{x}_j) = \min_{i=1, \dots, k-1} \|\mathbf{x}_j - \mu_i\| \quad (8.29)$$

其中, μ_i 是已选择的簇中心。

- 3) **根据距离的概率分布选择下一个簇中心:** 将每个数据点被选为新簇中心的概率定义为其距离平方的归一化值:

$$P(\mathbf{x}_j) = \frac{d(\mathbf{x}_j)^2}{\sum_{l=1}^N d(\mathbf{x}_l)^2} \quad (8.30)$$

根据该概率分布随机选择下一个簇中心。

- 4) **重复上述步骤:** 重复步骤 2 和 3, 直到选出所有 K 个簇中心。
- 5) **应用传统 K-Means 算法:** 使用选定的初始簇中心, 运行传统 K-Means 算法进行后续迭代。

3. K-Means++ 的优势

与传统 K-Means 相比, K-Means++ 具有以下优势:

- **更高的聚类质量:** 通过分散的初始簇中心选择, K-Means++ 能够避免局部最优, 从而提高聚类结果的质量。
- **更快的收敛速度:** 合理的初始簇中心分布减少了算法的迭代次数, 从而加快了收敛速度。
- **更稳定的结果:** 由于初始簇中心的选择具有确定性(基于概率分布), K-Means++ 的结果更加稳定。

4. K-Means++ 的实际案例

为更好地理解 K-Means++ 的工作原理, 通过一个简单的例子进行说明。假设有一个二维数据集, 包含 6 个数据点: (1, 1), (1, 2), (2, 1), (8, 8), (8, 9), (9, 8)。需要将其分为两个簇, 并使用 K-Means++ 选择初始簇中心。大概过程如下:

- 1) **随机选择第一个簇中心:** 假设选择 (1, 1) 作为第一个簇中心 μ_1 。
- 2) **计算每个数据点到 μ_1 的距离:**

$$\begin{aligned} d((1, 2)) &= \sqrt{(1-1)^2 + (2-1)^2} = 1 \\ d((2, 1)) &= \sqrt{(2-1)^2 + (1-1)^2} = 1 \\ d((8, 8)) &= \sqrt{(8-1)^2 + (8-1)^2} = \sqrt{98} \approx 9.90 \\ d((8, 9)) &= \sqrt{(8-1)^2 + (9-1)^2} = \sqrt{113} \approx 10.63 \\ d((9, 8)) &= \sqrt{(9-1)^2 + (8-1)^2} = \sqrt{113} \approx 10.63 \end{aligned} \quad (8.31)$$

3) 计算每个数据点被选为第二个簇中心的概率:

$$\begin{aligned}
 P((1,2)) &= \frac{1^2}{1^2 + 1^2 + 9.90^2 + 10.63^2 + 10.63^2} \approx 0.0025 \\
 P((2,1)) &= \frac{1^2}{1^2 + 1^2 + 9.90^2 + 10.63^2 + 10.63^2} \approx 0.0025 \\
 P((8,8)) &= \frac{9.90^2}{1^2 + 1^2 + 9.90^2 + 10.63^2 + 10.63^2} \approx 0.2475 \\
 P((8,9)) &= \frac{10.63^2}{1^2 + 1^2 + 9.90^2 + 10.63^2 + 10.63^2} \approx 0.3737 \\
 P((9,8)) &= \frac{10.63^2}{1^2 + 1^2 + 9.90^2 + 10.63^2 + 10.63^2} \approx 0.3737
 \end{aligned} \tag{8.32}$$

4) 根据概率分布随机选择第二个簇中心。假设选择 (8, 8) 作为第二个簇中心 μ_2 。

5) 使用选定的初始簇中心 (1, 1) 和 (8, 8)，运行传统 K-Means 算法进行后续迭代。

K-Means++ 通过对初始簇中心的选择进行优化，显著提高了 K-Means 算法的性能和稳定性。它不仅适用于大规模数据集，还可以与其他聚类方法结合使用（例如模糊 C 均值聚类）。尽管 K-Means++ 的初始化过程增加了计算开销，但其带来的性能提升通常能够弥补这一成本。

8.4.4 模糊 C 均值聚类

模糊 C 均值聚类 (Fuzzy C-Means, FCM) 是一种扩展的聚类方法，允许数据点同时属于多个簇，并通过隶属度 (membership degree) 来表示每个数据点对各个簇的归属程度。与硬聚类 (如 K-Means) 不同，FCM 的隶属度是一个介于 0 和 1 之间的值，且所有簇的隶属度之和为 1。这种软分配特性使得 FCM 能够更灵活地处理复杂分布的数据。

1. FCM 核心理念

FCM 的核心思想是通过优化目标函数，将数据点划分为 K 个簇，同时量化每个数据点对各簇的归属程度。其目标函数定义为：

$$J = \sum_{i=1}^K \sum_{j=1}^N u_{ij}^m \|\mathbf{x}_j - \mu_i\|^2 \tag{8.33}$$

其中， K 是簇的数量， N 是数据点的总数， $\mathbf{x}_j$ 是第 j 个数据点， μ_i 是第 i 个簇的中心， u_{ij} 是数据点 $\mathbf{x}_j$ 对簇 i 的隶属度， $m > 1$ 是模糊指数，用于控制隶属度的模糊程度。

2. FCM 的目标函数

FCM 的目标是最小化目标函数 J ，并通过迭代优化隶属度 u_{ij} 和簇中心 μ_i 。在 FCM 中，数据点 $\mathbf{x}_j$ 对第 i 个簇的隶属度记为 u_{ij} ，其计算公式为：

$$u_{ij} = \frac{1}{\sum_{k=1}^K \left(\frac{\|\mathbf{x}_j - \mu_i\|}{\|\mathbf{x}_j - \mu_k\|} \right)^{\frac{2}{m-1}}} \tag{8.34}$$

其中， $\|\mathbf{x}_j - \mu_i\|$ 表示数据点 $\mathbf{x}_j$ 与簇中心 μ_i 之间的欧氏距离； $m > 1$ 是模糊指数，较大的 m 值会使隶属度更加平滑，而较小的 m 值会使隶属度更接近硬分配。

3. FCM 的示例

隶属度的值越大，表示数据点越倾向于属于该簇；反之，隶属度越小，表示数据点对该簇的归属程度越低。为更好地理解隶属度的概念，这里通过一个简单的例子进行说明。假设有三个二维数据点： $\mathbf{x}_1 = (1, 1)$ ， $\mathbf{x}_2 = (3, 3)$ ， $\mathbf{x}_3 = (5, 5)$ ，以及两个簇中心： $\mu_1 = (1, 1)$ ， $\mu_2 = (5, 5)$ 。选择模糊指数 $m = 2$ ，并计算每个数据点对两个簇的隶属度。其计算过程如下：根据隶属度公式，首先计算每

个数据点到簇中心的距离：

- 对于 $\mathbf{x}_1 = (1, 1)$ ：

$$\begin{aligned}\|\mathbf{x}_1 - \boldsymbol{\mu}_1\| &= \sqrt{(1-1)^2 + (1-1)^2} = 0 \\ \|\mathbf{x}_1 - \boldsymbol{\mu}_2\| &= \sqrt{(1-5)^2 + (1-5)^2} = \sqrt{32} \approx 5.66\end{aligned}\quad (8.35)$$

- 对于 $\mathbf{x}_2 = (3, 3)$ ：

$$\begin{aligned}\|\mathbf{x}_2 - \boldsymbol{\mu}_1\| &= \sqrt{(3-1)^2 + (3-1)^2} = \sqrt{8} \approx 2.83 \\ \|\mathbf{x}_2 - \boldsymbol{\mu}_2\| &= \sqrt{(3-5)^2 + (3-5)^2} = \sqrt{8} \approx 2.83\end{aligned}\quad (8.36)$$

- 对于 $\mathbf{x}_3 = (5, 5)$ ：

$$\begin{aligned}\|\mathbf{x}_3 - \boldsymbol{\mu}_1\| &= \sqrt{(5-1)^2 + (5-1)^2} = \sqrt{32} \approx 5.66 \\ \|\mathbf{x}_3 - \boldsymbol{\mu}_2\| &= \sqrt{(5-5)^2 + (5-5)^2} = 0\end{aligned}\quad (8.37)$$

接下来，根据隶属度公式计算每个数据点对两个簇的隶属度。以 $\mathbf{x}_2$ 为例：

$$\begin{aligned}u_{21} &= \frac{1}{\left(\frac{\|\mathbf{x}_2 - \boldsymbol{\mu}_1\|}{\|\mathbf{x}_2 - \boldsymbol{\mu}_1\|}\right)^{\frac{2}{2-1}} + \left(\frac{\|\mathbf{x}_2 - \boldsymbol{\mu}_2\|}{\|\mathbf{x}_2 - \boldsymbol{\mu}_1\|}\right)^{\frac{2}{2-1}}} \\ &= \frac{1}{1 + \left(\frac{2.83}{2.83}\right)^2} = \frac{1}{1+1} = 0.5 \\ u_{22} &= \frac{1}{\left(\frac{\|\mathbf{x}_2 - \boldsymbol{\mu}_2\|}{\|\mathbf{x}_2 - \boldsymbol{\mu}_2\|}\right)^{\frac{2}{2-1}} + \left(\frac{\|\mathbf{x}_2 - \boldsymbol{\mu}_1\|}{\|\mathbf{x}_2 - \boldsymbol{\mu}_2\|}\right)^{\frac{2}{2-1}}} \\ &= \frac{1}{1 + \left(\frac{2.83}{2.83}\right)^2} = \frac{1}{1+1} = 0.5\end{aligned}\quad (8.38)$$

类似地，可以计算其他数据点的隶属度，结果如下表所示：

表 8.1: 数据点的隶属度计算结果

数据点	簇 1 隶属度 (u_{i1})	簇 2 隶属度 (u_{i2})
$\mathbf{x}_1 = (1, 1)$	1.00	0.00
$\mathbf{x}_2 = (3, 3)$	0.50	0.50
$\mathbf{x}_3 = (5, 5)$	0.00	1.00

4. 结果分析

从表 8.1中可以看出：

- 数据点 $\mathbf{x}_1$ 完全属于簇 1，因为它的隶属度为 1.00；
- 数据点 $\mathbf{x}_3$ 完全属于簇 2，因为它的隶属度为 1.00；
- 数据点 $\mathbf{x}_2$ 同时属于簇 1 和簇 2，且隶属度均为 0.50，表明它处于两个簇的边界区域。

5. 隶属度的意义

隶属度反映数据点对簇的归属程度，具有以下意义：

- **软分配：**与硬聚类不同，隶属度允许数据点同时属于多个簇，从而更灵活地处理复杂分布的数据。
- **边界点识别：**隶属度接近 0.5 的数据点通常位于簇的边界区域，可能需要进一步分析。
- **模糊性控制：**通过调整模糊指数 m ，可以控制隶属度的模糊程度。较大的 m 值会使隶属度更加平滑，而较小的 m 值会使隶属度更接近硬分配。

隶属度是模糊 C 均值聚类的核心概念，能够量化数据点对簇的归属程度。通过隶属度，可以更细致地分析数据的分布特性，并在实际应用中实现更灵活的分组策略。模糊 C 均值聚类适用于需要软分配的场景，例如图像分割、模式识别和模糊逻辑系统。

8.4.5 高斯混合模型

高斯混合模型（Gaussian Mixture Model, GMM）是一种基于概率分布的聚类方法，假设数据点由多个高斯分布生成。与 K-Means 等硬聚类算法不同，GMM 通过概率的方式为每个数据点分配簇归属，从而实现软聚类。GMM 在计算视觉、语音识别和异常检测等领域具有广泛的应用。

1. 基本思想

GMM 的核心思想是将数据集建模为多个高斯分布的线性组合。具体而言，假设数据点 $\mathbf{x}$ 由 K 个高斯分布生成，其概率密度函数可以表示为：

$$P(\mathbf{x}) = \sum_{i=1}^K \pi_i \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i) \quad (8.39)$$

其中， π_i 是第 i 个高斯分布的权重，满足 $\sum_{i=1}^K \pi_i = 1$ ， $\mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$ 表示均值为 $\boldsymbol{\mu}_i$ 、协方差矩阵为 $\boldsymbol{\Sigma}_i$ 的高斯分布， K 是高斯分布的数量。GMM 的目标是通过最大化数据的对数似然函数，估计每个高斯分布的参数（即 π_i 、 $\boldsymbol{\mu}_i$ 和 $\boldsymbol{\Sigma}_i$ ）。

2. 计算过程

GMM 通常使用期望最大化（Expectation-Maximization, EM）算法进行参数估计。EM 算法分为两个步骤：E 步（Expectation Step）和 M 步（Maximization Step）。具体过程如下：

1) **初始化**。随机初始化每个高斯分布的参数（ π_i 、 $\boldsymbol{\mu}_i$ 和 $\boldsymbol{\Sigma}_i$ ）。

2) **E 步：计算后验概率**。对于每个数据点 $\mathbf{x}_j$ ，计算其属于第 i 个高斯分布的后验概率（也称为“责任”）：

$$\gamma_{ij} = \frac{\pi_i \mathcal{N}(\mathbf{x}_j | \boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)}{\sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}_j | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)} \quad (8.40)$$

其中， γ_{ij} 表示数据点 $\mathbf{x}_j$ 对第 i 个高斯分布的责任。

3) **M 步：更新参数**。根据 E 步计算的责任，更新每个高斯分布的参数：

- 更新权重 π_i ：

$$\pi_i = \frac{\sum_{j=1}^N \gamma_{ij}}{N} \quad (8.41)$$

其中， N 是数据点的总数。

- 更新均值 $\boldsymbol{\mu}_i$ ：

$$\boldsymbol{\mu}_i = \frac{\sum_{j=1}^N \gamma_{ij} \mathbf{x}_j}{\sum_{j=1}^N \gamma_{ij}} \quad (8.42)$$

- 更新协方差矩阵 $\boldsymbol{\Sigma}_i$ ：

$$\boldsymbol{\Sigma}_i = \frac{\sum_{j=1}^N \gamma_{ij} (\mathbf{x}_j - \boldsymbol{\mu}_i)(\mathbf{x}_j - \boldsymbol{\mu}_i)^T}{\sum_{j=1}^N \gamma_{ij}} \quad (8.43)$$

4) **迭代**。重复 E 步和 M 步，直到对数似然函数收敛或达到最大迭代次数。

3. GMM 的优点

GMM 作为一种基于概率分布的聚类方法，具有以下优点：

- **软聚类能力**：GMM 允许数据点同时属于多个簇，并通过后验概率量化归属程度。
- **灵活性强**：GMM 能够处理复杂的分布形状（如椭圆形簇），而不仅限于球形簇。

- **生成模型特性：**GMM 不仅可以用于聚类，还可以生成新的数据点，模拟数据分布。

4. GMM 的局限性

尽管 GMM 具有诸多优点，但在实际应用中也存在一些局限性：

- **计算复杂度较高：**GMM 需要多次迭代计算协方差矩阵，计算开销较大，尤其对于高维数据。
- **对初始参数敏感：**初始参数的选择可能影响最终结果，导致陷入局部最优解。
- **需要指定簇数 K ：**与 K-Means 类似，GMM 也需要用户事先指定高斯分布的数量。

5. GMM 的实际案例

为更好地理解 GMM 的工作原理，这里通过一个简单的例子进行说明。假设有一个二维数据集，包含 6 个数据点：(1, 1), (1, 2), (2, 1), (8, 8), (8, 9), (9, 8)。需要将其分为两个簇，并使用 GMM 进行建模，具体建模过程如下：

1) 初始化两个高斯分布的参数，均值： $\mu_1 = (1, 1)$, $\mu_2 = (8, 8)$ ；协方差矩阵： $\Sigma_1 = \Sigma_2 = \mathbf{I}$ （单位矩阵）；权重： $\pi_1 = \pi_2 = 0.5$ 。

2) E 步：计算每个数据点对两个高斯分布的责任 γ_{ij} 。

3) M 步：根据责任更新高斯分布的参数（均值、协方差矩阵和权重）。

4) 重复 E 步和 M 步，直到对数似然函数收敛。最终，两个高斯分布的参数分别收敛为：

$$\begin{aligned} \mu_1 &= (1.33, 1.33), \quad \Sigma_1 = \begin{bmatrix} 0.22 & 0 \\ 0 & 0.22 \end{bmatrix}, \quad \pi_1 = 0.5 \\ \mu_2 &= (8.33, 8.33), \quad \Sigma_2 = \begin{bmatrix} 0.22 & 0 \\ 0 & 0.22 \end{bmatrix}, \quad \pi_2 = 0.5 \end{aligned} \quad (8.44)$$

6. GMM 应用

高斯混合模型是一种强大的概率聚类方法，能够处理复杂的分布形状并提供软聚类结果。尽管其计算复杂度较高，但在许多实际应用中表现出色。通过结合改进方法（如变分推断或深度学习），可以进一步提升 GMM 的性能和适用范围。GMM 在以下场景中具有广泛的应用：

- **图像分割：**通过建模像素的颜色分布，将图像划分为不同的区域。
- **语音识别：**通过建模语音信号的概率分布，实现声学建模。
- **异常检测：**通过拟合正常数据的分布，识别偏离分布的数据点。

8.5 密度聚类

8.5.1 密度聚类的基本概念

密度聚类是一类基于数据点局部密度的聚类方法，其核心思想是通过识别高密度区域中的数据点来形成簇。与划分聚类（如 K-Means）或原型聚类（如模糊 C 均值）不同，密度聚类不依赖于簇的形状假设，因此能够处理复杂分布的数据（如非凸形簇）。此外，密度聚类对噪声和异常值具有较强的鲁棒性，特别适合处理包含噪声的实际数据集。

密度聚类的核心机制是通过定义“密度可达性”和“密度连接性”来识别簇。具体而言：

- **密度可达性：**如果一个数据点位于另一个数据点的邻域内，并且该邻域内的数据点数量超过某个阈值，则称这两个点是密度可达的。
- **密度连接性：**如果两个数据点可以通过一系列密度可达的数据点连接起来，则称它们是密度相连的。

密度聚类的优点在于其灵活性和鲁棒性，但也存在一些局限性，例如对参数选择较为敏感，以及在高维数据上的性能下降。接下来，将详细介绍两种经典的密度聚类方法：DBSCAN（Density-Based Spatial Clustering of Applications with Noise）和 OPTICS（Ordering Points To Identify the Clustering Structure）。

8.5.2 DBSCAN 算法

DBSCAN 是一种经典的密度聚类算法，能够发现任意形状的簇，并对噪声和异常值具有较强的鲁棒性。与划分聚类（如 K-Means）或原型聚类（如模糊 C 均值）不同，DBSCAN 不依赖于簇的形状假设，因此特别适合处理复杂分布的数据。

1. DBSCAN 的思想

DBSCAN 的核心思想是通过数据点的局部密度来识别簇。具体而言，DBSCAN 将数据点分为三类：

- **核心点**：一个点的 ϵ -邻域内至少包含 $MinPts$ 个点，则该点为核心点。
- **边界点**：一个点的 ϵ -邻域内的点数少于 $MinPts$ ，但它位于某个核心点的 ϵ -邻域内，则该点为边界点。
- **噪声点**：既不是核心点也不是边界点的点，被视为噪声。

通过上面的三类点的定义，DBSCAN 通过“密度可达性”和“密度连接性”来形成簇。如果一个点 p 位于另一个点 q 的 ϵ -邻域内，且 q 是核心点，则称 p 从 q 密度可达。如果两个点 p 和 q 可以通过一系列密度可达的点连接起来，则称它们是密度相连的。

2. DBSCAN 的计算步骤

DBSCAN 将所有密度相连的点划分为同一个簇，以下为 DBSCAN 实现过程的几个核心步骤：

- 1) **初始化**：设置参数 ϵ （邻域半径）和 $MinPts$ （成为核心点所需的最小邻居数）。
- 2) **遍历数据点**：对于每个未访问的数据点 p ，执行以下操作：

- 计算 p 的 ϵ -邻域内的点数。
- 如果 p 的 ϵ -邻域内至少有 $MinPts$ 个点，则将 p 标记为核心点，并启动一个新的簇。
- 从 p 出发，递归地扩展簇，将所有密度可达的点加入该簇。

3) **标记边界点和噪声点**：对于非核心点，如果其位于某个核心点的 ϵ -邻域内，则将其标记为边界点；否则，将其标记为噪声点。

- 4) **重复上述过程**：直到所有数据点都被访问。

3. DBSCAN 的关键参数

DBSCAN 的性能和结果高度依赖于两个关键参数：

- ϵ （邻域半径）：控制每个点的邻域范围。较大的 ϵ 可能导致多个簇被合并，而较小的 ϵ 可能导致簇被分割或噪声点增多。
- $MinPts$ （最小邻居数）：控制核心点的定义。较大的 $MinPts$ 要求更高的密度，可能导致更多点被标记为噪声。

4. DBSCAN 的优势

在实际应用中，通常通过可视化或启发式方法选择合适的参数。DBSCAN 作为一种基于密度的聚类方法，具有以下优点：

- **能够发现任意形状的簇**：DBSCAN 不依赖于簇的形状假设，因此适合处理非凸形簇。
- **对噪声鲁棒**：DBSCAN 能够有效识别并排除噪声点。

- **无需预先指定簇数：**与 K-Means 等算法不同，DBSCAN 不需要用户事先指定簇的数量。

5. DBSCAN 的局限性

尽管 DBSCAN 具有诸多优点，但在实际应用中也存在一些局限性：

- **对参数敏感：** ϵ 和 $MinPts$ 的选择可能显著影响聚类结果。
- **难以处理高维数据：**在高维空间中，数据点之间的距离变得难以区分，导致 DBSCAN 的性能下降。
- **计算复杂度较高：**对于大规模数据集，计算每个点的 ϵ -邻域可能带来较高的计算开销。

6. DBSCAN 的实例

为更好地理解 DBSCAN 的工作原理，通过一个简单的例子进行说明。假设有一个二维数据集，包含以下数据点：(1, 1), (1, 2), (2, 1), (8, 8), (8, 9), (9, 8)。设置 $\epsilon = 2$ 和 $MinPts = 2$ ，应用 DBSCAN 进行聚类，其计算过程如下：

1) 遍历数据点，计算每个点的 ϵ -邻域内的点数：

- 对于 (1, 1)，其 ϵ -邻域内的点包括 (1, 1), (1, 2), (2, 1)，共 3 个点，满足 $MinPts$ 条件，因此 (1, 1) 为核心点。
- 类似地，(1, 2) 和 (2, 1) 也是核心点。
- 对于 (8, 8)，其 ϵ -邻域内的点包括 (8, 8), (8, 9), (9, 8)，共 3 个点，因此 (8, 8) 为核心点。
- 类似地，(8, 9) 和 (9, 8) 也是核心点。

2) 从核心点出发，扩展簇：

- 核心点 (1, 1)、(1, 2)、(2, 1) 密度相连，形成第一个簇。
- 核心点 (8, 8)、(8, 9)、(9, 8) 密度相连，形成第二个簇。

3) 没有孤立点被标记为噪声。

最终，数据被分为两个簇： $\{(1, 1), (1, 2), (2, 1)\}$ 和 $\{(8, 8), (8, 9), (9, 8)\}$ 。

DBSCAN 是一种强大的密度聚类算法，能够发现任意形状的簇并对噪声具有较强的鲁棒性。尽管其对参数选择较为敏感，但在许多实际场景中表现出色。通过结合参数调优和降维技术，可以进一步提升 DBSCAN 的性能和适用范围。

8.5.3 OPTICS 算法

OPTICS (Ordering Points To Identify the Clustering Structure) 是一种改进的密度聚类算法，旨在解决 DBSCAN 对参数 ϵ 的敏感性问题。与 DBSCAN 直接划分簇不同，OPTICS 通过生成一个可达距离图 (Reachability Plot) 来表示数据点的密度结构，从而允许用户在后续分析中灵活选择簇的划分方式。

1. 核心理念

OPTICS 的核心理念是通过计算每个数据点的“核心距离”和“可达距离”，揭示数据的层次化密度结构。具体而言：

- **核心距离：**一个点成为核心点所需的最小邻域半径。如果某点的 ϵ -邻域内至少包含 $MinPts$ 个点，则其核心距离为满足条件的最小半径；否则，其核心距离为未定义。
- **可达距离：**从一个点 p 到另一个点 q 的最小邻域半径，用于衡量两点之间的密度连接性。可达距离定义为：

$$Reachability-Distance(p, q) = \max(Core-Distance(q), \|p - q\|) \quad (8.45)$$

其中, $\|p - q\|$ 表示点 p 和点 q 之间的欧氏距离。

2. 计算过程

通过计算所有点的可达距离, OPTICS 生成一个可达距离图, 其中每个点按其访问顺序排列, 并标注其可达距离值。用户可以根据可达距离图中的低谷区域提取簇。OPTICS 的实现过程可以分为以下几个步骤:

1) **初始化**: 设置参数 $MinPts$ (成为核心点所需的最小邻居数), 并初始化一个优先队列 (通常使用堆结构) 来存储待处理的点及其可达距离。

2) **遍历数据点**: 对于每个未访问的数据点 p , 执行以下操作:

- 计算 p 的核心距离。如果 p 是核心点, 则将其 ϵ -邻域内的点加入优先队列, 并更新这些点的可达距离。
- 按照可达距离从小到大的顺序处理优先队列中的点, 记录每个点的访问顺序和可达距离。

3) **生成可达距离图**: 将所有点按其访问顺序排列, 并标注其可达距离值, 形成可达距离图。

4) **提取簇**: 根据可达距离图中的低谷区域, 动态选择阈值提取簇。

3. 关键概念

为更好地理解 OPTICS 的工作原理, 需要掌握以下几个关键概念:

- **核心距离**: 反映一个点成为核心点所需的最小密度要求。
- **可达距离**: 反映两个点之间的密度连接性。
- **可达距离图**: 一种可视化工具, 用于展示数据点的密度结构。低谷区域对应高密度区域, 而高峰区域对应低密度区域或噪声。

4. 优点

OPTICS 作为一种基于密度的层次化聚类方法, 具有以下优点:

- **揭示层次化密度结构**: OPTICS 能够生成一个全局的密度结构视图, 适合探索复杂分布的数据。
- **对参数不敏感**: 与 DBSCAN 相比, OPTICS 不需要预先指定 ϵ , 而是通过可达距离图动态选择簇的划分方式。
- **灵活性强**: 用户可以根据实际需求调整阈值, 提取不同粒度的簇。

4. 局限性

尽管 OPTICS 具有诸多优点, 但其在实际应用中也存在一些局限性:

- **计算复杂度较高**: OPTICS 需要维护一个优先队列, 并计算每个点的可达距离, 因此其计算开销较大。
- **结果解释依赖后处理**: OPTICS 生成的可达距离图需要进一步分析才能提取簇, 增加了用户的操作负担。
- **难以处理高维数据**: 与 DBSCAN 类似, 在高维空间中, 数据点之间的距离变得难以区分, 导致 OPTICS 的性能下降。

5. 计算实例

为更好地理解 OPTICS 的工作原理, 同样通过一个简单的例子进行说明。假设有一个二维数据集, 包含以下数据点: $(1, 1)$, $(1, 2)$, $(2, 1)$, $(8, 8)$, $(8, 9)$, $(9, 8)$ 。设置 $MinPts = 2$, 应用 OPTICS 生成可达距离图。

1) **遍历数据点**, 计算每个点的核心距离和可达距离:

- 对于 $(1, 1)$, 其核心距离为 $\sqrt{2}$ (即到 $(1, 2)$ 的距离), 因为其 ϵ -邻域内有 3 个点。

- 类似地, (1,2) 和 (2,1) 的核心距离也为 $\sqrt{2}$ 。
- 对于 (8,8), 其核心距离为 $\sqrt{2}$, 因为其 ϵ -邻域内有 3 个点。
- 类似地, (8,9) 和 (9,8) 的核心距离也为 $\sqrt{2}$ 。

2) 按访问顺序生成可达距离图:

- 数据点 (1,1), (1,2), (2,1) 的可达距离较低, 形成第一个高密度区域。
- 数据点 (8,8), (8,9), (9,8) 的可达距离较低, 形成第二个高密度区域。
- 两个高密度区域之间存在较高的可达距离, 表明它们属于不同的簇。

3) 根据可达距离图中的低谷区域, 提取两个簇: $\{(1,1), (1,2), (2,1)\}$ 和 $\{(8,8), (8,9), (9,8)\}$ 。

OPTICS 是一种强大的密度聚类算法, 能够揭示数据的层次化密度结构, 并允许用户灵活选择簇的划分方式。尽管其计算复杂度较高且结果解释需要额外的后处理步骤, 但在许多实际场景中表现出色。通过结合降维技术和自动化阈值选择方法, 可以进一步提升 OPTICS 的性能和适用范围。

8.6 分层聚类

8.6.1 分层聚类的基本概念

分层聚类 (Hierarchical Clustering) 是一种基于树状结构 (Dendrogram) 的聚类方法, 其核心思想是通过逐步合并或分裂数据点来构建簇的层次化结构。与划分聚类 (如 K-Means) 不同, 分层聚类不需要预先指定簇的数量, 而是生成一个完整的聚类树, 用户可以根据实际需求选择合适的粒度提取簇。分层聚类可以分为两种主要类型:

- **凝聚层次聚类 (Agglomerative Clustering):** 自底向上的方法, 从每个数据点作为一个独立的簇开始, 逐步合并最相似的簇, 直到所有数据点归为一个簇。
- **分裂层次聚类 (Divisive Clustering):** 自顶向下的方法, 从所有数据点作为一个簇开始, 逐步分裂成更小的簇, 直到每个数据点成为一个独立的簇。

分层聚类的优点在于其能够揭示数据的层次化结构, 适合探索性数据分析。然而, 其计算复杂度较高, 且对噪声和异常值较为敏感。

8.6.2 凝聚层次聚类

凝聚层次聚类 (Agglomerative Clustering) 是一种经典的分层聚类方法, 采用自底向上的策略构建簇的层次化结构。其核心思想是从每个数据点作为一个独立的簇开始, 逐步合并最相似的簇, 直到所有数据点归为一个簇或达到用户指定的停止条件。

1. 计算步骤

凝聚层次聚类的过程可以看作是构建一棵树状图 (Dendrogram), 其中每个叶节点代表一个数据点, 而每个非叶节点代表一个簇。通过逐步合并相似的簇, 最终形成一个完整的聚类树。用户可以根据实际需求选择合适的粒度提取簇, 例如通过切割树状图来确定簇的数量。其实现过程可以分为以下几个步骤:

- 1) **初始化:** 将每个数据点视为一个独立的簇, 初始簇数等于数据点的数量。
- 2) **计算簇间距离:** 根据某种距离度量 (如单链、全链或平均链), 计算每对簇之间的距离。常见的距离度量将在后续部分详细介绍。
- 3) **合并最相似的簇:** 选择距离最小的两个簇进行合并, 形成一个新的簇。
- 4) **更新簇间距离:** 重新计算新簇与其他簇之间的距离。
- 5) **重复上述过程:** 直到所有数据点归为一个簇, 或达到用户指定的簇数。

2. 度量方法

凝聚层次聚类的性能和结果高度依赖于簇间距离的度量方法。以下是几种常见的簇间距离度量方法：

- **单链 (Single Linkage)** 单链定义两个簇之间的距离为簇内最近两点之间的距离：

$$d(C_i, C_j) = \min_{\mathbf{x} \in C_i, \mathbf{y} \in C_j} \|\mathbf{x} - \mathbf{y}\| \quad (8.46)$$

单链倾向于生成细长形状的簇，适合处理链状分布的数据。

- **全链 (Complete Linkage)** 全链定义两个簇之间的距离为簇内最远两点之间的距离：

$$d(C_i, C_j) = \max_{\mathbf{x} \in C_i, \mathbf{y} \in C_j} \|\mathbf{x} - \mathbf{y}\| \quad (8.47)$$

全链倾向于生成紧凑形状的簇，适合处理球形分布的数据。

- **平均链 (Average Linkage)** 平均链定义两个簇之间的距离为簇内所有点对之间距离的平均值：

$$d(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{\mathbf{x} \in C_i} \sum_{\mathbf{y} \in C_j} \|\mathbf{x} - \mathbf{y}\| \quad (8.48)$$

平均链在单链和全链之间取得平衡，适合处理一般分布的数据。

- **Ward 方法** Ward 方法定义两个簇之间的距离为合并后簇的总方差增量：

$$d(C_i, C_j) = \frac{|C_i||C_j|}{|C_i| + |C_j|} \|\boldsymbol{\mu}_i - \boldsymbol{\mu}_j\|^2 \quad (8.49)$$

其中， $\boldsymbol{\mu}_i$ 和 $\boldsymbol{\mu}_j$ 分别是簇 C_i 和 C_j 的均值。Ward 方法倾向于生成大小均衡的簇，适合处理均匀分布的数据。

3. 优点

凝聚层次聚类作为一种经典的分层聚类方法，具有以下优点：

- **无需预先指定簇数：** 用户可以通过切割树状图灵活选择簇的数量。
- **揭示层次化结构：** 生成的树状图能够清晰地展示数据的层次化关系。
- **适用于小规模数据集：** 对于小规模数据集，凝聚层次聚类通常能够生成高质量的聚类结果。

4. 局限性

尽管凝聚层次聚类具有诸多优点，但在实际应用中也存在一些局限性：

- **计算复杂度较高：** 凝聚层次聚类的时间复杂度为 $O(N^3)$ （使用朴素实现）或 $O(N^2 \log N)$ （使用优化实现），因此不适合处理大规模数据集。
- **对噪声和异常值敏感：** 由于簇间距离的计算依赖于点对之间的距离，噪声和异常值可能显著影响聚类结果。
- **不可逆性：** 一旦两个簇被合并，无法撤销该操作，可能导致次优解。

5. 计算实例

为更好地理解凝聚层次聚类的工作原理，通过一个简单的例子进行说明。假设有一个二维数据集，包含以下数据点：(1, 1), (1, 2), (2, 1), (8, 8), (8, 9), (9, 8)。使用平均链作为距离度量方法，应用凝聚层次聚类进行分组。其计算过程如下：

1) 初始化：每个数据点作为一个独立的簇，初始簇为 $\{(1, 1)\}, \{(1, 2)\}, \{(2, 1)\}, \{(8, 8)\}, \{(8, 9)\}, \{(9, 8)\}$ 。

2) 计算簇间距离：根据平均链公式，计算每对簇之间的距离。例如：

$$\begin{aligned} d(\{(1,1)\}, \{(1,2)\}) &= \frac{\|(1,1) - (1,2)\|}{1 \cdot 1} = 1 \\ d(\{(1,1)\}, \{(8,8)\}) &= \frac{\|(1,1) - (8,8)\|}{1 \cdot 1} = \sqrt{(1-8)^2 + (1-8)^2} \approx 9.90 \end{aligned} \quad (8.50)$$

3) 合并最相似的簇：选择距离最小的两个簇进行合并。例如，(1,1) 和 (1,2) 的距离最小，因此将它们合并为一个新簇 $\{(1,1), (1,2)\}$ 。

4) 更新簇间距离：重新计算新簇 $\{(1,1), (1,2)\}$ 与其他簇之间的距离。例如：

$$d(\{(1,1), (1,2)\}, \{(2,1)\}) = \frac{\|(1,1) - (2,1)\| + \|(1,2) - (2,1)\|}{2 \cdot 1} = \frac{1 + \sqrt{2}}{2} \approx 1.21 \quad (8.51)$$

5) 重复上述过程：继续合并最相似的簇，直到所有数据点归为一个簇。最终生成的树状图如下所示：

- 第一层： $\{(1,1), (1,2), (2,1)\}$ 和 $\{(8,8), (8,9), (9,8)\}$ 。
- 第二层：所有数据点归为一个簇。

凝聚层次聚类是一种强大的分层聚类方法，能够揭示数据的层次化结构，并允许用户灵活选择簇的划分方式。尽管其计算复杂度较高且对噪声较为敏感，但在许多实际场景中表现出色。通过结合降维技术和自动化阈值选择方法，可以进一步提升凝聚层次聚类的性能和适用范围。

8.6.3 分裂层次聚类

分裂层次聚类 (Divisive Clustering) 是一种经典的分层聚类方法，采用自顶向下的策略构建簇的层次化结构。其核心思想是从所有数据点作为一个簇开始，逐步分裂成更小的簇，直到每个数据点成为一个独立的簇或达到用户指定的停止条件。

1. 计算步骤

分裂层次聚类的过程可以看作是逐步分解一棵树状图 (Dendrogram)，其中根节点代表所有数据点，而叶节点代表单个数据点。通过递归地选择当前簇中最适合分裂的点或子集，最终形成一个完整的聚类树。与凝聚层次聚类不同，分裂层次聚类从整体出发，逐步细化簇的划分。分裂层次聚类的实现过程可以分为以下几个步骤：

- 1) **初始化**：将所有数据点视为一个簇。
- 2) **选择分裂点**：根据某种分裂准则（如最大方差或最小相似性），选择当前簇中最适合分裂的点或子集。
- 3) **分裂簇**：将选定的簇分裂成两个子簇。
- 4) **递归分裂**：对每个子簇重复上述过程，直到每个数据点成为一个独立的簇，或达到用户指定的簇数。
- 5) **生成树状图**：记录每次分裂的过程，生成树状图以展示簇的层次化结构。

2. 分裂准则

分裂层次聚类的关键在于如何选择分裂点和分裂方式。以下是几种常见的分裂准则：

- **最大方差**：最大方差准则选择方差最大的维度进行分裂。具体而言，对于当前簇 C ，计算每个维度上的方差：

$$\text{Var}_d(C) = \frac{1}{|C|} \sum_{\mathbf{x} \in C} (x_d - \mu_d)^2 \quad (8.52)$$

- 其中， x_d 是数据点 $\mathbf{x}$ 在第 d 维上的值， μ_d 是簇 C 在第 d 维上的均值。选择方差最大的维度作为分裂方向，并根据该维度上的中位数或其他阈值将簇分为两部分。

- **最小相似性** 最小相似性准则选择相似性最低的点对进行分裂。具体而言，对于当前簇 C ，计算每对点之间的相似性（如欧氏距离或余弦相似度），并选择相似性最低的点对作为分裂点。
- **基于模型的方法** 基于模型的方法假设数据点由某种概率分布生成（如高斯分布），并通过最大化似然函数选择分裂点。例如，使用高斯混合模型（GMM）拟合当前簇，并根据模型参数确定分裂方向。

3. 优点

分裂层次聚类作为一种经典的分层聚类方法，具有以下优点：

- **揭示全局结构**：从整体出发，逐步细化簇的划分，能够更好地捕捉数据的全局分布特性。
- **灵活性强**：支持多种分裂准则，可以根据实际需求选择合适的分裂方式。
- **适用于小规模数据集**：对于小规模数据集，分裂层次聚类通常能够生成高质量的聚类结果。

4. 局限性

尽管分裂层次聚类具有诸多优点，但其在实际应用中也存在一些局限性：

- **计算复杂度较高**：分裂层次聚类的时间复杂度通常为 $O(N^2)$ 或更高，因此不适合处理大规模数据集。
- **对初始分裂点敏感**：初始分裂点的选择可能显著影响后续分裂过程，导致次优解。
- **难以处理噪声和异常值**：由于分裂过程依赖于簇内数据点的分布特性，噪声和异常值可能干扰分裂决策。

5. 计算实例

为更好地理解分裂层次聚类的工作原理，通过一个简单的例子进行说明。假设有一个二维数据集，包含以下数据点：(1, 1), (1, 2), (2, 1), (8, 8), (8, 9), (9, 8)。使用最大方差作为分裂准则，应用分裂层次聚类进行分组。其计算过程如下：

- 1) 初始化：将所有数据点视为一个簇，初始簇为 $\{(1, 1), (1, 2), (2, 1), (8, 8), (8, 9), (9, 8)\}$ 。
- 2) 计算方差：对于当前簇，计算每个维度上的方差。例如：

$$\begin{aligned}\text{Var}_x &= \frac{1}{6} ((1-5)^2 + (1-5)^2 + (2-5)^2 + (8-5)^2 + (8-5)^2 + (9-5)^2) \\ &= \frac{50}{6} \approx 8.33 \\ \text{Var}_y &= \frac{1}{6} ((1-5)^2 + (2-5)^2 + (1-5)^2 + (8-5)^2 + (9-5)^2 + (8-5)^2) \\ &= \frac{50}{6} \approx 8.33\end{aligned}\tag{8.53}$$

由于两个维度的方差相等，可以选择任意维度进行分裂。

- 3) 分裂簇：根据 x 维度上的中位数（即 5），将簇分为两个子簇：

- 子簇 1: $\{(1, 1), (1, 2), (2, 1)\}$
- 子簇 2: $\{(8, 8), (8, 9), (9, 8)\}$

4) 递归分裂：对每个子簇重复上述过程。例如，对于子簇 1，计算 x 和 y 维度上的方差，并根据方差最大的维度进行分裂。

- 5) 最终结果：经过多次分裂，所有数据点被划分为独立的簇，生成的树状图如下所示：

- 第一层: $\{(1, 1), (1, 2), (2, 1)\}$ 和 $\{(8, 8), (8, 9), (9, 8)\}$ 。
- 第二层: $\{(1, 1)\}$, $\{(1, 2)\}$, $\{(2, 1)\}$, $\{(8, 8)\}$, $\{(8, 9)\}$, $\{(9, 8)\}$ 。

分裂层次聚类是一种强大的分层聚类方法，能够从整体出发逐步细化簇的划分，并揭示数据的层次化结构。尽管其计算复杂度较高且对初始分裂点较为敏感，但在许多实际场景中表现出色。通过结合降维技术和自动化分裂准则选择方法，可以进一步提升分裂层次聚类的性能和适用范围。

8.7 实践练习

在本节中，将通过三个实际案例分别展示原型聚类、密度聚类和层次聚类的应用。每个案例将使用不同的聚类算法解决具体问题，并通过代码实现和结果分析验证算法的有效性。

8.7.1 示例：基于 K-Means 的客户分群

某电商平台希望根据用户的消费行为对其进行分群，以便制定个性化的营销策略。数据集包含每位用户的年消费金额（单位：千元）和消费频率（单位：次/月）。选择 K-Means 算法对用户进行分群。假设有以下二维数据点：用户 1: (10,2)，用户 2: (15,3)，用户 3: (20,4)，用户 4: (80,10)，用户 5: (90,12)，用户 6: (100,15)。首先，使用 K-Means 算法将用户分为两类。其次，对聚类结果进行可视化展示。最后分析每类用户的特征。以下是 Python 代码实现：

```
1 import numpy as np
2 from sklearn.cluster import KMeans
3 import matplotlib.pyplot as plt
4
5 # 数据集
6 data = np.array([[10, 2], [15, 3], [20, 4],
7                 [80, 10], [90, 12], [100, 15]])
8
9 # 应用K-Means算法
10 kmeans = KMeans(n_clusters=2, random_state=42)
11 labels = kmeans.fit_predict(data)
12
13 plt.rcParams['font.sans-serif'] = ['SimHei']
14 plt.figure(figsize=(6, 4))
15 # 可视化结果
16 plt.scatter(data[:, 0], data[:, 1], c=labels, cmap='viridis', s=50)
17 plt.scatter(kmeans.cluster_centers[:, 0], kmeans.cluster_centers[:, 1],
18             c='red', marker='x', s=200, label='原型点')
19 plt.xlabel("年度消费(千元)")
20 plt.ylabel("周期(次/月)")
21 plt.legend()
22 plt.grid()
23 plt.title("消费者分析")
24 plt.show()
25
```

运行上述代码后，能够得到下图8.1，可以看到用户被分为两类，第一类：低消费金额、低消费频率的用户。第二类：高消费金额、高消费频率的用户。电商平台可以根据这两类用户的特点制定不同的营销策略。

8.7.2 示例：基于 DBSCAN 的异常检测

某银行希望检测信用卡交易中的异常行为。数据集包含每笔交易的金额（单位：千元）和时间间隔（单位：小时）。由于异常交易通常具有孤立性和稀疏性，选择 DBSCAN 算法进行异常检测。

假设有以下二维数据点：交易 1: (1,2)，交易 2: (1.5,2.5)，交易 3: (2,3)，交易 4: (10,15)，交易 5: (11,16)，交易 6: (50,70)。首先，使用 DBSCAN 算法识别正常交易和异常交易。其次，可视化检测结果。最后，分析异常交易的特征。以下是 Python 代码实现：

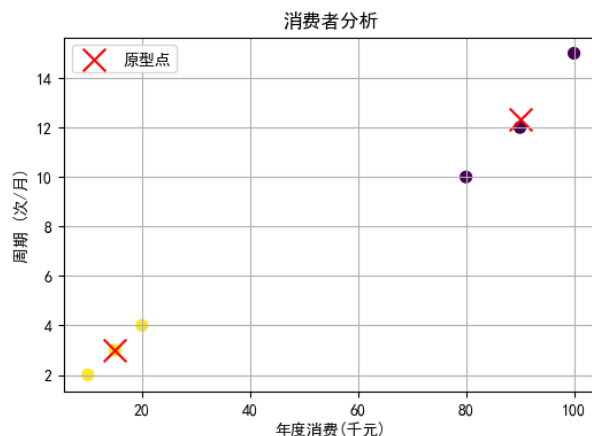


图 8.1: K-means 聚类结果

```

1  from sklearn.cluster import DBSCAN
2  import matplotlib.pyplot as plt
3
4  # 数据集
5  data = np.array([[1, 2], [1.5, 2.5], [2, 3],
6  [10, 15], [11, 16], [50, 70]])
7
8  # 应用DBSCAN算法
9  dbscan = DBSCAN(eps=5, min_samples=2)
10 labels = dbscan.fit_predict(data)
11
12 # 可视化结果
13 plt.scatter(data[:, 0], data[:, 1], c=labels, cmap='viridis', s=50)
14 plt.xlabel("Transaction Amount (thousand $)")
15 plt.ylabel("Time Interval (hours)")
16 plt.title("Anomaly Detection using DBSCAN")
17 plt.show()

```

运行上述代码后，能够得到下图8.2，可以看到，正常交易：形成两个簇（左下角），分别对应低金额短时间间隔和高金额长时间间隔的交易。异常交易：孤立点 (50,70) 被标记为噪声（右上角）。银行可以针对异常交易进一步调查其合法性。

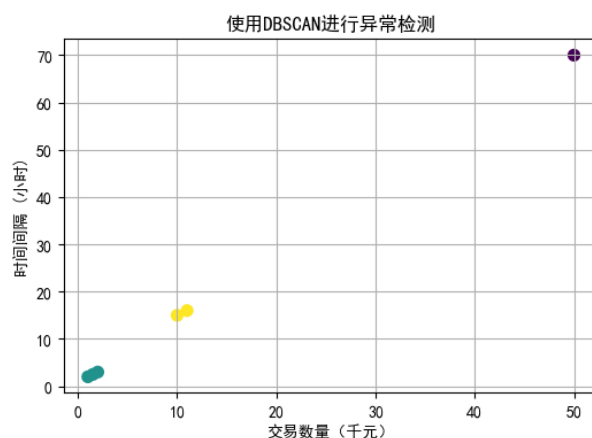


图 8.2: DBSCAN 异常检测

8.7.3 示例：基于凝聚层次聚类的文档分类

某新闻网站希望对一组新闻文章进行分类，以便更好地组织内容。数据集包含每篇文章的主题词向量（通过 TF-IDF 方法生成）。选择凝聚层次聚类算法对文章进行分类。

假设有以下二维主题词向量：文章 1: (0.1,0.2)，文章 2: (0.15,0.25)，文章 3: (0.2,0.3)，文章 4: (0.8,0.9)，文章 5: (0.85,0.95)，文章 6: (0.9,1.0)。首先，使用凝聚层次聚类算法对文章进行分类。其次，绘制树状图（Dendrogram）以展示层次化结构。最后，根据树状图选择合适的切割点提取簇。以下是 Python 代码实现：

```
1 from scipy.cluster.hierarchy import dendrogram, linkage
2 import matplotlib.pyplot as plt
3
4 # 数据集
5 data = [[0.1, 0.2], [0.15, 0.25], [0.2, 0.3],
6         [0.8, 0.9], [0.85, 0.95], [0.9, 1.0]]
7
8 plt.rcParams['font.sans-serif'] = ['SimHei']
9 plt.figure(figsize=(6, 4))
10 # 计算层次聚类的链接矩阵
11 Z = linkage(data, method='average')
12
13 # 绘制树状图
14 dendrogram(Z)
15 plt.title("层次化聚类树状图")
16 plt.xlabel("数据点")
17 plt.ylabel("距离")
18 plt.grid()
19 plt.show()
```

运行上述代码后，能够得到下图8.3，可以看到树状图清晰地展示了数据的层次化结构：前三篇文章形成一个簇，表示相似主题的文章；后三篇文章形成另一个簇，表示另一类主题的文章。新闻网站可以根据这些分类结果对文章进行归档或推荐。

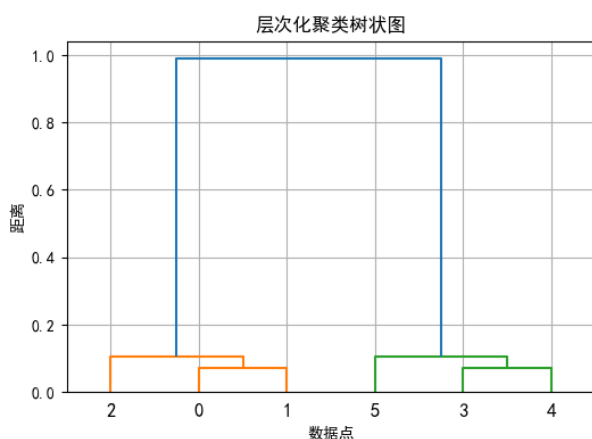


图 8.3: 层次聚类

8.7.4 示例：照片分类

回忆本章开头，石榴同学拍摄了各种照片，想要对这些照片进行分类，经过研究，石榴同学决定尝试两种经典的聚类算法：K-Means 和 DBSCAN。她希望通过实验比较这两种算法在照片分类任务中的表现，并找到最适合的方法，具体任务包括：

- 使用图像特征提取方法（如颜色直方图或深度学习特征）将照片转换为特征向量。
- 应用 K-Means 算法对照片进行分类，并分析其效果。
- 应用 DBSCAN 算法对照片进行分类，并分析其对噪声的鲁棒性。
- 对比两种算法的结果，选择更适合的算法并解释原因。

为简化问题，收集到一个包含 50 张照片的小型数据集，每张照片提取 3 维的灰度图像颜色直方图特征。这种特征提取方式相对简单高效，能够从灰度图像中捕捉到关于像素强度分布的关键信息。灰度图像颜色直方图记录了图像中不同灰度级像素的出现频率，通过将其量化为 3 维特征向量，可以在一定程度上保留图像的纹理和亮度等特性，尽管相较于更高维度的特征表示，可能会丢失一些细节信息，但对于初步的聚类分析来说，这种简化在计算效率和特征表达之间取得了一个平衡，能够快速地对图像进行分类和理解，为进一步的分析和处理提供基础。整个实现步骤如下：

1. 数据预处理：

- 读取照片特征向量，并对其进行归一化处理。
- 可视化部分照片及其特征分布，初步了解数据特性。

2. K-Means 聚类：

- 设置簇数 $K = 3$ ，分别对应“日出”、“森林”和“河流”。
- 应用 K-Means 算法对照片进行分类。
- 可视化聚类结果，并计算轮廓系数评估聚类质量。

3. DBSCAN 聚类：

- 设置参数 ϵ 和 $MinPts$ ，通过实验调整合适的值。
- 应用 DBSCAN 算法对照片进行分类。
- 分析噪声点的数量及其对聚类结果的影响。

4. 结果对比与分析：

- 比较两种算法的分类效果，重点关注对噪声的处理能力。
- 结合实际需求（如是否需要明确的簇数），选择更合适的算法。

以下是 Python 代码框架，供参考：

```
1 import numpy as np
2 from sklearn.cluster import KMeans, DBSCAN
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.metrics import silhouette_score
5 import matplotlib.pyplot as plt
6 # 读取照片特征向量（假设已提取）
7 features = np.load("photo_features.npy") # 假设特征存储在文件中
8 # 数据预处理
9 scaler = StandardScaler()
10 features_scaled = scaler.fit_transform(features)
11 # K-Means 聚类
12 kmeans = KMeans(n_clusters=3, random_state=42)
13 kmeans_labels = kmeans.fit_predict(features_scaled)
14 kmeans_score = silhouette_score(features_scaled, kmeans_labels)
15 # DBSCAN 聚类
16 dbscan = DBSCAN(eps=0.5, min_samples=3)
17 dbscan_labels = dbscan.fit_predict(features_scaled)
18 # 可视化聚类结果
19 plt.rcParams['font.sans-serif'] = ['SimHei']
```

```

20 plt.rcParams['axes.unicode_minus'] = False
21 plt.figure(figsize=(12, 4))
22 plt.subplot(1, 2, 1)
23 plt.scatter(features_scaled[:, 0], features_scaled[:, 1], c=kmeans_labels, cmap='viridis')
24 plt.title("K-Means 聚类")
25 plt.grid()
26 plt.subplot(1, 2, 2)
27 plt.scatter(features_scaled[:, 0], features_scaled[:, 1], c=dbscan_labels, cmap='viridis')
28 plt.title("DBSCAN 聚类")
29 plt.grid()
30 plt.show()
31 print(f"K-Means 轮廓系数: {kmeans_score}")
32 print(f"DBSCAN 噪声点个数: {np.sum(dbscan_labels == -1)}")

```

运行上述程序后，输出结果如下图所示：

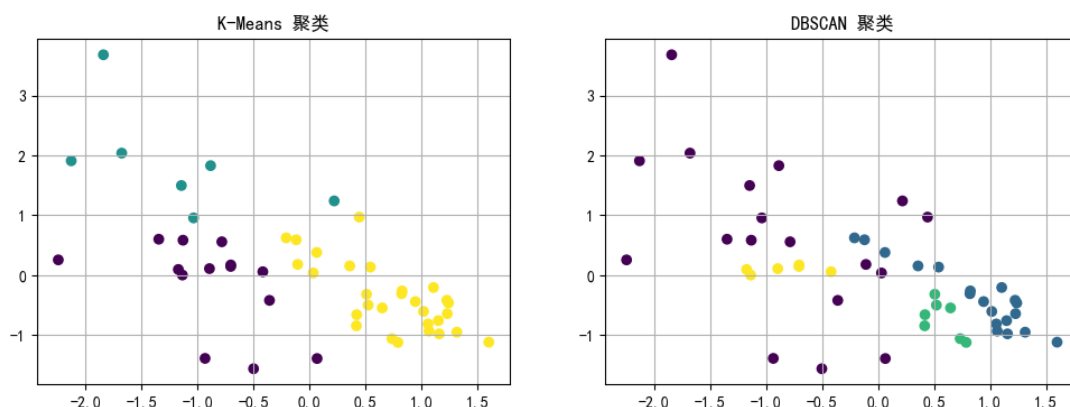


图 8.4: Kmeans 和 DBSCAN 聚类结果

在图 8.4 中，可以观察到两种聚类方法的显著差异。K-means 方法将数据明确地划分为 3 个类别，其结果清晰且易于理解，这得益于该算法在预设类别数量时的明确性。相比之下，DBSCAN 方法则不依赖于预先设定的类别数量，而是根据数据的分布密度和指定的参数，自主地将数据划分为 5 个类别。这种差异可以体现两种算法在处理数据时的不同策略：K-means 更适合于已知类别数量的场景，而 DBSCAN 则在探索数据内在结构方面更具灵活性。

通过本次实践，基本掌握 K-Means 和 DBSCAN 两种聚类算法的基本原理及其在实际问题中的应用。但在真正的实际应用中，你可能需要考虑以下几个问题：

1. 在 K-Means 算法中，如何选择合适的簇数 K ？可以使用哪些方法？
2. DBSCAN 算法中的参数 ϵ 和 $MinPts$ 对结果有何影响？如何调整这些参数？
3. 如果照片数量增加到数千张，如何优化聚类算法的效率？
4. 在实际应用中，除了颜色直方图，还可以使用哪些特征来描述照片？

8.8 习题

8.8.1 简答题

1. 什么是欧氏距离？它有哪些特点和应用场景？
2. 曼哈顿距离与欧氏距离的主要区别是什么？请举例说明其适用场景。

3. 余弦相似度适用于哪些类型的数据？为什么它不受向量长度的影响？
4. 马氏距离与欧氏距离相比有哪些优势？它的主要局限性是什么？
5. 杰卡德距离如何定义？它在文本聚类中有何应用？
6. 简述 K-Means 算法的基本思想及其局限性。
7. DBSCAN 算法中的核心点、边界点和噪声点是如何定义的？
8. 模糊 C 均值 (FCM) 与 K-Means 的主要区别是什么？
9. 层次聚类的优点和缺点分别是什么？
10. 在实际应用中，如何选择合适的聚类算法？

8.8.2 计算题

1. 给定两个二维点 $\mathbf{x} = (1, 2)$ 和 $\mathbf{y} = (4, 6)$ ，计算它们的欧氏距离和曼哈顿距离。
2. 给定两个向量 $\mathbf{x} = (3, 4)$ 和 $\mathbf{y} = (-1, 2)$ ，计算它们的余弦相似度。
3. 给定两个集合 $A = \{1, 2, 3\}$ 和 $B = \{2, 3, 4\}$ ，计算它们的杰卡德距离。
4. 使用 K-Means 算法对以下二维数据点进行聚类，假设簇数 $K = 2$ ，初始簇中心为 $(1, 1)$ 和 $(8, 8)$ 。写出每一步的簇分配和簇中心更新过程。数据点： $(1, 1), (1, 2), (2, 1), (8, 8), (8, 9), (9, 8)$
5. 给定以下数据点，使用 DBSCAN 算法进行聚类，参数设置为 $\epsilon = 2$ ， $MinPts = 2$ 。写出每个点的分类结果。数据点： $(1, 1), (1, 2), (2, 1), (8, 8), (8, 9), (9, 8)$
6. 给定以下隶属度矩阵，计算每个数据点的簇归属（取最大隶属度对应的簇）。隶属度矩阵：
$$\begin{bmatrix} 0.8 & 0.2 \\ 0.3 & 0.7 \\ 0.6 & 0.4 \end{bmatrix}$$
。

8.8.3 编程题

1. 编写 Python 代码实现 K-Means 算法，并将其应用于以下二维数据点，簇数 $K = 2$ 。数据点： $(1, 1), (1, 2), (2, 1), (8, 8), (8, 9), (9, 8)$
2. 使用 DBSCAN 算法对以下数据点进行聚类，参数设置为 $\epsilon = 2$ ， $MinPts = 2$ 。数据点： $(1, 1), (1, 2), (2, 1), (8, 8), (8, 9), (9, 8)$
3. 使用凝聚层次聚类算法对以下数据点进行聚类，并绘制树状图。数据点： $(0.1, 0.2), (0.15, 0.25), (0.2, 0.3), (0.8, 0.9), (0.85, 0.95), (0.9, 1.0)$

第九章 数据降维

在当今信息爆炸的时代，面对的数据往往具有极高的维度。高维数据虽然能够提供丰富的信息，但也带来了计算复杂度的增加和“维数灾难”的风险，使得模型训练效率低下，并且容易过拟合。因此，如何有效地降低数据维度，同时尽可能保持原始数据的主要特征，成为机器学习与数据分析中一个至关重要的课题。本章将介绍几种广泛使用的降维技术，包括主成分分析（PCA）、线性判别分析（LDA）等，并探讨它们的应用场景。

学习目标

本章系统介绍数据降维的基本概念与常用方法。理解降维在缓解维度灾难、降低计算复杂度、提升模型性能与可视化中的作用，掌握主成分分析和线性判别分析的原理及其在线性降维中的应用，了解非线性方法如 t-SNE 在高维数据可视化中的优势。通过 Python 实践，能够将这些方法应用于实际数据集，提升对高维数据的处理与分析能力。

9.1 水果营养分析问题

石榴同学正在处理一份关于水果营养成分的数据集，该数据集包含多种水果的大量营养元素，例如维生素 C 含量、矿物质种类等，并且数据集中有太多相似但又不完全相同的特征。她发现由于特征过多，导致模型训练时间过长，而且模型泛化能力也受到影响。她想：“如果能将这些特征合并成少数几个新的特征，那不是可以简化我的工作吗？”带着这个想法，石榴决定尝试一种叫做主成分分析（PCA）的方法来减少特征数量。

9.1.1 从故事到原理

想象一下，如果有一群点分布在二维平面上，但它们大致沿着一条直线排列。这时，可以找到这条直线作为新坐标轴，用它来表示所有点的位置，从而减少原本需要两个坐标才能描述的信息量。这就是 PCA 的核心思想——寻找最佳的方向（即主成分），以最大程度地保留数据的方差。

为更深入理解 PCA 的工作机制，需要考虑以下几点：

- **数据预处理**：在应用 PCA 之前，通常需要对原始数据进行标准化处理，使得每个特征具有零均值和单位方差。
- **协方差矩阵**：PCA 依赖于计算原始数据的协方差矩阵 Σ ，其定义为：

$$\Sigma = \frac{1}{n-1} \mathbf{X}^T \mathbf{X} \quad (9.1)$$

其中 n 是样本数， $\mathbf{X}$ 是中心化的数据矩阵。

- **特征值分解**：一旦得到协方差矩阵 Σ ，下一步就是对其进行特征值分解。对于一个 $d \times d$ 的矩阵 Σ ，存在一组正交向量 $\mathbf{v}_i$ 和对应的非负实数 λ_i ，满足

$$\Sigma \mathbf{v}_i = \lambda_i \mathbf{v}_i, \quad i = 1, \dots, d \quad (9.2)$$

这里 λ_i 是特征值，而 $\mathbf{v}_i$ 是相应的特征向量。

- **选择主成分**：根据特征值 λ_i 的大小排序，选择前 k 个特征值对应的特征向量组成投影矩阵 $\mathbf{W}$ ，其中 k 是希望降到的维度。因此，

$$\mathbf{W} = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k] \quad (9.3)$$

在这个过程中，石榴同学了解到，虽然 PCA 能够有效地降低数据维度，但它是一个无监督的方法，也就是说它并不考虑数据的标签信息。对于有标签的数据，还可以使用另一种方法——线性判别分析（LDA）。与 PCA 不同，LDA 旨在最大化类间差异最小化类内差异，因此更适合用于分类任务中的降维。

9.1.2 水果降维

为让理论更加直观，石榴同学给出一个简单的例子。假设有一个包含两种类型水果的数据集，分别是苹果和橙子，每种水果有两个特征：重量和甜度。这个数据集可以表示为一个 $m \times 2$ 的矩阵 $\mathbf{X}$ ，其中 m 是样本数。

表 9.1: 原始数据示例

类别	重量 (g)	甜度 (%)
苹果	150	13
苹果	140	12
苹果	160	14
橙子	180	10
橙子	170	9
橙子	190	8

石榴同学首先对这两个特征进行标准化处理，然后计算它们之间的协方差矩阵，并对其进行特征值分解。她找到两个特征值及其相应的特征向量。因为目标是将数据降至一维，所以石榴选择对应较大特征值的那个特征向量作为新的坐标轴方向。石榴同学利用 Python 编写了一段代码来完成上述过程：

```

1  import numpy as np
2  from sklearn.decomposition import PCA
3  # 原始数据（已省略标准化步骤）
4  X = np.array([[150, 13], [140, 12], [160, 14], [180, 10], [170, 9], [190, 8]])
5  # 创建PCA对象，设置n_components=1表示降维至1维
6  pca = PCA(n_components=1)
7  # 拟合并转换数据
8  X_pca = pca.fit_transform(X)
9  print("降维后的数据：")
10 print(X_pca)

```

运行这段代码后，石榴得到降维后的数据 $\mathbf{X}_{\text{pca}}$ ，这组数据只保留了一个特征，代表原始两个特征的主要变化方向。通过这种方式，石榴不仅可以简化数据结构，还为后续可能的机器学习任务打下良好的基础。

经过一番努力，石榴同学成功地运用 PCA 降低了数据集的维度，不仅可以提高模型训练的速度，还能够提升模型的性能。更重要的是，她还学到了一个重要的道理：有时候，少即是多；适当的简化可以更好地理解和解决问题。

9.2 主成分分析

9.2.1 PCA 目标函数

在本小节中，将深入探讨主成分分析（PCA）背后的数学原理，并通过直观解释帮助读者理解其计算过程。PCA 的核心思想是找到一个新坐标系，使得数据在这个新坐标系中的投影能够保留尽可能多的原始信息，同时减少维度。

想象一下，你有一堆散落在三维空间中的点，这些点代表你的高维数据集。你的任务是找到一种方法将这些点映射到一个二维平面上，同时尽量不丢失重要的信息。换句话说，你想要找到一张纸，当你把这张纸放在合适的位置和角度时，所有点在这个平面上的影子能最好地表示原来的点。

为简化问题，首先对数据进行中心化处理，即减去每个特征的均值，这样做的目的是让新的坐标系原点位于数据质心处，从而保证旋转不会改变数据的相对位置。

选择第一主成分的方向可以被看作是在所有可能的方向中，那个能够让投影后数据方差最大的方向。这就像在寻找那张“最佳平面”，使得当将点投影到这个平面上时，它们之间的距离尽可能保持不变。因此，需要解决以下优化问题：

$$\max_{\mathbf{w}} \sum_{i=1}^n (\mathbf{x}_i^T \mathbf{w})^2 \quad (9.4)$$

这里 $\mathbf{x}_i^T \mathbf{w}$ 表示第 i 个样本点 $\mathbf{x}_i$ 在向量 $\mathbf{w}$ 方向上投影后的长度。 $\mathbf{x}_i$ 是经过中心化处理后的样本点，是一个列向量；而 $\mathbf{w}$ 是单位长度的向量，也是一列向量，它指示着新的坐标轴方向。 $\mathbf{x}_i^T$ 表示 $\mathbf{x}_i$ 的转置，变成一行向量。当这两个向量相乘时，实际上是在计算两个向量之间的内积，结果是一个标量，它代表样本点在给定方向上的投影长度。

为确保 $\mathbf{w}$ 的长度为 1，添加约束条件 $\|\mathbf{w}\|_2^2 = 1$ 。这意味着要找的是一个单位向量，它指向数据变异最大的方向。则带有约束的目标函数就变为：

$$\begin{aligned} \max_{\mathbf{w}} \quad & \sum_{i=1}^n (\mathbf{x}_i^T \mathbf{w})^2 \\ \text{s.t.} \quad & \mathbf{w}^T \mathbf{w} = 1. \end{aligned} \quad (9.5)$$

现在来解释为什么求解该问题等价于找到协方差矩阵的最大特征值对应的特征向量。可以重写上述公式，以揭示其中隐藏的结构。因为 $\mathbf{x}_i^T \mathbf{w}$ 是一个标量，它的平方等于它自身的内积：

$$(\mathbf{x}_i^T \mathbf{w})^2 = (\mathbf{x}_i^T \mathbf{w})(\mathbf{w}^T \mathbf{x}_i) \quad (9.6)$$

将上式代入目标函数并展开求和，得到：

$$\sum_{i=1}^n (\mathbf{x}_i^T \mathbf{w})(\mathbf{w}^T \mathbf{x}_i) = \mathbf{w}^T \left(\sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^T \right) \mathbf{w} \quad (9.7)$$

括号内的表达式正好是协方差矩阵 $\mathbf{C}$ 的定义，如果假设数据已经中心化（即每个特征的平均值为 0），那么：

$$\mathbf{C} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^\top \quad (9.8)$$

于是，目标函数可以进一步简化为：

$$\begin{aligned} \max_{\mathbf{w}} \quad & \mathbf{w}^\top \mathbf{C} \mathbf{w} \\ \text{s.t.} \quad & \mathbf{w}^\top \mathbf{w} = 1. \end{aligned} \quad (9.9)$$

此时，优化问题变成寻找一个单位向量 $\mathbf{w}$ ，使得 $\mathbf{w}^\top \mathbf{C} \mathbf{w}$ 最大。根据线性代数的知识，这个问题等价于找到协方差矩阵 $\mathbf{C}$ 的最大特征值对应的特征向量作为最优解。这是因为对于任意单位向量 $\mathbf{w}$ ， $\mathbf{w}^\top \mathbf{C} \mathbf{w}$ 实际上是 $\mathbf{C}$ 的特征值的加权平均，权重为 $\mathbf{w}$ 对应的特征向量分量的平方。因此，当选择 $\mathbf{w}$ 为对应最大特征值的特征向量时，实际上是在最大化这个加权平均值，也就是最大化投影后的方差。

9.2.2 PCA 拉格朗日求解

从优化角度，为求解这个带约束的优化问题，可以采用拉格朗日乘数法。构造拉格朗日函数如下：

$$L(\mathbf{w}, \lambda) = \mathbf{w}^\top \mathbf{C} \mathbf{w} - \lambda(\mathbf{w}^\top \mathbf{w} - 1) \quad (9.10)$$

对 $\mathbf{w}$ 求偏导并令其等于零，得到：

$$\frac{\partial L}{\partial \mathbf{w}} = 2\mathbf{C} \mathbf{w} - 2\lambda \mathbf{w} = 0 \quad (9.11)$$

从而获得特征值问题：

$$\mathbf{C} \mathbf{w} = \lambda \mathbf{w} \quad (9.12)$$

这意味着要解决的问题转换为求解协方差矩阵 $\mathbf{C}$ 的特征值和特征向量。最大的特征值对应的特征向量给出了数据的最大方差方向，也就是所说的第一个主成分。

综上所述，求解该问题等价于找到协方差矩阵的最大特征值对应的特征向量，因为这正是最大化投影后数据方差的方法。这种方法不仅提供了计算上的便利，而且基于统计学的意义——特征值能够反映沿特定方向的数据分布的分散程度，特征向量则指出了这些方向。

对于第二、第三等后续主成分，它们应当满足与之前选择的所有主成分正交，并且最大化剩余数据的方差。这就好比你在找一张纸来放置那些已经被投影了一次的点的影子，你希望这张纸垂直于之前的纸，并且影子仍然尽可能分散。这可以通过依次选取次大的特征值对应的特征向量来实现，直到累积解释方差达到满意的程度或降至指定的维度。

PCA 不仅是一个降维工具，更是一种理解和探索数据结构的方法。它有助于识别出数据中的主要模式，使得复杂的数据变得易于可视化和解释。此外，通过去除冗余信息，PCA 还有助于提高机器学习模型的性能和效率。这样的方法不仅直观而且有效，在许多领域都有广泛的应用。

9.3 PCA 核心要素直观理解

基于上述分析，主成分分析的核心数学工具可归结为矩阵的特征值分解。本节将深入解析这一分解方法的数学机理及其在数据降维中的理论本质。

9.3.1 特征值分解理解

在探索数据集的过程中，石榴同学发现了一个有趣的现象：当对协方差矩阵 进行特征值分解后，得到的特征向量和特征值似乎隐藏着某种神秘的力量，它们有助于找到数据的主要结构。那么，这背后的原因是什么呢？这里通过一个更直观的方式来理解这个过程，并加入可视化代码以更好地展示。

想象一下，你面前有一群点分布在二维平面上，这些点代表你的数据样本。每个点的位置由两个坐标（比如重量和甜度）决定。现在，如果你试图用一条直线来概括这些点的分布情况，你会怎么做呢？最理想的情况下，这条直线应该尽可能多地穿过或靠近那些点，并且它还应该捕捉到数据的最大变化方向——也就是说，当沿着这条直线看时，你能看到点与点之间的最大差异。

在这个比喻中，协方差矩阵 就好比是描述这群点如何相互关联的地图。当对 进行特征值分解时，实际上是在问：“哪条线（特征向量）能够最好地反映数据的变化趋势，同时让这条线上点的分散程度（方差）达到最大？”这里，特征值就起到衡量作用——它指出沿着某个特征向量方向上，数据分散的程度有多大。因此，较大的特征值意味着更多的信息量被保留下来；相对应的特征向量则指示了该信息的方向。

回到石榴同学的故事里，她通过计算得到一组特征值和特征向量。其中，最大的那个特征值对应的特征向量指出了数据中最重要的变化方向——即第一个主成分。如果继续选择第二大的特征值及其对应的特征向量，那么可以获得第二个主成分，它与第一个主成分正交（垂直），并且表示次重要的变化方向。以此类推，对于 d 维的数据，理论上可以找到 d 个这样的主成分。

所以，简单来说，特征值分解后的结果之所以能用于主成分分析，是因为特征值和特征向量提供了一种数学上的方法，用来识别数据中最显著的变化模式。通过选取具有较大特征值的特征向量作为新的坐标轴，PCA 使得在减少维度的同时尽可能保持原始数据的重要特性。这样一来，即便降维后的数据只包含少数几个特征，它们依然能够很好地代表原来复杂多样的数据结构。

为更好地理解这一概念，将使用 Python 中的 matplotlib 库来进行可视化。以下是一个简单的例子，展示如何创建一个二维数据集、执行 PCA 并绘制原始数据及转换后的数据：

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.decomposition import PCA
4 # 创建一个模拟的二维数据集
5 np.random.seed(42) # 为了可重复性
6 X = np.dot(np.random.rand(2, 2), np.random.randn(2, 200)).T
7 # 执行PCA
8 pca = PCA(n_components=2)
9 X_pca = pca.fit_transform(X)
10 # 设置支持中文
11 plt.rcParams['font.sans-serif'] = ['SimHei']
12 # 设置支持负号
13 plt.rcParams['axes.unicode_minus'] = False
14 # 可视化原始数据和PCA变换后的数据
15 plt.figure(figsize=(12,4))
16 mean = np.mean(X, axis=0)
17 # 原始数据
18 plt.subplot(1, 2, 1)
19 plt.scatter(X[:, 0], X[:, 1], alpha=0.7)
20 plt.title('原始数据')
21 plt.xlabel('特征1')
22 plt.ylabel('特征2')
23 plt.grid()
24 for length, vector in zip(pca.explained_variance_, pca.components_):
25     # np.sqrt(length) 代表了沿着该特征向量方向上的标准差，反映了数据沿此方向的分散程度。
26     v = vector * 3 * np.sqrt(length)
27     plt.arrow(mean[0], mean[1], v[0], v[1], head_width=0.2, head_length=0.2, fc='red', ec='red')
```

```
28
29 # 绘制PCA变换后的数据
30 plt.subplot(1, 2, 2)
31 plt.scatter(X_pca[:, 0], X_pca[:, 1], alpha=0.7)
32 plt.title('PCA 变换后数据')
33 plt.xlabel('主成分 1')
34 plt.ylabel('主成分 2')
35
36 # 添加特征向量为箭头
37 for length, vector in zip(pca.explained_variance_, pca.components_):
38     # np.sqrt(length) 代表了沿着该特征向量方向上的标准差，反映了数据沿此方向的分散程度。
39     v = vector * np.sqrt(length)
40     # 绘制主成分变量的箭头
41     plt.arrow(mean[0], mean[1], v[0], v[1], head_width=0.2, head_length=0.2, fc='red', ec='red')
42
43 plt.tight_layout()
44 plt.grid()
45 plt.show()
```

9.3.2 特征向量可视化

上面代码首先生成一个随机的二维数据集，然后使用 sklearn 库中的 PCA 模块对其进行主成分分析。最后，它绘制两幅图：一幅显示原始数据点以及主要成分的方向（红色箭头），另一幅展示经过 PCA 转换后的数据点以及主要成分的方向（红色箭头）。其中不仅可以展示原始数据点的分布情况，而且还用红色箭头标出 PCA 计算得到的特征向量。现在来仔细分析一下这个可视化结果。

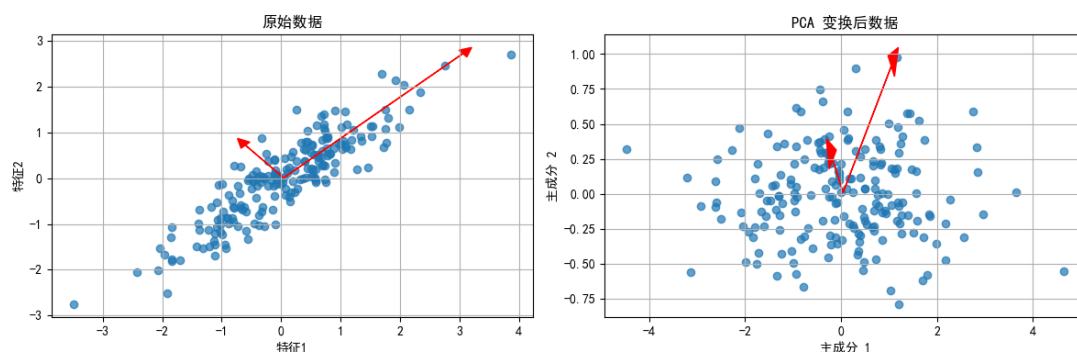


图 9.1: 主成分分析后的数据分布与特征向量方向

如图9.1所示，可以观察到以下几点：

1. **数据点的分布趋势：**从散点图中可以看出，数据点倾向于沿着某些特定的方向聚集，而不是均匀地分布在所有可能的位置上。这种现象表明，在高维空间中的数据实际上可以被投影到一个更低维度的空间中，而不会丢失太多信息。这就是降维技术能够发挥作用的基础。

2. **特征向量指示的主要变化方向：**红色箭头代表 PCA 算法找到的两个主要变化方向，即第一和第二主成分。这两个方向是通过最大化方差来确定的，因此它们能够捕捉到数据内部最显著的变化模式。具体来说，最长的那个箭头（通常对应于最大特征值）指出数据沿其分布最广泛的方向；而较短的箭头则表示次重要的变化方向。

3. **特征向量的长度与解释方差：**箭头的长度反映了相应主成分的重要性或影响力。根据前面代码中对特征向量进行缩放的方式，知道较长的箭头意味着更高的方差——也就是说，该方向上的数据波动更大，携带的信息更多。反之，如果某个特征向量很短，那说明在这个方向上的变化相对较小，它对于理解整体数据结构的作用也就不那么重要。

4. **新坐标系的理解：**当数据投影到由这些特征向量构成的新坐标系时，实际上是在重新定义数

据的空间布局。新的坐标轴不再是原来的维度，而是按照数据本身的内在结构来排列的。这样做的好处是可以更容易地发现隐藏在数据背后的规律，并且有助于去除噪声或者冗余信息。

综上所述，通过对 PCA 结果的可视化分析，不仅获得关于数据分布的直观认识，还加深了对降维过程中涉及的关键概念的理解。例如，现在知道，尽管原始数据可能是多维的，但其大部分变异往往集中在少数几个主成分上。这对于后续的数据探索、模型构建以及解释都有很大的帮助。此外，这样的可视化还可以用来检查 PCA 是否正确实施，因为异常的结果（如不合理的箭头方向或长度）可能会提示在预处理步骤中存在的问题。

9.4 核化线性降维

9.4.1 PCA 的局限性

在探索数据降维方法时，主成分分析（Principal Component Analysis, PCA）是一种非常流行的技术。它通过将原始数据投影到一个新的坐标系中，使得新坐标轴的方向与数据的方差最大方向对齐，从而实现数据压缩和特征提取的目的。然而，尽管 PCA 具有计算简单、易于理解的优点，但它并非适用于所有类型的数据集。

考虑一个非线性分布的数据点云，例如呈环状或螺旋状分布的数据。对于这类复杂结构的数据，PCA 可能会显得力不从心。因为 PCA 只能捕捉到数据中的线性相关性，即它寻找的是能够最大化方差的直线方向。当数据内部存在复杂的非线性关系时，PCA 可能无法有效地降低维度，甚至可能导致信息丢失。这是因为 PCA 假设数据分布在超平面上，而实际上很多现实世界的的数据并不满足这个条件。

9.4.2 核化线性降维

面对 PCA 的这些局限性，研究人员提出核化线性降维（Kernelized Linear Dimensionality Reduction）。这种技术的核心思想是利用核函数来映射原始数据到更高维的空间，在这个高维空间里，原本难以分离的数据点可以变得更加线性可分。通过选择合适的核函数，可以将非线性的模式转换成线性的，进而使用类似于 PCA 的方法进行降维处理。

具体来说，核化线性降维首先定义一个从输入空间到特征空间的隐式映射 $\phi(\mathbf{x})$ ，其中 $\mathbf{x}$ 表示原始数据向量。然后，不再直接操作于高维特征空间中的数据点，而是借助核函数 $K(\mathbf{x}, \mathbf{y}) = \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle$ 来计算两个数据点在这个空间里的内积。这里的 $\langle \cdot, \cdot \rangle$ 表示内积运算，而 $K(\mathbf{x}, \mathbf{y})$ 则代表在原始空间中计算得到的核矩阵元素。

这种方法的优势在于，它允许在不需要显式知道映射 $\phi(\mathbf{x})$ 的情况下，间接地在高维空间执行降维操作。这不仅简化了计算过程，而且由于高维空间中数据的线性特性增强，往往可以获得更好的降维效果。

9.4.3 核化 PCA

核化 PCA（Kernel PCA）继承了标准 PCA 的思想，但在计算协方差矩阵之前，先通过核函数将数据映射到一个更高维的空间。之后，再基于这个新的空间中的数据执行特征值分解，以找到最优的降维方向。

设有 N 个样本 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$ ，每个样本都是 d 维向量。在核化 PCA 中，首先定义一个非线性映射 $\phi: \mathbb{R}^d \rightarrow \mathcal{F}$ ，它将原始数据点映射到一个高维（可能是无穷维）特征空间 $\mathcal{F}$ 。在这个特征空间中，可以计算中心化的数据点 $\phi(\mathbf{x}_i) = \phi(\mathbf{x}_i) - \frac{1}{N} \sum_{j=1}^N \phi(\mathbf{x}_j)$ 。

由于直接在高维空间中进行计算往往是不可行的,因此使用核技巧来避免显式地构建映射 ϕ 。核函数 $K(\mathbf{x}, \mathbf{y})$ 定义为两个数据点在特征空间内的内积,即 $K(\mathbf{x}, \mathbf{y}) = \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle$ 。对于任意两个样本 $\mathbf{x}_i$ 和 $\mathbf{x}_j$, 它们之间的核函数值构成核矩阵 $\mathbf{K}$ 的元素, $\mathbf{K}_{ij} = K(\mathbf{x}_i, \mathbf{x}_j)$ 。

在标准 PCA 中, 对协方差矩阵 $\mathbf{C} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i \mathbf{x}_i^\top$ 进行特征值分解, 以寻找最大方差的方向。在核化 PCA 中, 处理的是特征空间中的协方差矩阵 $\mathbf{C}' = \frac{1}{N} \sum_{i=1}^N \phi(\mathbf{x}_i) \phi(\mathbf{x}_i)^\top$ 。但是, 并不需要直接计算 $\mathbf{C}'$, 而是利用核矩阵 $\mathbf{K}$ 来间接求解。

首先, 需要对核矩阵 $\mathbf{K}$ 进行中心化, 以确保在特征空间中也是基于中心化后的数据点来进行分析。这可以通过构造一个中心化矩阵 $\mathbf{H} = \mathbf{I} - \frac{1}{N} \mathbf{1} \mathbf{1}^\top$ 实现, 其中 $\mathbf{I}$ 是单位矩阵, 而 $\mathbf{1}$ 是所有元素均为 1 的列向量。中心化后的核矩阵记作 $\mathbf{K}_c = \mathbf{H} \mathbf{K} \mathbf{H}$ 。

接下来, 对中心化后的核矩阵 $\mathbf{K}_c$ 进行特征值分解, 得到一组特征值 $\lambda_1, \lambda_2, \dots, \lambda_N$ 和相应的特征向量 $\alpha_1, \alpha_2, \dots, \alpha_N$, 满足

$$\mathbf{K}_c \alpha_i = \lambda_i \alpha_i \quad (9.13)$$

这些特征向量实际上对应于特征空间中的主成分。为获取原始空间中的投影坐标, 可以使用核函数和特征向量 α_i 构造转换规则:

$$\mathbf{z}_i = \sqrt{\lambda_i} \alpha_i \quad (9.14)$$

这里, $\mathbf{z}_i$ 表示第 i 个样本在新特征空间下的表示。最后, 如果想要将一个样本 $\mathbf{x}_*$ 投影到由核化 PCA 确定的新坐标系上, 可以按照以下步骤操作:

首先, 计算新样本与训练集中所有样本的核函数值, 形成向量 $\mathbf{k}(\mathbf{x}_*)$ 。

其次, 对 $\mathbf{k}(\mathbf{x}_*)$ 进行中心化处理: $\mathbf{k}_c(\mathbf{x}_*) = \mathbf{H} \mathbf{k}(\mathbf{x}_*)$ 。

最后, 使用中心化后的核函数值向量 $\mathbf{k}_c(\mathbf{x}_*)$ 和特征向量 α_i 来计算新样本的投影坐标。

通过上述过程, 可以在不显式访问高维特征空间的情况下完成降维任务, 从而有效地保留数据的非线性结构。核化 PCA 不仅可以克服传统 PCA 的一些局限性, 而且能够提供更灵活的数据表示方式, 适用于更加广泛的应用场景。

9.4.4 核化 PCA 实例

为更好地理解核化主成分分析 (Kernel PCA), 通过一个简单的二维数据集来展示其计算过程。假设有如下二维数据点, 它们构成的数据矩阵 $\mathbf{X}$:

$$\mathbf{X} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{bmatrix} \quad (9.15)$$

将使用径向基函数 (RBF) 作为核函数进行核化 PCA。

1. 构建核矩阵

首先, 需要构建核矩阵 $\mathbf{K}$, 对于每个数据点对 i 和 j , 计算它们的核函数值 $\kappa(\mathbf{x}_i, \mathbf{x}_j)$ 。以 RBF 核为例, 其定义为:

$$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) \quad (9.16)$$

其中 γ 是一个参数, 通常需要调优。这里简单地取 $\gamma = 0.1$ 。计算得到的核矩阵 $\mathbf{K}$ 如下:

$$\mathbf{K} = \begin{bmatrix} 1.00 & 0.37 & 0.02 \\ 0.37 & 1.00 & 0.37 \\ 0.02 & 0.37 & 1.00 \end{bmatrix} \quad (9.17)$$

2. 中心化核矩阵

在应用 PCA 之前，需要确保数据是居中的。然而，在核空间中直接操作数据通常是不可能的，所以必须找到一种方法来中心化核矩阵 $\mathbf{K}$ 。这可以通过以下公式完成：

$$\mathbf{K} = (\mathbf{I} - \frac{1}{n}\mathbf{1})\mathbf{K}(\mathbf{I} - \frac{1}{n}\mathbf{1}) \quad (9.18)$$

其中 $\mathbf{I}$ 是单位矩阵， $\mathbf{1}$ 是所有元素都为 1 的矩阵， n 是样本数量。对于上述核矩阵，中心化后的结果为：

$$\mathbf{K} = \begin{bmatrix} 0.49 & -0.12 & -0.37 \\ -0.12 & 0.49 & -0.37 \\ -0.37 & -0.37 & 0.74 \end{bmatrix} \quad (9.19)$$

3. 特征分解

接下来，需要对中心化的核矩阵 $\mathbf{K}$ 进行特征分解。该步骤会给出特征值和对应的特征向量。这些特征向量实际上就是原始数据在核空间中的主成分方向。假设得到如下的特征值 λ_i 和特征向量 $\mathbf{v}_i$ ：

$$\lambda_1 = 1.28, \mathbf{v}_1 = \begin{bmatrix} -0.58 \\ -0.58 \\ 0.58 \end{bmatrix}, \lambda_2 = 0.74, \mathbf{v}_2 = \begin{bmatrix} -0.71 \\ 0.00 \\ 0.71 \end{bmatrix} \quad (9.20)$$

4. 选择主成分

根据特征值的大小，可以决定保留哪些主成分。在这个例子中，由于只有两个非零特征值，因此会选择这两个主成分。

5. 投影到低维空间

在核化 PCA 中，将原始数据点投影到由所选主成分构成的新坐标系中是最后一步。这一步骤可以通过将特征向量与中心化后的核矩阵相乘来完成，并且需要对特征值进行归一化处理。假设想要保留前两个主成分，则新数据点 $\mathbf{Z}$ 可以表示为：

$$\mathbf{Z} = \sqrt{\Lambda^{-1}}\mathbf{V}^T\mathbf{K} \quad (9.21)$$

其中 Λ 是包含所选特征值的对角矩阵， $\mathbf{V}$ 是相应的特征向量组成的矩阵，而 $\mathbf{K}$ 是中心化的核矩阵。

从前面的计算中知道，特征值和特征向量分别是：

$$\lambda_1 = 1.28, \mathbf{v}_1 = \begin{bmatrix} -0.58 \\ -0.58 \\ 0.58 \end{bmatrix}, \lambda_2 = 0.74, \mathbf{v}_2 = \begin{bmatrix} -0.71 \\ 0.00 \\ 0.71 \end{bmatrix} \quad (9.22)$$

以及中心化的核矩阵 $\mathbf{K}$ ：

$$\mathbf{K} = \begin{bmatrix} 0.49 & -0.12 & -0.37 \\ -0.12 & 0.49 & -0.37 \\ -0.37 & -0.37 & 0.74 \end{bmatrix} \quad (9.23)$$

构造特征值矩阵 Λ 和特征向量矩阵 $\mathbf{V}$ 如下：

$$\Lambda = \begin{bmatrix} 1.28 & 0 \\ 0 & 0.74 \end{bmatrix} \quad (9.24)$$

$$\mathbf{V} = \begin{bmatrix} -0.58 & -0.71 \\ -0.58 & 0.00 \\ 0.58 & 0.71 \end{bmatrix} \quad (9.25)$$

为得到 $\sqrt{\Lambda^{-1}}$ ，需要先求出 Λ 的逆矩阵 Λ^{-1} ，然后取其平方根。对于对角矩阵来说，这是非常简单的，只需要取每个对角元素的倒数并开方即可：

$$\sqrt{\Lambda^{-1}} = \begin{bmatrix} \frac{1}{\sqrt{1.28}} & 0 \\ 0 & \frac{1}{\sqrt{0.74}} \end{bmatrix} = \begin{bmatrix} 0.896 & 0 \\ 0 & 1.147 \end{bmatrix} \quad (9.26)$$

现在可以计算新的数据点 $\mathbf{Z}$ ：

$$\mathbf{Z} = \sqrt{\Lambda^{-1}} \mathbf{V}^T \mathbf{K} \quad (9.27)$$

执行上述矩阵运算，得到：

$$\mathbf{Z} = \begin{bmatrix} z_{11} & z_{12} \\ z_{21} & z_{22} \end{bmatrix} \quad (9.28)$$

具体地，

$$\begin{aligned} \mathbf{Z} &= \begin{bmatrix} 0.896 \cdot (-0.58 \cdot 0.49 + -0.58 \cdot -0.12 + 0.58 \cdot -0.37) & 0.896 \cdot (-0.58 \cdot -0.12 + -0.58 \cdot 0.49 + 0.58 \cdot -0.37) \\ 1.147 \cdot (-0.71 \cdot 0.49 + 0.00 \cdot -0.12 + 0.71 \cdot -0.37) & 1.147 \cdot (-0.71 \cdot -0.12 + 0.00 \cdot 0.49 + 0.71 \cdot -0.37) \end{bmatrix} \\ \mathbf{Z} &= \begin{bmatrix} 0.896 \cdot (-0.2842 + 0.0696 - 0.2146) & 0.896 \cdot (0.0696 - 0.2842 - 0.2146) \\ 1.147 \cdot (-0.3479 - 0.2627) & 1.147 \cdot (0.0852 - 0.2627) \end{bmatrix} \\ \mathbf{Z} &= \begin{bmatrix} 0.896 \cdot (-0.4292) & 0.896 \cdot (-0.4292) \\ 1.147 \cdot (-0.6106) & 1.147 \cdot (-0.1775) \end{bmatrix} \\ \mathbf{Z} &= \begin{bmatrix} -0.3845 & -0.3845 \\ -0.7009 & -0.2031 \end{bmatrix} \end{aligned} \quad (9.29)$$

因此，经过核化 PCA 变换后的新数据点 $\mathbf{Z}$ 就是上述矩阵，它表示原始数据在新特征空间中的位置。请注意，这里的数值是根据四舍五入的结果给出的近似值。在实际应用中，你可能会使用计算机软件来获得更精确的结果。

以上便是核化 PCA 的手动计算过程。通过这一系列步骤，可以将高维数据映射到低维空间，同时保持尽可能多的数据信息。这个过程有助于提高在处理复杂、非线性分布的数据时模型的表现。

9.5 线性判别分析降维

线性判别分析 (Linear Discriminant Analysis, LDA) 是一种监督学习的降维技术, 它不同于 PCA 这样的非监督方法。LDA 不仅考虑数据分布的信息, 还利用类别标签来最大化类间差异同时最小化类内差异, 因此它特别适用于分类问题中的特征提取和降维。

在数学上, LDA 试图找到一个投影矩阵 $\mathbf{W}$, 使得原始高维数据 $\mathbf{X}$ 投影到低维空间后的数据点 $\mathbf{Y} = \mathbf{X}\mathbf{W}$ 满足以下两个条件: 类间散布矩阵 $\mathbf{S}_B$ 的迹最大和类内散布矩阵 $\mathbf{S}_W$ 的迹最小。这两个散布矩阵定义如下:

类间散布矩阵 $\mathbf{S}_B$ 描述的是不同类别之间的散布情况, 其计算公式为:

$$\mathbf{S}_B = \sum_{i=1}^C N_i (\boldsymbol{\mu}_i - \boldsymbol{\mu})(\boldsymbol{\mu}_i - \boldsymbol{\mu})^T \quad (9.30)$$

其中, $\boldsymbol{\mu}_i$ 是第 i 类样本的均值向量, $\boldsymbol{\mu}$ 是所有样本的全局均值向量, N_i 是第 i 类样本的数量, C 是类别的总数。

类内散布矩阵 $\mathbf{S}_W$ 描述的是同一类别内部样本的散布情况, 其计算公式为:

$$\mathbf{S}_W = \sum_{i=1}^C \sum_{x \in X_i} (x - \boldsymbol{\mu}_i)(x - \boldsymbol{\mu}_i)^T \quad (9.31)$$

为找到最优的投影矩阵 $\mathbf{W}$, 需要解决广义特征值问题:

$$\mathbf{S}_W^{-1} \mathbf{S}_B \mathbf{w} = \lambda \mathbf{w} \quad (9.32)$$

这里 $\mathbf{w}$ 是想要找的投影方向, 而 λ 是对应的特征值。通过选择具有最大特征值的特征向量作为新的坐标轴, 可以将原始数据投影到这些新轴上, 从而实现降维。

LDA 的一个重要特性是它直接依赖于类别信息, 所以当数据集包含明确的类别标签时, LDA 可以提供比 PCA 更好的分类性能。然而, 这也意味着如果类别信息不可用或者不准确时, LDA 的效果会大打折扣。此外, LDA 假设每个类的数据服从高斯分布, 并且所有类有相同的协方差矩阵, 这在实际应用中可能是一个限制。

上述公式 (9.30)、(9.31) 和 (9.32) 分别是用于计算类间散布矩阵、类内散布矩阵以及求解最优投影方向的广义特征值问题的关键公式, 具体可以参考 4.5。

9.6 实践练习

为演示 PCA 的实际应用, 将使用 sklearn 库中的鸢尾花 (Iris) 数据集来演示如何使用各类降维方法进行降维。鸢尾花数据集是机器学习领域中一个非常受欢迎的数据集, 它包含 150 个样本, 每个样本有 4 个特征 (萼片长度、萼片宽度、花瓣长度和花瓣宽度), 以及 3 个类别 (Setosa, Versicolour 和 Virginica)。该数据集包含 150 个样本, 每个样本有 4 个特征以及对应的类别标签。

9.6.1 示例: PCA 降维

基于 Iris 数据集, 示例的任务是使用 PCA 将特征降到二维, 以便于可视化。具体的实现代码如下:

```
1 from sklearn.decomposition import PCA
2 from sklearn.datasets import load_iris
3 import matplotlib.pyplot as plt
```

```

4
5 # 加载Iris数据集
6 iris = load_iris()
7 X = iris.data
8 y = iris.target
9
10 plt.rcParams['font.sans-serif'] = ['SimHei']
11 plt.rcParams['axes.unicode_minus'] = False
12 plt.figure(figsize=(6,4))
13 # 使用PCA将数据降到2维
14 pca = PCA(n_components=2)
15 X_reduced = pca.fit_transform(X)
16
17 # 可视化降维后的数据
18
19 for i in range(len(iris.target_names)):
20     plt.scatter(X_reduced[y == i, 0], X_reduced[y == i, 1],
21               label=iris.target_names[i])
22 plt.legend()
23 plt.title('IRIS数据集主成分分析')
24 plt.xlabel('主成分1')
25 plt.ylabel('主成分2')
26 plt.grid()
27 plt.tight_layout()
28 plt.show()

```

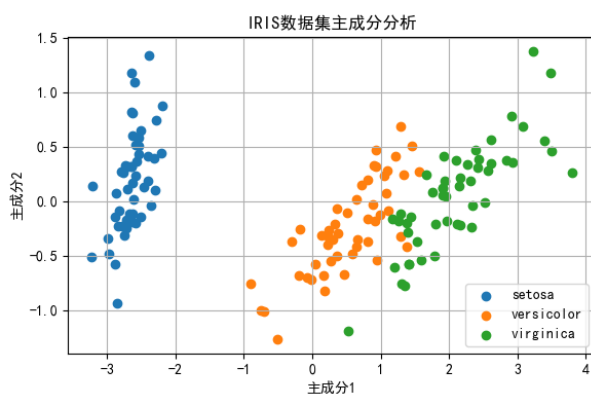


图 9.2: IRIS 数据集主成分分析

当执行给定的 Python 代码后，程序将会加载 Iris 数据集，并应用 PCA 算法将其从原始的四维空间降至二维空间。然后，它会创建一个散点图（如图9.2），其中每个点代表一朵鸢尾花，其位置由 PCA 降维后的两个主成分确定。不同的颜色和标记用于区分三种不同种类的鸢尾花：Setosa, Versicolor 和 Virginica。

从图形中可以观察到：

- Setosa 样本形成一个独立的集群，与其他两类明显分离。这表明 PCA 成功找到能够很好地区分这个类别的方向。
- Versicolor 和 Virginica 样本之间有一些重叠，但总体上还是可以区分开来。这表示即使是在降维之后，PCA 仍然保留大部分有用的信息，使得这两类之间的差异可见。

9.6.2 示例：LDA 在鸢尾花数据集上的应用

同样使用经典的鸢尾花（Iris）数据集来演示如何使用线性判别分析（LDA）进行降维。示例的目标是通过 LDA 将 4 维的特征空间降到 2 维，以便更好地可视化不同类别的分布情况。鸢尾花数

数据集可以直接从 sklearn 库中加载，该数据集已经过预处理，所有特征都具有数值型，并且已经被标准化。类别标签为整数 0, 1, 2，分别对应三个不同的鸢尾花品种。

首先，需要导入必要的 Python 库，然后加载鸢尾花数据集。接下来，执行 LDA 降维，最后对结果进行可视化。

```

1  from sklearn import datasets
2  from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
3  import matplotlib.pyplot as plt
4  import numpy as np
5
6  # 加载数据
7  iris = datasets.load_iris()
8  X = iris.data # 特征矩阵
9  y = iris.target # 类别标签
10
11 # 创建LDA模型实例并拟合数据
12 lda = LinearDiscriminantAnalysis(n_components=2)
13 X_r2 = lda.fit(X, y).transform(X)
14
15 # 可视化LDA降维后的数据
16 colors = ['navy', 'turquoise', 'darkorange']
17 target_names = iris.target_names
18 plt.rcParams['font.sans-serif'] = ['SimHei']
19 plt.rcParams['axes.unicode_minus'] = False
20 plt.figure(figsize=(6,4))
21
22 for color, i, target_name in zip(colors, [0, 1, 2], target_names):
23     plt.scatter(X_r2[y == i, 0], X_r2[y == i, 1], alpha=.8, color=color,
24               label=target_name)
25 plt.legend(loc='best', shadow=False, scatterpoints=1)
26 plt.title('IRIS数据集的LDA')
27 plt.xlabel('主成分1')
28 plt.ylabel('主成分2')
29 plt.grid()
30 plt.tight_layout()
31 plt.show()

```

上述代码段中，首先导入所需的 Python 库，包括用于加载数据集的“datasets”模块，用于执行 LDA 的“LinearDiscriminantAnalysis”类，用于绘图的“matplotlib.pyplot”，以及用于数值计算的“numpy”。

之后，加载鸢尾花数据集，并将其特征矩阵赋值给变量 X ，类别标签赋值给变量 y 。创建一个 LDA 模型实例，指定要保留的成分数目为 2（即 $n_components=2$ ），然后用训练数据拟合这个模型，并转换原始数据以获得降维后的表示 X_{r2} 。

最后，使用 matplotlib 绘制散点图，以可视化降维后的数据点分布。每个类别的点都用不同的颜色标识，并且在图上添加图例说明各个颜色对应的类别名称。通过这个图表，可以直观地看到 LDA 如何将多维数据投影到二维空间，并且保持类别之间的可分性。

上述代码段执行后，会生成一个二维散点图，该图表示通过 LDA 降维后的鸢尾花数据集，如图9.3所示。现在来详细分析一下这个结果。

- 数据分布：散点图中每个点代表一个鸢尾花样本，颜色根据其类别（Setosa、Versicolour 和 Virginica）而不同。由于使用了 LDA 将原始的 4 维特征空间降到 2 维，因此可以在二维平面上观察到所有样本点。
- 类别分离性：从图上可以明显看出，LDA 在降维的同时可以保持类别的可分性。特别是 Setosa 品种与另外两个品种之间的分离非常清晰。这表明 LDA 能够有效地找到能够最大化类别间差异的方向进行投影。

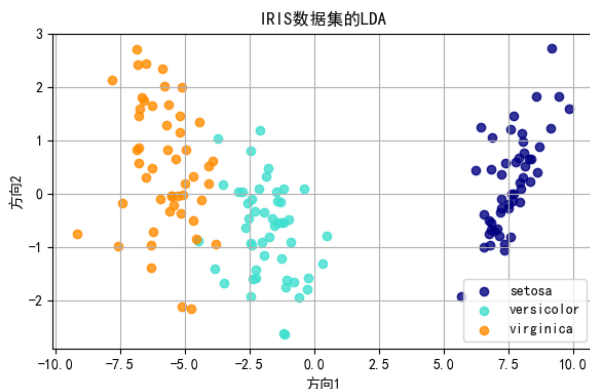


图 9.3: LDA 降维后的鸢尾花数据集可视化。

- **线性可分性：**LDA 假设类别是线性可分的，并尝试找到最佳的线性组合以区分不同的类别。对于鸢尾花数据集，可以看到，尽管 Versicolour 和 Virginica 之间存在一些重叠，但大部分情况下它们还是相对容易区分的。
- **维度解释：**图中的两个轴代表由 LDA 计算出的两个线性判别式。这些判别式是原始特征的线性组合，用于最大化类别间的方差同时最小化类别内的方差。因此，这两个轴捕捉到数据集中最重要的类别间信息。
- **可视化效果：**使用 matplotlib 创建的图表不仅可以直观地展示 LDA 的结果，而且也能够提供图例帮助理解每个颜色对应的类别名称。此外，alpha 参数控制点的透明度，使得重叠区域更加清晰可见。

需要注意的是，虽然 LDA 成功地将鸢尾花数据集降到二维，并且很好地分离了类别，但在实际应用中，LDA 可能并不总是达到如此理想的效果。例如，当类别不是线性可分时，或者当数据集包含更多的噪声或异常值时，LDA 的性能可能会受到影响。在这种情况下，可能需要考虑其他更复杂的降维方法或特征选择技术。此外，LDA 的降维能力受到类别数量的限制，即最多可以降到 $C - 1$ 个维度（其中 C 为类别数），在这个案例中就是二维，因为鸢尾花数据集有 3 个类别。

9.6.3 示例：核化 PCA 在 Iris 数据集的应用

在本节中，将使用 Python 语言和 scikit-learn 库来实现核化主成分分析（Kernel PCA），并应用于经典的 Iris 鸢尾花数据集。通过代码演示如何加载数据、应用核化 PCA 进行降维，并可视化降维后的结果。

```

1  # 导入必要的库
2  from sklearn.decomposition import KernelPCA # 核化PCA模型
3  from sklearn.datasets import load_iris     # 加载Iris数据集
4  import matplotlib.pyplot as plt           # 绘图工具
5  import numpy as np                        # 数值计算库
6
7  # 加载Iris数据集，该数据集包含150个样本，每个样本有4个特征，分为3类
8  iris = load_iris()
9  X = iris.data # 特征矩阵，每行代表一个样本，每列代表一个特征
10 y = iris.target # 类别标签向量
11
12 # 初始化KernelPCA模型，选择径向基函数(RBF)作为核函数，
13 # 并指定n_components参数为2以获取前两个主成分
14 kpca = KernelPCA(n_components=2, kernel='rbf', gamma=15)
15
16 # 拟合模型到数据，并将原始数据转换到由核化PCA定义的新坐标系中
17 X_kpca = kpca.fit_transform(X)

```

```
18
19 # 创建图表并设置尺寸
20 plt.figure(figsize=(5, 4))
21
22 # 使用for循环绘制不同类别的数据点，用不同颜色区分
23 for c, i, target_name in zip("rgb", [0, 1, 2], iris.target_names):
24     plt.scatter(X_kpca[y == i, 0], X_kpca[y == i, 1], c=c, label=target_name) # 绘制散点图
25
26 # 添加图表标题和轴标签
27 plt.title('Iris数据集上核化 PCA 应用')
28 plt.xlabel('主成分1') # 第一主成分
29 plt.ylabel('主成分2') # 第二主成分
30
31 # 显示图例，帮助识别类别
32 plt.legend()
33 plt.grid()
34 plt.tight_layout()
35 # 显示生成的图表
36 plt.show()
```

这段代码演示了如何应用核化主成分分析对鸢尾花数据集执行降维操作，并提供可视化的结果，有助于理解不同类别之间的关系。核化 PCA 是一种非线性降维技术，它允许在原始特征空间中捕捉复杂的模式，这些模式可能是线性方法所不能发现的。

当代码运行完毕后，会显示一个二维散点图，其中每个点代表一个样本实例。在这个图中，可以观察到以下几点：

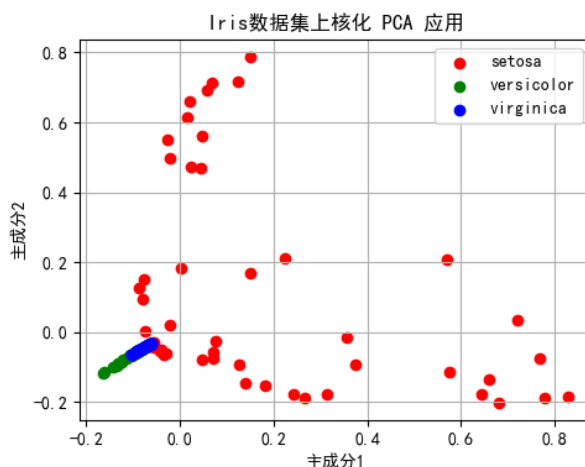


图 9.4: 核化 PCA 在 Iris 数据集上的应用。

- 类别的分离度：在降维后的二维空间中，不同类别的数据点被很好地分离开来。这表明核化 PCA 可以有效地找到能够很好地区分类别的新特征空间。
- 簇的紧凑性：同一类的数据点聚集在一起形成相对紧凑的簇，这意味着同一类内的相似性得到了保留。
- 非线性结构：如果在原始四维特征空间中，数据点之间存在非线性的关系，那么核化 PCA 通过映射到更高维度的空间再降维，可以更好地展现这种非线性结构。

从这个结果中可以得出结论，核化 PCA 对于处理具有非线性结构的数据集是一个有力的工具。它不仅可以简化数据的复杂性，还能够提高不同类别之间的可分性，这对于后续的机器学习任务，如分类或聚类，是非常有利的。此外，降维后的可视化也有助于直观地理解数据的分布特性。

9.7 补充知识

9.7.1 协方差矩阵

协方差矩阵是一种用来衡量多个随机变量之间关系的矩阵，它的每一个元素代表两个变量间的协方差。对于数据中心化的矩阵 $\mathbf{X}$ ，协方差矩阵 Σ 定义为：

$$\Sigma = \frac{1}{n-1} \mathbf{X}^T \mathbf{X} \quad (9.33)$$

这里 n 是样本数量。协方差矩阵的对角线元素表示各个特征的方差，而非对角线元素则表示特征之间的协方差。

在解释 $\mathbf{w}^T \mathbf{C} \mathbf{w}$ 为何实际上是协方差矩阵 $\mathbf{C}$ 的特征值的加权平均之前，先简单回顾一下相关的数学概念。

9.7.2 特征值和特征向量

1. 特征值与特征向量的定义

在数学中，尤其是线性代数领域，特征值 (Eigenvalue) 和特征向量 (Eigenvector) 是矩阵的重要属性。对于一个方阵 $\mathbf{A}$ ，如果存在非零向量 $\mathbf{v}$ 和标量 λ 使得下面的等式成立：

$$\mathbf{A} \mathbf{v} = \lambda \mathbf{v} \quad (9.34)$$

那么称 λ 为矩阵 $\mathbf{A}$ 的一个特征值，而 $\mathbf{v}$ 称为对应于 λ 的特征向量。

2. 求解特征值和特征向量

为求解给定矩阵 $\mathbf{A}$ 的特征值和特征向量，需要解如下的特征方程：

$$\det(\mathbf{A} - \lambda \mathbf{I}) = 0 \quad (9.35)$$

这里， $\mathbf{I}$ 是单位矩阵， $\det$ 表示行列式。这个方程通常被称为特征多项式，其根即为特征值。一旦找到所有特征值 $\lambda_1, \lambda_2, \dots, \lambda_n$ ，可以将每个特征值分别代入到原方程 $(\mathbf{A} - \lambda \mathbf{I}) \mathbf{v} = 0$ 中求解对应的特征向量 $\mathbf{v}$ 。需要注意的是，对于每一个特征值，可能存在多个线性无关的特征向量，但至少存在一个非零解。

3. 具体实例

为方便理解，这里给出一个求解二阶矩阵的特征值和特征向量的实例，考虑以下 2x2 矩阵 $\mathbf{A}$ ：

$$\mathbf{A} = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix} \quad (9.36)$$

要找到它的特征值和特征向量。首先，构建并求解特征方程：

$$\begin{aligned} \det(\mathbf{A} - \lambda \mathbf{I}) &= 0 \\ \det \begin{bmatrix} 3-\lambda & 1 \\ 1 & 3-\lambda \end{bmatrix} &= (3-\lambda)(3-\lambda) - 1 \\ &= \lambda^2 - 6\lambda + 8 \\ &= (\lambda - 4)(\lambda - 2) = 0 \end{aligned} \quad (9.37)$$

所以，特征值为 $\lambda_1 = 4$ 和 $\lambda_2 = 2$ 。

接下来，将每个特征值分别代入原方程来求解特征向量。

对于 $\lambda_1 = 4$:

$$\begin{bmatrix} 3-4 & 1 \\ 1 & 3-4 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad (9.38)$$

这给出 $x = y$ 的关系，因此，任何形如 $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$ 的向量都是对应于 $\lambda_1 = 4$ 的特征向量。

对于 $\lambda_2 = 2$:

$$\begin{bmatrix} 3-2 & 1 \\ 1 & 3-2 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad (9.39)$$

这给出了 $x = -y$ 的关系，因此，任何形如 $\mathbf{v}_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$ 的向量都是对应于 $\lambda_2 = 2$ 的特征向量。

4. 直观理解

从几何角度讲，特征值和特征向量描述了线性变换如何拉伸或压缩空间。具体来说，特征向量指示了那些在变换后仅被拉伸或压缩而不改变方向的特殊方向，而特征值则代表这些方向上拉伸或压缩的程度。例如，在上述例子中，当应用矩阵 $\mathbf{A}$ 进行变换时，特征向量 $\mathbf{v}_1$ 的方向上的点会沿着该方向被拉伸 4 倍，而 $\mathbf{v}_2$ 方向上的点会被拉伸 2 倍。如果特征值是负数，则意味着变换不仅改变长度还反转了方向。如果特征值为 0，则说明该方向上的所有点都被压缩到了原点。

9.7.3 协方差矩阵

1. 协方差矩阵

在多维数据分析中，协方差矩阵（Covariance Matrix）是一个非常重要的工具。它不仅能够描述各个维度上的数据的方差（即各维度数据的离散程度），还能够描述不同维度之间的线性关系。协方差矩阵是对称的，并且其对角线元素是每个变量的方差，而非对角线元素表示的是每两个变量之间的协方差。

对于一个包含 D 个特征的数据集 $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$ ，其中 $\mathbf{x}_i$ 是第 i 个观测值（样本），每个观测值都是一个 D 维列向量，可以计算出该数据集的协方差矩阵 $\mathbf{C}$:

$$\mathbf{C} = \frac{1}{N-1} \sum_{i=1}^N (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top \quad (9.40)$$

这里， $\bar{\mathbf{x}}$ 是所有观测值的平均值向量，也叫做样本均值。上述公式使用 $N-1$ 作为分母，这可以得到无偏估计，即期望值等于总体协方差的真实值。而使用 N 作为分母则得到有偏估计:

$$\mathbf{C}_{biased} = \frac{1}{N} \sum_{i=1}^N (\mathbf{x}_i - \bar{\mathbf{x}})(\mathbf{x}_i - \bar{\mathbf{x}})^\top \quad (9.41)$$

当样本数量 N 很大时，两种估计的结果相差不大；但在小样本的情况下，无偏估计更接近于总体的真实值。

2. 具体实例

这里，考虑一个简单的二维数据集，包括身高和体重两个特征，共有 3 个观测值:

$$\mathbf{X} = \begin{bmatrix} 170 & 65 \\ 180 & 70 \\ 160 & 58 \end{bmatrix} \quad (9.42)$$

首先计算每个特征的平均值：

$$\begin{aligned}\bar{x}_{height} &= \frac{170 + 180 + 160}{3} = 170 \\ \bar{x}_{weight} &= \frac{65 + 70 + 58}{3} = 64.33\end{aligned}\quad (9.43)$$

然后，将每个观测值减去相应的平均值以中心化数据：

$$\mathbf{X}_{centered} = \begin{bmatrix} 0 & 0.67 \\ 10 & 5.67 \\ -10 & -6.67 \end{bmatrix}\quad (9.44)$$

接下来，根据公式 (9.40) 计算无偏协方差矩阵。由于样本大小 $N = 3$ ，因此分母为 $N - 1 = 2$ 。将中心化的数据代入公式并进行计算：

$$\begin{aligned}\mathbf{C} &= \frac{1}{2} \begin{bmatrix} 0^2 + 10^2 + (-10)^2 & 0 \times 0.67 + 10 \times 5.67 + (-10) \times (-6.67) \\ 0 \times 0.67 + 10 \times 5.67 + (-10) \times (-6.67) & 0.67^2 + 5.67^2 + (-6.67)^2 \end{bmatrix} \\ &= \frac{1}{2} \begin{bmatrix} 200 & 123.4 \\ 123.4 & 76.96 \end{bmatrix} \\ &= \begin{bmatrix} 100 & 61.7 \\ 61.7 & 38.48 \end{bmatrix}\end{aligned}\quad (9.45)$$

如果使用公式 (9.41)，即有偏估计，则分母为 $N = 3$ ，计算结果会有所不同，但因为样本量较小，差异可能较为明显。在这个例子中，通常推荐使用无偏估计来获得更好的统计特性。

3. 直观理解协方差矩阵

要直观地理解协方差矩阵，不妨想象一个多维空间中的点云，每个点代表一个观测值，而每个维度对应一个特征。协方差矩阵就像是一个“指南针”，它告诉这些点是如何围绕它们的中心分布的。

- 数据云的方向：协方差矩阵的特征向量指出了数据云的最大变化方向，也就是数据点在多维空间中主要散布的方向。
- 数据云的伸展程度：特征值则对应于这些方向上的变化量或伸展程度，它们代表沿特征向量方向的数据变异大小。

例如，在二维空间中，如果有一个由身高和体重组成的数据集，协方差矩阵有助于了解这两者之间的相关性。如果协方差为正，意味着当一个变量增加时，另一个变量也倾向于增加；如果是负，则表示一个变量增加时另一个变量倾向于减少。而协方差的绝对值大小反映了这种趋势的强度。

为使用 Python 生成协方差矩阵与数据分布关系的示意图，可以使用 matplotlib 库来绘制。将创建一个简单的二维高斯分布的数据集，并计算其协方差矩阵，然后根据协方差矩阵的特征值和特征向量绘制出代表数据分布的椭圆。

以下是实现上述描述的 Python 代码示例：

```
1 # 导入必要的库
2 import numpy as np # 用于数值计算
3 import matplotlib.pyplot as plt # 用于绘制图表
4 from scipy.stats import multivariate_normal # 用于创建多元正态分布
5
6 # 定义二维高斯分布的参数
7 mean = [0, 0] # 数据的均值向量，表示数据集中心位置
8 cov_matrix = [[1, 0.8], [0.8, 3]] # 协方差矩阵，描述了数据点之间的关系和分布形状
```



```

9
10 # 使用numpy生成服从上述定义的高斯分布的数据点
11 data = np.random.multivariate_normal(mean, cov_matrix, 500) # 生成500个样本点
12
13 # 计算协方差矩阵的特征值和特征向量，它们可以用来构造代表数据分布的椭圆
14 eigenvalues, eigenvectors = np.linalg.eig(cov_matrix)
15
16 # 开始绘制图形
17 plt.figure(figsize=(5, 4)) # 设置画布大小为5x5英寸
18
19 # 绘制散点图来展示数据点
20 # 点的大小设置为5，透明度为0.5
21 plt.scatter(data[:, 0], data[:, 1], s=5, alpha=0.5, label='Data Points')
22
23 # 创建一个多元正态分布对象，用于后续绘制等高线图
24 rv = multivariate_normal(mean, cov_matrix)
25
26 # 创建网格坐标，用于绘制等高线图，范围是-4到4，间隔0.01
27 x, y = np.mgrid[-4:4:.01, -4:4:.01]
28
29 # 将网格坐标转换成适合输入给pdf函数的形式
30 pos = np.dstack((x, y))
31
32 # 绘制等高线图，它显示了不同概率密度的区域
33 plt.contour(x, y, rv.pdf(pos), levels=10, colors='k', alpha=0.5) # 黑色轮廓线，透明度为0.5
34 plt.contourf(x, y, rv.pdf(pos), levels=10, cmap='Blues', alpha=0.5) # 蓝色调填充，透明度为0.5
35
36 # 计算椭圆的角度，这是由特征向量的方向决定的
37 theta = np.degrees(np.arctan2(*eigenvectors[:, 0][::-1])) # 特征向量的方向角转换为度数
38
39 # 椭圆的宽度和高度是特征值的两倍平方根（对应于两个标准差）
40 width, height = 2 * np.sqrt(eigenvalues)
41
42 # 创建椭圆对象，并添加到当前的轴上
43 ellipse = plt.matplotlib.patches.Ellipse(xy=mean, width=width, height=height, angle=theta,
44     edgecolor='r', fc='None', lw=2)
45 plt.gca().add_patch(ellipse)
46
47 for length, vector in zip(sorted(eigenvalues, reverse=True), np.fliplr(eigenvectors)):
48     # np.sqrt(length) 代表了沿着该特征向量方向上的标准差，反映了数据沿此方向的分散程度。
49     v = vector * np.sqrt(length)
50     # 绘制主成分变量的箭头
51     plt.arrow(0, 0, v[0], v[1], head_width=0.3, head_length=0.3, fc='purple', ec='purple')
52
53 plt.title('协方差矩阵与数据分布')
54 plt.xlabel('X-轴')
55 plt.ylabel('Y-轴')
56 plt.legend()
57 plt.axis('equal')
58 plt.tight_layout()
59 plt.grid()
60 plt.show()

```

在以上代码中，首先导入所需的 Python 库。然后，定义一个具有特定均值和协方差矩阵的二维高斯分布，并从这个分布中随机抽取 500 个样本点作为数据集。接着，计算该协方差矩阵的特征值和特征向量，这些信息对于理解数据的分布特性非常重要，因为特征向量指出了数据的最大方差方向，而特征值则量化了沿这些方向的方差。

之后，使用 matplotlib 库来可视化数据。首先绘制散点图来展示原始数据点的位置。为更直观地了解数据分布，还绘制等高线图以展示不同概率密度的区域。最后，根据协方差矩阵的特征值和

特征向量，计算并绘制一个椭圆，该椭圆的中心位于数据的均值处，其长轴和短轴长度分别与特征值有关，角度则是由特征向量的方向决定的。这提供了一种可视化方式来展示数据的分布特性和相关性。

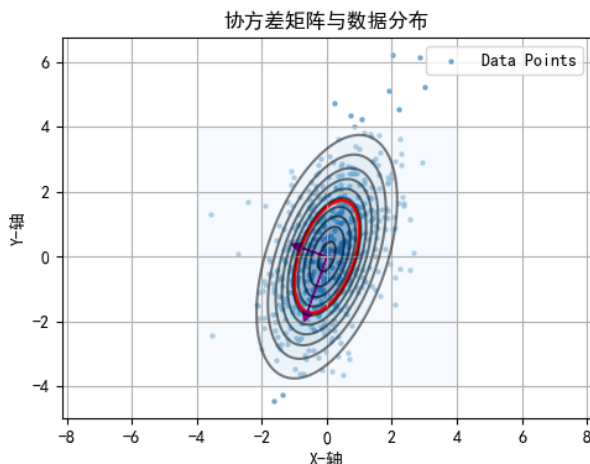


图 9.5: 协方差矩阵与数据分布的关系示意图。椭圆的轴长和方向分别对应于特征值及其对应的特征向量。

在三维或多维情况下,协方差矩阵同样可以视为定义了一个高维空间中的超椭球体(hyper-ellipsoid),该超椭球体的主轴由协方差矩阵的特征向量给出,轴的长度由相应的特征值决定。因此,协方差矩阵不仅揭示了各个维度上的方差(即每个变量自身的变异性),还包含不同维度之间相互关系的信息。

此外,对于标准化后的数据(即均值为0,标准差为1的数据),协方差矩阵实际上就是相关系数矩阵,此时对角线元素都为1,非对角线元素则直接给出变量间的皮尔逊相关系数,范围从-1到+1,进一步简化对变量间关系的理解。

综上所述,协方差矩阵提供了一种紧凑且信息丰富的形式来描述多维数据集的内部结构和各变量间的关联特性。通过分析协方差矩阵,可以获取关于数据形状、方向以及变量间依赖性的深刻见解。

9.7.4 矩阵特征分解

1. 矩阵特征分解

矩阵的特征分解(也称为谱分解)是一种将矩阵表示为其特征值和特征向量之间关系的方法。对于一个 $n \times n$ 的方阵 $\mathbf{A}$, 如果它有 n 个线性无关的特征向量, 则可以进行特征分解。

假设 $\mathbf{A}$ 是一个对称矩阵(在很多实际应用中,如协方差矩阵,处理的都是对称矩阵),那么它的特征分解可以写作:

$$\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^{-1} \quad (9.46)$$

其中 $\mathbf{V}$ 是 $n \times n$ 的矩阵,其列是 $\mathbf{A}$ 的单位正交特征向量; $\mathbf{\Lambda}$ 是对角矩阵,其对角元素是对应的特征值。因为 $\mathbf{A}$ 是对称矩阵,所以 $\mathbf{V}$ 实际上是一个正交矩阵,即 $\mathbf{V}^T = \mathbf{V}^{-1}$, 这意味着

$$\mathbf{A} = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T \quad (9.47)$$

当矩阵 $\mathbf{A}$ 不是对称的时候,仍然可能找到一组特征向量使得上述等式成立,但是这些特征向量可能不是正交的,并且可能存在复数特征值。然而,在很多情况下,特别是讨论实对称矩阵时,通常能够获得一组实数特征值和一组正交特征向量。

特征分解在数学和工程领域有着广泛的应用,包括但不限于:

- **矩阵幂次计算**：一旦拥有特征分解，计算矩阵的幂就变得简单，因为 $(\mathbf{V} \mathbf{V}^\top)^k = \mathbf{V}^k \mathbf{V}^\top$ 。
- **方程求解**：用于求解线性系统、最小二乘问题等。
- **数据分析**：例如主成分分析（PCA）依赖于协方差矩阵的特征分解来降维。
- **物理学中的对角化**：在量子力学等领域，哈密顿算符的特征分解对应于系统的能量本征态和本征值。

因此，特征分解不仅在理论上提供了关于矩阵结构的重要信息，而且在实践中为解决各种问题提供了一种有效的方法。通过将复杂的问题简化到与特征值和特征向量相关的操作上，可以更高效地处理许多涉及矩阵运算的任务。

2. 具体实例

为更好地理解这个过程，通过一个具体的例子来说明如何进行特征分解。考虑以下对称矩阵 $\mathbf{A} = \begin{bmatrix} 4 & 2 \\ 2 & 3 \end{bmatrix}$ 作为特征分解的具体示例。为找到矩阵 $\mathbf{A}$ 的特征值，需要解特征方程 $|\mathbf{A} - \lambda \mathbf{I}| = 0$ ，其中 $\mathbf{I}$ 是单位矩阵， λ 是未知的特征值。

$$|\mathbf{A} - \lambda \mathbf{I}| = \begin{vmatrix} 4 - \lambda & 2 \\ 2 & 3 - \lambda \end{vmatrix} = (4 - \lambda)(3 - \lambda) - 4 = \lambda^2 - 7\lambda + 8 \quad (9.48)$$

将上述二次多项式设置为零并求解 λ ，得到两个特征值：

$$\lambda_{1,2} = \frac{7 \pm \sqrt{(-7)^2 - 4 \cdot 1 \cdot 8}}{2 \cdot 1} = \frac{7 \pm \sqrt{49 - 32}}{2} = \frac{7 \pm \sqrt{17}}{2} \quad (9.49)$$

因此，有：

$$\begin{aligned} \lambda_1 &= \frac{7 + \sqrt{17}}{2} \approx 5.5616 \\ \lambda_2 &= \frac{7 - \sqrt{17}}{2} \approx 1.4384 \end{aligned} \quad (9.50)$$

现在，对于每个特征值，求解对应的特征向量。以 $\lambda_1 \approx 5.5616$ 为例，有：

$$(\mathbf{A} - \lambda_1 \mathbf{I}) \mathbf{v}_1 = \mathbf{0} \quad (9.51)$$

这给出一个线性方程组，可以用它来解出 $\mathbf{v}_1$ 。具体有：

$$\begin{bmatrix} 4 - \lambda_1 & 2 \\ 2 & 3 - \lambda_1 \end{bmatrix} \begin{bmatrix} v_{1,1} \\ v_{1,2} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad (9.52)$$

代入 λ_1 的具体数值，得到：

$$\begin{bmatrix} -1.5616 & 2 \\ 2 & -2.5616 \end{bmatrix} \begin{bmatrix} v_{1,1} \\ v_{1,2} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad (9.53)$$

由于这是一个齐次方程系统，它总是有一个平凡解（所有变量都等于零），但这里寻找的是非零解。通过行简化或直接观察，可以找到该方程的一个基础解系。例如，如果 $v_{1,1} = 1$ ，那么 $v_{1,2} = \frac{1.5616}{2}$ 。因此，可以取特征向量 $\mathbf{v}_1$ 为：

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0.7808 \end{bmatrix} \quad (9.54)$$

同样地，对于 $\lambda_2 \approx 1.4384$ ，可以找到另一个特征向量 $\mathbf{v}_2$ 。因为 $\mathbf{A}$ 是对称的，由于 $\mathbf{v}_1$ 和 $\mathbf{v}_2$ 正交。为方便起见，通常会将特征向量标准化，使得它们的长度为 1。

最后，把特征向量作为列向量组合成矩阵 $\mathbf{V}$ ，并且将特征值放在对角矩阵 Λ 中，这样就完成矩阵 $\mathbf{A}$ 的特征分解。

$$\mathbf{A} = \mathbf{V}\Lambda\mathbf{V}^T \quad (9.55)$$

这里， $\mathbf{V}$ 是由特征向量构成的正交矩阵，而 Λ 是对角线上包含特征值的对角矩阵。注意，由于在计算中使用了近似值，实际的数值可能会略有不同。在实践中，应该使用更高的精度来保证结果的准确性。

通过这样的分解，可以更深入地了解矩阵的性质，并且在许多应用中，如数据压缩、主成分分析（PCA）等，这种分解方法是非常有价值的。

3. 矩阵特征值分解的直观理解

矩阵的特征值分解听起来可能有点抽象，但其实它有着非常直观的几何意义，并且在许多机器学习 and 数据科学任务中扮演着重要角色。

想象一下，你有一个由点组成的云状图形，这些点表示多维空间中的数据样本。现在，假设这个点云不是随机分布的，而是倾向于沿着某些方向延伸，就像橄榄球或椭圆的形状。如果想要描述这种倾向性，或者说找出点云的主要扩展方向，那么就需要用到特征值分解。

特征值分解可以被看作是将原始数据进行旋转和平移的过程，以找到最能表达数据变异性的坐标轴。对于任意给定的方阵（比如协方差矩阵），可以找到一组特殊的向量，叫做特征向量，它们指出了数据沿哪些方向伸展得最远。与每个特征向量相关联的是一个数值，称为特征值，它表示数据沿该特征向量方向上的拉伸程度或变化幅度。

更具体地说，如果将一个矩阵应用到其特征向量上，结果会是这个特征向量被缩放到一个特定的比例，而这个比例正好就是对应的特征值。这意味着特征向量定义了新的坐标系的方向，而特征值则表示在这个新坐标系下各个维度的重要性。

例如，在二维空间中，如果有一个非对角化的协方差矩阵，通过特征值分解，可以得到两个正交的特征向量，这两个向量指向数据点云的最大和次大方差方向。同时，也得到两个特征值，分别对应于沿着这两个特征向量方向的数据变异性大小。如果将点云投影到由这两个特征向量构成的新坐标系中，会发现点云看起来像是沿着主轴被拉长或压缩过的样子，而且此时协方差矩阵变成对角形式，其中对角线元素即为特征值。

因此，特征值分解提供了一种方法来简化复杂的数据结构，揭示出数据内部的本质特性。在计算视觉领域，这一技术经常用于降维、主成分分析（PCA）、图像处理等任务中，帮助更好地理解 and 操作图像数据。

9.7.5 协方差矩阵特征值的加权平均

现在，可以来解释为什么 $\mathbf{w}^T \mathbf{C} \mathbf{w}$ 实际上是协方差矩阵 $\mathbf{C}$ 的特征值的加权平均。

假设有一个协方差矩阵 $\mathbf{C}$ ，并且知道它的特征值为 $\lambda_1, \lambda_2, \dots, \lambda_d$ ，以及对应的正交单位特征向量 $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_d$ 。因为这些特征向量是正交的，所以它们构成一个基底，任何 d 维向量 $\mathbf{w}$ 都可以表示为这个基底的线性组合：

$$\mathbf{w} = c_1 \mathbf{v}_1 + c_2 \mathbf{v}_2 + \dots + c_d \mathbf{v}_d \quad (9.56)$$

如果考虑 $\mathbf{w}^T \mathbf{C} \mathbf{w}$ ，可以将 $\mathbf{C}$ 用它的特征分解表达，即 $\mathbf{C} = \mathbf{V}\Lambda\mathbf{V}^T$ ，其中 $\mathbf{V}$ 是包含所有特征向量的矩阵， Λ 是对角矩阵，其对角元素是相应的特征值。因此，

$$\mathbf{w}^T \mathbf{C} \mathbf{w} = \mathbf{w}^T (\mathbf{V} \mathbf{V}^T) \mathbf{w} \quad (9.57)$$

通过代入 $\mathbf{w}$ 的特征向量基底表示，并利用特征向量的正交性，可以得到

$$\mathbf{w}^T \mathbf{C} \mathbf{w} = (c_1^2 \lambda_1 + c_2^2 \lambda_2 + \dots + c_d^2 \lambda_d) \quad (9.58)$$

这里 c_i^2 就是权重，它们是 $\mathbf{w}$ 在各个特征向量方向上的投影长度的平方。由于 $\mathbf{w}$ 是一个单位向量（通常情况下会规范化 $\mathbf{w}$ ），所以 $c_1^2 + c_2^2 + \dots + c_d^2 = 1$ 。这意味着 $\mathbf{w}^\top \mathbf{C} \mathbf{w}$ 确实是协方差矩阵 $\mathbf{C}$ 的特征值的加权平均，其中权重 c_i^2 对应于 $\mathbf{w}$ 在每个特征向量方向上的贡献比例。这在主成分分析 (PCA)、线性判别分析 (LDA) 等降维技术中是非常重要的一个概念。

9.8 习题

9.8.1 简答题

1. 解释核化主成分分析 (Kernel PCA) 与标准 PCA 的主要区别是什么？
2. 为什么在应用核化 PCA 时需要选择适当的核函数？

9.8.2 计算题

假设有一个二维数据集，其中每个样本点都位于单位圆周上。请说明为什么在这种情况下，标准 PCA 无法有效地进行降维，而核化 PCA 可以？

9.8.3 编程题

使用 Python 和 scikit-learn 库，编写一段代码来加载 MNIST 手写数字数据集，并应用核化 PCA 对其进行降维。绘制出前两个主成分的散点图，并解释你观察到了什么？

第十章 集成学习

在机器学习领域，单一模型（如决策树、线性回归等）虽然能够解决许多问题，但在面对复杂任务时，其性能往往受到限制。例如，单一模型可能容易过拟合或欠拟合数据，导致泛化能力不足。为克服这些局限性，研究者提出**集成学习（Ensemble Learning）**的思想。

集成学习是一种通过结合多个基学习器（Base Learners）来提升模型性能的方法。其核心理念是“集体智慧优于个体智慧”，即通过整合多个模型的预测结果，可以显著提高模型的准确性和鲁棒性。集成学习已经在许多实际应用中取得巨大成功，例如图像分类、自然语言处理、金融预测等领域。本章将深入探讨集成学习的基本原理、常用算法以及实际应用场景。通过学习本章内容，读者将掌握如何设计和实现高效的集成学习模型，并理解其在解决复杂问题中的优势。

学习目标

本章系统介绍集成学习的基本原理与主要方法。理解集成学习的核心思想，即通过结合多个基学习器提升模型整体性能，掌握其“集体智慧优于个体”的理念及其与单一模型的区别。熟悉 Bagging、Boosting 和 Stacking 三种主流框架，了解随机森林、AdaBoost 和 Gradient Boosting 等典型算法的原理与特点，并能够使用 scikit-learn 库实现相关模型。同时，认识各类集成方法的优势与局限，为构建高性能机器学习模型奠定理论与实践基础。

10.1 竞赛问题

10.1.1 竞赛策略问题

石榴同学是某中学的一名学生，他所在的班级即将参加一场知识竞赛。竞赛规则很简单：每个班级派出一名代表回答问题，答对得分，答错扣分。然而，石榴发现，单靠自己一个人的知识储备并不足以应对所有问题。于是，他决定召集班里的几位同学组成一个团队，并制定以下策略：

- **策略 1：** 让每位同学独立准备，然后在比赛中随机选择一位同学回答问题（类似于随机采样）。
- **策略 2：** 根据每位同学的强项和弱项，安排他们依次补充彼此的答案，确保最终答案更加准确（类似于迭代修正）。
- **策略 3：** 邀请一位“智囊团”成员，负责综合所有同学的答案并给出最终决策（类似于整合多个模型）。

最终，石榴的班级凭借团队合作赢得比赛。这个故事告诉我们，单一的力量可能有限，但通过合理的协作与整合，集体智慧可以超越个体表现。这种“集体智慧优于个体智慧”的理念正是集成学习（Ensemble Learning）的核心思想。

10.1.2 集成学习的核心思想

集成学习是一种通过结合多个基学习器（Base Learners）来提升模型性能的方法。其核心思想是利用多个模型的优势，弥补单一模型的不足，从而提高预测的准确性与鲁棒性。常见的集成学习方法包括：

- **Bagging**: 通过对训练集进行多次采样并训练多个基模型，然后将它们的预测结果进行平均或投票。
- **Boosting**: 通过迭代训练多个基模型，每个模型专注于修正前一个模型的错误。
- **Stacking**: 通过使用元模型（Meta-model）整合多个基模型的预测结果，进一步提升性能。

接下来，将分别详细介绍这三种方法的工作原理及其应用场景。

10.1.3 Bagging: 独立训练，集体决策

回到石榴同学的故事中，策略 1 对应的就是 Bagging 方法。在这种方法中，每位同学独立准备，互不干扰，最终通过投票或平均的方式得出答案。类似地，在 Bagging 中，通过对训练集进行多次随机采样（Bootstrap Sampling），生成多个子数据集，并在每个子数据集上训练一个基模型。最终，这些基模型的预测结果通过投票（分类任务）或平均（回归任务）的方式整合。随机森林是 Bagging 的一种经典实现。它不仅对训练集进行随机采样，还对特征进行随机选择，从而进一步降低模型之间的相关性，提升整体性能。Bagging 的优点就是能够有效降低模型的方差，适合处理高维数据和噪声数据。但是对于偏差较大的基模型（如浅层决策树），Bagging 的效果可能有限。

10.1.4 Boosting: 相互补强，构建更强预测力

策略 2 对应的是 Boosting 方法。在这种方法中，每位同学根据自己的强项和弱项，依次补充彼此的答案，确保最终答案更加准确。类似地，在 Boosting 中，通过迭代训练多个基模型，每个模型专注于修正前一个模型的错误。具体来说，Boosting 会为错误分类的样本赋予更高的权重，使得后续模型更加关注这些样本。典型算法有 AdaBoost、Gradient Boosting 和 XGBoost：

- **AdaBoost**: 通过调整样本权重，逐步修正错误。
- **Gradient Boosting**: 通过梯度下降优化损失函数，逐步改进模型。
- **XGBoost**: 一种高效的 Gradient Boosting 实现，广泛应用于工业界。

Boosting 能够有效降低模型的偏差，适合处理复杂任务。但容易过拟合，尤其是当基模型数量过多时。

10.1.5 Stacking: 多元视角，统一判断

策略 3 对应的是 Stacking 方法。在这种方法中，智囊团成员负责综合所有同学的答案，并给出最终决策。类似地，在 Stacking 中，使用一个元模型（Meta-model）来整合多个基模型的预测结果。具体来说，基模型的预测结果作为元模型的输入特征，元模型通过学习这些特征与真实标签之间的关系，生成最终预测。Stacking 常用于竞赛场景（如 Kaggle 比赛），因为它能够充分利用不同模型的优势，进一步提升性能。Stacking 能够灵活整合不同类型模型的优点，适合处理复杂的多模态数据。但计算成本较高，且模型的解释性较差。

10.2 集成学习策略及经典实现

在集成学习中，Bagging、Boosting 和 Stacking 是三种核心策略。它们通过不同的方式整合多个基学习器，以提升模型的性能。接下来将分别详细介绍这三种策略，并结合经典算法进行具体说明。

10.2.1 Bagging 和随机森林

Bagging (Bootstrap Aggregating) 是一种通过并行训练多个基学习器来降低模型方差的方法。其核心思想是通过训练集进行多次随机采样 (Bootstrap Sampling)，生成多个子数据集，并在每个子数据集上独立训练一个基模型。最终，这些基模型的预测结果通过投票 (分类任务) 或平均 (回归任务) 的方式整合。

1. 随机森林关键步骤

随机森林 (Random Forest) 是 Bagging 的一种经典实现，广泛应用于分类和回归任务。与普通的 Bagging 不同，随机森林不仅对训练集进行随机采样，还对特征进行随机选择，从而进一步降低模型之间的相关性，提升整体性能。随机森林的工作原理可以分为以下几个关键步骤：

1) **数据采样**：随机森林通过对原始数据集进行有放回的随机抽样 (即 Bootstrap Sampling)，生成多个子数据集。这些子数据集的大小通常与原始数据集相同，但由于是有放回抽样，每个子数据集中可能会包含重复样本，同时也可能遗漏部分样本。这种采样方式不仅可以增加模型的多样性，并且还能为每棵决策树提供不同的训练数据，从而弱化模型之间的相关性。

2) **特征选择**：在构建每棵决策树时，随机森林引入额外的随机性。具体来说，在每次分裂节点时，算法并不会使用所有特征，而是从所有特征中随机选择一部分特征作为候选分裂特征。这种随机特征选择的方式能够有效减少特征之间的共线性影响，进一步降低模型的相关性，增强泛化能力。通常，随机选择的特征数量是一个超参数，可以根据任务需求进行调整 (例如取特征总数的平方根或对数)。

3) **模型集成**：随机森林通过训练多棵决策树，并将它们的预测结果进行整合来生成最终输出。对于分类任务，通常采用投票法，即选择多数决策树预测的类别作为最终结果；而对于回归任务，则通常采用平均法，即将所有决策树的预测值取平均作为最终结果。这种集成方式不仅能够有效降低单一决策树可能带来的过拟合风险，还能显著提升模型的稳定性和预测性能。

2. 随机森林优缺点

随机森林的优点和缺点较为明显：

1) 优点：

- 能够有效降低模型的方差，适合处理高维数据和噪声数据。
- 对异常值和缺失值具有较强的鲁棒性。
- 可以并行化训练，计算效率较高。

2) 缺点：

- 对于偏差较大的基模型 (如浅层决策树)，Bagging 的效果可能有限。
- 模型的解释性较差，尤其是当基模型数量较多时。

3. 随机森林示例

以下是一个使用 `scikit-learn` 实现随机森林的简单示例：

```
1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.datasets import load_iris
3 from sklearn.model_selection import train_test_split
4 from sklearn.metrics import accuracy_score
```

```
5
6 # 加载数据集
7 X, y = load_iris(return_X_y=True)
8 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
9
10 # 构建随机森林模型
11 clf = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42)
12 clf.fit(X_train, y_train)
13
14 # 测试集预测
15 y_pred = clf.predict(X_test)
16 accuracy = accuracy_score(y_test, y_pred)
17 print("测试集准确率:", accuracy)
```

随机森林通过引入随机性（数据采样和特征选择），显著降低模型的方差，同时保持较高的预测性能。它是一种简单且高效的集成学习方法，适用于多种场景。

10.2.2 Boosting 和 AdaBoost 模型

Boosting 的基本思想是通过构建一系列弱学习器（即那些仅略优于随机猜测的模型），并将它们按顺序组合起来形成一个强学习器。在 Boosting 过程中，每个新加入的弱学习器都会针对前序模型的错误进行特别训练，以纠正之前的误分类情况。这一过程通常通过调整样本权重来实现：每次迭代后，被误分类的样本权重增加，使得后续模型更加关注这些难处理的实例。最终，所有弱学习器的预测结果通过加权投票或加权求和的方式结合起来，形成对新数据点的最终预测。Boosting 算法不仅能够提高模型的整体性能，还可以增强其泛化能力，有效避免过拟合问题。该方法通过逐步改进和团队协作来显著提升学习效果，能够充分体现“团结就是力量”的理念。

1. AdaBoost 计算步骤

AdaBoost (Adaptive Boosting) 是 Boosting 方法中的一种经典算法，广泛应用于分类任务。它通过迭代训练多个弱分类器，并赋予它们不同的权重，逐步修正错误，最终生成一个强分类器。以下将详细介绍 AdaBoost 的基本思想、工作原理、优缺点以及代码实现。AdaBoost 的工作过程可以用以下几个步骤来直观描述：

1) **初始化样本权重**：在开始时，所有样本的权重是相同的，表示每个样本对模型训练的重要性相同。

2) **训练第一个弱分类器**：使用当前样本权重训练一个简单的弱分类器（例如决策树桩）。这个弱分类器可能无法很好地拟合数据，但它的预测结果会被记录下来。

3) **调整样本权重**：根据弱分类器的表现，调整样本的权重。对于那些被正确分类的样本，权重会降低；而对于那些被错误分类的样本，权重会提高。这样，在下一轮训练中，新的弱分类器会更加关注之前被错误分类的样本。

4) **重复训练弱分类器**：不断重复上述过程，每一轮都会训练一个新的弱分类器，并根据其表现调整样本权重。每次训练的弱分类器都会专注于修正前一个分类器的错误。

5) **组合弱分类器**：最终，所有的弱分类器会被整合成一个强分类器。每个弱分类器的预测结果会根据其表现的好坏被赋予不同的权重，表现越好的分类器权重越高。

6) **生成最终预测**：强分类器通过综合所有弱分类器的预测结果，给出最终的分类结果。如果多数弱分类器认为某个样本属于某一类，那么强分类器也会将其归为这一类。

2. AdaBoost 优缺点

通过这种方式，AdaBoost 能够逐步改进模型的性能，最终生成一个强大的分类器。AdaBoost 的优缺点如下：

1) 优点：

- 能够有效降低模型的偏差，适合处理复杂任务。

- 在许多实际问题中表现优异，尤其是在结构化数据上。
- 对噪声较少的数据具有较强的鲁棒性。

2) 缺点:

- 容易过拟合，尤其是当弱分类器数量过多或数据中存在较多噪声时。
- 训练过程较慢，因为弱分类器需要按顺序训练。
- 对异常值较为敏感，可能导致模型性能下降。

3. AdaBoost 示例

以下是一个使用 `scikit-learn` 实现 AdaBoost 的简单示例:

```
1 from sklearn.ensemble import AdaBoostClassifier
2 from sklearn.tree import DecisionTreeClassifier
3 from sklearn.datasets import load_breast_cancer
4 from sklearn.model_selection import train_test_split
5 from sklearn.metrics import accuracy_score
6
7 # 加载数据集
8 X, y = load_breast_cancer(return_X_y=True)
9 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
10
11 # 构建 AdaBoost 模型
12 base_estimator = DecisionTreeClassifier(max_depth=1) # 弱分类器
13 ada = AdaBoostClassifier(base_estimator=base_estimator, n_estimators=50, learning_rate=1.0,
14                           random_state=42)
15 ada.fit(X_train, y_train)
16
17 # 测试集预测
18 y_pred = ada.predict(X_test)
19 accuracy = accuracy_score(y_test, y_pred)
20 print("测试集准确率:", accuracy)
```

AdaBoost 是一种经典的 Boosting 方法，通过迭代训练多个弱分类器并赋予它们不同的权重，逐步修正错误，最终生成一个强分类器。它能够有效降低模型的偏差，适合处理复杂的非线性问题。然而，AdaBoost 对噪声数据较为敏感，容易过拟合，因此在实际应用中需要注意数据预处理和超参数调优。

10.2.3 Stacking 和元模型整合

Stacking 是一种通过使用元模型 (Meta-model) 整合多个基模型的预测结果来提升性能的方法。其核心思想是将基模型的预测结果作为输入特征，训练一个元模型来生成最终预测。在 Stacking 中，基模型可以是不同类型的模型 (如决策树、支持向量机、神经网络等)。元模型的作用是对基模型的预测结果进行综合分析，从而得出更优的预测。

1. 元模型整合步骤

Stacking 是一种通过元模型整合多个基模型预测结果的集成学习方法。其核心思想是将基模型的预测结果作为新的特征 (即元特征)，训练一个元模型以生成最终预测。具体来说，Stacking 的工作原理可以分为以下几个关键步骤:

1) **训练基模型**: 在训练集上训练多个基模型，这些基模型可以是不同类型的算法 (如决策树、支持向量机、神经网络等)。每个基模型独立地从数据中学习模式，并生成各自的预测结果。为确保基模型之间的多样性，可以选择不同类型的模型或调整超参数。此外，为避免过拟合，通常会对基模型的复杂度进行限制，例如使用浅层决策树或正则化方法。

2) **生成元特征**: 使用验证集或交叉验证生成基模型的预测结果，作为元模型的输入特征。具体来说，在每一轮交叉验证中，基模型在训练集的一个子集上训练，并在验证集上生成预测结果。这

些预测结果被收集起来，形成一个新的特征矩阵（即元特征矩阵）。元特征矩阵的每一列对应一个基模型的预测结果，而每一行对应一个样本的预测值。通过这种方式，元特征能够捕捉到基模型对数据的不同视角，从而为元模型提供丰富的信息。

3) **训练元模型**：在元特征上训练一个元模型（如线性回归、逻辑回归、岭回归等），生成最终预测。元模型的作用是对基模型的预测结果进行综合分析，从而得出更优的预测。例如，在回归任务中，元模型可以通过加权平均的方式整合基模型的预测值；在分类任务中，元模型可以通过学习基模型预测的概率分布来生成最终类别标签。由于元模型直接作用于基模型的预测结果，因此它能够有效平衡基模型之间的偏差和方差，进一步提升整体性能。

2. 元模型优缺点

元模型的优点与缺点如下：

1) 优点：

- 能够灵活整合不同类型模型的优点，适合处理复杂的多模态数据。
- 在竞赛场景中表现优异，能够显著提升模型性能。

2) 缺点：

- 计算成本较高，尤其是当基模型数量较多时。
- 模型的解释性较差，难以理解元模型的决策过程。

3. 元模型示例

以下是一个使用 `scikit-learn` 实现 Stacking 的简单示例：

```
1  from sklearn.ensemble import StackingClassifier
2  from sklearn.linear_model import LogisticRegression
3  from sklearn.tree import DecisionTreeClassifier
4  from sklearn.svm import SVC
5  from sklearn.datasets import load_digits
6  from sklearn.model_selection import train_test_split
7  from sklearn.metrics import accuracy_score
8
9  # 加载数据集
10 X, y = load_digits(return_X_y=True)
11 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
12
13 # 定义基模型
14 base_models = [
15     ('dt', DecisionTreeClassifier(max_depth=5, random_state=42)),
16     ('svc', SVC(probability=True, random_state=42))
17 ]
18
19 # 定义元模型
20 meta_model = LogisticRegression()
21
22 # 构建 Stacking 模型
23 clf = StackingClassifier(estimators=base_models, final_estimator=meta_model)
24 clf.fit(X_train, y_train)
25
26 # 测试集预测
27 y_pred = clf.predict(X_test)
28 accuracy = accuracy_score(y_test, y_pred)
29 print("测试集准确率:", accuracy)
```

Stacking 通过整合多个基模型的预测结果，进一步提升模型的性能。它是一种灵活且强大的集成学习方法，尤其适合处理复杂的多模态数据。

10.3 集成学习数学描述

10.3.1 随机森林回归的数学描述

随机森林 (Random Forest) 是一种基于决策树的集成学习方法, 广泛应用于分类和回归任务。在回归问题中, 随机森林通过整合多棵决策树的预测结果来生成最终输出。以下从数学角度详细描述随机森林回归的核心思想和实现过程。

1. 基本思想

随机森林的核心思想是通过对训练数据进行随机采样 (Bootstrap Sampling) 和对特征进行随机选择, 降低模型之间的相关性, 从而提升整体性能。

- **Bootstrap Sampling:** 给定一个训练集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$, 其中 $x_i \in \mathbb{R}^d$ 是输入特征, $y_i \in \mathbb{R}$ 是目标值。随机森林从 D 中有放回地抽取 B 个子数据集 $D_1, D_2, \dots, D_B$, 每个子数据集的大小通常与原始数据集相同。
- **特征选择:** 在每棵树的分裂过程中, 随机森林从所有 d 个特征中随机选择 m 个特征 (通常 $m = \sqrt{d}$ 或 $m = \log_2(d)$), 并仅在这 m 个特征中选择最佳分裂点。

2. 基学习器: 决策树回归

随机森林中的每一棵决策树是一个回归树, 其目标是最小化预测值与真实值之间的误差。对于一棵决策树 $T_b(x)$, 其预测值可以表示为:

$$T_b(x) = \sum_{j=1}^J c_j \cdot \mathbb{I}(x \in R_j) \quad (10.1)$$

其中, J 是叶节点的数量。 R_j 是第 j 个叶节点对应的区域。 c_j 是第 j 个叶节点的预测值, 通常取该区域内所有样本目标值的均值:

$$c_j = \frac{1}{|R_j|} \sum_{i: x_i \in R_j} y_i \quad (10.2)$$

3. 模型集成: 平均预测

随机森林通过构建 B 棵决策树 $T_1(x), T_2(x), \dots, T_B(x)$, 并将它们的预测结果进行平均, 生成最终的预测值:

$$f(x) = \frac{1}{B} \sum_{b=1}^B T_b(x) \quad (10.3)$$

其中, $f(x)$ 是随机森林的最终预测值, $T_b(x)$ 是第 b 棵决策树的预测值。

4. 损失函数与优化

随机森林的目标是最小化预测值与真实值之间的均方误差 (MSE)。对于训练集 D , 损失函数定义为:

$$L(f) = \frac{1}{n} \sum_{i=1}^n (y_i - f(x_i))^2 \quad (10.4)$$

由于随机森林中的每棵树是独立训练的, 因此可以通过最小化每棵树的局部损失函数来优化模型。具体来说, 对于每棵树 $T_b(x)$, 其损失函数为:

$$L(T_b) = \frac{1}{|D_b|} \sum_{(x_i, y_i) \in D_b} (y_i - T_b(x_i))^2 \quad (10.5)$$

5. 方差与偏差分析

随机森林通过引入随机性（数据采样和特征选择）降低模型的方差，同时保持较低的偏差。从数学上看：

- **单棵树的偏差：**决策树的偏差较高，因为它容易过拟合训练数据。
- **随机森林的偏差：**随机森林的偏差接近于单棵树的偏差，因为每棵树的预测结果被平均后仍然能够捕捉到数据的整体趋势。
- **单棵树的方差：**决策树的方差较高，因为它对训练数据的变化非常敏感。
- **随机森林的方差：**随机森林通过平均多个树的预测结果显著降低方差。根据统计学中的方差分解公式，随机森林的方差可以近似表示为：

$$\text{Var}(f(x)) = \frac{1}{B} \text{Var}(T_b(x)) + \frac{B-1}{B} \text{Cov}(T_b(x), T_{b'}(x)) \quad (10.6)$$

其中， $\text{Var}(T_b(x))$ 是单棵树的方差， $\text{Cov}(T_b(x), T_{b'}(x))$ 是不同树之间的协方差。

由于随机森林中的树之间具有较低的相关性（通过数据采样和特征选择实现），因此协方差项较小，从而显著降低整体方差。

6. 特征重要性计算

随机森林的一个重要特性是能够评估特征的重要性。对于每个特征 j ，其重要性可以通过以下公式计算：

$$\text{Importance}(j) = \frac{1}{B} \sum_{b=1}^B \Delta L_b(j) \quad (10.7)$$

其中， $\Delta L_b(j)$ 表示在第 b 棵树中，使用特征 j 进行分裂时损失函数的减少量。通过这种方式，随机森林可以量化每个特征对模型预测的贡献，从而帮助用户理解数据中的关键因素。

7. 总结

从数学的角度来看，随机森林回归的核心思想是通过数据采样和特征选择引入随机性，降低模型之间的相关性，从而有效降低模型的方差。通过平均多棵决策树的预测结果，随机森林能够在保持较低偏差的同时显著提高模型的泛化能力。此外，随机森林同时提供特征重要性评估功能，进一步增强模型的解释性。

10.3.2 AdaBoost 的数学描述

AdaBoost (Adaptive Boosting) 是一种基于 Boosting 思想的集成学习方法，通过迭代训练多个弱分类器并赋予它们不同的权重，逐步修正错误，最终生成一个强分类器。以下从数学角度详细描述 AdaBoost 的核心思想和实现过程。

1. 基本思想

AdaBoost 的核心思想是通过调整样本权重和模型权重，使每个弱分类器专注于修正前一个分类器的错误。具体来说：

- 在每一轮迭代中，AdaBoost 会为错误分类的样本赋予更高的权重，使得后续分类器更加关注这些样本。
- 最终的强分类器通过对多个弱分类器的加权投票生成。

2. 算法步骤

假设有一个二分类问题，目标是通过 T 轮迭代构建一个强分类器。以下是 AdaBoost 的主要步骤：

1) **初始化样本权重**：给定训练集 $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ ，其中 $x_i \in \mathbb{R}^d$ 是输入特征， $y_i \in \{-1, +1\}$ 是目标值。初始样本权重为：

$$w_i^{(1)} = \frac{1}{n}, \quad i = 1, 2, \dots, n \quad (10.8)$$

2) **迭代训练弱分类器**：对于第 t 轮迭代 ($t = 1, 2, \dots, T$)：

- 使用当前样本权重 $w_i^{(t)}$ 训练一个弱分类器 $h_t(x)$ 。
- 计算弱分类器的错误率：

$$\epsilon_t = \sum_{i=1}^n w_i^{(t)} \cdot \mathbb{I}(h_t(x_i) \neq y_i) \quad (10.9)$$

- 计算弱分类器的权重 α_t ：

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right) \quad (10.10)$$

- 更新样本权重：

$$w_i^{(t+1)} = \frac{w_i^{(t)} \cdot \exp(-\alpha_t \cdot y_i \cdot h_t(x_i))}{Z_t} \quad (10.11)$$

其中 Z_t 是归一化因子，确保权重之和为 1：

$$Z_t = \sum_{i=1}^n w_i^{(t)} \cdot \exp(-\alpha_t \cdot y_i \cdot h_t(x_i)) \quad (10.12)$$

3) **生成强分类器**：最终的强分类器通过对所有弱分类器的加权投票生成：

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t \cdot h_t(x) \right) \quad (10.13)$$

3. 损失函数与优化

AdaBoost 的目标是最小化指数损失函数 (Exponential Loss)，其定义为：

$$L(H) = \frac{1}{n} \sum_{i=1}^n \exp(-y_i \cdot H(x_i)) \quad (10.14)$$

在每一轮迭代中，AdaBoost 通过调整样本权重和模型权重，逐步降低损失函数的值。具体来说：

- 当弱分类器 $h_t(x)$ 的预测结果接近真实值时，样本权重 $w_i^{(t)}$ 会减小。
- 当弱分类器 $h_t(x)$ 的预测结果偏离真实值时，样本权重 $w_i^{(t)}$ 会增大。

通过这种方式，AdaBoost 能够逐步修正错误，最终生成一个性能较强的分类器。

4. 方差与偏差分析

AdaBoost 的核心思想是通过迭代训练多个弱分类器来降低模型的偏差。从数学上看：

- **单个弱分类器的偏差**：弱分类器的偏差较高，因为它通常是一个简单的模型（如决策树桩）。
- **AdaBoost 的偏差**：通过迭代修正错误，AdaBoost 显著降低整体模型的偏差。
- **AdaBoost 的方差**：由于 AdaBoost 不断关注错误分类的样本，可能会导致模型对噪声数据过度拟合，从而增加方差。因此，在实际应用中需要控制迭代次数 T 或引入正则化方法。

5. 总结

从数学的角度来看, AdaBoost 的核心思想是通过调整样本权重和模型权重, 使每个弱分类器专注于修正前一个分类器的错误, 从而逐步降低模型的偏差。通过加权投票的方式, AdaBoost 能够将多个弱分类器整合为一个性能较强的分类器。然而, AdaBoost 对噪声数据较为敏感, 容易过拟合, 因此在实际应用中需要注意参数调优和正则化。

10.3.3 Stacking 模型的数学描述

Stacking 是一种通过元模型 (Meta-model) 整合多个基模型预测结果的集成学习方法。其核心思想是将基模型的预测结果作为输入特征, 训练一个元模型以生成最终预测。以下从数学角度详细描述 Stacking 的核心思想和实现过程。

1. 基本思想

Stacking 的核心思想是利用多个基模型的预测结果作为新的特征, 训练一个元模型来生成更优的预测。具体来说:

- 基模型可以是不同类型的模型 (如决策树、支持向量机、神经网络等)。
- 元模型的作用是对基模型的预测结果进行综合分析, 从而得出更优的预测。

2. 算法步骤

假设有 M 个基模型 $f_1(x), f_2(x), \dots, f_M(x)$, 以及一个元模型 $g(x)$ 。以下是 Stacking 的主要步骤:

1) **训练基模型:** 对于每个基模型 $f_m(x)$, 在训练集上进行训练:

$$f_m(x) = \arg \min_f \sum_{i=1}^n L(y_i, f(x_i)) \quad (10.15)$$

其中 $L(y_i, f(x_i))$ 是损失函数 (如均方误差或交叉熵损失)。

2) **生成元特征:** 使用验证集或交叉验证生成基模型的预测结果, 作为元模型的输入特征。对于第 m 个基模型, 其预测结果为:

$$z_{m,i} = f_m(x_i), \quad i = 1, 2, \dots, n \quad (10.16)$$

将所有基模型的预测结果组合成一个新的特征矩阵:

$$Z = [z_1, z_2, \dots, z_M] \quad (10.17)$$

其中 $Z \in \mathbb{R}^{n \times M}$ 表示元特征矩阵。

3) **训练元模型:** 在元特征矩阵 Z 上训练元模型 $g(x)$:

$$g(Z) = \arg \min_g \sum_{i=1}^n L(y_i, g(Z_i)) \quad (10.18)$$

其中 Z_i 是第 i 个样本的元特征向量。

4) **生成最终预测:** 最终的预测值通过对基模型的预测结果和元模型的输出进行整合生成:

$$\hat{y} = g(Z) \quad (10.19)$$

3. 损失函数与优化

Stacking 的目标是最小化最终预测值与真实值之间的误差。对于回归问题，通常使用均方误差（MSE）作为损失函数：

$$L(y, \hat{y}) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (10.20)$$

对于分类问题，通常使用交叉熵损失函数：

$$L(y, \hat{y}) = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{i,k} \cdot \log(\hat{y}_{i,k}) \quad (10.21)$$

其中 K 是类别数量， $y_{i,k}$ 是第 i 个样本的真实标签， $\hat{y}_{i,k}$ 是预测概率。

4. 方差与偏差分析

Stacking 的核心思想是通过整合多个基模型的预测结果，降低模型的整体偏差和方差。从数学上看：

- **基模型的偏差与方差：**不同基模型可能具有不同的偏差和方差特性。例如，复杂模型（如深度神经网络）通常具有低偏差和高方差，而简单模型（如线性回归）通常具有高偏差和低方差。
- **Stacking 的偏差与方差：**通过元模型对基模型的预测结果进行整合，Stacking 能够有效平衡偏差和方差，从而提高模型的泛化能力。

5. 总结

从数学的角度来看，Stacking 的核心思想是通过元模型整合多个基模型的预测结果，从而生成更优的预测。通过引入元特征和元模型，Stacking 能够灵活整合不同类型模型的优点，适合处理复杂的多模态数据。然而，Stacking 的计算成本较高，且模型的解释性较差，因此在实际应用中需要权衡性能与效率。

10.4 实践练习

10.4.1 示例：鸢尾花二分类

本节将使用经典的鸢尾花（Iris）数据集进行二分类任务（将类别“Setosa”与“Versicolor”区分开）。目标是通过集成学习模型提高分类性能，并比较不同模型的表现。

以下是完整的代码实现，其中每一步都包含详细的注释以解释其功能：

```

1  # 导入必要的库
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from sklearn.datasets import load_iris
5  from sklearn.model_selection import train_test_split
6  from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
7  from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay
8
9  # 1. 加载鸢尾花数据集，并将其简化为一个二分类问题（只保留前两类“Setosa”和“Versicolor”）
10 iris = load_iris()
11 X = iris.data[iris.target != 2] # 只保留前两类
12 y = iris.target[iris.target != 2]
13
14 # 2. 数据集被划分为训练集和测试集，比例为7:3
15 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
16
17 # 3. 使用随机森林（Random Forest）模型进行训练
18 rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
19 rf_model.fit(X_train, y_train)

```

```

20
21 # 使用梯度提升树 (Gradient Boosting) 模型进行训练
22 gb_model = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, random_state=42)
23 gb_model.fit(X_train, y_train)
24
25 # 4. 训练完成后, 对测试集进行预测, 并计算每个模型的准确率
26 rf_pred = rf_model.predict(X_test)
27 gb_pred = gb_model.predict(X_test)
28
29 print("随机森林准确率:", accuracy_score(y_test, rf_pred))
30 print("梯度提升树准确率:", accuracy_score(y_test, gb_pred))
31
32 # 5. 最后, 绘制两个模型的混淆矩阵图, 用于直观比较它们的分类性能
33 fig, axes = plt.subplots(1, 2, figsize=(12, 5))
34
35 # 随机森林混淆矩阵
36 ConfusionMatrixDisplay(confusion_matrix(y_test, rf_pred), display_labels=["Setosa", "Versicolor
37     "]).plot(ax=axes[0], cmap="Blues")
38 axes[0].set_title("Random Forest Confusion Matrix")
39
40 # 梯度提升树混淆矩阵
41 ConfusionMatrixDisplay(confusion_matrix(y_test, gb_pred), display_labels=["Setosa", "Versicolor
42     "]).plot(ax=axes[1], cmap="Greens")
43 axes[1].set_title("Gradient Boosting Confusion Matrix")
44
45 plt.tight_layout()
46 plt.show()

```

运行上述代码后, 您将看到以下输出:

- 随机森林和梯度提升树模型的准确率:

随机森林准确率: 1.0

梯度提升树准确率: 1.0

- 两个模型的混淆矩阵图, 分别展示它们在测试集上的分类性能, 如下图所示。

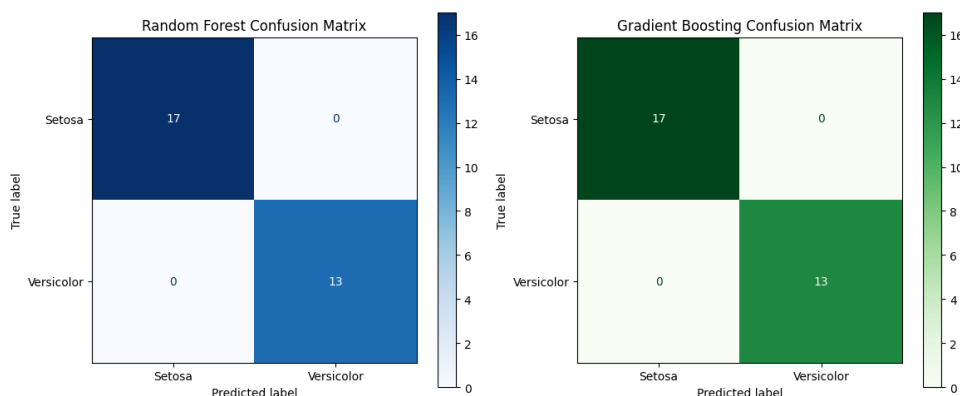


图 10.1: 随机森林和梯度提升树的混淆矩阵对比

本次实践通过深入探讨随机森林与梯度提升树两种集成学习模型在分类问题中的应用, 详细展示其解决过程与效果。实验结果表明, 这两种模型在处理鸢尾花数据集时均表现出较高的分类性能。此外, 通过可视化混淆矩阵, 可以更直观地分析模型的分类效果, 为进一步优化提供参考。

10.4.2 示例：房价预测

房价预测是回归任务中的经典问题，广泛应用于房地产评估和市场分析。本实践使用加州房价数据集（California Housing Dataset），通过集成学习模型（如随机森林和梯度提升树）进行房价预测，并比较不同模型的性能。以下是完整的代码实现，其中每一步都包含详细的注释以解释其功能：

```
1  # 导入必要的库
2  import numpy as np
3  import pandas as pd
4  from sklearn.datasets import fetch_california_housing
5  from sklearn.model_selection import train_test_split
6  from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
7  from sklearn.metrics import mean_squared_error, r2_score
8  import matplotlib.pyplot as plt
9
10 # 1. 加载加州房价数据集
11 california = fetch_california_housing()
12 X = pd.DataFrame(california.data, columns=california.feature_names) # 特征
13 y = california.target # 目标变量（房价）
14
15 # 2. 数据集被划分为训练集和测试集，比例为8:2
16 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
17
18 # 3. 使用随机森林（Random Forest）模型进行训练
19 rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
20 rf_model.fit(X_train, y_train)
21
22 # 使用梯度提升树（Gradient Boosting）模型进行训练
23 gb_model = GradientBoostingRegressor(n_estimators=100, learning_rate=0.1, random_state=42)
24 gb_model.fit(X_train, y_train)
25
26 # 4. 训练完成后，对测试集进行预测，并计算每个模型的性能指标
27 rf_pred = rf_model.predict(X_test)
28 gb_pred = gb_model.predict(X_test)
29
30 # 计算均方误差（MSE）和决定系数（R2）
31 rf_mse = mean_squared_error(y_test, rf_pred)
32 rf_r2 = r2_score(y_test, rf_pred)
33
34 gb_mse = mean_squared_error(y_test, gb_pred)
35 gb_r2 = r2_score(y_test, gb_pred)
36
37 print("随机森林 - 均方误差 (MSE):", rf_mse)
38 print("随机森林 - 决定系数 (R2):", rf_r2)
39
40 print("梯度提升树 - 均方误差 (MSE):", gb_mse)
41 print("梯度提升树 - 决定系数 (R2):", gb_r2)
42
43 # 5. 可视化真实值与预测值的对比
44 plt.figure(figsize=(12, 6))
45
46 # 随机森林预测结果
47 plt.subplot(1, 2, 1)
48 plt.scatter(y_test, rf_pred, alpha=0.7, color='blue')
49 plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red', linestyle='--')
50 plt.title("Random Forest: True vs Predicted")
51 plt.xlabel("True Values")
52 plt.ylabel("Predicted Values")
53
54 # 梯度提升树预测结果
```

```

55 plt.subplot(1, 2, 2)
56 plt.scatter(y_test, gb_pred, alpha=0.7, color='green')
57 plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color='red', linestyle='--')
58 plt.title("Gradient Boosting: True vs Predicted")
59 plt.xlabel("True Values")
60 plt.ylabel("Predicted Values")
61
62 plt.tight_layout()
63 plt.show()

```

运行上述代码后，您将看到以下输出：

- 随机森林和梯度提升树模型的均方误差（MSE）和决定系数（ R^2 ），如下所示。

随机森林—均方误差（MSE）：0.2553684927247781

随机森林—决定系数（ R^2 ）：0.8051230593157366

梯度提升树—均方误差（MSE）：0.2939973248643864

梯度提升树—决定系数（ R^2 ）：0.7756446042829697

- 两个模型的真实值与预测值对比图，分别展示不同算法在测试集上的回归性能，如下图所示。

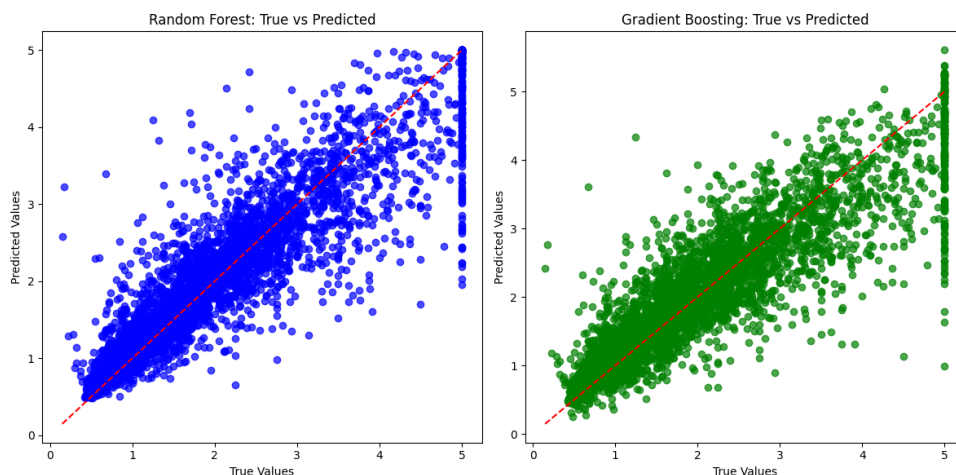


图 10.2: 随机森林和梯度提升树的真实值与预测值对比

本次实践深入探讨如何使用随机森林和梯度提升树两种集成学习模型在回归问题的应用，展示对应算法的强大建模能力。实验结果表明，这两种模型在处理加州房价数据集时均表现出较高的预测性能。此外，通过可视化真实值与预测值的对比，可以更直观地分析模型的回归效果，为进一步优化提供参考。这种集成学习方法可以广泛应用于其他回归任务，如股票价格预测、销售量预测等。

10.4.3 示例：集成学习方法对比

集成学习是一种通过组合多个基学习器来提高模型性能的方法，广泛应用于分类和回归任务。本实践将对三种常见的集成学习方法：随机森林（Random Forest）、梯度提升树（Gradient Boosting）和 AdaBoost。本小节使用糖尿病数据集（Pima Indians Diabetes Dataset）进行二分类任务（预测患者是否患有糖尿病），并通过准确率、混淆矩阵等指标评估每种方法的性能。以下是完整的代码实现，其中每一步都包含详细的注释以解释其功能：

```

1  # 导入必要的库
2  import numpy as np
3  import pandas as pd
4  import matplotlib.pyplot as plt
5  from sklearn.model_selection import train_test_split
6  from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier,
   AdaBoostClassifier
7  from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay
8
9  # 1. 加载糖尿病数据集并准备数据
10 # 数据集可以从以下链接下载: https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-
   database, 假设数据存放data文件夹
11 data = pd.read_csv("../data/diabetes.csv") # 假设数据文件名为 diabetes.csv
12 X = data.drop(columns=["Outcome"]) # 特征矩阵
13 y = data["Outcome"] # 目标变量 (0: 未患病, 1: 患病)
14
15 # 数据集被划分为训练集和测试集, 比例为7:3
16 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
17
18 # 2. 使用随机森林 (Random Forest) 进行训练和预测
19 rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
20 rf_model.fit(X_train, y_train)
21 rf_pred = rf_model.predict(X_test)
22 print("随机森林准确率:", accuracy_score(y_test, rf_pred))
23
24 # 3. 使用梯度提升树 (Gradient Boosting) 进行训练和预测
25 gb_model = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, random_state=42)
26 gb_model.fit(X_train, y_train)
27 gb_pred = gb_model.predict(X_test)
28 print("梯度提升树准确率:", accuracy_score(y_test, gb_pred))
29
30 # 4. 使用AdaBoost进行训练和预测
31 ada_model = AdaBoostClassifier(n_estimators=100, random_state=42)
32 ada_model.fit(X_train, y_train)
33 ada_pred = ada_model.predict(X_test)
34 print("AdaBoost准确率:", accuracy_score(y_test, ada_pred))
35
36 # 设置支持中文
37 plt.rcParams['font.sans-serif'] = ['SimHei']
38 # 设置支持负号
39 plt.rcParams['axes.unicode_minus'] = False
40 # 5. 绘制混淆矩阵以直观比较三种方法的分类效果
41 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
42
43 # 随机森林混淆矩阵
44 ConfusionMatrixDisplay(confusion_matrix(y_test, rf_pred), display_labels=["未患病", "患病"]).
   plot(ax=axes[0], cmap="Blues")
45 axes[0].set_title("随机森林混淆矩阵")
46
47 # 梯度提升树混淆矩阵
48 ConfusionMatrixDisplay(confusion_matrix(y_test, gb_pred), display_labels=["未患病", "患病"]).
   plot(ax=axes[1], cmap="Greens")
49 axes[1].set_title("梯度提升树混淆矩阵")
50
51 # AdaBoost混淆矩阵
52 ConfusionMatrixDisplay(confusion_matrix(y_test, ada_pred), display_labels=["未患病", "患病"]).
   plot(ax=axes[2], cmap="Reds")
53 axes[2].set_title("AdaBoost混淆矩阵")
54
55 plt.tight_layout()
56 plt.show()
57

```

```

58 # 如果需要保存上述图，可以使用以下代码：
59 # plt.savefig('confusion_matrices.png', dpi=300, bbox_inches='tight')

```

运行上述代码后，您将看到以下输出：

- 随机森林、梯度提升树和 AdaBoost 三种方法的准确率，如下所示。

随机森林准确率：0.7532467532467533

梯度提升树准确率：0.7489177489177489

AdaBoost准确率：0.7142857142857143

- 三种方法的混淆矩阵图，分别展示它们在测试集上的分类性能，如下图10.3。

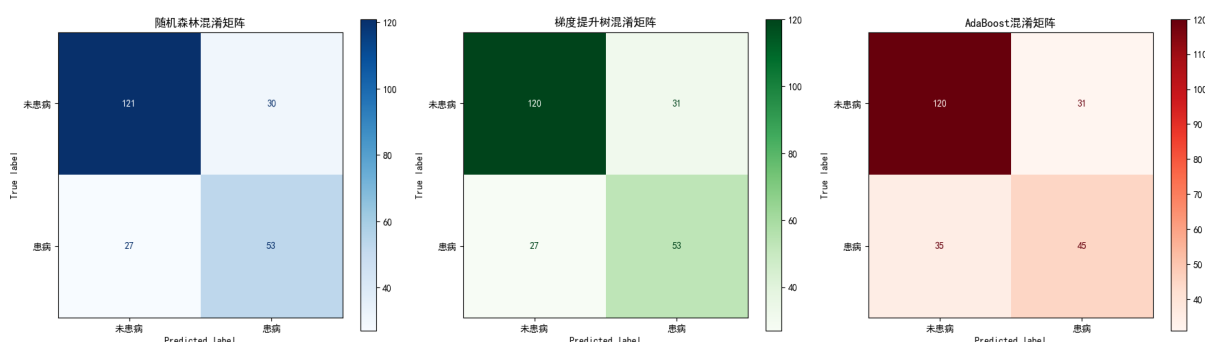


图 10.3: 随机森林、梯度提升树和 AdaBoost 的混淆矩阵对比

本次实践对比三种常见的集成学习方法在糖尿病数据集上的分类效果：

- **随机森林**：基于 Bagging 思想，通过构建多棵决策树并投票决策。优点是鲁棒性强，适合处理高维数据；缺点是对噪声数据可能过拟合。
- **梯度提升树**：基于 Boosting 思想，通过逐步优化损失函数构建强学习器。优点是精度高，但对超参数敏感，训练时间较长。
- **AdaBoost**：通过加权调整样本权重，逐步提升弱学习器的性能。优点是简单高效，但对噪声数据较为敏感。

实验结果表明，不同集成学习方法在性能上各有优劣，实际应用中应根据任务需求和数据特点选择合适的方法。此外，通过可视化混淆矩阵，可以更直观地分析每种方法的分类效果。

10.5 补充知识

假设你正在练习射击，目标是击中靶心。每次射击的结果可以看作模型对数据的预测，而靶心则是真实的目标值（即希望模型预测得越准越好）。根据射击结果的不同情况，可以用**偏差（Bias）**和**方差（Variance）**来描述模型的表现。

1. 偏差（Bias）：瞄准不准

- **定义**：偏差衡量的是模型预测值与真实值之间的差距。换句话说，偏差反映模型是否“瞄准靶心”。
- **直观解释**：如果你的子弹总是偏离靶心（比如集中在靶子的左下角），说明你的“瞄准”有问题，模型存在高偏差。在机器学习中，高偏差通常意味着模型过于简单（如线性回归模型处理非线性数据），无法捕捉数据的真实规律。这种情况称为**欠拟合**。

- **生活中的类比：**假设你在玩飞镖游戏，但你的眼睛被蒙住，只能凭感觉投掷。结果发现，所有的飞镖都偏离靶心，而且偏离的方向一致。这说明你的“瞄准系统”有问题——这就是高偏差的表现。

2. 方差 (Variance)：射击不稳定

- **定义：**方差衡量的是模型对训练数据变化的敏感程度。换句话说，方差可以反映模型是否“每次射击都稳定地击中同一个地方”。
- **直观解释：**如果你的子弹散布得很开（比如有的打在靶心附近，有的打到靶子外面甚至墙上），说明你的射击非常不稳定，模型存在高方差。在机器学习中，高方差通常意味着模型过于复杂（如深度决策树或高阶多项式回归），过度对训练数据中的噪声进行拟合。这种情况称为**过拟合**。
- **生活中的类比：**假设你是一个新手射手，每次射击时手都在抖，导致飞镖落点非常分散。虽然偶尔可能有一两支飞镖击中靶心，但整体表现很不稳定——这就是高方差的表现。

3. 理想状态：低偏差 + 低方差

- **定义：**一个好的模型应该既能够准确瞄准靶心（低偏差），又能够在多次射击中保持稳定（低方差）。
- **直观解释：**回到射击的例子，如果你每次射击都能稳稳地命中靶心附近的小范围区域，那么你的射击技术就是优秀的。在机器学习中，这意味着模型既能很好地拟合训练数据，又能很好地泛化到新数据。
- **生活中的类比：**假设你是一位经验丰富的射手，每次射击都精准地击中靶心附近，且落点非常集中。这说明你的“瞄准系统”既准确又稳定——这就是低偏差和低方差的理想状态。

4. 偏差和方差的权衡 (Bias-Variance Tradeoff)

在实际建模过程中，偏差和方差往往是相互制约的：

- 如果模型太简单（如线性模型），它可能会有**高偏差**和**低方差**，因为模型无法捕捉复杂的模式。
- 如果模型太复杂（如深度决策树），它可能会有**低偏差**和**高方差**，因为模型过度拟合了训练数据中的噪声。

因此，目标是找到一个平衡点，使得模型在偏差和方差之间达到最佳权衡，从而在训练集和测试集上都表现良好。

5. 总结

- **偏差：**模型是否“瞄准靶心”。高偏差意味着模型太简单，无法捕捉数据的真实规律（欠拟合）。
- **方差：**模型是否“射击稳定”。高方差意味着模型太复杂，过度对训练数据中的噪声进行拟合（过拟合）。
- **理想模型：**低偏差 + 低方差，既准确又稳定。

通过射击靶子的比喻，可以更直观地理解偏差和方差的概念，并在实际建模中更好地调整模型参数，找到最佳的平衡点。

10.6 习题

10.6.1 简答题

1. 什么是集成学习？它与单一模型的主要区别是什么？
2. Bagging 和 Boosting 的主要区别是什么？
3. Stacking 的元模型有什么作用？它的输入特征是如何生成的？

10.6.2 计算题

1. 假设使用 Bagging 方法构建一个随机森林模型，其中有 100 棵决策树。每棵树的分类准确率为 70%。如果采用多数投票的方式进行集成，求最终模型的分类准确率。（假设每棵树的预测相互独立）
2. 假设使用 AdaBoost 方法训练一个二分类模型，初始样本权重为 $w_1 = w_2 = \cdots = w_n = \frac{1}{n}$ 。第一轮训练后，分类器的错误率为 $\epsilon = 0.3$ 。求该分类器的权重 α 。

10.6.3 编程题

1. 使用“scikit-learn”实现一个基于随机森林的分类模型，并完成以下任务：
 - 加载鸢尾花数据集并划分为训练集和测试集。
 - 构建随机森林模型并训练。
 - 计算测试集的分类准确率，并绘制特征重要性图。
2. 使用“scikit-learn”实现一个基于梯度提升树的回归模型，并完成以下任务：
 - 加载加利福尼亚房价数据集并划分为训练集和测试集。
 - 构建梯度提升树模型并训练。
 - 计算测试集的均方误差（MSE）和决定系数（ R^2 ）。

第十一章 特征选择

特征选择 (Feature Selection) 是机器学习和计算视觉中一个至关重要的步骤。在实际问题中, 数据通常包含大量冗余或无关的特征, 这些特征不仅会增加模型的复杂性, 还可能导致过拟合, 降低模型的泛化能力。通过特征选择, 可以从原始特征集中挑选出最具代表性和信息量的特征, 从而提高模型的性能和效率。

在计算视觉领域, 特征选择的应用尤为广泛。例如, 在图像分类任务中, 可能需从数千个像素值中提取出能够有效区分不同类别的关键特征; 在目标检测任务中, 特征选择有助聚焦于与目标对象最相关的区域。本章将通过石榴同学的学习历程, 带领读者了解特征选择的基本原理、常用方法以及如何在实际任务中应用这些技术。

学习目标

本章系统介绍特征选择的基本概念与实践方法。理解特征选择的定义及其在提升模型性能、降低计算复杂度和缓解维度灾难中的作用, 掌握其与特征提取的区别与联系。在实践方面, 学习过滤法、包裹法和嵌入法等常用特征选择策略, 能够使用 Python 实现相关算法, 并将其应用于计算机视觉任务中, 有效提升模型泛化能力并减少过拟合风险。

11.1 图像分类问题

石榴同学最近在研究一个图像分类任务, 目标是区分猫和狗的照片。她一开始非常兴奋, 直接将原始像素值输入到一个简单的机器学习模型中, 但结果却让她大失所望——模型的准确率只有 50

“问题出在哪里呢?” 石榴陷入沉思。经过一番查阅资料, 她意识到, 原始像素值包含大量的冗余信息, 例如背景、光照条件和噪声, 而这些信息对于区分猫和狗并没有太大帮助。于是, 她决定尝试一种新的方法: 特征选择。

石榴首先提取一些手工设计的特征, 例如边缘检测 (Edge Detection)、颜色直方图 (Color Histogram) 和纹理特征 (Texture Features)。然后, 她使用一种叫做“过滤法”的特征选择技术, 计算每个特征与目标变量之间的相关性, 并只保留那些相关性较高的特征。通过这种方法, 模型的准确率提升到 75%!

然而, 石榴并不满足于此。她又尝试包裹法 (Wrapper Method), 利用递归特征消除 (Recursive Feature Elimination, RFE) 来动态地选择最优特征子集。最终, 她的模型准确率达到 85%! 石榴感叹道: “原来特征选择这么重要! 它不仅能提高模型性能, 还能让模型更加高效。”

通过石榴的故事, 可以总结出特征选择的几个关键点:

- **去除冗余特征:** 数据中的许多特征可能是无关或冗余的, 这些特征会增加模型的复杂性并降低性能。
- **提升模型效率:** 通过减少特征数量, 可以加快模型训练速度并降低存储需求。
- **增强模型泛化能力:** 去除噪声和无关特征有助于避免过拟合, 使模型在测试数据上表现更好。

根据石榴的学习经历，特征选择方法可以分为以下三类：过滤法、包裹法和嵌入法。本节将重点介绍过滤法，并通过平顺的叙述方式，结合实际应用场景，帮助读者理解如何灵活运用这些方法。

11.2 过滤法

过滤法（Filter Methods）是一种基于统计学的方法，它独立于机器学习模型，通过评估特征与目标变量之间的相关性来筛选特征。这种方法的优点是计算效率高，适合处理大规模数据集；缺点是忽略了特征之间的交互作用。接下来，同样以石榴同学的实际操作为例，逐步讲解三种常用的过滤法及其应用。

11.2.1 皮尔逊相关系数

皮尔逊相关系数（Pearson Correlation Coefficient）用于衡量两个连续变量之间的线性相关性。简单来说，它有助于判断某个特征是否对目标变量的变化有显著影响。其公式如下：

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}} \quad (11.1)$$

其中， $\mathbf{x}$ 和 $\mathbf{y}$ 分别表示特征向量和目标变量， $\bar{x}$ 和 $\bar{y}$ 是它们的均值。当 r 接近 1 或 -1 时，说明特征与目标变量高度相关；当 r 接近 0 时，说明两者之间几乎没有线性关系。

在研究图像分类任务时，石榴发现原始数据中包含许多冗余特征，例如背景像素值和噪声。为简化问题，她决定使用皮尔逊相关系数来筛选特征。她首先计算每个特征与目标变量之间的相关性，然后保留相关性较高的特征。具体步骤如下：

```
1 import pandas as pd
2 # 计算特征与目标变量的相关系数
3 correlation_matrix = data.corr()
4 target_correlation = correlation_matrix['target']
5
6 # 筛选相关性大于阈值的特征
7 threshold = 0.5
8 selected_features = target_correlation[abs(target_correlation) > threshold].index
9 print("Selected Features:", selected_features)
```

通过这种方法，石榴成功地将特征数量从几百个减少到几十个，大大提高了模型的训练效率。

11.2.2 互信息

互信息（Mutual Information）是一种非参数化的度量方法，能够捕捉特征与目标变量之间的非线性关系。与皮尔逊相关系数不同，互信息不仅适用于连续变量，还可以处理离散变量。其定义为：

$$I(X; Y) = \sum_{y \in Y} \sum_{x \in X} p(x, y) \log \frac{p(x, y)}{p(x)p(y)} \quad (11.2)$$

其中， $p(x, y)$ 是联合概率分布， $p(x)$ 和 $p(y)$ 是边缘概率分布。互信息越大，说明特征与目标变量的相关性越强。

在后续的实验，石榴意识到某些特征与目标变量之间的关系可能是非线性的。于是，她尝试使用互信息来评估特征的重要性。她利用 Scikit-learn 中的 `mutual_info_classif` 函数计算每个特征的互信息分数，并根据分数高低筛选出最有价值的特征：

```
1 from sklearn.feature_selection import mutual_info_classif
2 import pandas as pd
```

```

3 # 计算每个特征与目标变量的互信息
4 mi_scores = mutual_info_classif(X, y)
5 # 将互信息分数转换为DataFrame以便排序
6 mi_df = pd.DataFrame({'Feature': X.columns, 'MI Score': mi_scores})
7 mi_df = mi_df.sort_values(by='MI Score', ascending=False)
8
9 # 筛选互信息分数较高的特征
10 threshold = 0.2
11 selected_features = mi_df[mi_df['MI Score'] > threshold]['Feature']
12 print("Selected Features:", selected_features)

```

通过这种方法，石榴能够发现一些之前被忽略的重要特征，进一步提升了模型的性能。

11.2.3 卡方检验

卡方检验（Chi-Square Test）适用于分类任务，用于评估特征与类别之间的独立性。简单来说，它可以帮助判断某个特征是否对分类结果有显著影响。其公式为：

$$\chi^2 = \sum_{i=1}^n \frac{(O_i - E_i)^2}{E_i} \quad (11.3)$$

其中， O_i 是观测频数， E_i 是期望频数。如果 χ^2 值较大，则说明特征与目标变量之间存在显著关联。

在处理一个文本分类任务时，石榴遇到一些离散型特征，例如词频统计。为评估这些特征的重要性，她选择卡方检验。她利用 Scikit-learn 中的 `chi2` 函数计算每个特征的卡方统计量和 p 值，并根据结果筛选出最有价值的特征：

```

1 from sklearn.feature_selection import chi2
2 import pandas as pd
3 # 计算每个特征的卡方统计量和p值
4 chi_scores, p_values = chi2(X, y)
5
6 # 将卡方统计量和p值转换为DataFrame以便排序
7 chi_df = pd.DataFrame({'Feature': X.columns, 'Chi2 Score': chi_scores, 'P-Value': p_values})
8 chi_df = chi_df.sort_values(by='Chi2 Score', ascending=False)
9
10 # 筛选卡方统计量较高且p值较小的特征
11 threshold_chi2 = 10
12 threshold_p = 0.05
13 selected_features = chi_df[(chi_df['Chi2 Score'] > threshold_chi2) & (chi_df['P-Value'] <
14                               threshold_p)]['Feature']
15 print("Selected Features:", selected_features)

```

通过这种方法，石榴成功消除大量无关特征，同时保留对分类任务最有贡献的特征。

通过上述三种方法，可以从不同角度评估特征的重要性，并根据实际需求灵活选择合适的工具。过滤法的核心思想是利用统计指标量化特征与目标变量之间的关系，从而实现高效的特征选择。虽然这种方法计算效率高，但它未考虑特征之间的交互作用，因此在复杂任务中可能需要结合其他方法使用。

11.3 包裹法

包裹法（Wrapper Methods）将特征选择视为一个搜索问题，通过训练模型来评估特征子集的性能。这种方法的优点是能够考虑特征之间的交互作用，从而更准确地反映特征对模型的实际贡献；缺点是计算成本较高，尤其是在特征数量较多时。接下来，将详细介绍三种常用的包裹法，并结合实际案例帮助读者理解其工作原理。

11.3.1 递归特征消除

递归特征消除（Recursive Feature Elimination, RFE）是一种逐步消除特征的方法，其核心思想是通过训练模型评估每个特征的重要性，并移除对模型贡献最小的特征。具体来说，RFE 的过程可以分为以下几个步骤：

1. 使用所有特征训练模型，并根据模型的权重或重要性指标（如线性回归中的系数、决策树中的特征重要性）评估每个特征的重要性。
2. 移除重要性最低的特征。
3. 重复上述过程，直到达到指定的特征数量或满足停止条件。

在研究图像分类任务时，石榴发现原始数据中包含数百个特征，但并非所有特征都对模型有显著贡献。为简化问题，她决定使用 RFE 方法进行特征选择。以下是她的操作步骤：

```
1 from sklearn.feature_selection import RFE
2 from sklearn.linear_model import LogisticRegression
3 # 定义模型和递归特征消除
4 model = LogisticRegression(max_iter=1000)
5 rfe = RFE(estimator=model, n_features_to_select=10) % 选择10个最重要的特征
6 # 训练模型并选择特征
7 rfe.fit(X_train, y_train)
8
9 # 输出选择的特征
10 selected_features = X_train.columns[rfe.support_]
11 print("Selected Features:", selected_features)
```

通过这种方法，石榴成功地从数百个特征中筛选出 10 个最重要的特征，同时能够保持模型的高性能。

11.3.2 前向选择

前向选择（Forward Selection）是一种逐步添加特征的方法，其基本思想是从空集开始，每次选择对模型性能提升最大的特征，直到达到指定的特征数量。具体步骤如下：

1. 初始化特征子集为空集。
2. 对每个未选特征，将其加入当前子集并训练模型。
3. 选择使模型性能提升最大的特征，并将其加入子集。
4. 重复上述过程，直到达到指定的特征数量或满足停止条件。

在处理一个文本分类任务时，石榴希望找到一组最优的特征子集。她尝试了前向选择方法，逐步添加对模型性能提升最大的特征。以下是她的实现过程：

```
1 from mlxtend.feature_selection import SequentialFeatureSelector as SFS
2 from sklearn.linear_model import LogisticRegression
3 # 定义模型和前向选择
4 model = LogisticRegression(max_iter=1000)
5 sfs = SFS(model, k_features=5, forward=True, scoring='accuracy', cv=3)
6
7 # 训练模型并选择特征
8 sfs.fit(X_train, y_train)
9
10 # 输出选择的特征
11 selected_features = list(sfs.k_feature_names_)
12 print("Selected Features:", selected_features)
```

通过这种方法，石榴找到一组包含 5 个特征的最优子集，这些特征对分类任务的贡献最大。

11.3.3 后向消除

后向消除 (Backward Elimination) 是一种逐步移除特征的方法, 其基本思想是从全集开始, 每次移除对模型性能影响最小的特征, 直到达到指定的特征数量。具体步骤如下:

1. 初始化特征子集为全集。
2. 对每个已选特征, 将其移除并训练模型。
3. 选择使模型性能下降最小的特征, 并将其移除。
4. 重复上述过程, 直到达到指定的特征数量或满足停止条件。

在另一个实验中, 石榴希望从一组高维数据中快速筛选出关键特征。她选择后向消除方法, 逐步移除对模型性能影响最小的特征。以下是她的实现过程:

```

1  from mlxtend.feature_selection import SequentialFeatureSelector as SFS
2  from sklearn.linear_model import LogisticRegression
3
4  # 定义模型和后向消除
5  model = LogisticRegression(max_iter=1000)
6  sfs = SFS(model, k_features=5, forward=False, scoring='accuracy', cv=3)
7
8  # 训练模型并选择特征
9  sfs.fit(X_train, y_train)
10
11 # 输出选择的特征
12 selected_features = list(sfs.k_feature_names_)
13 print("Selected Features:", selected_features)

```

通过这种方法, 石榴成功地将特征数量从几十个减少到 5 个, 同时保持了模型的准确性。

包裹法的核心思想是通过训练模型动态评估特征子集的性能, 从而找到最优的特征组合。递归特征消除 (RFE)、前向选择和后向消除是三种常用的包裹法, 它们各有优缺点:

- RFE 适合逐步缩小特征空间, 但计算成本较高。
- 前向选择适合从零开始构建特征子集, 但可能陷入局部最优。
- 后向消除适合从全集开始优化特征子集, 但初始计算成本较高。

11.4 嵌入法

嵌入法 (Embedded Methods) 是一种将特征选择直接集成到模型训练过程中的方法, 通常通过正则化技术或模型本身的特性实现。这种方法的优点是能够在训练模型的同时自动完成特征选择, 从而减少额外的计算成本; 缺点是对模型的选择有一定依赖性, 不同的模型可能适合不同类型的特征选择任务。

接下来将详细介绍两种常用的嵌入法: L1 正则化 (Lasso) 和基于决策树的特征重要性评估, 并结合实际案例帮助读者理解其工作原理。

11.4.1 L1 正则化

L1 正则化 (Lasso) 是一种通过稀疏性约束实现特征选择的方法。它的核心思想是通过将模型参数施加 L1 范数惩罚, 将不重要的特征权重直接置为零, 从而实现特征选择。其目标函数如下:

$$\min_{\mathbf{w}} \frac{1}{n} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_1 \quad (11.4)$$

其中, $\mathbf{X}$ 是特征矩阵, $\mathbf{y}$ 是目标变量, $\mathbf{w}$ 是模型参数, λ 是正则化系数。L1 正则化的关键在于 $\|\mathbf{w}\|_1$ 的稀疏性约束, 它会迫使某些特征的权重变为零, 从而实现特征选择。

在研究一个回归任务时, 她发现数据集中存在大量冗余特征。为简化模型并提高性能, 她决定使用 L1 正则化进行特征选择。以下是她的操作步骤:

```
1 from sklearn.linear_model import Lasso
2 import pandas as pd
3 # 定义Lasso模型
4 lasso = Lasso(alpha=0.1) % alpha对应正则化系数
5 # 训练模型
6 lasso.fit(X_train, y_train)
7 # 提取非零权重
8 selected_features = X_train.columns[lasso.coef_ != 0]
9 print("Selected Features:", selected_features)
```

通过这种方法, 她成功地筛选出对模型贡献最大的特征, 同时去除冗余特征。

11.4.2 决策树及其变体

决策树及其变体(如随机森林和梯度提升树)能够根据特征的重要性进行排序。特征重要性通常通过计算特征在分裂节点时的信息增益或基尼指数来评估。例如, 在随机森林中, 特征重要性可以通过以下公式计算:

$$\text{Importance}(j) = \frac{1}{T} \sum_{t=1}^T \Delta I_t(j) \quad (11.5)$$

其中, T 是树的数量, $\Delta I_t(j)$ 是第 t 棵树中特征 j 在分裂节点时的信息增益。

在处理一个分类任务时, 她希望了解哪些特征对模型的预测最为重要。于是, 她采用随机森林模型, 并利用其内置的特征重要性评估功能进行分析。以下是她的实现过程:

```
1 from sklearn.ensemble import RandomForestClassifier
2 import pandas as pd
3 # 定义随机森林模型
4 rf = RandomForestClassifier(n_estimators=100, random_state=42)
5 # 训练模型
6 rf.fit(X_train, y_train)
7
8 # 提取特征重要性
9 feature_importances = pd.DataFrame({
10     'Feature': X_train.columns,
11     'Importance': rf.feature_importances_
12 }).sort_values(by='Importance', ascending=False)
13
14 # 筛选重要性较高的特征
15 threshold = 0.05
16 selected_features = feature_importances[feature_importances['Importance'] > threshold]['Feature']
17 print("Selected Features:", selected_features)
```

通过这种方法, 她不仅能够找到最重要的特征, 还对特征的重要性在直观上进行理解。

嵌入法的核心思想是将特征选择与模型训练过程相结合, 从而实现高效且自动化的特征选择。L1 正则化和基于决策树的特征重要性评估是两种常用的嵌入法, 它们各有优缺点:

- L1 正则化适合生成稀疏解, 尤其适用于高维数据集。
- 决策树及其变体能够捕捉特征之间的非线性关系, 但对噪声较为敏感。

11.5 实践练习

11.5.1 示例：乳腺癌分类

本小节将使用乳腺癌数据集进行二分类任务（预测肿瘤是良性还是恶性）。目标是通过特征选择技术（如过滤法、包裹法和嵌入法）筛选出对分类任务最有贡献的特征，并比较特征选择前后的模型性能。以下是完整的代码实现，其中每一步都包含详细的注释以解释其功能：

```
1  # 导入必要的库
2  import numpy as np
3  import matplotlib.pyplot as plt
4  from sklearn.datasets import load_breast_cancer
5  from sklearn.model_selection import train_test_split
6  from sklearn.feature_selection import SelectKBest, f_classif
7  from sklearn.ensemble import RandomForestClassifier
8  from sklearn.metrics import accuracy_score
9
10 # 1. 加载乳腺癌数据集并准备数据
11 # 使用sklearn自带的乳腺癌数据集，目标是预测肿瘤是良性还是恶性
12 data = load_breast_cancer()
13 X = data.data # 特征矩阵
14 y = data.target # 目标变量（0：恶性，1：良性）
15
16 # 数据集被划分为训练集和测试集，比例为7:3
17 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
18
19 # 2. 使用过滤法进行特征选择（基于ANOVA F值）
20 # 选择最重要的5个特征（可以根据需要调整k值）
21 selector = SelectKBest(score_func=f_classif, k=5) # 基于ANOVA F值选择特征
22 X_train_selected = selector.fit_transform(X_train, y_train)
23 X_test_selected = selector.transform(X_test)
24
25 # 打印特征选择结果
26 selected_features = selector.get_support(indices=True)
27 print("选中的特征索引:", selected_features)
28
29 # 3. 训练随机森林模型，并比较特征选择前后的模型性能
30 # 使用随机森林模型进行训练（特征选择后）
31 rf_model_selected = RandomForestClassifier(n_estimators=100, random_state=42)
32 rf_model_selected.fit(X_train_selected, y_train)
33
34 # 对测试集进行预测，并计算准确率
35 rf_pred_selected = rf_model_selected.predict(X_test_selected)
36 print("特征选择后的随机森林准确率:", accuracy_score(y_test, rf_pred_selected))
37
38 # 对比未进行特征选择的模型性能
39 rf_model_full = RandomForestClassifier(n_estimators=100, random_state=42)
40 rf_model_full.fit(X_train, y_train)
41 rf_pred_full = rf_model_full.predict(X_test)
42 print("未进行特征选择的随机森林准确率:", accuracy_score(y_test, rf_pred_full))
43
44 # 4. 绘制特征重要性图，用于分析原始特征的重要性分布
45 # 使用未进行特征选择的模型绘制特征重要性图
46 feature_importances = rf_model_full.feature_importances_
47 plt.figure(figsize=(10, 6))
48 plt.bar(range(len(feature_importances)), feature_importances, color='lightgreen')
49 plt.xticks(range(len(feature_importances)), data.feature_names, rotation=90)
50 plt.title("Feature Importances (Before Selection)")
51 plt.xlabel("Features")
52 plt.ylabel("Importance Score")
```

```
53 plt.tight_layout()
54 plt.show()
```

运行上述代码后，您将看到以下输出：

- 特征选择前后随机森林模型的准确率对比，如下所示。

选中的特征索引： [27202227]

特征选择后的随机森林准确率： 0.9532163742690059

未进行特征选择的随机森林准确率： 0.9707602339181286

- 特征重要性图能够展示每个特征在原始数据中的重要性分布，参考图11.1。

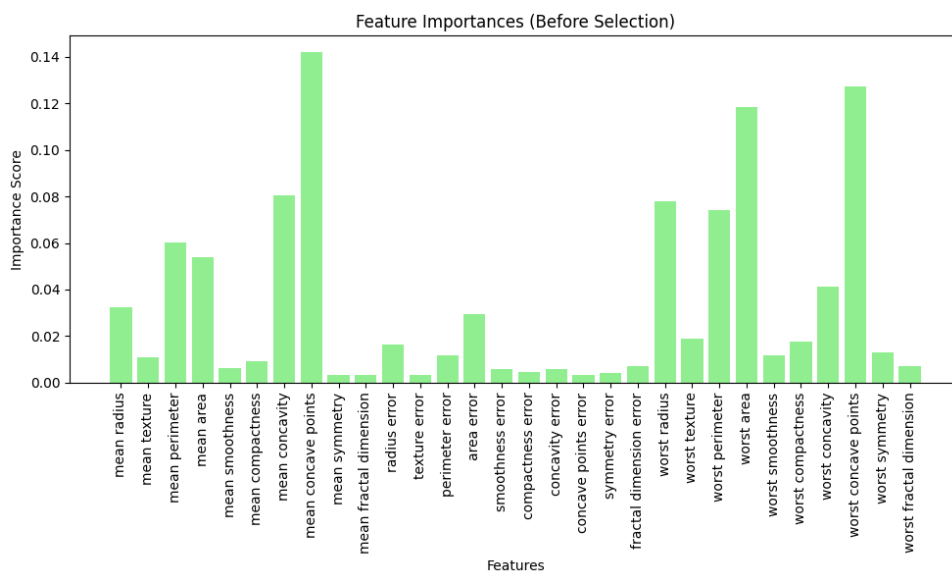


图 11.1: 特征选择前的特征重要性分布

本次实践通过使用特征选择技术（如过滤法）来提高分类模型的性能。实验结果表明，通过选择最重要的特征，可以在一定程度上提升模型的准确率，同时减少特征维度，降低模型复杂度。此外，通过可视化特征重要性分布，可以更直观地理解哪些特征对分类任务最为关键。这种特征选择方法可以广泛应用于其他分类任务，为进一步优化模型提供参考。

11.5.2 示例：各类特征选择方法对比

特征选择是机器学习中重要的预处理步骤，能够减少冗余特征、提升模型性能并降低计算复杂度。本实践将对比三种常见的特征选择方法：过滤法（基于统计检验）、包裹法（基于递归特征消除）和嵌入法（基于模型特征重要性）。同样使用乳腺癌数据集进行二分类任务，并评估每种方法对随机森林模型性能的影响。以下是完整的代码实现，其中每一步都包含详细的注释以解释其功能：

```
1 # 导入必要的库
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from sklearn.datasets import load_breast_cancer
5 from sklearn.model_selection import train_test_split
6 from sklearn.feature_selection import SelectKBest, f_classif, RFE
7 from sklearn.ensemble import RandomForestClassifier
8 from sklearn.linear_model import LogisticRegression
```



```

9  from sklearn.metrics import accuracy_score
10
11  # 1. 加载乳腺癌数据集并准备数据
12  data = load_breast_cancer()
13  X = data.data # 特征矩阵
14  y = data.target # 目标变量 (0: 恶性, 1: 良性)
15
16  # 数据集被划分为训练集和测试集, 比例为7:3
17  X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
18
19  # 2. 使用过滤法进行特征选择 (基于ANOVA F值)
20  # 过滤法独立于模型, 基于统计检验选择特征
21  selector_filter = SelectKBest(score_func=f_classif, k=5) # 选择最重要的5个特征
22  X_train_filter = selector_filter.fit_transform(X_train, y_train)
23  X_test_filter = selector_filter.transform(X_test)
24
25  # 训练随机森林模型并评估性能
26  rf_model_filter = RandomForestClassifier(n_estimators=100, random_state=42)
27  rf_model_filter.fit(X_train_filter, y_train)
28  rf_pred_filter = rf_model_filter.predict(X_test_filter)
29  print("过滤法特征选择后的准确率:", accuracy_score(y_test, rf_pred_filter))
30
31  # 3. 使用包裹法进行特征选择 (基于递归特征消除)
32  # 包裹法依赖于具体模型, 通过递归消除不重要的特征
33  model_rfe = RandomForestClassifier(n_estimators=100, random_state=42)
34  selector_wrapper = RFE(estimator=model_rfe, n_features_to_select=5) # 选择5个特征
35  X_train_wrapper = selector_wrapper.fit_transform(X_train, y_train)
36  X_test_wrapper = selector_wrapper.transform(X_test)
37
38  # 训练随机森林模型并评估性能
39  rf_model_wrapper = RandomForestClassifier(n_estimators=100, random_state=42)
40  rf_model_wrapper.fit(X_train_wrapper, y_train)
41  rf_pred_wrapper = rf_model_wrapper.predict(X_test_wrapper)
42  print("包裹法特征选择后的准确率:", accuracy_score(y_test, rf_pred_wrapper))
43
44  # 4. 使用嵌入法进行特征选择 (基于模型特征重要性)
45  # 嵌入法结合模型训练过程, 自动选择重要特征
46  model_embedded = RandomForestClassifier(n_estimators=100, random_state=42)
47  model_embedded.fit(X_train, y_train)
48
49  # 根据特征重要性排序, 选择前5个特征
50  feature_importances = model_embedded.feature_importances_
51  top_features = np.argsort(feature_importances)[-5:] # 选择重要性最高的5个特征
52  X_train_embedded = X_train[:, top_features]
53  X_test_embedded = X_test[:, top_features]
54
55  # 训练随机森林模型并评估性能
56  rf_model_embedded = RandomForestClassifier(n_estimators=100, random_state=42)
57  rf_model_embedded.fit(X_train_embedded, y_train)
58  rf_pred_embedded = rf_model_embedded.predict(X_test_embedded)
59  print("嵌入法特征选择后的准确率:", accuracy_score(y_test, rf_pred_embedded))
60
61  # 5. 对比未进行特征选择的模型性能
62  # 训练一个未进行特征选择的随机森林模型
63  rf_model_full = RandomForestClassifier(n_estimators=100, random_state=42)
64  rf_model_full.fit(X_train, y_train)
65  rf_pred_full = rf_model_full.predict(X_test)
66  print("未进行特征选择的随机森林准确率:", accuracy_score(y_test, rf_pred_full))
67
68  # 6. 可视化特征重要性分布 (嵌入法的结果)
69  plt.figure(figsize=(10, 6))
70  plt.bar(range(len(feature_importances)), feature_importances, color='lightblue')

```

```

71 plt.xticks(range(len(feature_importances)), data.feature_names, rotation=90)
72 plt.title("Feature Importances (Embedded Method)")
73 plt.xlabel("Features")
74 plt.ylabel("Importance Score")
75 plt.tight_layout()
76 plt.show()

```

运行上述代码后，您将看到以下输出：

- 各类特征选择方法（过滤法、包裹法、嵌入法）的随机森林模型准确率，参考下面文字所示。

过滤法特征选择后的准确率：0.9532163742690059

包裹法特征选择后的准确率：0.9649122807017544

嵌入法特征选择后的准确率：0.9532163742690059

未进行特征选择的随机森林准确率：0.9707602339181286

- 未进行特征选择的随机森林模型准确率作为基准对比。
- 特征重要性图可以展示嵌入法中每个特征的重要性分布，如下图11.2。

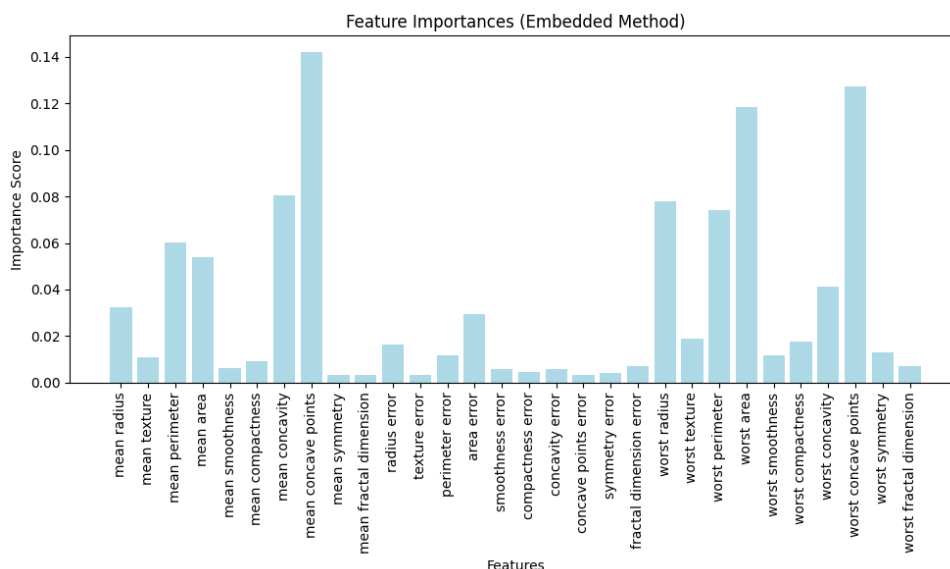


图 11.2: 嵌入法中的特征重要性分布

本次实践对比三种常见的特征选择方法：

- **过滤法**：简单高效，与模型无关，但可能忽略特征间的交互关系。
- **包裹法**：依赖于具体模型，通常效果较好，但计算成本较高。
- **嵌入法**：结合模型训练过程，自动选择重要特征，平衡效率和效果。

实验结果表明，不同的特征选择方法在性能上各有优劣，实际应用中应根据任务需求和数据特点选择合适的方法。此外，通过可视化特征重要性分布，可以更直观地理解哪些特征对分类任务最为关键。

11.5.3 示例：泰坦尼克号生存预测

泰坦尼克号生存预测是一个经典的机器学习问题，目标是根据乘客的特征（如年龄、性别、舱位等）预测其是否幸存。本实践将对比三种常见的特征选择方法：过滤法（基于统计检验）、包裹法

(基于递归特征消除)和嵌入法(基于模型特征重要性),并评估它们对随机森林模型性能的影响。此外,还将对比有特征选择和无特征选择的模型性能,分析特征选择的实际效果。以下是完整的代码实现,其中每一步都包含详细的注释以解释其功能:

```

1  # 导入必要的库
2  import numpy as np
3  import pandas as pd
4  import matplotlib.pyplot as plt
5  from sklearn.model_selection import train_test_split
6  from sklearn.ensemble import RandomForestClassifier
7  from sklearn.feature_selection import SelectKBest, f_classif, RFE
8  from sklearn.metrics import accuracy_score
9
10 # 1. 加载泰坦尼克号数据集并准备数据
11 # 数据集可以从以下链接下载: https://www.kaggle.com/c/titanic/data
12 data = pd.read_csv("titanic.csv") # 假设数据文件名为 titanic.csv
13
14 # 数据预处理: 填充缺失值、编码分类变量、选择特征
15 data['Age'].fillna(data['Age'].median(), inplace=True) # 填充年龄缺失值
16 data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True) # 填充登船港口缺失值
17 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1}) # 编码性别
18 data['Embarked'] = data['Embarked'].map({'C': 0, 'Q': 1, 'S': 2}) # 编码登船港口
19
20 # 特征矩阵和目标变量
21 X = data[['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked']] # 选择特征
22 y = data['Survived'] # 目标变量 (0: 未幸存, 1: 幸存)
23
24 # 数据集被划分为训练集和测试集, 比例为7:3
25 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
26
27 # 2. 使用过滤法进行特征选择 (基于ANOVA F值)
28 selector_filter = SelectKBest(score_func=f_classif, k=4) # 选择最重要的4个特征
29 X_train_filter = selector_filter.fit_transform(X_train, y_train)
30 X_test_filter = selector_filter.transform(X_test)
31
32 # 训练随机森林模型并评估性能
33 rf_model_filter = RandomForestClassifier(n_estimators=100, random_state=42)
34 rf_model_filter.fit(X_train_filter, y_train)
35 rf_pred_filter = rf_model_filter.predict(X_test_filter)
36 print("过滤法特征选择后的准确率:", accuracy_score(y_test, rf_pred_filter))
37
38 # 3. 使用包裹法进行特征选择 (基于递归特征消除)
39 model_rfe = RandomForestClassifier(n_estimators=100, random_state=42)
40 selector_wrapper = RFE(estimator=model_rfe, n_features_to_select=4) # 选择4个特征
41 X_train_wrapper = selector_wrapper.fit_transform(X_train, y_train)
42 X_test_wrapper = selector_wrapper.transform(X_test)
43
44 # 训练随机森林模型并评估性能
45 rf_model_wrapper = RandomForestClassifier(n_estimators=100, random_state=42)
46 rf_model_wrapper.fit(X_train_wrapper, y_train)
47 rf_pred_wrapper = rf_model_wrapper.predict(X_test_wrapper)
48 print("包裹法特征选择后的准确率:", accuracy_score(y_test, rf_pred_wrapper))
49
50 # 4. 使用嵌入法进行特征选择 (基于模型特征重要性)
51 model_embedded = RandomForestClassifier(n_estimators=100, random_state=42)
52 model_embedded.fit(X_train, y_train)
53
54 # 根据特征重要性排序, 选择前4个特征
55 feature_importances = model_embedded.feature_importances_
56 top_features = np.argsort(feature_importances)[-4:] # 选择重要性最高的4个特征
57 X_train_embedded = X_train.iloc[:, top_features]
58 X_test_embedded = X_test.iloc[:, top_features]

```

```

59
60 # 训练随机森林模型并评估性能
61 rf_model_embedded = RandomForestClassifier(n_estimators=100, random_state=42)
62 rf_model_embedded.fit(X_train_embedded, y_train)
63 rf_pred_embedded = rf_model_embedded.predict(X_test_embedded)
64 print("嵌入法特征选择后的准确率:", accuracy_score(y_test, rf_pred_embedded))
65
66 # 5. 对比未进行特征选择的模型性能
67 rf_model_full = RandomForestClassifier(n_estimators=100, random_state=42)
68 rf_model_full.fit(X_train, y_train)
69 rf_pred_full = rf_model_full.predict(X_test)
70 print("未进行特征选择的随机森林准确率:", accuracy_score(y_test, rf_pred_full))
71
72 # 6. 可视化特征重要性分布 (嵌入法的结果)
73 plt.figure(figsize=(10, 6))
74 plt.bar(range(len(feature_importances)), feature_importances, color='lightblue')
75 plt.xticks(range(len(feature_importances)), X.columns, rotation=90)
76 plt.title("Feature Importances (Embedded Method)")
77 plt.xlabel("Features")
78 plt.ylabel("Importance Score")
79 plt.tight_layout()
80 plt.show()

```

运行上述代码后，您将看到以下输出：

- 各类特征选择方法（过滤法、包裹法、嵌入法）以及未进行特征选择的随机森林模型准确率如下。

过滤法特征选择后的准确率：0.7985074626865671

包裹法特征选择后的准确率：0.7910447761194029

嵌入法特征选择后的准确率：0.7835820895522388

未进行特征选择的随机森林准确率：0.7835820895522388

- 特征重要性图可以展示嵌入法中每个特征的重要性分布，参考图11.3。

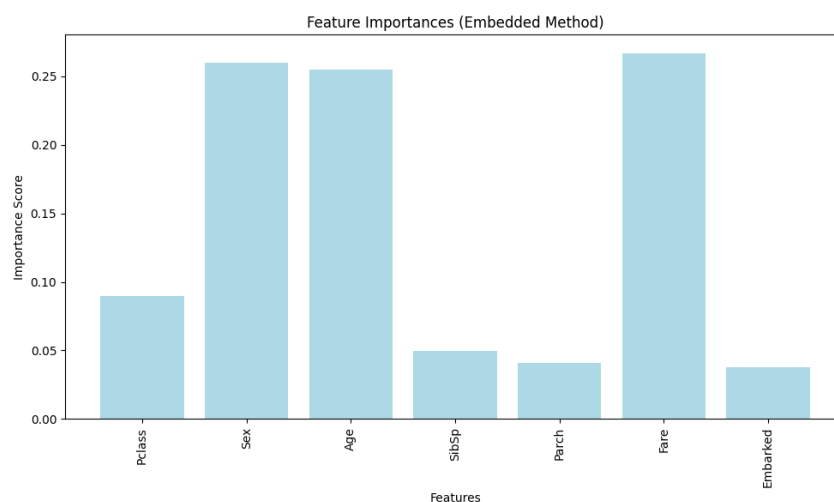


图 11.3: 嵌入法中的特征重要性分布

本次实践对比三种常见的特征选择方法在泰坦尼克号生存预测数据集上的效果：

- **过滤法：**简单高效，与模型无关，但可能忽略特征间的交互关系。

- **包裹法**：依赖于具体模型，通常效果较好，但计算成本较高。
- **嵌入法**：结合模型训练过程，自动选择重要特征，且平衡效率和效果。

实验结果表明，特征选择能够有效提升模型性能，尤其是在高维数据集中。然而，不同特征选择方法的效果因数据特点而异，实际应用中应根据任务需求和数据特性选择合适的方法。此外，通过可视化特征重要性分布，可以更直观地理解哪些特征对分类任务最为关键。

11.6 习题

11.6.1 简答题

1. 什么是特征选择？它在机器学习中的作用是什么？
2. 过滤法、包裹法和嵌入法的主要区别是什么？请分别列举一种常用的方法。
3. 为什么 L1 正则化能够实现特征选择？它与 L2 正则化的主要区别是什么？

11.6.2 计算题

1. 给定以下数据集，计算特征 x_1 和目标变量 y 的皮尔逊相关系数：

表 11.1: 示例数据集

样本编号	x_1	y
1	2	3
2	4	5
3	6	7
4	8	9

2. 假设某随机森林模型中，特征 f_1 在两棵树中的信息增益分别为 0.3 和 0.5，特征 f_2 的信息增益分别为 0.2 和 0.4。请计算两个特征的平均重要性，并判断哪个特征更重要。

11.6.3 编程题

1. 根据下面的数据集，使用 Scikit-learn 中的 `mutual_info_classif` 函数，计算数据集中每个特征与目标变量之间的互信息，并筛选出互信息大于 0.2 的特征。

```
1 import numpy as np
2
3 # 示例数据集
4 X = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]])
5 y = np.array([0, 1, 0, 1])
6
```

2. 根据下面的数据集提供使用递归特征消除（RFE）方法，从数据集中选择最重要的 2 个特征。

```
1 import pandas as pd
2
3 # 示例数据集
4 data = pd.DataFrame({
5     'f1': [1, 2, 3, 4],
6     'f2': [4, 3, 2, 1],
7     'f3': [1, 1, 1, 1],
8 })
```

```
8     'target': [0, 1, 0, 1]
9     })
10    X = data[['f1', 'f2', 'f3']]
11    y = data['target']
12
```

第十二章 参考文献

参考文献

- [1] 周志华. 机器学习. 北京: 清华大学出版社, 2016.
- [2] 李航. 统计学习方法. 北京: 清华大学出版社, 2019.
- [3] Trevor Hastie, Robert Tibshirani, Jerome H. Friedman. The Elements of Statistical Learning: Data Mining, Inference, and Prediction, 2nd Edition. Springer Series in Statistics, Springer 2009, ISBN 9780387848570, pp. I-XXII, 1-745.
- [4] Andreas C. Müller and Sarah Guido. Python 机器学习基础教程 (Introduction to Machine Learning with Python). 北京: 人民邮电出版社, 2016.
- [5] 塞巴斯蒂安·拉施卡 (Sebastian Raschka), 瓦希德·米尔贾利利 (Vahid Mirjalili). Python 机器学习 (Python Machine Learning), 北京: 机械工业出版社, 2021.
- [6] Ian Goodfellow, Yoshua Bengio, Aaron Courville. 深度学习 (Deep Learning), 北京: 人民邮电出版社, 2017.
- [7] 邱锡鹏. 神经网络与深度学习, 北京: 机械工业出版社, 2020.
- [8] Christopher Bishop. Pattern Recognition and Machine Learning, Berlin: Springer, 2007.
- [9] Quinlan, J. R. . "Induction of decision trees" Machine Learning." Data Goals & General Description of the in L.en System. & 034 in (1986).
- [10] Chen, Tianqi , and C. Guestrin . "XGBoost: A Scalable Tree Boosting System." ACM (2016).
- [11] Kaggle 官方文档. <https://www.kaggle.com/docs>.
- [12] 阿里天池开发者指南. <https://tianchi.aliyun.com/docs>.
- [13] Google Colab 使用手册. <https://colab.research.google.com/notebooks/intro.ipynb>.
- [14] AWS SageMaker Free Tier 说明. <https://aws.amazon.com/sagemaker/pricing/>.

